STM32 多功能小仪表 · 固件技术白皮书

固件版本 20261010E · 十页仪表的代码级解读

源码包	STM32多功能小仪表_最新源码_20261010E.zip（1,499,645 字节）
源码指纹	SHA-256 8A964F3B26790F557579C61FC3327B9CCCB64C37B94A635C97BDF70194BEC65D
固件版本	20261010E（`dashboard.h:11`），十页仪表（`dashboard.h:4`）
平台	STM32F103 + ST 标准外设库（SPL），Keil MDK 工程 `level/MDK-ARM/level.uvprojx`
代码规模	14 个 .c / 8 个 .h，15 章节，正文约 361,373 字符
证据约定	行号一律指向 `level/USER/*`（GBK 原件），与 UTF-8 阅读副本 1:1 对应
版本边界	本书解读 **20261010E**；工作区源码此后已被作者升级为 **20261011F**（十一页），差异说明见「摘要」末节
生成方式	Markdown 分章撰写 → 合并 → A4 排版 → 无头 Edge 打印为 PDF

本文只描述源码中确实存在的行为；未实现的机制（看门狗、DMA、互斥锁、RTOS）都在正文中显式标注为「未实现」。
所有断言附 `文件:行号`，可按行复核。凡源码中找不到证据的推断，正文均明确标注为推断。

目录

第 1 章　项目速览与目录结构 7

1.1 这个仪表是什么 7

1.2 硬件形态与引脚占用总表 7

1.3 仓库与目录结构 8

1.4 代码规模与分层现状 9

1.5 模块依赖矩阵 9

1.6 数据流鸟瞰 10

1.7 读代码的建议路线 11

本章发现的问题与建议 11

第 2 章　平台基础：时基、SysTick 与 TIM2 12

2.1 三层时基各管什么 12

2.2 时钟推导：8MHz 晶体如何变成 TIM2 的 1MHz 12

2.3 delay_micros16()：返回计数、差值由调用者算 13

2.4 delay_us / delay_ms：忙等实现、代价与隐患 13

2.5 中断向量、优先级与 ISR 职责边界 14

2.6 时间语义与回绕：四种判定原语 15

2.7 系统滴答与功耗：没有睡眠，也没有看门狗 15

本章发现的问题与建议 11

第 3 章　软件 I2C：位操作、卡总线恢复与错误熔断 17

本章要点 6

3.1 为什么用软件 I2C 17

3.2 初始化：SoftI2C_Init（i2c.c:34-47） 17

3.3 位级原语 18

3.4 卡总线守护：scl_released()（i2c.c:12-22） 19

3.5 故障熔断：i2c_fault 的真实作用域 19

3.6 总线恢复：SoftI2C_BusRecover()（i2c.c:55-81） 20

3.7 事务 API 20

3.8 流式接口：SoftI2C_Begin/SendByte/End（i2c.c:283-296） 21

3.9 并发模型：没有锁的协作式互斥 21

3.10 时序与电气风险清单 21

本章发现的问题与建议 11

第 4 章　器件驱动：MPU6050 与 DHT11 23

本章要点 6

4.1 MPU6050 初始化：逐寄存器精读 23

4.2 写后回读验收 24

4.3 WHO_AM_I 与 MPU6050_ReadID 24

4.4 数据通路：一次突发读 14 字节 24

4.5 DHT11：单总线非阻塞状态机 25

4.6 EXTI1_IRQHandler：中断里做位解码 26

4.7 解码、单位与失败语义 26

4.8 时间常量与"帧丢失"时上层看到什么 —

4.9 DHT11_Busy()：全工程的准互锁 27

本章发现的问题与建议 11

第 5 章　SPI NOR Flash 驱动：W25QXX 29

本章要点 6

5.1 引脚、总线与时钟 29

5.2 上电识别：唤醒 + JEDEC ID + 三重校验 30

5.3 命令集与地址宽度 30

5.4 超时熔断：transfer() 与 bus_error 锁存器 32

5.5 重试全部在上层：退避链 32
5.6 容量语义与三块区域的地址布局 33
5.7 工程立场：不做隐式破坏 34
本章发现的问题与建议 11
第 6 章　业务服务：主循环调度器与十页仪表盘 37
6.1 main() 启动序列：把最坏的等待放在最前 37
6.2 主循环：一个 5ms 节拍的相位式调度器 37
6.3 没有 RTOS 的并发：截止时间 + busy 门控 39
6.4 due() 与时间预算 39
6.5 状态放在哪儿：file-static、栈帧与直读寄存器 39
6.6 DashboardView：单结构体契约，但不是唯一数据通路 40
6.7 十页渲染 41
6.8 sprintf 审计 42
本章发现的问题与建议 11
第 7 章　存储服务：传感器记录日志与开机基准标定 44
7.1 记录区：容量算式与 Flash 地图 44
7.2 32 字节记录格式：CRC 与提交字节的分工 44
7.3 上电扫描：三种终止态与"绝不隐式擦除" —
7.4 写入状态机：四相位、写后回读、单点失败即终止 45
7.5 读取路径与 recent 环形缓冲 46
7.6 flags 位域与"没有有效数据就不记" —
7.7 开机基准标定：后台步进的 100 点 47
7.8 倾角换算：5730 的来历、误差与除零审计 47
7.9 pose_check：翻转与超范围 48
7.10 lpf 的更新率与滤波强度 48
本章发现的问题与建议 11
第 8 章　会话事件日志与复位溯源 50
本章要点 6
8.1 区域位置与容量算式 50
8.2 硬依赖：传感器日志不 READY，事件日志就不启动 50
8.3 记录格式：32 字节逐字段 50
8.4 扫描与恢复：开机第一次读 Flash 51
8.5 追加写状态机：四个相位、三次回读 52
8.6 槽位选择与双扇区轮换 52
8.7 心跳与时长聚合：断电只能给出下界 52
8.8 复位溯源链：从 RCC->CSR 到屏幕文本 —
8.9 互锁与调用点 53
8.10 写入寿命与计数上限 53
本章发现的问题与建议 11
第 9 章　人机交互：OLED 显示子系统 55
本章要点 6
9.1 显存模型：8 页 × 128 列与位序约定 55
9.2 从 7 位地址说起：0x3C 与丝印 0x78 55
9.3 初始化：16 条短事务与 41 字节命令 55
9.4 差分短块刷新（核心机制） 56
9.5 失败与恢复：need_recovery() 与 16 步重放 57
9.6 调光与现实中的"空闲"定义 —
9.7 刷新调度与"主循环唯一重绘点" —
9.8 绘图与字体 API 一览 59
9.9 一个 32 字节块的时间预算 60
9.10 本章发现的问题与建议 60

第 10 章　人机交互：编码器、按键、RGB、蜂鸣器与板载灯 62
本章要点 6
硬件映射与有效电平 62
编码器解码：1ms 轮询加一张格雷表 62
按键：20ms 去抖与 800ms 长按 62
跨中断传递：为什么这里必须关中断 63
TIM1 硬件 PWM：2kHz 载波与 3600 级分辨率 63
软件包络：200Hz 的呼吸与 gamma 校正 64
六种灯光模式与五档频闪 64
蜂鸣器：低电平触发与上电毛刺 65
板载 LED：心跳、报码与三方仲裁 65
主循环如何消费这些输入 65
本章发现的问题与建议 11
第 11 章　专题一：总线共享与 32 字节差分块刷新 67
本章要点 6
11.1 拓扑与电气前提 67
11.2 三个参与者的真实周期 67
11.3 三道（实为四道）互锁 67
11.4 量化：从两个实测锚点到每个块 68
11.5 为什么是 32 字节而不是 16 字节 69
11.6 故障路径：need_recovery()、SoftI2C_BusRecover() 与 i2c_fault 69
11.7 为什么全工程没有 I2C 互斥原语 70
本章发现的问题与建议 11
第 12 章　专题二：两条 Flash 日志的掉电安全与区域布局 72
本章要点 6
12.1 物理前提：4.5MHz、容量探测与页/扇区约束 72
12.2 区域布局：三块区域与它们之间的依赖 73
12.3 记录格式：两套 32 字节格式逐字段对照 73
12.4 提交协议：四阶段状态机与原子性论证 74
12.5 两套"陌生数据"判据的差异 —
12.6 扇区轮换策略：两种相反的取舍 76
12.7 断电损失边界：一条会话能证明什么 77
12.8 容量换算与寿命 78
12.9 两种"事件日志消失"的可区分性，与恢复路径的不对称 —
本章发现的问题与建议 11
本章小结 83
第 13 章　专题三：姿态解算与标定的整数数学 84
本章要点 6
13.1 从 14 个字节到屏幕上的小数点：单位链 84
13.2 为什么是 Q8 乘法而不是左移 85
13.3 一阶低通的离散化：τ、f_c、极限环 85
13.4 标定：100×10 ms 的整数统计 86
13.5 stable_text：0.15° 迟滞与舍入方向 87
13.6 两份逐字节相同的整数开方 88
13.7 ±150 径向钳位与它的消费者 89
13.8 气泡映射：15.0° 满量程，之后只剩方向 89
13.9 一次没有副作用的初始化：写-回读-校验 91
13.10 误差总账与替代方案 91
本章发现的问题与建议 11
第 14 章　专题四：开机自检 POST 与故障处置 96
本章要点 6

14.1 三个执行体与一条时间线 96
14.2 十一步怎么被推进：三种节拍 97
14.3 11 项自检的证据强度分级 99
14.4 第 1 项 CLOCK 的第一个合取项是恒真式 99
14.5 两项时基检查只证明"在走"，不证明"走得准" —
14.6 第 9 项 ENCODER：被验证的是"配置写进去了" —
14.7 第 10 项 OUTPUTS 的确定性假阴性：0xAAA 与 0xBBB 100
14.8 6.5 秒窗口：最坏路径逐项累加 101
14.9 两个扫描等待的 guard 是自旋计数，不是时间界 102
14.10 两处"标签即公式"：64KB READ 与 8KB READ —
14.11 结果字面量的全集与两处边界 103
14.12 本章发现的问题与建议 103
第 15 章　全局缺陷清单、改进路线与工程结论 105
本章要点 6
15.1 总评 105
15.2 按严重度排序的前 10 条 105
15.3 缺陷总表 105
15.4 可复用的工程资产 110
15.5 如果重写：架构层面的六个决定 110
15.6 三阶段改进路线 111
15.7 工程结论 112
15.8 最小验收清单 112
本章发现的问题与建议 11

摘要

本文是一份**代码级**的固件技术白皮书。它不满足于描述"这个仪表有哪些功能"，而是逐个模块 打开源码，把每一个关键函数的签名、寄存器取值、时间常数、状态迁移与错误码写成可核对的清单， 并把"为什么这么写"讲清楚。全文所有技术断言都指向 `文件:行号`，读者可以逐条回源码复核。

被解读的对象是 `STM32多功能小仪表_最新源码_20261010E.zip`（1,499,645 字节， SHA-256 `8A964F3B…E65D`）解出的固件源码：14 个 `.c` + 8 个 `.h`， 共 **2,900 余行**业务代码，跑在 STM32F103 上，用 ST 标准外设库（SPL）而非 HAL， 无 RTOS、无动态内存、无看门狗喂狗、无 DMA。它有 10 个显示页面、一条软件 I2C 总线（OLED 与 MPU6050 共享）、一条 SPI 总线（W25QXX NOR Flash 上跑着两套掉电安全的日志）、一个编码器加一个按键 的极简输入，以及一套用 TIM1 硬件 PWM 驱动的 RGB 状态灯。

阅读顺序建议：第 1 章建立全貌与依赖矩阵；第 2–5 章是平台与驱动层（时基、软件 I2C、传感器、 Flash）；第 6–8 章是业务与存储层（主循环调度、仪表盘、两套日志与标定）；第 9–10 章是人机交互； 第 11–14 章是四个跨模块专题（总线共享的时间切片、掉电安全协议、整数姿态数学、开机自检）； 第 15 章是缺陷总表与改进路线。只想快速了解结论的读者，可以直接读第 1 章的「本章要点」与第 15 章。

版本边界：本书描述哪一版代码

本书逐行解读的对象是**交付包** **`20261010E`**（十页仪表）。写作期间，工作区源码已由作者 升级为 **`20261011F`**（十一页），因此正文中的行号对应 `20261010E` 那一次快照，而不是当前 `level/USER/` 的最新内容。作者修改了 5 个文件： `dashboard.h`（29→32 行）、 `dashboard.c`（360→399）、 `main.c`（664→671）、 `eventlog.c`（206→223）、 `eventlog.h`（22→23）；其余 22 个源文件与快照逐行一致。

新版做的是**功能增量**而非重构： `DASHBOARD_FW_VERSION` 由 `"20261010E"` 改为 `"20261011F"`（`dashboard.h:13`）、 `DASHBOARD_PAGE_COUNT` 由 10 改为 11（`dashboard.h:4`），新增第 11 页 `POST_PAGE`（`dashboard.h:8`）、 `Dashboard_SetPostResults()` 与 `EventLog_CompareUptime()`， 让开机自检的 11 项结果在自检结束后仍可翻看。构建规模随之从 `Code=28912 / RO=2524 / RW=420 / ZI=7628` （这是本书引用的 20261010E 构型）变为 `Code=29728 / RO=2616 / RW=520 / ZI=8720`； `level.map` 也重排过一次，所以书中链接器映像行号（如 `level.map:943`、 `:2272`、 `:2585`）同样属于旧构建：同一批符号在当前映像里已移到 `:947-950`（i2c 四个被删函数）、 `:951-955`（oled 五个）、 `:957`（`MPU6050_TempToC`）， `OLED_F8x16` 从 `:2272 / 0x080071dc` 变为 `:2302 / 0x0800750c`， `Image component sizes` 从 `:2585` 移到 `:2619`。

怎么读这两版：第 2、3、5、9、10、11 章的驱动与专题分析对两版都成立（这些文件逐行未变）； 凡引用 `main.c` / `dashboard.c` / `dashboard.h` / `eventlog.c` / `eventlog.h` 的行号，请以本书所附的 `20261010E` 快照（ `.archify/architecture-stm32-meter-20261010-213835/src-utf8/`）为准。第 15 章的缺陷结论按 `20261010E` 判定——其中第 14 章「POST 结论只存活一帧」这一条，正好被新版的 POST 结果页 部分解决，这从反面印证了该缺陷的诊断是对的。

第 1 章　项目速览与目录结构

本章要点

- 这是一块 STM32F103C8 上的桌面小仪表：MPU6050 姿态 + DHT11 温湿度 + W25QXX 双区日志 + SSD1306 OLED 十页界面。固件版本 `20261010E`（`dashboard.h:11`），页数上限 `DASHBOARD_PAGE_COUNT 10`（`dashboard.h:4`）。
- 外设集中在 GPIOA/GPIOB/GPIOC 三组 I/O 上：一条软件 I2C（PB6/PB7）同时挂 OLED 与 MPU6050，SPI1（PA4–PA7）挂 Flash，DHT11 占 PA1 + EXTI1，人机交互占 PB0/PB1/PB10 与 PA8/9/10，蜂鸣器 PB8、板载灯 PC13。
- 业务代码 14 个 `.c` 共 2864 行、15 个 `.h` 共 667 行。Keil ARM Compiler 5.06 update 7 构建 `0 Error(s), 0 Warning(s)`，`Code=28912`（`build.log:1`、`build.log:5`、`build.log:7`）。
- **未实现**：没有 RTOS、没有动态内存分配、没有看门狗、没有 DMA、没有互斥锁、主循环没有 `__WFI`。全工程搜 `malloc|free|WWDG|IWDG|DMA_|osThread|mutex` 只有 2 处命中，且都是把复位原因位翻译成字符串（`main.c:52-53`）。
- 模块间唯一的结构化数据契约是 `DashboardView`（`dashboard.h:12-25`）；但 `main.c` 直接读 `SysTick` / `TIM2` / `TIM1` / `GPIOB` / `RCC` 的寄存器做上电自检（`main.c:247-283`、`main.c:505`），这是全工程最明显的分层破口。

1.1 这个仪表是什么

一句话：**一块 STM32F103C8 驱动的小型桌面仪表——用一枚 MPU6050 做水平仪与姿态显示、一枚 DHT11 采温湿度、一片 W25QXX Flash 存两类日志，在 0.96 寸 SSD1306 OLED 上翻十页界面，靠旋转编码器、按钮、RGB 灯和蜂鸣器完成全部人机交互。**

它的功能边界可以从"页"看出来：十个页面就是它愿意展示的全部能力。页名总表是权威清单，它在渲染入口里以数组形式硬编码：

```
static const char *titles[DASHBOARD_PAGE_COUNT]={"LEVEL","CLIMATE","HISTORY","SYSTEM","RAW DATA","TREND","DEBUG","STROBE","RUNTIME","SESSIONS"};
```

（`dashboard.c:321`）

页号	页名	内容	页宏
0	LEVEL	水平仪：气泡 + 俯仰/横滚角度，开机默认页	无（裸数字）
1	CLIMATE	温湿度实时值与是否在记录	无（裸数字）
2	HISTORY	最近若干条传感器日志（32 字节记录解码后显示）	无（裸数字）
3	SYSTEM	固件版本、Flash 状态、采样率/帧率、最大循环耗时	无（裸数字）
4	RAW DATA	MPU6050 原始三轴加速度/角速度/温度与姿态合法性	无（裸数字）
5	TREND	俯仰/横滚的历史曲线（环形缓冲）	无（裸数字）
6	DEBUG	传感器日志内部状态、错误计数、复位原因	无（裸数字）
7	STROBE	红蓝频闪演示页，翻到即触发	`STROBE_PAGE`（`dashboard.h:5`）
8	RUNTIME	本次运行时长（HH:MM:SS）、启动次数、上一条会话时长	`RUNTIME_PAGE`（`dashboard.h:6`）
9	SESSIONS	会话事件表，可纯旋转浏览	`EVENT_PAGE`（`dashboard.h:7`）

页宏只覆盖了后三页。渲染分派里前七页写的是裸数字 `0`–`6`（`dashboard.c:328-334`），只有 STROBE/RUNTIME 走宏、SESSIONS 靠 `else` 兜底（`dashboard.c:335-336`）。这不算错，但增删页面时要手工对齐三处（`titles[]`、分派链、页数宏），本章末尾会把它列为问题。

1.2 硬件形态与引脚占用总表

一张表交代全部对外连接。每一行都对应源码里实际写下的配置，而不是接线图的转述。

外设	引脚	电气/参数	关键证据
软件 I2C 总线	SCL = PB6，SDA = PB7	两脚均为开漏输出（`GPIO_Mode_Out_OD`），读时直接取 IDR；两模块自带的上拉并联后约 2.35 kΩ	`i2c.h:24-26`、`i2c.h:12-16`
总线速率	—	`I2C_DELAY_LOOPS 8`，72 MHz、`-O0` 下约 250 kHz；整屏刷新 85 ms → 37 ms	`i2c.h:29-36`
OLED	复用上面两个脚	SSD1306 0.96"，128×64，4 针 I2C；7 位地址 `0x3C`（8 位 `0x78`）	`oled.h:7`、`oled.h:21`、`i2c.h:20`
MPU6050	复用上面两个脚	7 位地址 `0x68`，AD0 悬空（模块自带 4.7 kΩ 下拉）；不用 DMP	`mpu6050.h:19`、`mpu6050.h:10`、`mpu6050.h:12-14`
DHT11	PA1	上拉输入 + 外部中断 EXTI1 下降沿捕获；起始脉冲期间用 `EXTI->IMR &= ~EXTI_Line1` 屏蔽捕获；中断优先级抢占 1 / 子 0	`dht11.c:32-51`
DHT11 采样窗口	—	位判定：间隔 55–150 µs 为 0、120–210 µs 为 1（越界即判错）；收到 42 个边沿即置 `rx_done=1` 并屏蔽 EXTI	`dht11.c:63-70`
W25QXX Flash	SCK = PA5，MISO = PA6，MOSI = PA7，CS = PA4	软件片选；PA5/PA7 配 `AF_PP`，**PA6 配 `IN_FLOATING`**；SPI1 主机、分频 16 → 72 MHz / 16 = 4.5 MHz	`w25qxx.h:4`、`w25qxx.c:33-40`
旋转编码器 A/B	PB0，PB1	上拉输入；16 项格雷码查表，4 个边沿记 1 步，非法跳变清零；步数上限 ±20	`controls.h:4`、`controls.c:150-152`、`controls.c:170-179`
独立按钮	PB10	上拉输入，低有效；在 SysTick 中断里消抖	`controls.h:4`、`controls.c:150-152`、`controls.c:165`

外设	引脚	电气/参数	关键证据
RGB LED	PA8 = R，PA9 = G，PA10 = B	三个都是 TIM1 的复用推挽输出，对应 CH1/CH2/CH3；PWM 2 kHz、周期 3600 计数	`controls.c:160-162`、`controls.c:59-61`、`controls.c:16-17`
蜂鸣器	PB8	推挽输出，有源、低电平触发；初始化时先置高再切输出以避免上电毛刺	`buzzer.h:6-16`、`buzzer.c:4-5`、`buzzer.c:16-18`
板载 LED	PC13	推挽输出，低电平点亮；身兼心跳与"闪灯报码"两职	`led.c:3-8`、`led.c:16-19`、`led.c:24-27`
系统时钟	—	外部 8 MHz 晶振前提，`SYSCLK_FREQ_72MHz` = 72 MHz	`system_stm32f10x.c:65-68`、`system_stm32f10x.c:83`

三点值得单独记住：

第一，**OLED 与 MPU6050 共享一条软件 I2C 总线**，这是全工程资源冲突的根源，也是中断里只计时、不做总线传输的原因（`main.c:16`）。DHT11 采集期间主循环还会主动让出 OLED 服务（`main.c:661` 的 `if(!DHT11_Busy())`）。

第二，**PA6 被配成浮空输入而不是复用推挽**（`w25qxx.c:37`）。功能上可行——SPI 主机模式下 MISO 本就该是输入——但它与相邻两根线的写法不同，读代码时容易被误认为笔误。

第三，**蜂鸣器与板载 LED 都是低电平有效**，两者初始化顺序有讲究：`main()` 第一步就是 `Buzzer_Init()`，把"PB8 悬空可能上电嘀一声"的窗口压到最短（`buzzer.h:12-14`、`main.c:508`）。

1.3 仓库与目录结构

工作区里唯一在开发的是 `level/` 这一层。它下面只有四个子目录，加四个文件：

路径	内容	规模
`level/USER/`	**全部业务源码**，14 个业务 `.c` + 15 个 `.h` + `system_stm32f10x.c`	30 个文件
`level/Libraries/`	ST 标准外设库（SPL）与 CMSIS	82 个文件：CMSIS 36、`STM32F10x_StdPeriph_Driver/inc` 23、`src` 23
`level/MDK-ARM/`	Keil 工程与构建产物：`level.uvprojx`、`build.log`、`Obj/`、`List/`、`RTE/` 等	11 项
`level/docs/`	项目文档：`dashboard.md` 207 行、`validation.md` 181 行、`wiring-beginner.md` 259 行、`sessions.md` 25 行、`strobe.md` 43 行，另有 6 张图与一份说明书 docx/pdf	13 项
`level/README.md`、`level/readme.txt`	接线表、设计决定及理由	—
`level/build.cmd`	一键编译脚本（自动查找 UV4）	—

`Libraries/` 只有 `CMSIS` 与 `STM32F10x_StdPeriph_Driver` 两项；CMSIS 下面又分成 `CM3/CoreSupport`（`core_cm3.c/.h`）和 `CM3/DeviceSupport/ST/STM32F10x/`（`stm32f10x.h`、`system_stm32f10x.h`），后者再带一个 `startup/` 放五种工具链的启动文件——本工程用的是 `startup_stm32f10x_md.s`（`level.uvprojx:670`）。

按"文件 - 行数 - 职责"过一遍 `USER/`：

文件	行数	职责
`main.c`	664	初始化顺序、11 项上电自检、主循环调度（采样/滤波/标定/姿态/重绘/存储/重试）、Flash 退避重试、RGB 状态选择
`dashboard.c`	360	十页渲染、页面分发表、趋势环形缓冲、气泡与曲线绘制
`oled.c`	420	SSD1306 初始化、显存 `OLED_GRAM`、32 字节差分块刷新、对比度渐变与掉电重建、绘图原语与字符
`i2c.c`	296	软件 I2C 位操作与超时、总线恢复 `SoftI2C_BusRecover`、地址探针
`controls.c`	225	编码器格雷码解码、按钮消抖与长按、TIM1 三通道 PWM、STROBE 灯效、Q8 定点调光
`eventlog.c`	206	事件区布局、扫描与校验、BOOT/ALIVE 记录、提交字节 + CRC + 回读、会话聚合
`stm32f10x_it.c`	163	中断向量实现；`SysTick_Handler` 里调用 `delay_tick()` 与 `Controls_Tick()`（`stm32f10x_it.c:136-139`）
`logger.c`	140	传感器日志区：32 字节记录编解码、扇区擦除、环形覆盖
`dht11.c`	105	DHT11 状态机、EXTI1 边沿捕获、校验与失效判定
`mpu6050.c`	99	MPU6050 寄存器初始化与 14 字节连读
`w25qxx.c`	82	SPI1 + 软件片选、JEDEC 识别、状态轮询、按页写、4 KB 擦除
`buzzer.c`	39	PB8 高低电平控制与阻塞式短鸣
`delay.c`	33	TIM2 1 MHz 自由运行计时 + SysTick 毫秒计数
`led.c`	32	PC13 点亮/熄灭
`system_stm32f10x.c`	1033	CMSIS 时钟配置模板（本工程走 `SYSCLK_FREQ_72MHz` 分支）

头文件 15 个、共 667 行，其中 `oledfont.h` 293 行（字模数据）最大，其余是各模块约 10–56 行的接口。

> 行数口径：本章全部行号以文件内 LF 字节逐字节计数为准，`read` 工具报告的"total N lines"与之一致。**不要用 `(Get-Content <file>).Count` 统计**——在多字节编码的文件上它会少算（例如会报 `oled.h` 38 行，实际 48 行）。本白皮书其余章节若见口径差异，均以此为准。

1.4 代码规模与分层现状

业务代码总量：14 个业务 `.c` 共 **2864 行**，15 个 `.h` 共 **667 行**，合计 3531 行（不含 `USER/system_stm32f10x.c` 这 1033 行 CMSIS 模板，也不含 `Libraries/`）。

编译器给出的实际占用（`build.log:5`）：

```
Program Size: Code=28912 RO-data=2524 RW-data=420 ZI-data=7628
```

（`build.log:5`）

按此行相加，Flash 占用 = Code + RO-data + RW-data = 31856 B ≈ 31.1 KiB，占 STM32F103C8 的 64 KiB 约 48.6%；静态 RAM 占用 = RW-data + ZI-data = 8048 B ≈ 7.86 KiB，占 20 KiB 约 39.3%。同一份日志确认编译器是 `V5.06 update 7 (build 960)`（AC5），结果 `0 Error(s), 0 Warning(s)`（`build.log:1`、`build.log:7`）。

工程设置里能读到的事实（不臆测命令行开关）：目标名 `level`（`level.uvprojx:10`）、器件 `STM32F103C8`（`level.uvprojx:17`）、CPU 描述 `IRAM(0x20000000,0x00005000) IROM(0x08000000,0x00010000) CPUTYPE("Cortex-M3") CLOCK(12000000)`（`level.uvprojx:21`）、`<uAC6>0</uAC6>` 即使用 AC5（`level.uvprojx:14`）、C99 打开（`level.uvprojx:329`）、生成 HEX（`level.uvprojx:55`）。

分层现状：代码按"驱动 → 存储 → UI → 应用"四层组织，但只有 UI 层是干净的。

- 驱动层：`delay`、`i2c`、`w25qxx`、`mpu6050`、`dht11`、`oled`、`controls`、`led`、`buzzer`。它们之间只有 `delay` 这一个公共底座，互不调用。
- 存储层：`logger` 建立在 `w25qxx` 之上；`eventlog` 建立在 `w25qxx` **和** `logger` 之上（`eventlog.c:2-3`）。
- UI 层：`dashboard` 只通过 `DashboardView` 收数据、只调用 `oled` 输出，界面与业务解耦得比较彻底。
- 应用层：`main.c` 负责一切调度，也负责一切"跨界"判断。

明确未实现的机制（这是本章的结论之一，不是遗漏）：

机制	结论	证据
RTOS	未实现	搜 `osThread` / `FreeRTOS` / `vTask` / `mutex` / `Semaphore` 在 `level/USER` 下 0 命中
动态内存	未实现	搜 `malloc` / `calloc` / `realloc` / `free(` 在 `level/USER` 下 0 命中
看门狗	未实现	搜 `IWDG` / `WWDG` 全工程只有 2 处命中，都是复位原因解码：`main.c:52` 的 `if(flags&RCC_CSR_IWDGRSTF)return "IWD";` 与 `main.c:53` 的 `if(flags&RCC_CSR_WWDGRSTF)return "WWD";`
DMA	未实现	搜 `DMA_` 在 `level/USER` 下 0 命中
`__WFI` 低功耗等待	未实现	搜 `__WFI` 在 14 个业务 `.c` 下 0 命中

复现命令与结果：

```
grep -rE "malloc|calloc|realloc|WWDG|IWDG|DMA_|osThread|mutex|FreeRTOS" level/USER
→ 2 matches: main.c:52 (RCC_CSR_IWDGRSTF), main.c:53 (RCC_CSR_WWDGRSTF)

grep -rE "__WFI" level/USER/*.c
→ 0 matches
```

`level/USER` 里没有任何一处写 `IWDG_Enable()` 或喂狗动作，所以 `main.c:52-53` 那两行只是"读复位原因"——它在 DEBUG 页告诉使用者"上次是看门狗复位"，而本固件自己永远不会触发看门狗。这一点在稳定性盘点上很关键：**一旦主循环卡在某处，没有任何机制会自动复位。**

中断面同样极窄：只有两个中断源。`SysTick_Handler` 每 1 ms 调用 `delay_tick()` 累加毫秒、调用 `Controls_Tick()` 采样按键与编码器（`stm32f10x_it.c:136-139`）；`EXTI1_IRQHandler` 只做 DHT11 的边沿时间戳记录（`dht11.c:53-71`）。**所有总线传输（I2C 读写、OLED 刷新、Flash 读写、DHT11 状态机推进）都发生在主循环里**（`main.c:16`），所以不需要互斥锁——这是用"单线程 + 非阻塞状态机"换掉了 RTOS 与锁，代价是主循环里任何一次阻塞调用都会直接拉长整个系统的响应时间。

1.5 模块依赖矩阵

依据是每个 `.c` 的 `#include` 列表加跨模块符号的实际调用统计（按词界前缀匹配，含该文件自身的函数定义）。

模块	直接依赖（`#include`）	实际调用的外部模块
`delay`	`delay.h`	无
`led`	`led.h`	无
`controls`	`controls.h`	无（自管 TIM1/GPIO）
`buzzer`	`buzzer.h`、`delay.h`	`delay`（阻塞短鸣）
`i2c`	`i2c.h`、`delay.h`	`delay`（超时计时）
`w25qxx`	`w25qxx.h`、`delay.h`	`delay`
`dht11`	`dht11.h`、`delay.h`、`<string.h>`	`delay`
`mpu6050`	`mpu6050.h`、`i2c.h`、`delay.h`	`i2c`、`delay`
`oled`	`oled.h`、`oledfont.h`、`i2c.h`	`i2c`
`logger`	`logger.h`、`w25qxx.h`、`<string.h>`	`w25qxx`

模块	直接依赖（#include）	实际调用的外部模块
eventlog	eventlog.h、logger.h、w25qxx.h、<string.h>	w25qxx、logger
dashboard	dashboard.h、oled.h、dht11.h、logger.h、w25qxx.h、controls.h、eventlog.h、<stdio.h>、<string.h>	oled、dht11、logger、w25qxx、controls、eventlog
main	上述全部头文件 + stm32f10x.h、<string.h>	全部模块
stm32f10x_it	stm32f10x_it.h、delay.h、controls.h	delay、controls

符号级统计印证同一张图（数字是该 .c 里出现的次数，含自身定义）：

调用方	主要被调模块与次数
main.c	post_ 70、Logger_ 41、OLED_ 31、delay_ 23、EventLog_ 23、DHT11_ 18、Controls_ 14、SoftI2C_ 9、LED_ 7、MPU6050_ 5、Buzzer_ 5、Dashboard_ 3、W25Q_ 2
dashboard.c	OLED_ 81、EventLog_ 12、Logger_ 7、DHT11_ 6、W25Q_ 4、Dashboard_ 3、Controls_ 2
eventlog.c	W25Q_ 13、EventLog_ 9、Logger_ 3
logger.c	W25Q_ 14、Logger_ 13
mpu6050.c	MPU6050_ 31、SoftI2C_ 10、delay_ 3
oled.c	OLED_ 70、SoftI2C_ 7

几处不是"干净分层"的真实耦合，必须在读代码前知道：

1. main.c 绕开驱动层，直接读外设寄存器做上电自检：SysTick->CTRL 与 SysTick->LOAD（main.c:247）、TIM2->CR1 与 TIM2->PSC（main.c:263-264）、GPIOB->CRL 与 GPIOB->ODR（main.c:272）、TIM1->CR1/BDTR/CCER/ARR/CCMR1/CCMR2 与 GPIOA->CRH（main.c:278-283）；另有一处是复位原因 RCC->CSR（main.c:505）。后果是——**改任何一个驱动的引脚配置或 PWM 参数，都必须同步改 main.c 里的自检期望值，否则上电自检会失败并把设备卡在闪灯循环里。**
2. eventlog 硬依赖 logger。不只是 #include（eventlog.c:2），而是状态机互锁：eventlog.c:91 写着 if(Logger_State()!=LOG_READY||W25Q_Capacity()<SENSOR_BYTES+EVENT_BYTES)return;——传感器日志没进入 READY，事件日志就不启动。main.c:612-615 也只在 Logger_State()==LOG_READY 时才调用 EventLog_Init。这是**单点故障**：传感器日志区一旦损坏，复位溯源功能跟着一起失效，而两者的数据在 Flash 上本来是分开的两块区域（eventlog.c:92 用 W25Q_Capacity()-SENSOR_BYTES-EVENT_BYTES 算事件区基址）。
3. dashboard 与外界只通过 DashboardView 交换数据（dashboard.h:12-25），但它反过来直接 #include 了六个驱动/存储模块的头文件（dashboard.c:1-9）并直接调用它们的 getter。方向是"UI 拉数据"，不是"驱动推数据"——干净的是数据契约，不是依赖方向。

1.6 数据流鸟瞰

整台仪器的运行可以拆成一条采样链和一条存储链，两条链都挂在同一个主循环上。

```
if(!OLED_RefreshBusy()&&due(now,frame_next)){
    view.mpu_valid=calibrated&&!stale&&pose_valid;view.calibrating=cal_active;view.cal_error=cal_error;
    view.pose_error=stale?POSE_OK:pose_error;
    view.raw_valid=sensor_ready&&!stale;
    view.cal_percent=cal_attempts;view.level=level_latched;
    view.pitch_x10=pitch;view.roll_x10=roll;view.pitch_text=pitch_text;view.roll_text=roll_text;
    view.uptime_s=uptime_seconds;view.toast=Controls_ButtonDown()?"KEY DOWN":toast;
    view.now_ms=now;
    view.strobe_hint=(uint8_t)(view.page==STROBE_PAGE&&(uint32_t)(now-strobe_since)<2000UL);
    Dashboard_Draw(&view);OLED_StartRefresh();rate_frames++;
    frame_next=now+(view.page==0?LEVEL_DISPLAY_INTERVAL_MS:DISPLAY_INTERVAL_MS);
}
if(!DHT11_Busy())OLED_Service(now);
OLED_RefreshStep();
```

（main.c:649-662）

采样链（传感器 → 屏幕）：

1. 上电初始化按固定顺序铺开：Dashboard_Init() → 读 RCC->CSR 得到复位原因 → Buzzer_Init() / LED_Init() / Controls_Init() / delay_init() → 开机短鸣 → SoftI2C_Init() + SoftI2C_BusRecover() → 用地址探针分别探测 OLED 0x3C 与 MPU6050 0x68 → OLED_Init() → post_startup() 跑 11 项自检（main.c:506-517）。任何一项致命失败都会退化成"闪板载灯报码"的死循环。
2. 进入主循环（main.c:531 的 while(1)，一直到 663 行）。
3. 每 5 ms（SAMPLE_INTERVAL_MS）采一次 MPU6050：MPU6050_ReadAll() 读回三轴加速度、角速度与温度（main.c:575），写入 view.accel_raw/gyro_raw/temperature_raw（main.c:578-579），随后依次进滤波器（main.c:581）、进开机基准标定（main.c:582）、做姿态合法性检查（main.c:584），再用整数运算换算出百分之一度的俯仰/横滚：px100=PITCH_DIR*((lpf[0]/256-base_ax)*5730L/base_az);（main.c:586-587）。
4. DHT11 走自己的状态机：主循环每轮调用 DHT11_Task(now) 推进（main.c:532），真正的位捕获在 EXTI1_IRQHandler 里完成（dht11.c:53-71），收到 42 个边沿后回到主循环解码。
5. 渲染只发生在一个地方——main.c:649-660，并且有两个门：OLED_RefreshBusy() 必须为假（上一帧已发完），due(now,frame_next) 到期。到期后把十几项状态一次性写进 DashboardView（main.c:650-657），调用 Dashboard_Draw(&view) 重新绘制显存，再 OLED_StartRefresh() 启动一次刷新；下一帧周期是 33 ms（LEVEL 页）或 50 ms（其它页）（main.c:658-659）。
6. 出屏是分片的：OLED_RefreshStep() 每轮只发送**一个 32 字节块**（oled.c:179-206），32 个块走完才结束一帧。分片的意义在于块与块之间主循环还能继续跑——特别是还能去读 MPU6050，所以慢速的软件 I2C 不会把采样节拍拖死。代价是一条硬约定：**刷新期间不能改 OLED_GRAM**（oled.c:132-134）。
7. 同一轮里 OLED_Service(now) 负责与画面无关的杂事：对比度渐变、空闲变暗，以及 OLED 掉电后按初始化命令表逐步重建（main.c:661、oled.c:143-172）。

存储链（检查点 → 两类日志）：

1. 每 60 s 一条检查点：`log_next` 在初始化时被设为 `now+60000`（`main.c:527`），到期且满足 `Logger_State()==LOG_READY`、`!EventLog_Busy()`、DHT11 数据有效且无错这四个条件时，调用 `save_snapshot()` 排队一条 32 字节传感器记录；返回 0 表示"已受理"，此时把 `log_next` 再推 60 s（`main.c:616-619`、`main.c:148`）。受理不等于落盘成功——真正的确认发生在后续几轮里，靠 `Logger_LastSequence()` 是否变化来判定并给出 `SAVED` / `SAVE ERROR`（`main.c:620-622`）。
2. 会话事件日志独立推进：`EventLog_Task(now, uptime_seconds)` 每轮被调用（`main.c:615`），记录本次 BOOT 一条（`EVENT_BOOT`）、之后按心跳周期记 `EVENT_ALIVE`（`eventlog.h:7`）。
3. 两条链共用同一个"出错"状态源：`Logger_State()` / `EventLog_State()` 落到错误态时，RGB 灯切到错误色（`main.c:645-646`），SYSTEM/DEBUG 页把它显示成可读文字（`dashboard.c:45-52`）。
4. 掉电语义：突然断电写不进最终时长，HISTORY/SESSIONS 只能显示最近一条检查点形成的下界（约一分钟误差），没有检查点则显示 `NO CHECKPOINT`（`main.c:616-619`）。

人机交互是第三条短线：`Controls_Tick()` 在 SysTick 中断里每 1 ms 采样编码器与按钮（`stm32f10x_it.c:139`），主循环每轮取走累积结果 `Controls_TakeTurn()` / `Controls_TakeButton()`（`main.c:541`），据此翻页、翻历史、翻转 STROBE、或触发一次手动存盘（`main.c:545-566`）。

1.7 读代码的建议路线

按下面顺序读，每一步都能独立跑通、且为下一步准备好概念：

1. **`delay.h` + `delay.c`（33 行）** —— 全工程最短的时间底座：`delay_millis()` 来自 SysTick 毫秒计数，`delay_micros16()` 直接返回 TIM2 的 1 MHz 自由运行计数器（`delay.c:12-19`）。后面每一个 `due(now, next)` 判断都建立在这两个函数上，先读它，后面的超时逻辑才不神秘。
2. **`i2c.h` + `i2c.c`** —— 总线层。重点看开漏配置与读 IDR 的写法（`i2c.h:12-16`）、时基常量 `I2C_DELAY_LOOPS` 的来历（`i2c.h:29-36`），以及卡总线恢复 `SoftI2C_BusRecover`——OLED 掉电重建全靠它。
3. **`controls.h` + `controls.c`（225 行）** —— 纯 GPIO/定时器，不碰任何总线，可以一次读完。它同时展示了两种典型手法：格雷码查表消抖（`controls.c:170-179`）和 TIM1 三通道 PWM 的自管初始化（`controls.c:49-72`，`PSC` 由运行时 `RCC_GetClocksFreq` 反推，不写死 72）。
4. **`main.c:1-60`** —— 常量表就是全工程的参数面板：采样间隔、显示间隔、标定样本数、滤波移位、各页门限都在这里（`main.c:17-41`）。读完它再读主循环，就不会被魔数绊住。
5. **`main.c:488-531` 然后 `main.c:531-663`** —— 初始化顺序和主循环。这是全工程的骨架，也是所有模块第一次被串起来的地方；建议对照 1.6 节的两条链一起看。
6. **`dashboard.h`（29 行）然后 `dashboard.c`** —— 先读 `DashboardView`（`dashboard.h:12-25`），它是全系统唯一的结构化数据契约；再去 `dashboard.c:319-359` 看页面分发表，最后按需读单个页面函数。不要把 `dashboard.c` 从头读到尾，它是十个互不相干的页面拼起来的。
7. **`oled.c:119-210`** —— 差分块刷新的完整实现。理解"为什么刷新期间不能改 `OLED_GRAM`"（`oled.c:132-134`）和"32 个块是怎么切出来的"（`oled.c:185-195`），是理解整个系统实时性的关键。
8. **`w25qxx.c` → `logger.c` → `eventlog.c`** —— 由浅入深读存储：先裸 Flash 驱动（识别、轮询、按页写、4 KB 擦除），再 32 字节记录的编解码与扇区覆盖（`logger.c`），最后事件区布局与"提交字节 + CRC + 回读"的协议（`eventlog.c`）。注意最后一步会撞上 `eventlog.c:91` 对 `Logger_State()` 的硬依赖。

`mpu6050.c`（99 行）与 `dht11.c`（105 行）可以插在第 3 步之后随时补读：前者是标准的"寄存器初始化 + 连续读"，后者是"状态机 + 外部中断捕获"的完整例子，两者都不长。

本章发现的问题与建议

问题 1：页面宏只定义了三页，其余七页靠裸数字分派。

- 现象：`dashboard.h:5-7` 只给出 `STROBE_PAGE 7`、`RUNTIME_PAGE 8`、`EVENT_PAGE 9`，页 0–6 没有名字；渲染分派里写的是 `if(v->page==0) level_page(v);`、`else if(v->page==1) climate_page();` 一直到 `else if(v->page==6)debug_page(v);`（`dashboard.c:328-334`）。
- 影响：页顺序号在 `titles[]`（`dashboard.c:321`）、分派链（`dashboard.c:328-336`）以及各处 `page==N` 的显示判断里重复出现；在中间插入一页需要同时改动多处，且页名与裸数字之间没有编译器可检查的关系。
- 建议：在 `dashboard.h` 里补齐 `LEVEL_PAGE` / `CLIMATE_PAGE` /…/ `DEBUG_PAGE` 七个宏，把 `dashboard.c:328-334` 的裸数字替换为宏名；后续若要继续加页，可进一步把页面函数收成一张函数指针表。

问题 2：上电自检直接读寄存器，与驱动层各自为政。

- 现象：`main.c` 用 `SysTick->CTRL/LOAD`（`main.c:247`）、`TIM2->CR1/PSC`（`main.c:263-264`）、`GPIOB->CRL/ODR`（`main.c:272`）、`TIM1->CR1/BDTR/CCER/ARR/CCMR1/CCMR2` 与 `GPIOA->CRH`（`main.c:278-283`）逐位核对期望值。
- 影响：这些期望值（例如 `TIM1->ARR+1UL==3600UL` 对应 `controls.c:16` 的 `LED_PWM_PERIOD 3600U`）是驱动层私有常量在应用层的复制品。修改 `controls.c` 的 PWM 周期或 `delay.c` 的 SysTick 配置后，如果忘了改 `main.c`，上电自检会失败并把设备锁在闪灯死循环里——症状看起来像"硬件坏了"。
- 建议：把每项检查下沉为对应驱动模块的自检函数（例如 `Controls_SelfTest()` 返回布尔值），`main.c` 只负责调度与报码；驱动层自己拥有"什么算配置正确"的知识。

问题 3：事件日志被传感器日志的状态机绑架。

- 现象：`eventlog.c:91` 是 `if(Logger_State()!=LOG_READY||W25Q_Capacity()<SENSOR_BYTES+EVENT_BYTES)return;`，而 `main.c:612-615` 也只在 `Logger_State()==LOG_READY` 时才调用 `EventLog_Init`。
- 影响：两类日志在 Flash 上是相邻但独立的两块区域，却共享一个"传感器日志是否就绪"的启动条件。传感器区一旦持续报错，复位溯源（BOOT 记录、掉电下界）就一起不可用，而这恰恰是排障时最需要的东西。
- 建议：把事件日志的启动条件改为只依赖 `W25Q_Capacity()` 与自身区域校验结果，删除对 `Logger_State()` 的判断；若确需互斥（避免两块区域同时擦写），应改为"总线忙的时候让一步"，而不是"对方没就绪我就不启动"。

第 2 章　平台基础：时基、SysTick 与 TIM2

本章要点：

- 全系统只有两个硬件时基：SysTick 提供 32 位毫秒计数（`delay.c:2`、`delay.c:15`），TIM2 提供 1MHz 自由运行的 16 位微秒计数（`delay.c:12-13`）；业务层的"到期"判断全部建立在这两个计数器的**差值**上（`main.c:49`）。
- 定时器时钟由 `RCC_GetClocksFreq` 推导：PCLK1 = 36MHz，APB1 非 1 分频故再 ×2 得 72MHz，于是 `PSC = 71`、每计数 1μs；`ARR = 0xFFFF` 使 TIM2 每 **65.536ms** 回绕一次（`delay.c:7-13`、`delay.h:10`）。
- `delay_micros16()` 只返回计数器的**瞬时值**，不返回差值；模 65536 的减法天然给出正确的微秒差，但前提是被测间隔必须小于 65.536ms（`delay.c:19`、`dht11.c:60`）。
- `delay_us` 与 `delay_ms` 都是纯忙等：不睡眠、不进 WFI、不喂狗。主循环 `main.c:531-663` 是单线程相位式轮询，全工程**未实现**低功耗与看门狗。
- 中断只有两个入口：`SysTick_Handler`（`stm32f10x_it.c:136-140`）与 `EXTI1_IRQHandler`（`dht11.c:53`）。EXTI1 抢占优先级为 1，高于按最低优先级安装的 SysTick；中断里刻意不碰 I2C/Flash（`main.c:16`）。

2.1 三层时基各管什么

资源	频率 / 粒度	用途	关键行号
SysTick（内核）	72MHz / 1ms，回绕 49.71 天	`tick_ms` 毫秒时刻表，所有 `due()` 判定与 `delay_ms` 的地基	`delay.c:2`、`delay.c:15`、`stm32f10x_it.c:138`
TIM2（APB1）	1MHz / 1μs，16 位自由运行，65.536ms 回绕	脉冲宽度测量、总线超时、POST 的 μs 级自检；**不开中断**	`delay.c:11-13`、`delay.c:19`
业务层截止时刻	以 ms 为单位，最大 60s	`due(now, deadline)` 比较，实现"无 RTOS 的并发"	`main.c:49`、`main.c:527`、`main.c:625-630`
TIM1（APB2，输出用）	72MHz / 2kHz PWM，ARR = 3600	RGB 灯 PWM（第 10 章主题，此处只作时钟推导对照）	`controls.c:53-61`

三者职责不重叠：SysTick 管"多久之后做事"，TIM2 管"刚才那段有多长"，业务层只做整数比较。TIM2 因此完全不进中断，微秒时基的 CPU 开销为零。

2.2 时钟推导：8MHz 晶体如何变成 TIM2 的 1MHz

系统时钟在启动阶段的 `SystemInit` 里建立：`SYSCLK_FREQ_72MHz`（`system_stm32f10x.c:83`）使 `SystemCoreClock = 72000000`（`:130`），PLL 分支用 `PLLSRC_HSE | PLLMULL9` 得 8MHz × 9 = 72MHz（`:992-995`，HSE 取值见 `stm32f10x.h:119`）。总线分频为 AHB ÷1、APB2 ÷1（PCLK2 = 72MHz）、**APB1 ÷2（PCLK1 = 36MHz）**（`system_stm32f10x.c:961`、`964`、`967`）。

`delay_init()` 是唯一的时基初始化入口：

```
void delay_init(void)
{
    RCC_ClocksTypeDef clocks;
    uint32_t timer_clock;
    RCC_GetClocksFreq(&clocks);
    timer_clock = clocks.PCLK1_Frequency;
    if (clocks.PCLK1_Frequency != clocks.HCLK_Frequency) timer_clock *= 2;
    /* TIM2 在 1MHz 自由运行；SysTick 每 1ms 仅计时。 */
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);
    TIM2->CR1 = 0; TIM2->PSC = timer_clock / 1000000UL - 1;
    TIM2->ARR = 0xFFFF; TIM2->EGR = 1; TIM2->SR = 0; TIM2->CNT = 0; TIM2->CR1 = 1;
    tick_ms = 0;
    SysTick_Config(SystemCoreClock / 1000UL);
}
```

（`delay.c:3-16`）

推导链条与数值：

1. `RCC_GetClocksFreq` 从 `RCC->CFGR` 的 `PPRE1` 位段查表算出 `PCLK1_Frequency`（`stm32f10x_rcc.c:1012-1017`，分频表 `stm32f10x_rcc.c:192`）→ 72MHz ÷ 2 = **36MHz**。
2. APB1 预分频不为 1 时，STM32F1 的定时器时钟是 PCLK1 的 **2 倍**（ST 的刻意设计：APB 总线可为兼容外设降频，TIMxCLK 要保持高速）。代码用 `PCLK1 != HCLK` 作为"APB1 非 1 分频"的等价判据（`delay.c:9`）：分频为 1 时 PCLK1 恒等于 HCLK，否则必然不等。于是 `timer_clock = 36MHz × 2 = 72MHz`。
3. `PSC = 72 000 000 / 1 000 000 − 1 = 71`，计数频率 72MHz/(71+1) = **1MHz**，即每个计数正好 1μs。
4. `ARR = 0xFFFF = 65535`，周期 = 65536 × 1μs = **65.536ms**（`delay.h:10` 的注释与之一致）。

TIM2 挂在 APB1 上却按 72MHz 计数，正是第 2 步的 ×2 造成的：漏掉 ×2 会得到 `PSC = 35`，微秒计数变成 2μs 一步，所有超时与脉宽都会差一倍。

寄存器顺序也有讲究：先 `CR1 = 0` 停机配置，再写 `PSC/ARR`；`EGR = 1`（UG）产生一次更新事件，把预分频器与重装值**立即**装入影子寄存器——否则新 `PSC` 要等到第一次溢出才生效，起始阶段会短暂跑错频率；`SR = 0` 清掉 UG 顺带置起的 UIF；`CNT = 0` 后可 `CR1 = 1`（CEN）启动（`delay.c:12-13`）。

TIM1 走同一条公式但挂在 APB2 上：PCLK2 = 72MHz = HCLK，不满足 ×2 条件，故 `timer_clock` 仍是 72MHz（`controls.c:53-55`），`PSC = 72 000 000/(2000 × 3600) − 1 = 9`、`ARR = 3600 − 1 = 3599`，PWM 频率 72MHz/((9+1) × 3600) = **2kHz**（`controls.c:60-61`，周期常量 `controls.c:16-17`）。

启动自检会把推导重算一遍：`post_clock_check()` 断言 `SystemCoreClock/SYSCLK/HCLK == 72MHz`、`PCLK1 == 36MHz`、`PCLK2 == 72MHz`（`main.c:234-241`）；`post_systick_check()` 断言 `SysTick->CTRL&7 == 7`、`SysTick->LOAD+1 == SystemCoreClock/1000`，并用 `tick_ms` 前进 1 证明中断在跑（`main.c:243-252`）；`post_tim2_check()` 断言 `CEN` 已置、`timer_clock ≥ 1MHz`、**能被 1MHz 整除**、`PSC+1 == timer_clock/1000000`，再实测 100μs（`main.c:254-268`）。三项都过才置 `post_timebase_ok`（`main.c:390`），失败则停在闪灯死循环（`main.c:518-521`）。

读代码时的一个坑：`system_stm32f10x.c:966` 注释写 `/* PCLK1 = HCLK */`，紧接的 `:967` 却是 `RCC_CFGR_PPRE1_DIV2`；这是 V3.5.0 原版库自带的矛盾（原版同一处 `system_stm32f10x.c:1027-1028`），以 `main.c:240` 的断言为准。

2.3 `delay_micros16()`：返回计数、差值由调用者算

```
uint16_t delay_micros16(void) { return (uint16_t)TIM2->CNT; }
```

（`delay.c:19`；头文件注释为 `/* 1MHz TIM2 wraps every 65.536ms */`，`delay.h:10`）

签名与语义：无参数，返回 TIM2 计数器的当前值，单位 1µs，值域 0…65535，**不是**"距上次调用过去了多久"。调用者必须自己保存起点再做减法：

```
stamp=delay_micros16(); span=(uint16_t)(stamp-edge_time); edge_time=stamp;
```

（`dht11.c:60`，DHT11 的下降沿间隔测量）

这样设计是因为同一原语要服务两种用法：**长期起点**（`w25qxx.c:11` 取 `start`，`:14`/`:17` 判超时；`i2c.c:17` 取 `start`，`:19` 判超时；`main.c:265` 取 `start`，`:266-267` 判 100µs 自检）与**逐边沿差值**（每次进 ISR 取时间戳，与 `edge_time`（`dht11.c:8`）相减，`dht11.c:60`）。若 API 直接返回"距上次调用"的差值，前者会把每次轮询当成新测量，逻辑立刻失效。

回绕正是这个 API 的核心约定：`ARR = 0xFFFF` 使 `CNT` 在 0…65535 间循环，用**无符号 16 位**减法后 `(uint16_t)(t_end - t_start)` 恰好等于真实间隔，模 65536 的算术天然正确，不需要任何回绕分支，代价是间隔必须小于 65536µs。这就是"差值必须由上层算"的原因：只有调用者知道两次采样的先后关系，也只有调用者能保证这个上界。

实际安全裕度很大：DHT11 的合法窗口是 120…210µs（`dht11.c:63`）与 55…150µs（`dht11.c:67`），阈值 `span >= 100`（`dht11.c:69`），最大值只占 65.536ms 的 0.32%；总线超时是 1000µs（`w25qxx.c:14`/`17`、`i2c.c:19`）与 100µs（`main.c:266-267`）。**混叠**只在两次边沿相隔 65536µs 以上时发生，届时差值折回小数、落在窗口外被判成帧错误（`dht11.c:63`/`67`），是可接受的降级。

中断安全是它另一个性质：读的是硬件自由运行的计数器，单次 APB 读操作，不需要关中断或临界区，`dht11.c:60` 直接在 ISR 里调用无任何保护也成立。帧首边沿会丢弃第一次算出的跨度（`dht11.c:61` 的 `if (!edge_count)` 早退），顺带掩盖了 `edge_time` 从不复位的事实。

2.4 `delay_us` / `delay_ms`：忙等实现、代价与隐患

```
void delay_us(uint32_t us)
{
    while (us) {
        uint16_t span = (uint16_t)(us > 60000UL ? 60000UL : us);
        uint16_t start = (uint16_t)TIM2->CNT;
        while ((uint16_t)((uint16_t)TIM2->CNT - start) < span) { }
        us -= span;
    }
}
void delay_ms(uint32_t ms)
{
    uint32_t start = delay_millis();
    while ((uint32_t)(delay_millis() - start) < ms) { }
}
```

（`delay.c:20-33`）

`delay_us` 的关键常数是 **60000**：单片上限必须严格小于 65536，取值留了 5536 个计数的余量。分片使 `us` 可以任意长（例如 120000µs 会被拆成两片 60000）。轮询条件是"当前计数减去起点"的 16 位无符号值，写法与 2.3 节完全一致；这里隐含一个前提：`ARR` 必须保持 `0xFFFF`、计数频率必须保持 1MHz，否则 60000 上限与 µs 语义会同时失效。

`delay_ms` 则完全不看 TIM2：它以 `delay_millis()`（即 `tick_ms`）的差值为准，而 `tick_ms` 只在 `SysTick_Handler` 里自增（`delay.c:17`、`stm32f10x_it.c:138`）。这意味着：

- 两个函数都是**纯忙等**：CPU 以 72MHz 空转，期间不执行其他工作，也不进入任何低功耗状态。
- `delay_ms` 的精度等于 1ms 的 SysTick 粒度，且会**向上取整**（差值达到 `ms` 才退出，最坏多等约 1ms）。
- `delay_us` 不受中断影响（计数在硬件里跑），`delay_ms` 则依赖中断使能——这是它的隐式前提，详见本章问题清单第 1 条。

两者都不会被优化掉：循环读的是 `volatile` 的 `tick_ms`（`delay.c:2`）与硬件寄存器，"空循环被优化掉"的老坑在这里不存在；唯一刻意使用自减空循环的是时基故障分支（`main.c:519-520`）。

表 2-2 `delay_*` API 表：

签名	单位 / 语义	精度与上限	调用点
`void delay_init(void)`	一次性建立两层时基	假定 `timer_clock` 能被 1MHz 整除且商不超 16 位	`main.c:508`（唯一调用）
`uint32_t delay_millis(void)`	ms，32 位**绝对**时刻	1ms 粒度；49.71 天回绕一次	`main.c:226/248/249/250/392/407/411/429/467/523/532/601/629`、`delay.c:31-32`
`uint16_t delay_micros16(void)`	µs，TIM2 计数**瞬时值**	1µs 粒度；65.536ms 回绕一次	`dht11.c:60`、`w25qxx.c:11/14/17`、`i2c.c:17/19`、`main.c:265/266/267/409/414`
`void delay_us(uint32_t us)`	µs 忙等	1µs 粒度；按 60000 计数分片	`w25qxx.c:41`（`delay_us(10)`，唯一调用）
`void delay_ms(uint32_t ms)`	ms 忙等	1ms 粒度；依赖 SysTick 中断在跑	`buzzer.c:37`、`mpu6050.c:24/28/46`、`main.c:509/512/513`
`void delay_tick(void)`	`tick_ms++`	只在 ISR 中调用	`stm32f10x_it.c:138`（唯一调用）

表 2-3 `delay_ms` 全部调用点与阻塞时长：

调用点	阻塞时长	场景
`main.c:508` → `buzzer.c:37`	60ms（`BOOT_BEEP_MS`，`main.c:35`）	开机短鸣，此刻 I2C 尚未初始化，是全系统第一次阻塞
`main.c:509`	300ms	`SoftI2C_Init` + 总线恢复之后、探测 OLED/MPU 之前的固定等待（`main.c:510`）；该行无注释
`main.c:512`	每轮 200ms（80ms 亮 + 120ms 灭）	OLED 未就绪时的闪灯报码
`main.c:513`	1200ms + 一轮闪灯	每轮重探前的等待；`while(!oled_ok)`（`main.c:511`）没有重试上限
`mpu6050.c:24`	100ms	写 `PWR_MGMT_1 = 0x80` 软复位后等待
`mpu6050.c:28`	10ms	解除休眠、切到陀螺 PLL 后等待
`mpu6050.c:46`	10ms	5 个配置寄存器写完后、回读校验前等待

为什么这个项目里可以接受：

1. 阻塞总量小且有界：正常运行时主循环里一次 `delay_ms` 都不会发生——延时场合都用"截止时刻 + 标志位"代替。开机短鸣是唯一一次阻塞式 `Buzzer_Beep`（`main.c:508`），运行期改用非阻塞的 `beep_start()`（`main.c:76-79`）加到期关断（`main.c:538`），LED 心跳同理（`main.c:540`）。
2. 单线程相位式架构本来就没有"被阻塞的别的任务"：没有 RTOS 和需要及时响应的协议栈，唯一对时间敏感的是**中断驱动**的 DHT11 边沿捕获（`dht11.c:53-71`），不会因主循环忙等丢边沿。

哪里是隐患：

- `MPU6050_Init()` 内含 100 + 10 + 10 = **120ms** 阻塞，运行期会被读超时路径调用（`main.c:600-602`，`due(now,retry_next)` 保证最多 1s 一次）。设计者算过这笔账——`main.c:599` 的注释正是 `/* 避免 120ms MPU 复位跨越 DHT 的 20ms 起始脉冲。 */`，数字与 `mpu6050.c:24/28/46` 对得上。代价是这 120ms 内 5ms 采样节拍（`main.c:21`）与 OLED 帧都会停摆，而主循环的 `max_sample_gap_ms`（`main.c:571-573`）会把它记录在案——这正是该统计量存在的价值。
- 开机自检第 5 步（DHT11）会**忙等最长 6500ms**：`while((!dht->attempts||DHT11_Busy())&&(uint32_t)(delay_millis()-now)<6500UL&&timer_elapsed<6500000UL&&guard)`（`main.c:410-411`），三重上限是毫秒 6500、TIM2 累计微秒 6500000、`guard = 30000000`（`main.c:409`）。等待期间只推进四个泵（`post_pump()`，`main.c:224-232`），**不消费**编码器与按键；`turns` 在 ISR 里饱和到 ±20（`controls.c:177`、`controls.c:180`），所以自检期转旋钮最多积压 20 格且无视觉反馈——这解释了"上电后前几秒旋钮无反应"。
- 时基故障分支用 `volatile` 自减空循环而非 `delay_ms`（`main.c:519-520`）是**正确**的选择：走到这里说明 `systick_ok` 可能已失败（`main.c:390` 要求三项全过），用 `delay_ms` 会永久死等。闪灯周期（约 25 万与 100 万次空转）只求肉眼可辨，是可观测的故障指示。

2.5 中断向量、优先级与 ISR 职责边界

工程使用中容量启动文件（`level.uvprojx:668-670` 指向 `startup_stm32f10x_md.s`）：向量表里 SysTick 在 `:76`、EXTI1 在 `:86`，两者的弱定义在 `:178-179` 与 `:236`，被项目里的强定义覆盖。

```
void SysTick_Handler(void)
{
  delay_tick();
  Controls_Tick();
}
```

（`stm32f10x_it.c:136-140`）

表 2-4 中断资源表：

中断	向量表项	抢占 / 子优先级	触发源	ISR 职责	行号
SysTick	`startup_stm32f10x_md.s:76`	3 / 3（即 0xF0，最低）	内核 SysTick，每 1ms	`delay_tick()` 加 1；`Controls_Tick()` 跑编码器、按键、灯光状态机	`stm32f10x_it.c:136-140`、`delay.c:17`、`controls.c:168-196`
EXTI1	`startup_stm32f10x_md.s:86`	1 / 0	PA1 下降沿（DHT11 DATA）	记录边沿时间戳、按跨度窗口解码 40 位数据	`dht11.c:47-49`、`dht11.c:53-71`

全工程只配置了这**一个**外设中断：`NVIC_Init` 只在 `dht11.c:48-49` 出现一次，没有任何其他 `xxx_IRQHandler` 实现（`stm32f10x_it.c:154` 起的 PPP 模板仍是注释）。

SysTick 优先级的来源容易被忽略：`SysTick_Config()` 内部就写了 `NVIC_SetPriority(SysTick_IRQn, (1<<__NVIC_PRIO_BITS) - 1)`（`core_cm3.h:1699`），`__NVIC_PRIO_BITS = 4`（`stm32f10x.h:160`），故写入 15；系统异常写 `SCB->SHP[]` 而非 `NVIC->IP[]`（`core_cm3.h:1589` 与 `:1591` 的分支），寄存器值 = 15<<4 = 0xF0。`delay_init` 传入 72000，重装值 71999（`core_cm3.h:1698`），周期恰 1.000ms，`CTRL = 7`（`core_cm3.h:1701-1703`）。

DHT11 侧先调用 `NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2)`，再设置抢占 1、子 0（`dht11.c:47-49`）。分组 2 意味着 4 位优先级字段中高 2 位是抢占、低 2 位是子优先级，于是 0xF0 被解释为抢占 3，抢占 1 的 EXTI1 因此可以**打断** SysTick，反之不行。两点结论：

- SysTick 的 1ms 节拍会被 DHT11 的边沿中断推迟，但**不会丢也不漂**：SysTick 是硬件递减计数器，溢出后中断挂起、等 EXTI1 返回立刻执行，且下一次重装与时基无关，因此没有累积误差，只有微秒级的抖动（EXTI1 的 ISR 只有几十条指令）。`tick_ms` 因此是"略被推迟但不丢也不漂"的毫秒表。
- 微秒测量完全不受这个优先级安排影响，因为时间戳取自硬件计数器而非软件节拍（`dht11.c:60`）。

一个顺序细节：`delay_init()` 在 `main.c:508` 就装了 SysTick 优先级，而改分组的 `NVIC_PriorityGroupConfig` 到 `main.c:392` 才执行；本例两种解释下 SysTick 都是最低、EXTI1 都高于它，所以无害（0xF0 在分组 0 下为抢占 15，分组 2 下为抢占 3），但这个顺序不该照抄。

为什么中断里绝不做 I2C/Flash，`main.c:16` 的一句话就是设计纲领：

```
/* 中断只计时；共用 I2C 的采样和短块刷新均在主循环执行。 */
```

配套的具体理由（都能在代码里指到）：

1. 软件 I2C 靠位翻转 + `volatile` 空转半周期延时（`i2c.c:24-32`），电平持续时长决定从机能否识别，被更高优先级中断打断会拉长占空比。
2. 同一条总线挂着 OLED（0x3C）与 MPU6050（0x68），全工程没有总线互斥锁——互斥靠"只有主循环碰总线"这条架构约束实现，中断一旦也访问就必须引入锁，而 ISR 不可能阻塞等待。
3. W25QXX 的写/擦除要轮询忙位，量级是毫秒（第 5、12 章），放进中断会让 SysTick 与 DHT11 边沿捕获一起停摆。
4. 中断里也不能用 `delay_ms`——它的时钟正是被自己推迟的 SysTick。
5. 唯一允许的共享是"短、位级、无外设访问"：`Controls_TakeTurn()/Controls_TakeButton()` 用 `__disable_irq()`/`__set_PRIMASK()` 做几拍临界区取走 `volatile` 队列（`controls.c:198-207`），主循环读到的是快照，不会撕裂。

2.6 时间语义与回绕：四种判定原语

`due()` 是全工程唯一被命名的到期判定原语，以文件级 `static` 形式定义在 `main.c` 内部：

```
static uint8_t due(uint32_t now, uint32_t deadline) { return (int32_t)(now - deadline) >= 0; }
```

（`main.c:49`）

语义是"现在是否已经到达或越过 `deadline`"，实现是**有符号差值**：把 32 位无符号减法重解释为 `int32_t`，只要真实间隔小于 2^31ms，符号位就正确表示先后——`now` 略小于 `deadline` 时差值是接近 2^32 的大正数（重解释为负 → 假），越过时是小正数（→ 真）。它同样跨越 `tick_ms` 回绕点：`now` 从 0xFFFFFFFF 跳到 0 时差值仍在小范围内。

`due()` 的调用点共 11 处：`main.c:117`（校准 10ms 节拍）、`538`/`539`（蜂鸣器与 LED 关断）、`540`（1s 心跳）、`570`（5ms 采样）、`600`（MPU 重初始化节流）、`604`（重校准重试）、`616`（60s 存档）、`628`（Flash 退避重试）、`634`（toast 到期）、`649`（重绘）；deadline 全部取自 `now + 常数`，最大 60s（`main.c:527` 与 `main.c:630`）。以 2^31ms ÷ 60s ≈ **35 791 倍**裕度，`due()` 在这个项目里不可能失效。

但 `due()` 只是**一种**写法，代码里实际并存四种判定原语，混用是有意的：

判定原语	语义	代表位置	回绕行为	何时会失效
`due(now, deadline)` / 内联 `(int32_t)(now-x)>=0`	是否到达截止时刻	`main.c:49`（定义）与 11 处调用；内联重写见 `dht11.c:76/83/90`、`logger.c:95/108`、`eventlog.c:164/175/182/191`	模 2^32 正确，可跨回绕	真实间隔 ≥ 2^31ms（约 24.86 天）
内联 `(int32_t)(now-x)<0` → 提前返回	未到期就返回	`dashboard.c:38`、`oled.c:150`、`oled.c:163`	同上	同上
`(uint32_t)(now-start) >= X` 或 `< X`	自 `start` 起的已流逝量	`main.c:92/98/163/202/208/214/411/596/606/657`、`dht11.c:102`	模 2^32 正确	若 `start` 被写成未来时刻，条件立即为真
`(uint16_t)(delay_micros16()-start)` 与常数比较	µs 级已流逝量	`dht11.c:60`、`w25qxx.c:14/17`、`i2c.c:19`、`main.c:266/267`	模 2^16 正确	间隔 ≥ 65.536ms 时混叠
`(int32_t)(a-b)>0`（非时间，序号）	32 位序号的新旧比较	`logger.c:46`、`logger.c:100`、`eventlog.c:46/50/150`	模 2^32 正确	序号差 ≥ 2^31

第三行那种"裸 elapsed 比较"是 `due()` 之外的写法，`main.c:596` 与 `main.c:606` 是最典型的两处：

```
if((uint32_t)(now-last_good)>=READ_TIMEOUT_MS){
```

（`main.c:596`，`READ_TIMEOUT_MS = 100`，`main.c:29`）

```
if((uint32_t)(now-rate_since)>=1000){
```

（`main.c:606`，1Hz 速率窗口）

这两处**不是缺陷**：参照点来自过去——`last_good` 只在采样成功时写入当时时刻（`main.c:581`），`rate_since` 只在统计后重置（`main.c:609`），无符号差值表述"自那以后过了多久"，只要间隔小于 49.71 天就精确。真正的区别在**失效方向**：`due()` 在"未到期"时返回假；裸 `elapsed >= X` 在参照点被误写成未来时刻时会立即为真（把"还没到"当成"早已过期"）。所以"未来截止"必须用 `due()` 或有符号差值，"过去起点"才可用裸 elapsed。

一致性上要留意：`due()` 是 `main.c` 的 `static` 函数，其他模块无法复用，同一套有符号逻辑被内联重写至少 13 处（`dht11.c:76/83/90`、`logger.c:95/108`、`eventlog.c:164/175/182/191` 等），另有 3 处反向写法（`dashboard.c:38`、`oled.c:150/163`）；逻辑都对，但没有共享名称，详见问题清单第 3 条。

另外两个与回绕直接相关的计数：

- `uptime_seconds` 用**累加**维护：取 `elapsed=now-uptime_last` 后累加到秒与毫秒余数（`main.c:533-535`）。它只用相邻两次差值，毫秒回绕不会污染秒计数；秒计数是 32 位，可记到 136 年。
- DHT11 的数据有效期是裸 elapsed 加严格大于：`if (data.valid && (uint32_t)(now-data.last_good_ms)>6000) data.valid=0;`（`dht11.c:102`），`last_good_ms` 在解码成功时写入（`dht11.c:99`），6s 后作废，避免显示陈旧温湿度。

2.7 系统滴答与功耗：没有睡眠，也没有看门狗

对 `level\USER` 全量检索 `WFI`/`__WFI`/`PWR_`/`IWDG`/`WWDG`/`Sleep`/`STANDBY`，命中的只有两行字符串比较：

```
if(flags&RCC_CSR_IWDGRSTF)return "IWD";
if(flags&RCC_CSR_WWDGRSTF)return "WWD";
```

（`main.c:52-53`）

这两行是**读取复位原因**（`RCC->CSR`，`main.c:505`、`main.c:507`），不是配置看门狗——`reset_reason()` 只把六个标志位译成 `IWD/WWD/SW/POR/LPW/PIN` 短字符串（`main.c:50-58`）；启动代码与各驱动里都没有 `IWDG_Init`/`WWDG_Init`。

由此可以给出两条明确结论：

- **未实现低功耗。**全工程从不执行 `__WFI()`，也不进入 Sleep/Stop/Standby，外设时钟只有使能没有关闭。主循环 `main.c:531-663` 是纯时间片轮询：每个周期把二十来项到期检查跑一遍，没有事可做时也在 72MHz 上继续空转，唯一的周期性唤醒源就是 SysTick 的 1ms 中断。唯一与"省电"沾边的是**屏幕调光**：`IDLE_DIM_MS = 45000UL`（`main.c:24`）配合 `OLED_SetIdle((uint32_t)(now-last_activity)>=IDLE_DIM_MS)`（`main.c:163`），降的是 OLED 对比度，MCU 功耗完全没动。
- **未实现看门狗。**直接后果是：固件里三处"永不返回"的路径都只能靠断电或复位引脚恢复——时基故障闪灯循环（`main.c:518-521`）、OLED 探测失败重试循环（`main.c:511-515`，无重试上限）、以及所有 Fault 处理函数里的 `while (1) {}` 死循环（`stm32f10x_it.c:57-63`，HardFault/MemManage/BusFault/UsageFault 四个都是同样的写法）。这也解释了时基故障分支为何要闪灯而不是静默：没有看门狗兜底时，唯一手段就是让故障可见。

取舍是"用电流换确定性"：不睡眠、不门控时钟、不做动态调频，换来任何时刻的寄存器状态都可解释、任何延时都可预测——市电/USB 供电的桌面仪表场景下合理；一旦改电池供电，第 2.4 节的忙等（尤其 120ms 的 MPU 复位与开机 300 + 1200ms 等待）与这里的"永不睡眠"就是首先要处理的两笔账。

本章发现的问题与建议

1. **`delay_ms()` 的隐式前提没有任何文档或保护。**现象：`delay_ms` 的时钟来自 SysTick 中断（`delay.c:29-33`、`delay.c:17`），但 `delay.h:6-11` 只写了单位，没写"必须在中断使能且优先级低于 SysTick 的上下文里调用"。影响：在 ISR 或 `__disable_irq()` 区间内调用（`controls.c:200-207` 已有临界区写法）会永久死等，又因为没有看门狗（第 2 条）只能断电。建议：在 `delay.h` 写明契约，或让 `__get_PRIMASK()` 非零时改走 TIM2 的 µs 延时。
2. **未实现看门狗。**现象：全工程无 `IWDG_Init` / `WWDG_Init`，`main.c:52-53` 只是解读复位标志。影响：`main.c:511-515`、`main.c:518-521`、`stm32f10x_it.c:57-63` 三处死循环不可自愈，只能人工断电。建议：加入 IWDG（约 1–2s），只在主循环相位末尾喂狗，绝不在 ISR 内喂——第 1 条的死等也能被自动复位抢救。
3. **到期判定原语分散且无统一名称。**现象：`due()` 只以 `static` 形式存在于 `main.c:49`，其余模块内联重写了至少 13 处同类表达式（`dht11.c:76/83/90`、`logger.c:95/108`、`eventlog.c:164/175/182/191`），另有 3 处反向写法（`dashboard.c:38`、`oled.c:150/163`）。影响：语义靠人复制，某处误改成裸 elapsed 不会被编译器发现，方向性错误（把"未到期"判成"已过期"）会让任务提前触发。建议：把 `delay_due(now, deadline)` 与 `delay_elapsed(now, start)` 提升到 `delay.h`，各模块只调用这两个名字。
4. **`delay_init()` 不校验推导结果。**现象：`delay.c:8-12` 直接做 `timer_clock / 1000000UL - 1` 并写入 `PSC`，不检查能否整除 1MHz、也不检查商是否超 16 位。影响：主频或 PLL 配置变化时 `PSC` 被静默截断/取整，微秒时基变成"看起来正常但数值错误"；唯一能发现它的是启动自检（`main.c:263-264`），而自检只在上电跑一次、本身也要靠 `PSC` 才能测出 TIM2 在计数。建议：`delay_init` 内部校验 `% 1000000` 与商值范围，不满足则进入可观测的故障状态（复用 `main.c:518-521` 的闪灯）。
5. **`delay_us()` 与 `ARR = 0xFFFF` 硬耦合。**现象：`delay.c:23` 的 60000 分片上限成立的前提是 `delay.c:13` 的 `ARR = 0xFFFF` 且计数频率为 1MHz。影响：将来为获得更长的微秒量程而改 `ARR`（或改预分频）时，`delay_us` 不会报错，只会给出错误时长；`delay_micros16()` 的 65.536ms 契约（`delay.h:10`）同样只写在注释里。建议：把回绕周期定义成宏（如 `DELAY_US_WRAP`），让分片上限与注释都由它派生。
6. **注释与实现不符：`main.c:16` 说"中断只计时"，但 SysTick ISR 同时在驱动输出。**现象：`SysTick_Handler` 调用的 `Controls_Tick()`（`controls.c:168-196`）不只计数——每 5 次调用（200Hz 包络，`controls.c:111`）会走完整的灯光状态机并直接写 `TIM1->CCR1/2/3`（`controls.c:113-121`、`controls.c:141-143`），还包含频闪推进（`controls.c:107-110`）与 800ms 长按判定（`controls.c:192-194`）。影响：这不是缺陷（ISR 内只做整数运算和寄存器写），但"中断只计时"会让接手人低估 ISR 的职责，进而认为"在 ISR 里加点外设操作也无妨"——而第 2.5 节的理由（位翻转 I2C、共享总线无锁、Flash 毫秒级忙等）恰恰禁止那样做。建议：把这行注释改成"中断只做计数与短时输出驱动；I2C 采样、Flash 与屏幕刷新均在主循环执行"。

第 3 章　软件 I2C：位操作、卡总线恢复与错误熔断

本章要点

- 整条 I2C 总线由 GPIO 位翻转实现，全部代码只有 `i2c.c`（296 行）+ `i2c.h`（56 行）；PB6=SCL、PB7=SDA（`i2c.h:24-26`），OLED（7 位地址 0x3C）与 MPU6050（0x68）共用（`i2c.h:7`、`i2c.h:20-21`）。
- 两个引脚都必须是开漏输出 `GPIO_Mode_Out_OD`（`i2c.c:41`），高电平完全靠模块自带的 4.7k 上拉（并联约 2.35k，`i2c.h:12-14`）；开漏模式下输入通道仍有效，所以读线直接读 IDR（`i2c.c:8-9`），不需要切换输入/输出模式。
- 半周期延时是纯循环 `I2C_DELAY_LOOPS 8`（`i2c.h:36`），注释给了 20→约 110kHz、8→约 250kHz、5→约 380kHz 三个实测点（`i2c.h:30-32`）；按这三点反推，每个循环约 10.4 个 CPU 周期（0.145μs），位周期 ≈ 0.52μs + 0.435μs × loops，N=8 时约 4.0μs/位 → 250kHz。
- `scl_released()`（`i2c.c:12-22`）是唯一的在线守护：SCL 被从机持续拉低超过 1000μs（`i2c.c:19`）就置 `i2c_fault=1`（`i2c.c:20`），全文 6 个调用点（`i2c.c:89,104,122,133,154,164`）。
- `i2c_fault` 不是永久锁存器：它在每次 `i2c_start()` 里被清零（`i2c.c:85`），作用域是"一次事务"；置位后本次事务剩余步骤全部短路返回失败（`i2c.c:115,146,206,223,227,267,271`），总线恢复要靠 `SoftI2C_BusRecover()`（`i2c.c:55-81`）显式调用（`oled.c:37,152`、`main.c:513,601`）。
- 全模块**没有互斥锁、没有临界区、没有中断内调用**（`stm32f10x_it.c:136-140`、`dht11.c:53-70` 内均无 I2C），一切事务在主循环里串行执行（`main.c:16`）——这是本设计成立的前提，也是它最脆弱的地方。

3.1 为什么用软件 I2C

3.1.1 拓扑与电气前提

`i2c.h:6-22` 的文件头注释就是设计说明：两条线的电气前提是"两个脚都配成开漏输出，靠模块上自带的 4.7k 上拉电阻把线拉高"，OLED 模块与 MPU6050 模块各带一组 4.7k，并联后约 **2.35k**（`i2c.h:12-14`）。这个并联值决定了上升沿时间与最高速率，也解释了为什么 `i2c.h:35` 提示"换成长杜邦线以后出现花屏，把 loops 调回 12 到 20"。

开漏带来的两个直接好处在代码里清晰可见：一是**总线可以被任何一方拉低**，主从都不会出现推挽对推挽的短路；二是 **STM32 在开漏输出模式下输入通道仍然有效**（`i2c.h:15-16`），因此 `SDA_READ()` / `SCL_READ()` 直接读 `GPIOB->IDR`（`i2c.c:8-9`），省掉了每比特切换方向的麻烦与毛刺。

硬件 I2C 外设在本工程里**未被使用**：标准外设库的 `stm32f10x_i2c.c` 虽然在工程文件清单里（`level.uvprojx:603-605`），但 USER 目录全部 `.c` 中检索不到任何 `I2C_Init` / `I2C_GenerateSTART` / `I2C_SendData` 之类的调用，所有总线流量都走 GPIO。软件 I2C 的代价是 CPU 全程占用（N=8 时每位约 4.0μs，见 3.1.2），收益是时序完全可控、不需要处理 STM32F1 硬件 I2C 的已知缺陷，也没有 DMA/中断复杂度。

3.1.2 速率：从三个实测点反推位时序

`i2c.h:28-35` 给出的三个数据点是本节唯一的一手证据：

I2C_DELAY_LOOPS	注释给出的 SCL	换算位周期	注释位置
20	约 110kHz	9.09μs	`i2c.h:30`
8（当前取值）	约 250kHz	4.00μs	`i2c.h:31`
5	约 380kHz	2.63μs	`i2c.h:32`

一个 SCL 位周期由三段 `i2c_delay()` 加固定开销组成（见 `i2c.c:119-126`：写 SDA → 延时 → 拉高 SCL → 等释放 → 延时 → 拉低 SCL → 延时）。设 `T(N) = C + 3b·N`，其中 b 是 `i2c_delay` 每循环一次的成本：

- 由 20→8：(9.09 − 4.00) ÷ (3 × 12) = **0.141μs/循环**；
- 由 8→5：(4.00 − 2.63) ÷ (3 × 3) = **0.152μs/循环**。

两段独立相减得到一致结果，取 b ≈ **0.145μs**，即 72MHz 下每循环约 **10.4 个 CPU 周期**。代回求固定项：C = 4.00 − 3×0.145×8 ≈ **0.52μs**（约 37 周期，含 3 次函数调用、4 次 BSRR/BRR 写和一次 `scl_released()`）。

于是 **T_SCL ≈ 0.52μs + 0.435μs × I2C_DELAY_LOOPS**，用它回代三个点：N=8 → 4.00μs（250kHz，与注释一致）；N=20 → 9.22μs（**108.5kHz**，注释 110kHz，差 1.4%）；N=5 → 2.70μs（**370kHz**，注释 380kHz，差 2.6%）。模型三点全对，可以放心使用。

按这个模型，"N=8 → 约 250kHz" 的具体含义是：**每位 ≈ 4.0μs ≈ 288 个 CPU 周期，其中 8 次循环延时占 24 次迭代（约 83 周期/次调用）**；一个字节含 8 位数据 + 1 个 ACK 时钟，共 **9 个位槽 → 36.0μs/字节**。这也解释了 `i2c.h:33` 的收益："整屏刷新从 85ms 降到约 37ms"正是 1024 字节显存的推屏时间：N=20 时 1024×82.9μs ≈ **84.9ms**，N=8 时 1024×36.0μs ≈ **36.9ms**，与注释数字吻合到 0.1ms。注意当前 E 版 OLED 已改差分短块（`oled.c:182-197`），每次事务是 7 字节命令 + 33 字节数据 ≈ 1.44ms/块，这里引用的是"整屏"这个量级的原始对照。

需要指出：`i2c.h:29` 那句"每减小 1，SCL 大约快 15kHz"是线性近似，与它自己的表格不自洽——110→250kHz 段平均 11.7kHz/单位，250→380kHz 段却是 43.3kHz/单位（因为 f = 1/(C+3bN) 是双曲线）。按 15kHz/单位从 250kHz 外推"loops=5 应为 295kHz"，与注释自身的 380kHz 差 85kHz。

3.2 初始化：`SoftI2C_Init`（`i2c.c:34-47`）

```
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);
    GPIO_InitStructure.GPIO_Pin   = I2C_SCL_PIN | I2C_SDA_PIN;
    GPIO_InitStructure.GPIO_Mode  = GPIO_Mode_Out_OD;   /* 开漏，必须 */
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(I2C_PORT, &GPIO_InitStructure);
```

（`i2c.c:38、40-43`，逐字摘录，省略了中间的空白行）

逐句要点：

- `RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);`（`i2c.c:38`）：PB6/PB7 属 GPIOB，挂在 APB2；不使能时钟时对 CRH/BSRR 的写入会被忽略，引脚保持复位后的浮空输入——表现为"接线完全正确却扫不到器件"。
- `GPIO_Mode_Out_OD`（`i2c.c:41`）：**必须开漏**。I2C 是多主/多从的线与总线，从机在应答周期会把 SDA 拉低；若配成推挽输出，主机把 SDA 驱高、从机同时把 SDA 拉低，就是一个 3.3V 到地的低阻通路，轻则数据错，重则烧引脚。开漏下主机只能"拉低或释放"，拉低优先权天然让给任何一方。注释里直接写了"开漏，必须"。
- `GPIO_Speed_50MHz`（`i2c.c:42`）：输出翻转速率档位，不是"输出频率"。250kHz 的位速率对 50MHz 档毫无压力，选高档位只是为了让边沿更陡。
- 最后两行 `SCL_HIGH(); SDA_HIGH();`（`i2c.c:45-46`）把总线放回**空闲态**（两条线都由外部上拉为高），这是任何一次 START 之前的合法状态。

一个可以改进的细节：STM32 复位后 ODR = 0，而 `GPIO_Init()` 只改 CRH、不写 ODR，所以 `i2c.c:43` 一执行，两个引脚立刻处于"开漏 + 输出寄存器为 0"=**主动拉低**状态，直到 `i2c.c:45-46` 才释放。这段窗口只有几条指令（不到 0.2μs），且 SCL/SDA 同时为低不构成 START/STOP，`SoftI2C_Init()` 又只在开机调用一次（`main.c:509`），因此当前无害；但把 BSRR 预置放到 `GPIO_Init()` 之前是更稳妥的写法。

3.3 位级原语

`i2c.c` 一共 7 个内部原语（宏 + static 函数），对外只暴露 3.7 节的事务 API：

原语	行号	签名	作用	返回值语义
`SCL_HIGH/LOW`、`SDA_HIGH/LOW`	`i2c.c:4-7`	宏	`GPIO_SetBits/ResetBits`，写 BSRR/BRR	—
`SDA_READ`、`SCL_READ`	`i2c.c:8-9`	宏	读 `GPIOB->IDR` 对应位	非 0 = 线为高
`scl_released`	`i2c.c:12-22`	`static uint8_t scl_released(void)`	SCL 是否被释放的守护检测	**1 = 已释放，0 = 失败**（并置 `i2c_fault=1`）
`i2c_delay`	`i2c.c:25-32`	`static void i2c_delay(void)`	半周期延时	—
`i2c_start`	`i2c.c:83-97`	`static uint8_t i2c_start(void)`	产生起始条件	0 = 成功，1 = 失败（并置 fault）
`i2c_stop`	`i2c.c:99-109`	`static void i2c_stop(void)`	产生停止条件	无；失败只体现在 `i2c_fault`
`i2c_send_byte`	`i2c.c:112-140`	`static uint8_t i2c_send_byte(uint8_t byte)`	高位先发 1 字节 + 收第 9 位 ACK	**0 = 从机应答，1 = 无应答/失败**
`i2c_read_byte`	`i2c.c:143-171`	`static uint8_t i2c_read_byte(uint8_t send_ack)`	收 1 字节 + 发 ACK/NACK	数据字节；**0xFF 兼作错误码**

`i2c_delay()`（`i2c.c:25-32`） 用 `volatile uint8_t i = I2C_DELAY_LOOPS;` 加空 `while (i--)`。`volatile` 是必需的——否则在 -O2/-O3 下编译器会把整个空循环删掉，总线直接跑到 MHz 级。注释"用 volatile 计数器，防止编译器把空循环优化掉"（`i2c.c:24`）就是在说明这一点。

`i2c_start()`（`i2c.c:83-97`） 五步：`i2c_fault=0`（85，清错误标志）→ SDA/SCL 都置高并等 SCL 真正释放（86-90）→ **检查 SDA 是否为高**，低则说明总线被占用/挂死，置 fault 并返回 1（91）→ 在 SCL 为高时把 SDA 拉低，形成起始边沿（92）→ SCL 拉低，进入数据传输（94）。第 91 行是全文唯一的总线忙检测，但只在 START 时刻做一次。

`i2c_stop()`（`i2c.c:99-109`） 相反的顺序：SDA 先拉低（101）→ SCL 拉高（103，这里 `(void)scl_released();` 丢弃了返回值）→ 在 SCL 高时释放 SDA，形成停止边沿（106）→ 延时后**回读 SDA，若没回到高就置 `i2c_fault=1`**（108）。第 108 行是"事务结束后总线没恢复空闲"的唯一记录点：因为 `i2c_stop()` 返回 void，这个故障只能通过调用者在事务末尾返回的 `i2c_fault`（`i2c.c:206,227,271`）或 `SoftI2C_Probe` 里的显式检查（`i2c.c:182`）被看到。

`i2c_send_byte()`（`i2c.c:112-140`） 先短路检查 `if(i2c_fault)return 1;`（115），然后 8 位高位先发（117-127：`if (byte & 0x80) SDA_HIGH(); else SDA_LOW();` + 三个延时），再处理第 9 个时钟：

```
    /* 第 9 个时钟：放开 SDA，读从机的应答 */
    SDA_HIGH();
    i2c_delay();
    SCL_HIGH();
    if(!scl_released())return 1;
    i2c_delay();
    ack = (SDA_READ() == 0) ? 0 : 1;
    SCL_LOW();
    i2c_delay();

    return ack;
```

（`i2c.c:129-139`，逐字摘录）

这段是整个驱动最关键的 11 行。ACK 周期里**主机必须完全释放 SDA**（`SDA_HIGH()`，130），否则主机的"高"会和从机的"低"打架；随后拉高 SCL（132），在 SCL 高电平期间延时后采样（134-135）。**采样含义**：SDA 被从机拉低 = 从机收到字节，记 `ack = 0`；SDA 保持高 = 无应答，记 `ack = 1`。即 `0 = ACK`、`1 = NACK`。所有上层调用都按这个语义判断（`i2c.c:190,198,214,246,251,258`）。采完样立刻拉低 SCL（136），第 9 个时钟结束。

`i2c_read_byte(uint8_t send_ack)`（`i2c.c:143-171`）：先 `SDA_HIGH()` 释放 SDA 交出控制权（148），然后 8 次"先移位再采样"（152、156：`byte <<= 1; ... if (SDA_READ()) byte |= 0x01;`），确认是高位先收。收完由主机发第 9 位：`send_ack=1` 拉低 SDA 表示"还要继续读"，`send_ack=0` 释放 SDA 表示"这是最后一字节"（161），最后释放 SDA（168）。调用方按 `(i < len - 1) ? 1 : 0` 计算（`i2c.c:222,266`），即**最后一个字节发 NACK**——这是 I2C 规范要求主机在最后一个字节后发 NACK，从机才会停止驱动数据线，紧接着的 STOP 才能成功。

三处 `return 0xFF`（`i2c.c:146,154,164`）是故障出口，但 0xFF 同时也是合法的数据取值，语义上存在歧义；之所以没出问题，是因为唯一的调用者 `SoftI2C_ReadBytes` / `SoftI2C_ReadRegs` 在每次读取后立刻检查 `i2c_fault` 并丢弃结果（`i2c.c:223,267`）。

3.4 卡总线守护：`scl_released()`（`i2c.c:12-22`）

```
static uint8_t scl_released(void)
{
    uint16_t start;
    volatile uint32_t guard=10000UL;
    if(SCL_READ())return 1;
    start=delay_micros16();
    while(!SCL_READ() && guard--)
        if((uint16_t)(delay_micros16()-start)>=1000U)break;
    if(!SCL_READ()){i2c_fault=1;return 0;}
    return 1;
}
```

（`i2c.c:12-22`，逐字摘录）

它保护的是 **clock stretching 失控**：I2C 允许从机把 SCL 拉低来"减速"主机，这是合法行为；但若从机因为复位、掉电、总线被打断而**永久**拉住 SCL，主机侧没有任何硬件超时（这里是 GPIO 模拟，不是硬件 I2C 外设），整个系统就会死在一条 `while` 里。这个函数就是补上那个超时。

设计要点：

- 快速路径 `if(SCL_READ())return 1;`（16）：线已经是高，立刻返回，零开销。慢路径只在真的读到低电平时才读 TIM2 时间戳。
- `start=delay_micros16()`（17）与 `(uint16_t)(delay_micros16()-start)`（19）利用 16 位无符号减法天然处理回绕——`delay_micros16()` 是 1MHz 的 16 位时间戳，每 65.536ms 回绕一次（`delay.h:10`），只要单次等待远小于 65.536ms，差值就总是正确。
- **双阈值**：`guard=10000`（15）是循环次数上限，1000µs（19）是时间上限。正常情况时间上限先到（每次迭代要读一次 IDR、调用一次 `delay_micros16()`，按 -O0 每迭代 20-40 周期估算，10000 次约 3-6ms），所以 guard 是"TIM2 时间戳不动了"（例如 TIM2 意外停摆）时的兜底。两条路径不会漏报：循环退出后第 20 行**再检查一次** `SCL_READ()`，无论是因为 1000µs 还是因为 guard 耗尽退出，只要 SCL 仍为低就置 `i2c_fault=1` 并返回 0。
- 因为线是开漏的，**主机不可能自己把 SCL 读成低**：`SCL_HIGH()` 之后仍读到低，只可能来自从机拉低（或短路）。所以这个超时几乎不会误报；从机只是慢一点点（OLED 与 MPU6050 正常工作都不做 clock stretching）不会触发，1ms 的余量非常宽裕。中断抢占只会让检测循环变慢（每次迭代更久），不会凭空造出一个低电平；反过来，真正拉住 SCL 的场景一定会在 1000µs 后被抓住。
- TIM2 时基本身在开机时被校验：`post_tim2_check()`（`main.c:254-268`）确认 TIM2 已使能、时钟 ≥1MHz、预分频正好等于 `timer_clock/1000000`，校验失败时主循环会停在闪灯死循环里（`main.c:518-521`），根本不会进入 I2C 通信。

6 个调用点（全部在 `i2c.c`，别处不调用）：

行号	所在函数	调用时机	失败后果
89	`i2c_start`	拉起 SCL 之后、发 START 之前	返回 1，事务不开始
104	`i2c_stop`	拉起 SCL 之后、发 STOP 之前	返回值被 `(void)` 丢弃；fault 由 20 行置位
122	`i2c_send_byte`	每一位数据 SCL 拉高之后	返回 1 = NACK 语义，事务中止
133	`i2c_send_byte`	第 9 个 ACK 时钟 SCL 拉高之后	同上
154	`i2c_read_byte`	每一位数据 SCL 拉高之后	返回 0xFF，事务中止
164	`i2c_read_byte`	读取方 ACK/NACK 时钟	同上

调用点都在 `SCL_HIGH()` 之后**立刻**执行，没有先等上升沿；由于函数内部的循环会一直等到线真的为高（或超时），这实际上同时充当了**上升沿等待**——按 2.35k 上拉、总线电容 100pF 量级估算，上升时间常数约 0.24µs，这一设计让上升沿慢的场合（长杜邦线）也不会误采样。唯一的语义瑕疵是命名反向：`scl_released()` 自己返回 **1 = 正常**，而调用者写的是 `if(!scl_released())return 1;`（返回 1 = 失败），同一个"1"在两层含义相反，阅读时容易看错。

3.5 故障熔断：`i2c_fault` 的真实作用域

`i2c_fault` 是 `i2c.c:11` 定义的 `static uint8_t`，全文的读写点如下：

动作	行号	说明
清零	`i2c.c:85`	`i2c_start()` 的第一句，**每次事务开始都清**
置位	`i2c.c:20`、`i2c.c:91`、`i2c.c:108`	SCL 拉死超时、START 前 SDA 为低、STOP 后 SDA 未回高
短路检查（发送侧）	`i2c.c:115`	`i2c_send_byte` 入口，置位则直接返回 1
短路检查（接收侧）	`i2c.c:146`	`i2c_read_byte` 入口，置位则直接返回 0xFF
作为返回值上报	`i2c.c:206`、`i2c.c:227`、`i2c.c:271`	三个字节级 API 在 STOP 之后 `return i2c_fault;`
显式判定	`i2c.c:182`、`i2c.c:223`、`i2c.c:267`	`Probe` 要求 `ack==0 && !i2c_fault`；读循环每字节后发现 fault 立即 `i2c_stop()` 并返回 1

必须澄清一个容易误读的点：**它不是"一旦置位就永久锁死"的锁存器。** `i2c.c:85` 在每次 `i2c_start()` 里无条件清零，因此它的作用域是"上一次 START 到本次事务结束"：

- 事务内部，第 115/146 行的短路检查保证**不会拿半个事务的状态继续污染从机**——例如第 3 个字节 NACK 后，剩下的字节不会被继续发出，读循环（`i2c.c:223`）也会立刻收尾。这是"熔断"的真实价值。
- 事务之间它**不提供任何保护，也不提供任何自动恢复**：下一个事务照样会在一条已经挂死的总线上重新开始，`i2c_start()` 第 91 行读到 SDA 为低、置 fault、返回 1，整条事务立刻失败。也就是说系统会"每次都以失败告终"，但不会永久锁死。
- 真正把总线救回来的是显式的 `SoftI2C_BusRecover()`（`i2c.c:55-81`），它**不读也不清** `i2c_fault`，恢复完成后靠下一次 `i2c_start()` 的 85 行完成清零。

工程里实际的恢复路径有 5 条：

触发点	行号	行为
开机	`main.c:509`	`SoftI2C_Init(); SoftI2C_BusRecover();` 先补时钟再探测
开机扫不到 OLED	`main.c:511-515`	闪 LED 报 3 或 5 次（有/无 MPU6050），`delay_ms(1200)` 后重发 `BusRecover()` 再 `Probe` 两个地址，循环直到 OLED 在
POST 的 OLED 项	`main.c:396`	直接用 `SoftI2C_Probe(OLED_ADDR7)` 做功能性判定（不是查寄存器配置）
运行中 MPU 读失败	`main.c:596-602`	数据超时（`READ_TIMEOUT_MS 100`）后每 1s 重试：`SoftI2C_BusRecover(); MPU6050_Init()`，且必须 `!DHT11_Busy()`
运行中 OLED 写失败	`oled.c:33-38`、`oled.c:151-152`、`oled.c:168-169`、`oled.c:196-199`	任一次 `SoftI2C_WriteBytes` 失败 → `need_recovery()`：作废已发送缓存、置恢复态、调 `BusRecover()`，随后每 500ms 重发**一条**初始化命令（共 16 条，`oled.c:146-159`），全部成功后重新整屏刷新

结论：`i2c_fault` 承担"**让本次事务立刻止血**"的职责，`SoftI2C_BusRecover()` 承担"**把物理总线救回来**"的职责，两者缺一不可，且后者完全没有自动触发——**谁失败谁负责调恢复**，没有任何模块在后台兜底。

3.6 总线恢复：`SoftI2C_BusRecover()`（`i2c.c:55-81`）

函数的注释（`i2c.c:49-54`）已经写清了原理：上一次通信被中途打断（例如 MCU 在从机正要吐字节时复位），从机会**一直把 SDA 拉低**，之后的通信全部作废；标准解法是补 9 个时钟让从机把没发完的字节走完，再补一个 STOP。上电时先调一次能省掉很多"明明接对了却扫不到器件"的怪问题。

```
	if (SDA_READ() == 0)		/* SDA 被拉死了才需要补时钟 */
	{
		for (i = 0; i < 9; i++)
		{
			SCL_LOW();
			i2c_delay();
			SCL_HIGH();
			i2c_delay();
		}
	}
```

（`i2c.c:63-72`，逐字摘录）

逐段精读：

1. `SDA_HIGH(); SCL_HIGH(); i2c_delay();`（59-61）：先把两条线释放为空闲，等一个半周期让电平稳定，再去判断 SDA。
2. `if (SDA_READ() == 0)`（63）：**只在 SDA 真的被拉死时才补时钟。**SDA 为高说明总线本来就是空闲的，补时钟没必要（但下面的 STOP 仍会发）。
3. `for (i = 0; i < 9; i++)`（65-71）：9 个完整的 SCL 脉冲，每个脉冲都是"先拉低、延时、再拉高、延时"。**为什么是 9 个**：从机被打断时可能停在一个字节的任意一位上。一个字节 8 位加 1 个 ACK 槽共 9 个时钟，是"最坏情况下把当前这个字节连同它的应答位走完"所需的最大时钟数；走完之后从机的移位寄存器必然吐空，它就会释放 SDA。少补一个时钟可能仍停在最后一位，多补则可能被从机误当成下一个字节的起始——9 是 I2C 恢复序列的通用取值。注意循环里 SCL 拉低后先延时再拉高，保证 SDA 在时钟低电平期间已经稳定。
4. `SDA_LOW(); i2c_delay(); SCL_HIGH(); i2c_delay(); SDA_HIGH(); i2c_delay();`（75-80）：**补一个 STOP。**这是规范要求的一步：没有 STOP，从机内部的状态机仍停在"传输中"，即便它这一刻释放了 SDA，也可能在下一次 START 时把它当成一个"重复起始"而延续旧事务；STOP 是唯一能把所有从机的位计数器、移位寄存器和应答状态一起复位、让总线回到"空闲"的显式信号。此外 STOP 后 SDA 必须由外部上拉拉高，才能满足总线空闲时间的定义。
5. 整个函数**没有任何超时**，但也不会卡死：时钟脉冲是固定 9 次（不依赖 SDA/SCL 的状态），所以即使从机把 SCL 也拉着不放，函数同样会在固定次数后走完并发 STOP。代价是恢复是否有效无从判定——函数不校验 SDA 是否真的回到了高，无法把"恢复了"和"没恢复"区分开告诉调用者。

3.7 事务 API

签名	语义	返回码	行号
`void SoftI2C_Init(void)`	使能 GPIOB 时钟、PB6/PB7 配开漏 50MHz、两线置空闲高	无	`i2c.c:34-47`
`void SoftI2C_BusRecover(void)`	补 9 时钟 + 补 STOP，的总线复位	无	`i2c.c:55-81`
`uint8_t SoftI2C_Probe(uint8_t addr7)`	只发地址字节（写方向）判断器件是否存在	**1 = 在**，0 = 不在	`i2c.c:174-183`
`uint8_t SoftI2C_WriteBytes(uint8_t addr7, const uint8_t *buf, uint16_t len)`	START + 地址(w) + len 字节 + STOP，逐字节查 ACK	**0 = 成功，1 = 失败**	`i2c.c:185-207`
`uint8_t SoftI2C_ReadBytes(uint8_t addr7, uint8_t *buf, uint16_t len)`	START + 地址(r) + 读 len 字节（末字节 NACK）+ STOP	0 = 成功，1 = 失败	`i2c.c:209-228`
`uint8_t SoftI2C_WriteReg(uint8_t addr7, uint8_t reg, uint8_t val)`	写 `reg` 再写 `val`，即 2 字节的 `WriteBytes`	同 `WriteBytes`	`i2c.c:230-238`
`uint8_t SoftI2C_ReadRegs(uint8_t addr7, uint8_t reg, uint8_t *buf, uint16_t len)`	写 `reg` → **重复起始条件** → 读 `len` 字节	同 `ReadBytes`	`i2c.c:241-272`
`uint8_t SoftI2C_ReadReg(uint8_t addr7, uint8_t reg, uint8_t *val)`	`ReadRegs(...,1)` 的封装	同上	`i2c.c:274-277`

地址位：所有 API 一律接收 **7 位地址**，函数内部自行拼接读写位——写方向 `(uint8_t)(addr7 << 1)`（`i2c.c:179,190,246,285`），读方向 `(uint8_t)((addr7 << 1) | 0x01)`（`i2c.c:214,258`）。注释（`i2c.h:18-21`）特意提醒不要把模块丝印的 8 位地址混进来：OLED 标 0x78（8 位写地址）→ 7 位 0x3C，MPU6050 标 0x68 → 7 位 0x68。两个常量分别定义在 `oled.h:21`（`OLED_ADDR7 0x3C`）与 `mpu6050.h:19`（`MPU6050_ADDR7 0x68`）。

字节序：字节内 MSB 先发（`i2c.c:117` 注释"高位先发"）、先收（`i2c.c:152-156`）；寄存器访问是"寄存器地址字节在前，数据在后"（`i2c.c:234-235`）。`ReadRegs` 用**重复起始条件**（`i2c.c:257`，注释见 `i2c.c:240`）而不是"STOP + 重新 START"：中间不发 STOP 就不会把总线交还给其他潜在主机，也不会让从机把内部寄存器指针复位，是读寄存器序列的标准写法。多字节读是设备内部地址自增，字节序由器件决定（例如 `mpu6050.c:62` 以 14 字节从 `ACCEL_XOUT_H` 起读，得到的是高字节在前的 7 个 16 位量）。

参数边界：`len` 为 `uint16_t` 且无上限校验（`(i < len - 1)` 在 `len == 0` 时不进入循环，无下溢）；三个 API 都不检查空指针，也不返回"写入了几个字节"，失败时输出缓冲区可能已被部分填充，调用者必须依据返回值整体丢弃数据。

3.8 流式接口：`SoftI2C_Begin/SendByte/End`（`i2c.c:283-296`）

`SoftI2C_Begin(addr)` 只做 `i2c_start()` + 发地址（`i2c.c:285`，且把地址字节的 ACK 用 `(void)` 丢弃），`SoftI2C_SendByte(val)` 直接转发（`i2c.c:290`，返回 0 = 从机应答），`SoftI2C_End()` 只发 STOP（`i2c.c:295`，返回 void）。它把"事务边界"交给调用者，适合"一次事务里连发几千字节"。

`i2c.h:45-47` 明确写了：**当前 OLED 用短块差分刷新，不用它。**原因是流式接口缺少错误处理通道：

- 流一旦中途失败**无法回滚**。上半帧已经推进到 OLED 的显存里，`SoftI2C_End()` 又是 void，连 STOP 是否成功（`i2c.c:108` 的故障）都无法反馈给调用者；只能在每次 `SendByte` 的返回值里发现"某字节没被应答"，而那时损坏已经发生。
- 短块事务（`oled.c:182-197`）每次只发"7 字节窗口命令 + 33 字节数据"，每一条都通过 `SoftI2C_WriteBytes` 的返回值判定成功与否，失败立刻 `need_recovery()`（`oled.c:199`）——**损坏被限制在一个 32 字节块内**，并且可以重发整条初始化序列把面板恢复到已知状态。这正是 `i2c.h:46` 说的"短块差分刷新，以便块间采样"：块与块之间主循环还能去读 MPU6050（`oled.c:132-134`），长事务做不到这一点，也做不到逐块校验。
- 流式接口在当前固件里是**未被使用的通用能力**：USER 目录全部 `.c` 中检索 `SoftI2C_Begin` / `SoftI2C_SendByte` / `SoftI2C_End` 只命中 `i2c.c:283/288/293` 三个定义，没有任何调用点。

3.9 并发模型：没有锁的协作式互斥

`i2c.c` 里**没有互斥锁、没有忙标志、没有临界区**；整个模块假定"同一时刻只有一个调用者在用总线"。这个假定靠三件事维持：

1. **主循环单线程串行。** `main.c:16` 的注释就是这条约束的书面化："中断只计时；共用 I2C 的采样和短块刷新均在主循环执行。"OLED 刷新、MPU6050 读取、`SoftI2C_Probe` 全部由主循环按 `due(now, ...)` 的时间片驱动（`main.c:596-613`），永不并发。
2. **中断里确实没有 I2C。** `stm32f10x_it.c` 里唯一启用的中断处理是 `SysTick_Handler`（`stm32f10x_it.c:136-140`），只调用 `delay_tick()` 与 `Controls_Tick()`；模板里的外设中断处理是被注释掉的（`stm32f10x_it.c:154-156`）；DHT11 的 `EXTI1_IRQHandler`（`dht11.c:53-70`）内部只有 `delay_micros16()` 取时间戳（`dht11.c:60`）和计数器。全工程没有任何中断服务程序碰 I2C。
3. **只做时间层面的协调，不做锁。**典型例子是 `main.c:599-601`：MPU6050 重初始化（会拉长总线活动）被 `!DHT11_Busy()` 拦住，注释写明"避免 120ms MPU 复位跨越 DHT 的 20ms 起始脉冲"。这是跨模块的时间互让，不是资源锁。

这套模型在什么情况下会破：

- **一旦有中断（或任何优先级更高的执行流）调用 I2C API，立即失效。**位翻转引擎的每一次拉低/拉高都必须连续；被抢占后，两次事务的位相位会交错在同一对引脚上，双方数据同时损坏，而 `i2c_fault` 是一个被两边共享的全局变量，一方的故障会立刻让另一方短路返回失败。代码里没有任何断言或忙标志能发现这件事。
- 中断**抢占本身**（SysTick 每 1ms 一次，`main.c:248-251` 的 POST 校验确认了 1ms 粒度；DHT11 接收期 42 个边沿，`dht11.c:70`）会拉长位的半周期。由于 I2C 规范不限制时钟高/低电平的最长时间，这与从机的 clock stretching 等价，协议上无害，只是速率略有下降——留出的余量是：单次中断服务几微秒，远小于 `scl_released()` 的 1000µs 阈值，因此不会误判为卡总线。
- 工程里**有**临界区惯用法，只是没用在 I2C 上：`controls.c:200-206` 用 `__get_PRIMASK()` / `__disable_irq()` / `__set_PRIMASK(mask)` 把编码器计数与按键事件从 SysTick 里原子取出。I2C 没做同样的保护，因为当前确实没有第二个执行流访问它。

3.10 时序与电气风险清单

#	风险	证据行号	影响	建议
1	开漏输出**未启用内部上拉**，高电平完全依赖模块上的 4.7k	`i2c.c:41`、`i2c.h:12-13`	模块上拉缺失/虚焊时两条线浮空，读数不确定，总线全废且没有明确报错	保留外部上拉；或在诊断页把"两线空闲高"作为开机自检项
2	**无总线仲裁**：只在 START 前查一次 SDA	`i2c.c:91`；发送期间 `i2c.c:119-124` 拉高 SDA 后不回读比对	若真的接上第二个主机，数据被破坏且无人察觉（典型的多主冲突）	单主机设计可接受；再多一个主机就必须逐位回读 SDA → SCL 做仲裁
3	**无时钟同步检测**：只有"被拉死"超时，没有周期测量	`i2c.c:12-22`	从机拉长 SCL（合法 stretching）不会被统计，速率异常无迹可查	可把 `scl_released()` 触发次数累加进仪表（DEBUG 页）作为电气健康指标
4	速率是**纯循环延时**，随编译选项漂移	`i2c.h:29`（注明 72MHz/-O0）；`level.uvprojx:317` `<Optim>1</Optim>` = Level 0 (-O0)，`level.uvprojx:14` `<uAC6>0` = ARM Compiler 5，`level.uvprojx:340` 附加选项为空	注释里那三个实测点只对当前编译配置成立；一旦改成 -O1/-O2 或换编译器，实际 SCL 频率会整体偏移	加编译期检查（如 `#if I2C_DELAY_LOOPS != 8 时 #warning 需重新测量速率`），或把半周期改为基于 TIM2 的定时延时
5	250kHz 距 400kHz 上限余量不大，长杜邦线+2.35k 上拉使上升沿变慢	`i2c.h:33-35`	有效高电平变窄，OLED 可能花屏、MPU 可能读出错误数据	按注释把 `I2C_DELAY_LOOPS` 调回 12-20，并用示波器确认 SCL 高电平时间
6	`SoftI2C_Init` 未预置 ODR	`i2c.c:43` 与 `i2c.c:45-46` 之间	开漏 + ODR=0 会让两线被主动拉低数条指令（当前 SCL/SDA 同低，不构成 START/STOP，实际无害）	在 `GPIO_Init()` 前先写 `GPIOB->BSRR = I2C_SCL_PIN | I2C_SDA_PIN`
7	无事务忙碌标志/重入断言	`i2c.c` 全文无 busy 变量	未来若在中断里调用 I2C，会静默破坏两条执行流的时序	加 `static uint8_t in_txn`，在 `i2c_start` 置位、`i2c_stop` 清零，并用断言暴露重入

另外一处与代码无关但会误导人的配置：Keil 目标里登记的外部晶振是 12MHz（`level.uvprojx:21` 的 `CLOCK(12000000)`），而工程的 `HSE_VALUE` 走的是 8MHz 分支（`stm32f10x.h:115-121`，工程 Define 只有 `STM32F10X_MD,USE_STDPERIPH_DRIVER`，见 `level.uvprojx:341`，未定义 `STM32F10X_CL`）。这个字段只影响 Keil 的仿真/时间显

示，不改变编译产物与位翻转速率，但容易让后来人误算 loops 与频率的对应关系。

本章发现的问题与建议

1. **`i2c_fault` 的"锁存"语义被误解，实际是事务级标志。** 现象：`i2c.c:85` 每次 `i2c_start()` 无条件清零。影响：一方面它不会提供"永久熔断"保护（挂在总线上的下一个事务仍会重试并失败），另一方面它也不会永久锁死系统；任何依赖"置位后所有后续事务都短路"的推理都是错的。建议：改名为 `i2c_txn_error` 并在 `i2c.h` 里写明作用域；若确实想要真正的锁存保护，应只在 `SoftI2C_BusRecover()` 成功后清零。
2. **STOP 的失败无法从返回值层面区分。** 现象：`i2c_stop()` 返回 void（`i2c.c:99`），STOP 后 SDA 未回高只置 `i2c_fault`（`i2c.c:108`），而 `SoftI2C_Probe` 之后立即返回，最终上报为"0 = 不在"（`i2c.c:178,182`）。影响：调用方无法区分"器件真的不在"与"总线挂死"。建议：给 `i2c_stop()` 加 `uint8_t` 返回值并在 API 层用一个独立的错误码上报总线故障。
3. **`i2c_read_byte` 用 0xFF 兼作错误码。** 现象：`i2c.c:146,154,164` 三处返回 0xFF，与合法数据同值。影响：直接调用者（含未来的流式接口用户）会把故障当成数据 0xFF。建议：增加状态出参，或返回 `int16_t`（-1 表示失败）。
4. **失败时输出缓冲区已被部分写入。** 现象：`i2c.c:222` 先赋值、223 行才检查 fault。影响：`mpu6050.c:62` 这类调用若只看数据不看返回值，会读到半新半旧的采样。建议：读进临时数组，成功后整块 `memcpy` 出去。
5. **`scl_released()` 的返回值与命名反向，且一处被丢弃。** 现象：函数返回 1 = 正常（`i2c.c:16,21`），调用者返回 1 = 失败（`i2c.c:89,122,133,154,164`）；`i2c.c:104` 的 `(void)scl_released();` 直接丢弃结果。影响：可读性差，容易在后续维护中写反判断。建议：改名 `scl_released_ok()` 或反转返回值，让 `i2c_stop()` 也返回该状态。
6. **流式接口是无人使用的死代码，且没有错误通道。** 现象：`i2c.c:283/288/293` 三个定义在 USER 目录内没有任何调用点；`SoftI2C_Begin` 丢弃地址 ACK（`i2c.c:285`），`SoftI2C_End` 返回 void（`i2c.c:293`）。影响：接口看起来可用，实际无法安全用于长帧。建议：要么删除，要么补齐 `Begin` / `End` 的返回值并明确"中途失败必须由调用者作废整帧"的契约。
7. **每个循环的延时成本没有运行时校验。** 现象：速率完全由 `I2C_DELAY_LOOPS 8`（`i2c.h:36`）加编译选项决定，`i2c.h:29` 的实测条件（72MHz、-O0）恰好与 `level.uvprojx:317` 一致，但代码里没有任何保护。影响：换优化等级或编译器后总线速率静默漂移，可能越过器件或 400kHz 规范上限。建议：加 `#warning` /静态断言锁定配置，或改用 `delay_micros16()` 定时的半周期（代价是每半周期多一次 TIM2 读）。

第 4 章　器件驱动：MPU6050 与 DHT11

本章要点

- `mpu6050`（`mpu6050.c` 99 行 / `mpu6050.h` 45 行）用 5 条寄存器写加 3 段延时完成初始化，写后逐寄存器回读验收（`mpu6050.c:18-53`），返回 `0 = 成功 / 1 = 失败`。
- 最关键的设计决策是 `ACCEL_CONFIG(0x1C)=0x00`：写 `0x01` 会打开加速度高通滤波、把重力分量滤掉，倾角将完全失效（`mpu6050.c:41-44`）。
- `MPU6050_ReadAll` 从 `0x3B` 一次突发读 14 字节，使加速度、温度、角速度来自同一采样点（`mpu6050.c:58-73`）。
- `MPU6050_TempToC` 是纯整数实现，常数比参考公式少 0.53 且做除法截断（`mpu6050.c:96-99`）；它与 `MPU6050_ReadAccel`、`MPU6050_ACCEL_LSB_PER_G` 在本工程中均为零引用。
- `dht11`（`dht11.c` 105 行）把单总线时序拆成 3 段非阻塞状态机（`dht11.c:73-103`），位解码全部在 `EXTI1_IRQHandler` 内完成（`dht11.c:53-71`），输出统一为 **×10 单位**（`dht11.h:5-6`）。
- `DHT11_Busy()`（`dht11.c:105`）不是互斥锁，而是"总线忙"标志，被主循环用于回避长阻塞（`main.c:230`、`main.c:600`、`main.c:628`、`main.c:661`）。

4.1 MPU6050 初始化：逐寄存器精读

`mpu6050` 只走"直接读寄存器"路线，不碰 DMP：`mpu6050.h:12-14` 写明理由是 DMP 要往芯片灌固件、要拉进一大坨库代码，而水平仪是静态应用，用加速度计直接算姿态角就够了。器件在软件 I2C 总线上从机地址 `0x68`（`mpu6050.h:19`，AD0 悬空、模块自带 4.7k 下拉，`mpu6050.h:10`），寄存器地址定义在 `mpu6050.h:21-29`。

初始化把 5 个寄存器目标值集中成两个常量数组：

```
static const uint8_t regs[5] = {MPU6050_REG_PWR_MGMT_1, MPU6050_REG_SMPLRT_DIV,
       MPU6050_REG_CONFIG, MPU6050_REG_GYRO_CONFIG, MPU6050_REG_ACCEL_CONFIG};
static const uint8_t values[5] = {0x01, 0x04, 0x03, 0x00, 0x00};
```

（出处 `mpu6050.c:18-20`）

同一份数组既用于写入阶段，也用于写后回读的比对基准（`mpu6050.c:50-52`），避免"写一套值、校验另一套值"的漂移。逐寄存器含义如下。

寄存器	地址	写入值	位域解释	效果	行号
`PWR_MGMT_1`	`0x6B`	`0x80`	`DEVICE_RESET` (bit7)=1	软复位，回到确定的初始状态	`mpu6050.c:23`
—	—	延时 100ms	—	等复位完成（复位后 `SLEEP` 位回到默认的 1）	`mpu6050.c:24`
`PWR_MGMT_1`	`0x6B`	`0x01`	`SLEEP` (bit6)=0，`CLKSEL` (bits2:0)=001	解除休眠，时钟源选 X 轴陀螺 PLL	`mpu6050.c:27`
—	—	延时 10ms	—	等 PLL 稳定	`mpu6050.c:28`
`SMPLRT_DIV`	`0x19`	`0x04`	采样率分频	1kHz/(1+4)=**200Hz** 采样率	`mpu6050.c:33`
`CONFIG`	`0x1A`	`0x03`	`DLPF_CFG` (bits2:0)=3，`EXT_SYNC_SET` =0	数字低通滤波，加速度带宽 **44Hz**（陀螺带宽 42Hz、陀螺延迟 4.9ms 为器件手册值；源码注释只写了 44Hz）	`mpu6050.c:36`
`GYRO_CONFIG`	`0x1B`	`0x00`	自检位 bits7:5=0，`FS_SEL` (bits4:3)=00	陀螺满量程 **±250°/s**，灵敏度 131 LSB/(°/s)	`mpu6050.c:39`
`ACCEL_CONFIG`	`0x1C`	`0x00`	自检位 bits7:5=0，`AFS_SEL` (bits4:3)=00，`ACCEL_HPF` (bits2:0)=000	加速度 **±2g**（16384 LSB/g），**关闭高通滤波**	`mpu6050.c:44`
—	—	延时 10ms	—	等量程与滤波生效后再读	`mpu6050.c:46`

三处值得单独说明的细节：

1. **`0x80` 与 `0x01` 分两次写同一个寄存器**（`mpu6050.c:23`、`mpu6050.c:27`）。第一笔是复位命令，芯片执行后 `SLEEP` 位回到默认的 1；第二笔才把 `SLEEP` 清 0 并把时钟源切到 X 轴陀螺 PLL（源码注释 `mpu6050.c:26` 给的取舍理由："比内部 8MHz 稳"）。注意 `regs[]/values[]` 里保存的是第二笔的 `0x01` 而不是复位用的 `0x80`——复位命令会自清，回读值只可能是复位后的默认值，所以复位动作本身无法用"回读==写入值"验证，校验点只能落在复位之后的状态上。
2. **采样率注释里的"分频 4"要按寄存器语义读**：注释（`mpu6050.c:30-32`）写"陀螺输出 1kHz，分频 4 -> 200Hz"、寄存器值是 `0x04`，而手册公式 `f = 1kHz/(1+SMPLRT_DIV)` 的实际除数是 5——结论 200Hz 不变。选 200Hz 的理由也在这条注释里：主循环每 5ms 采一次（`main.c:21` 的 `SAMPLE_INTERVAL_MS 5`），采样率远高于读取率，每次读到的都是新样本。
3. **`GYRO_CONFIG=0x00` 是 ±250°/s，不是 ±2000°/s**（后者要写 `0x18`、即 `FS_SEL=11`、16.4 LSB/(°/s)）；`mpu6050.c:38` 的注释"陀螺 ±250 度/秒，不自检"与位域一致，可以采信。

4.1.1 关键决策：`ACCEL_CONFIG` 必须写 `0x00`

这是整个驱动里被注释反复强调的一处，值得逐字引用：

```
/* 加速度 ±2g，不自检。
   注意这里必须是 0x00：原厂 51 例程给的是 0x01，
   那个值会打开加速度的高通滤波，把重力分量滤掉，算倾角就废了。 */
if (SoftI2C_WriteReg(MPU6050_ADDR7, MPU6050_REG_ACCEL_CONFIG, 0x00)) return 1;
```

（出处 `mpu6050.c:41-44`）

`ACCEL_CONFIG` 的位域是：bits7:5 为三轴自检开关，bits4:3 为 `AFS_SEL` 满量程选择，bits2:0 为 `ACCEL_HPF` 高通滤波配置（000 = 复位/不使用，001 = 5Hz，…）。写 `0x01` 只动最低位，`AFS_SEL` 仍是 ±2g，但 `ACCEL_HPF=001` 会把加速度信号的直流分量滤掉——而"水平静止时某个轴上的重力投影"恰恰就是直流分量。对一个靠重力分量算倾角的静态水平仪来说，这等于把被测信号本身滤掉，读数会缓慢衰减到 0。

这条注释的价值不只是"避免抄错一个字节"，而是记录了一个**器件语义级别的坑**：同样的 `AFS_SEL`、相近的数值，只因低三位不同就从"可用"变成"不可用"，且故障表现是"读数缓慢归零"这种看起来不像 I2C 故障的现象。本工程写 `0x00`，把高通彻底关掉，让原始重力分量原样进入后续的整数滤波与倾角计算（`main.c:586-587`）。

4.2 写后回读验收

初始化收尾是一段"写完必须读回来比对"的逻辑：

```
        if (MPU6050_ReadID() != MPU6050_ADDR7) return 1;
        /* ACK 只表示收到字节：回读确认量程、滤波和时钟真正生效。 */
        for (i = 0; i < 5; i++)
        {
                if (SoftI2C_ReadReg(MPU6050_ADDR7, regs[i], &value) || value != values[i]) return 1;
        }
```

（出处 `mpu6050.c:48-53`）

设计动机写在 `mpu6050.c:49` 的注释里：**"ACK 只表示收到字节"**。一次写事务只证明从机在第 9 个时钟拉低了 SDA（`SoftI2C_WriteReg` 的返回码只覆盖这一层，`i2c.c:230`），不证明寄存器真的接受了值——从机可能仍在复位、内部状态机未就绪，或被总线毛刺干扰成另一个字节。回读把"写成功"提升为"配置生效"，并在开机自检阶段失败暴露：`MPU6050_Init()` 的返回值直接决定 POST 第 5 步结果（`main.c:400-401`，步骤名 `"MPU6050"` 见 `main.c:286-290`）与全局 `sensor_ready`。

代价是初始化多出 6 次单字节读事务（1 次 `WHO_AM_I` + 5 次回读），逐位翻转的软件 I2C 上每次约 27 个 SCL 时钟（`mpu6050.c:46` 另有 10ms 稳定等待）；更重要的代价是**校验偏严**——回读必须精确等于写入值，若某颗芯片 `CLKSEL` 回读不是 001（如 PLL 未锁定而回落内部振荡器），`MPU6050_Init()` 直接失败、POST 报 `ERR`，而非"降级可用"。取舍可接受：姿态不准的仪表没有使用价值，早失败早暴露。

4.3 `WHO_AM_I` 与 `MPU6050_ReadID`

`MPU6050_ReadID`（`mpu6050.c:5-14`）读 `WHO_AM_I(0x75)`，读失败返回 `0xFF`（`mpu6050.c:11`），成功返回寄存器内容。`mpu6050.c:48` 要求它等于 `MPU6050_ADDR7` 即 `0x68`。MPU6050 的 `WHO_AM_I` 返回 7 位从机地址的高 6 位，因此"等于从机地址"这一比较对这颗芯片成立（器件手册）。这条校验还顺带覆盖了 `0xFF` 分支：总线读失败同样让初始化失败。

需要纠正一处容易想当然的印象：**`MPU6050_ReadID` 的唯一调用点就是 `mpu6050.c:48`**，`main.c` 中没有任何 `MPU6050_ReadID` 调用（按 `MPU6050_` 前缀检索，`main.c` 只命中 `MPU6050_ADDR7` 探测 `main.c:510/514`、`MPU6050_Init` `main.c:400/601`、`MPU6050_ReadAll` `main.c:575`）。所以器件身份校验是初始化内部的一环、不是独立的自检步骤，POST 的 `MPU6050` 步骤实际测的是"初始化整条链路"；也正因为校验做在 `Init` 里，POST 无法区分"器件不是 MPU6050"与"寄存器没写进去"——两者都只表现为 `ERR`。

4.4 数据通路：一次突发读 14 字节

```
uint8_t MPU6050_ReadAll(int16_t *acc, int16_t *gyro, int16_t *temp_raw)
{
        uint8_t buf[14];

        if (SoftI2C_ReadRegs(MPU6050_ADDR7, MPU6050_REG_ACCEL_XOUT_H, buf, 14) != 0)
        {
                return 1;
        }

        acc[0] = (int16_t)((buf[0] << 8) | buf[1]);     /* AX */
        acc[1] = (int16_t)((buf[2] << 8) | buf[3]);     /* AY */
        acc[2] = (int16_t)((buf[4] << 8) | buf[5]);     /* AZ */
        *temp_raw = (int16_t)((buf[6] << 8) | buf[7]);  /* 温度 */
        gyro[0] = (int16_t)((buf[8] << 8) | buf[9]);    /* GX */
        gyro[1] = (int16_t)((buf[10] << 8) | buf[11]);  /* GY */
        gyro[2] = (int16_t)((buf[12] << 8) | buf[13]);  /* GZ */
```

（出处 `mpu6050.c:58-73`，返回 `0 = 成功 / 1 = 失败`）

三件事一起成立，这段代码才有意义：

- **起点与长度**：从 `MPU6050_REG_ACCEL_XOUT_H = 0x3B`（`mpu6050.h:25`，注释"从这开始连读 14 字节"）起读 14 字节 = 加速度 6 + 温度 2 + 角速度 6，正好覆盖到 `0x48`。`mpu6050.h:36` 把目的写得很直白："一次连读 14 字节…保证三轴是同一时刻的"。
- **原子性**：`SoftI2C_ReadRegs`（`i2c.c:241-272`）在写完寄存器指针后用**重复起始条件**（`i2c.c:257`）转读，整段只有一个停止条件（`i2c.c:270`）。也就是说 14 字节落在同一个 I2C 事务里，MPU6050 在这期间冻结采样寄存器。若改成"先 `MPU6050_ReadAccel` 再单独读陀螺"，两次事务之间隔着数毫秒的软件 I2C 位翻转，加速度与角速度就来自不同采样点，姿态数据会出现时间错位——在本仪表 200Hz 采样、5ms 读取的节奏下，错位量级正好等于读取周期。
- **大端拼装**：`(int16_t)((buf[0] << 8) | buf[1])` 把高字节左移 8 位后或上低字节，再整体转 `int16_t` 完成符号扩展；缓冲区 14 字节的语义以 `mpu6050.c:60` 的 `uint8_t buf[14]` 为准，与 `mpu6050.h:36` 的说明一一对应。

`MPU6050_ReadAccel`（`mpu6050.c:78-92`）是同样手法的 6 字节版本（只读加速度），`mpu6050.h:39` 注释"只读加速度，够水平仪用了"；但它在本工程中**零调用**——`main.c` 统一走 `MPU6050_ReadAll`（`main.c:575`），因为原始数据页、`stale` 判定与倾角计算共用同一次读取结果。

量程、分辨率与实际用途

通道	量程（配置行号）	灵敏度	本仪表的实际用途	证据
加速度 X/Y/Z	±2g（`mpu6050.c:44`）	16384 LSB/g（`mpu6050.h:31`），1 LSB ≈ 0.061 mg	X/Y 相对开机基准算 pitch/roll；Z 参与姿态合法性判定	`main.c:586-587`、`main.c:141-142`
陀螺 X/Y/Z	±250°/s（`mpu6050.c:39`）	131 LSB/(°/s)，1 LSB ≈ 0.0076°/s	仅上 RAW DATA 页显示原始整数，不参与解算	`main.c:575-578`、`dashboard.c:196`

通道	量程（配置行号）	灵敏度	本仪表的实际用途	证据
温度	340 LSB/°C（`mpu6050.c:94`）	1 LSB ≈ 0.0029°C	仅上 RAW DATA 页显示原始值，未换算	`main.c:579`、`dashboard.c:200`

换算：±2g 满量程下 32768/2 = 16384 LSB/g，故 1 LSB = 2/32768 g = 61.0 μg ≈ 0.061 mg；±250°/s 满量程下 32768/250 ≈ 131 LSB/(°/s)，故 1 LSB = 250/32768 ≈ 0.0076°/s。

驱动层的"缺位"也在这里显形：`MPU6050_ACCEL_LSB_PER_G` 定义了 16384（`mpu6050.h:31`）却在全工程**零引用**（全树检索只命中定义行），真正用到的换算被写成业务层魔数 `5730L`（`main.c:586-587`，即 180/π×100），器件语义（量程、灵敏度）散落在使用点。

4.5 DHT11：单总线非阻塞状态机

DHT11 用一根数据线（PA1）完成"主机拉低起始 → 释放 → 传感器回 40 位"的全过程，时序以微秒计。本驱动没有用 `delay_us` 阻塞等待，而是把整个过程拆成 3 个跨调用的状态，由主循环反复调用 `DHT11_Task(now)` 推进（`main.c:532`；POST 阶段由 `post_pump` 调用，`main.c:227`）。

状态	进入/退出条件	动作	时限	行号
0 空闲	`!state && (int32_t)(now-next_read)>=0`	屏蔽 EXTI1、清 `receiving` / `rx_done`、采样总线空闲电平 `idle_high`、`attempts++`、切开漏输出、拉低 PA1、设定超时与下次周期	起始脉冲 20ms	`dht11.c:76-82`
1 起始脉冲	`state==1 && (int32_t)(now-deadline)>=0`	清 `rx` / `edge_count` / `rx_done`、清 EXTI 挂起位、`receiving=1`、开 EXTI1、释放总线（置高）、切上拉输入、缩短超时	收帧窗口 10ms	`dht11.c:83-89`
2 收帧	`rx_done || (int32_t)(now-deadline)>=0`	屏蔽 EXTI1、清 `receiving`、记录边沿数 `edges`、成功则拷贝 5 字节并解码、失败则算错误码、回到状态 0	同上	`dht11.c:90-101`
— 有效性维护	每次调用末尾	距上次好帧超过 6s 则 `valid=0`	6000ms	`dht11.c:102`

状态 0 与状态 1 的逐字实现：

```
    if (!state && (int32_t)(now-next_read)>=0) {
        EXTI->IMR &= ~EXTI_Line1; receiving=0; rx_done=0;
        data.idle_high=(GPIOA->IDR & GPIO_Pin_1)!=0;
        data.attempts++;
        data_mode(GPIO_Mode_Out_OD);
        GPIO_ResetBits(GPIOA,GPIO_Pin_1);
        deadline=now+20; next_read=now+2500; state=1;
    } else if (state==1 && (int32_t)(now-deadline)>=0) {
        memset((void *)rx,0,sizeof(rx)); edge_count=rx_done=0;
        EXTI_ClearITPendingBit(EXTI_Line1);
        receiving=1; EXTI->IMR |= EXTI_Line1;
        GPIO_SetBits(GPIOA,GPIO_Pin_1);
        data_mode(GPIO_Mode_IPU); /* Release and explicitly enable the input path. */
        deadline=now+10; state=2;
    }
```

（出处 `dht11.c:76-89`）

逐句说明：

- **先屏蔽 EXTI 再拉低**（`dht11.c:77`、`dht11.c:81`）：主机自己拉低产生的下降沿不能当成数据位，否则 `edge_count` 起步就多一个、后续所有位错位一格；状态 1 重新使能中断前也先清一次挂起位（`dht11.c:85` 在 `dht11.c:86` 之前），保证开中断瞬间没有残留标志。
- **`idle_high` 在拉低之前采样**（`dht11.c:78`）：这是"发起前总线是否空闲为高"的诊断位，直接决定错误码是 3 还是 5（`dht11.c:97`）；在破坏总线状态之前抓下诊断信息，是低成本的可观测性设计。
- **`GPIO_Mode_Out_OD` + `GPIO_ResetBits`**（`dht11.c:80-81`）：开漏输出只能拉低、不能推高，配合 4.7k 上拉（`dht11.c:39` 注释提醒裸传感器仍需外部 4.7k）避免主机与传感器的输出打架。释放时先 `GPIO_SetBits`（`dht11.c:87`，开漏下等价于高阻）再切回 `GPIO_Mode_IPU`（`dht11.c:88`，注释 "Release and explicitly enable the input path"）——先置高再改模式，避免切换瞬间的低电平毛刺。
- **两个 deadline 分开设**（`dht11.c:82` 的 `now+20`、`dht11.c:89` 的 `now+10`）：20ms 是起始脉冲宽度（器件手册只规定拉低 ≥18ms，本驱动取 20ms、留 2ms 余量）；10ms 是收帧预算，真正帧长约 5ms（算式见 4.6 节），留了约 2 倍余量。
- **`next_read=now+2500` 写在"发起"处而不是"收帧结束"处**（`dht11.c:82`）：周期从发起时刻起算，保证两次尝试至少间隔 2.5s，满足器件手册 ≥1s 的采样间隔要求；首次等待 2s 由 `DHT11_Init` 设定（`dht11.c:50`），给传感器上电稳定留余量。
- **`memset((void *)rx, 0, sizeof(rx))`**（`dht11.c:84`）：那个 `(void *)` 不是风格问题，而是 `rx` 声明为 `volatile`（`dht11.c:7`）、标准库原型不接受 volatile 指针——这是 volatile 带来的一处可见副作用。

初始化注释（`dht11.c:42-45`）还记录了一个标准外设库层面的坑：

```
    /* SPL configures trigger registers only when LineCmd is ENABLE. */
    exti.EXTI_Trigger=EXTI_Trigger_Falling; exti.EXTI_LineCmd=ENABLE;
    EXTI_Init(&exti);
    EXTI->IMR &= ~EXTI_Line1; /* Keep capture masked until the start pulse ends. */
```

（出处 `dht11.c:42-45`）

SPL 的 `EXTI_Init()` 只在 `LineCmd == ENABLE` 时才写触发边沿寄存器；若为了"先不开中断"而传 `DISABLE`，下降沿触发配置根本不会生效、EXTI 永远不触发。正确做法是三步：先用 `ENABLE` 完成配置（`dht11.c:43-44`），再手动清 `EXTI->IMR` 把捕获屏蔽到起始脉冲结束（`dht11.c:45`），随后清一次挂起位（`dht11.c:46`）——项目文档记录的"EXTI 初始化修复后温湿度恢复"改的就是这一处。PA1→EXTI1 的映射见 `dht11.c:40`，中断优先级为抢占 1 / 子优先级 0（`dht11.c:47-49`）。

4.6 `EXTI1_IRQHandler`：中断里做位解码

```
    if (EXTI_GetITStatus(EXTI_Line1) == RESET) return;
    EXTI_ClearITPendingBit(EXTI_Line1);
    if (!receiving) return;
    stamp=delay_micros16(); span=(uint16_t)(stamp-edge_time); edge_time=stamp;
    if (!edge_count) { edge_count=1; return; }
    if (edge_count==1) {
        if (span < 120 || span > 210) { receiving=0; rx_done=2; }
        else edge_count=2;
        return;
    }
    if (span < 55 || span > 150) { receiving=0; rx_done=2; return; }
    bit=edge_count-2;
    rx[bit/8]=(uint8_t)((rx[bit/8]<<1) | (span >= 100));
    if (++edge_count==42) { receiving=0; rx_done=1; EXTI->IMR &= ~EXTI_Line1; }
```

（出处 `dht11.c:57-70`）

为什么第 2 个边沿之后才开始取样。 传感器在第 0 个下降沿开始响应（先 80µs 低、再 80µs 高），第 0 位数据的下降沿才是第 2 个边沿：

- 第 0 个下降沿（`edge_count` 0→1）只记时间戳、不判位（`dht11.c:61`）；`edge_time` 已在上一行更新（`dht11.c:60`），为下一次测量建立基准。
- 第 1 个下降沿（`edge_count==1`）测得的间隔 = 响应低 80µs + 高 80µs = **160µs**，落在 120..210µs 帧头窗口内（`dht11.c:63`），充当"这确实是一帧 DHT11 响应"的握手确认。
- 从第 2 个下降沿起，相邻两沿间隔 = 位低 50µs + 位高：位"0"的高电平为 26~28µs（合计 76~78µs），位"1"为 70µs（合计 120µs）；窗口 55..150µs（`dht11.c:67`）同时容纳两者，判决门限 100µs（`dht11.c:69`）落在两个分布之间（下侧余量 22~24µs、上侧 20µs）。**窗口负责剔除毛刺与断帧，门限负责判 0/1**，分工不同。
- 位序与结束：`bit = edge_count - 2`（`dht11.c:68`）使 `edge_count=2` 时拼第 0 位，每来一位左移一格再或上最低位（`dht11.c:69`）天然实现 MSB 优先的字节拼装；累计 42 个下降沿（第 0 个记时间戳、第 1 个验帧头、第 2~41 个拼 40 位）即满 5 字节，`dht11.c:70` 置 `rx_done=1` 并再次屏蔽 EXTI。

实时性代价。 每条边沿要执行：SPL 读/清 EXTI 挂起位、`receiving` 判空、读 `TIM2->CNT` 取微秒时间戳（`delay.c:19`）、16 位减法、volatile 数组的读-改-写，加上 Cortex-M3 的异常进出栈。按每条 120~200 个 CPU 周期估算，72MHz 下约 **1.7~2.8µs**，42 条边沿合计约 **70~120µs**（由周期数与主频推算，非源码实测值）。最坏帧长是 40 位全为 1：40×(50+70)+160 = 4960µs ≈ **5.0ms**，中断总开销约占该帧 CPU 时间的 2%，单条中断相对 76µs 的最短位宽有 25 倍以上余量；主循环在约 5ms 内被切 42 次、每次 2µs 量级，对 5ms 的采样周期（`main.c:21`）与 SysTick 毫秒时基没有实质影响（被抢占只会推迟、不会丢计数）。代价真正体现在**纪律约束**上：这段代码必须保持"只打时间戳、只移位"的形态，一旦有人在中断里加打印、浮点或 I2C 访问，42 次调用的开销会立刻膨胀。ISR 里没有屏蔽全局中断（`dht11.c:5` 注释 "No global IRQ masking"），也是刻意的低干扰做法。

`volatile` 的必要性。 `dht11.c:7` 声明了 `static volatile uint8_t rx[5], edge_count, rx_done, receiving;`，`dht11.c:8` 是 `static volatile uint16_t edge_time;`。这 5 个对象都被中断写、被主上下文读（`dht11.c:76-97`）：不声明 `volatile`，编译器有权把 `edge_count` / `rx_done` 缓存在寄存器里、或重排清缓冲与随后的判断，状态 2 就永远看不到"帧收完了"，表现为间歇性收不到数据。反之，`state` / `next_read` / `deadline`（`dht11.c:9-10`）**不**加 `volatile` 是正确的——中断里从不访问它们，加了只是平白阻止优化。

4.7 解码、单位与失败语义

```
uint8_t DHT11_Decode(const uint8_t b[5], int16_t *temperature, uint16_t *humidity)
{
    int16_t t;
    uint16_t h;
    if ((uint8_t)(b[0]+b[1]+b[2]+b[3]) != b[4]) return 1;
    if (b[1] > 9 || (b[3] & 0x7F) > 9) return 2;
    h = b[0]*10U + b[1];
    t = (int16_t)(b[2]*10 + (b[3] & 0x7F));
    if (b[3] & 0x80) t = -t;
    if (h > 1000 || t < -200 || t > 600) return 2;
    *temperature = t; *humidity = h; return 0;
}
```

（出处 `dht11.c:19-30`，返回 `0 = 成功 / 1 = 校验和错 / 2 = 字段非法或超量程`）

- **校验和**（`dht11.c:23`）：前四字节之和强制转 `uint8_t`，天然完成 mod 256，再与第 5 字节比较；不匹配返回 1。这是唯一的完整性校验，**没有 CRC**。
- **小数位合法性**（`dht11.c:24`）：两个小数字节必须 ≤9，否则返回 2。这是一道"字节内容本身不可能存在"的防线，用于发现错位或乱码。
- **拼装与单位**：`h = b[0]*10 + b[1]`（`dht11.c:25`）、`t = b[2]*10 + (b[3]&0x7F)`（`dht11.c:26`）、符号位 `b[3]&0x80` 决定取负（`dht11.c:27`）。**全篇统一：驱动输出的温度与湿度都是 ×10 单位**，例如 `265` 表示 26.5°C。字段名自带单位（`temperature_x10` / `humidity_x10`，`dht11.h:5-6`），显示侧按 ×10 格式化（`dashboard.c:146-147`），存档侧直接写入记录（`main.c:150`）。
- **量程检查**（`dht11.c:28`）：湿度 >1000（>100.0%RH）、温度 <-200 或 >600（-20.0~+60.0°C）判为非法。这个窗口比 DHT11 的标称量程（20~90%RH、0~50°C，器件手册值）更宽，作用是"拦住明显的坏数据"而不是"标称范围断言"。
- **器件事实**：DHT11 的湿度小数位 `b[1]` 恒为 0（分辨率 1%RH），温度小数位同理恒为 0，所以 `(b[3]&0x7F)` 实际只贡献 0。代码没有依赖这两点，只把它们当合法位参与拼装与校验。（此为器件常识，源码中无对应断言。）

错误码在驱动内还有一个统一出口：`data.error` 取 `DHT11_Decode` 的返回值，或在收帧失败时按下式合成（`dht11.c:97`）：

```
        } else result=rx_done ? 4 : (data.idle_high ? 3 : 5);
```

`error`	含义	产生处
0	成功	`dht11.c:96`（`Decode` 返回 0）

error	含义	产生处
1	校验和错	dht11.c:23
2	小数位非法或超量程	dht11.c:24、dht11.c:28
3	10ms 内没有收到任何帧，且发起前总线空闲为高（传感器没响应）	dht11.c:97
4	收到了边沿但时序校验失败（rx_done==2）	dht11.c:63、dht11.c:67 → dht11.c:97
5	10ms 内没有收到帧，且发起前总线已被拉低（疑似线被拉死）	dht11.c:97

data.error 每次尝试都会被重写（dht11.c:98），成功后自然清零；data.valid 只在成功时置 1（dht11.c:99），并在超过 6s 没有好帧时清 0（dht11.c:102）。3 与 5 的区分完全靠 idle_high 这个发起前采样的诊断位，这是本驱动可维护性的亮点：同一个"没数据"的现象被拆成"传感器没理我"和"线被拉死了"两种，现场排查方向完全不同。

4.8 时间常量与"帧丢失"时上层看到什么

常量/时限	值	含义	行号
首次等待	2000ms	DHT11_Init 后第一次发起	dht11.c:50
尝试周期	2500ms	每次发起时设定的下一次	dht11.c:82
起始脉冲	20ms	主机拉低时长（state 1 的 deadline）	dht11.c:82
收帧窗口	10ms	state 2 的 deadline	dht11.c:89
帧头窗口	120..210µs	响应低 + 响应高的两沿间隔	dht11.c:63
数据位窗口	55..150µs	位低 + 位高的两沿间隔	dht11.c:67
位判决门限	≥100µs 记为 1	span >= 100	dht11.c:69
有效保持	6000ms	无好帧后 valid=0	dht11.c:102
尝试计数	uint16_t attempts	65535 次后回绕（约 45.5 小时 @2.5s）	dht11.c:79、dht11.h:10

一帧失败时，上层看到的是分层的可视信号：

- **单帧失败**（error=1..5，valid 仍为 1）：CLIMATE 页的大数字继续显示上一次的温湿度，右上角出现 OLD（dashboard.c:150，条件是 d->error && d->valid）；该页同时被排除在"屏幕活动"判定之外（main.c:637），并让 RGB 进入 LIGHT_ERROR（main.c:644）。
- **连续失败超过 6s**（valid=0）：CLIMATE 页大数字变成 --（dashboard.c:148），SYSTEM 页的 DHT 状态从 OK 变为 WAIT（dashboard.c:181），main.c:644 继续保持错误指示灯。
- **永久失效**：60s 周期的传感器存档条件 DHT11_Get()->valid && !DHT11_Get()->error（main.c:617）永远为假，传感器日志停止增长（logger 本身仍在运行）；save_snapshot 也会因 d->valid&&!d->error 不成立而不置 LOG_ENV_VALID（main.c:152），HISTORY 页对应字段显示 --（dashboard.c:164-165）。
- **开机自检阶段**：POST 的 DHT11 步骤最多等 6.5s（main.c:411），把错误码渲染成 E<n> 文本、成功渲染成 CRC OK、一帧都没等到渲染成 NO FRAME（main.c:420-423）。

语义上要留意：**valid 表示"数据新鲜度"而不是"本次通信成功"**——它由 6s 定时器维护（dht11.c:102），而驱动每 2.5s 就该出一帧，故正常运行时 valid 恒为 1，只有连续两帧以上都没成功才翻成 0；因此 OLD 标记（error!=0 && valid==1）是现场唯一能区分"正在用陈旧数据"的信号。

4.9 DHT11_Busy()：全工程的准互锁

uint8_t DHT11_Busy(void) { return state != 0; }（dht11.c:105）只有一行，却被主循环当作跨模块的调度信号使用。

调用点	门控对象	为什么必须让路
main.c:230	POST 阶段 OLED_Service(now)	OLED 刷新要占用主循环与共享的软件 I2C（OLED 与 MPU6050 同总线），必须避开 20ms 起始脉冲与 10ms 收帧窗口
main.c:410、main.c:419	POST 等待首帧的循环条件与结果判定	轮询"已尝试过且不再忙"
main.c:600	MPU 故障重初始化	MPU6050_Init() 内部有 100+10+10 = 120ms 延时（mpu6050.c:24/28/46），远超 20ms 起始脉冲；main.c:599 的注释把这条约束写死了："避免 120ms MPU 复位跨越 DHT 的 20ms 起始脉冲。"
main.c:628	Flash 重试 Logger_Init(now)	日志扫描/擦除耗时远超 20ms
main.c:661	主循环 OLED_Service(now)	同 main.c:230

反面例子同样重要：**60s 周期存档（main.c:616-619）不用 DHT11_Busy()**，门控是 !EventLog_Busy() && DHT11_Get()->valid && !DHT11_Get()->error——只要求手上有新鲜的温湿度可写，因为存档只是把内存里的数据写进 Flash，不需要等总线空闲。

必须强调它不是互斥锁：它只回答"DHT 状态机是否处于非空闲"，不保护任何共享资源、也没有超时或阻塞语义；真正保证不冲突的是调度纪律——驱动只有一个发起者（main.c:532，POST 阶段 main.c:227），所有可能长时间占用主循环的调用点都自觉先查这个标志。

本章发现的问题与建议

1. **MPU6050_TempToC 的常数与截断偏差**（mpu6050.c:96-99）。实现是 (int16_t)(temp_raw / 340 + 36)，而参考公式（源码注释 mpu6050.c:94）是 TEMP_OUT/340 + 36.53：常数少了 0.53，且整数除法向零截断、小数被丢掉。以 temp_raw = 0（0°C）为例返回 36，偏差 -0.53°C；正温区综合偏差约 -0.53 ~ -1.53°C，负温区因向零截断偏差略小。**影响**：该函数当前零调用（main.c:579 直接把未换算的 temperature_raw 交给 dashboard.c:200 显示），缺陷暂时只存在于 API 层面；一旦被 UI

或存档引用就会写入带偏差的温度。**建议**：改成 ×10 单位以与 DHT11 的 `temperature_x10` 统一，并做四舍五入：`return (int16_t)((temp_raw * 10L + 170L) / 340 + 365);`（`temp_raw=0` → 365 即 36.5°C）。

2. **驱动层量纲缺位**：`MPU6050_ReadAccel`（`mpu6050.c:78-92`）与 `MPU6050_TempToC`（`mpu6050.c:96-99`）零调用，`MPU6050_ACCEL_LSB_PER_G`（`mpu6050.h:31`）零引用，真正需要的换算常数 `5730L` 写在业务层（`main.c:586-587`）。**影响**：器件语义分散在使用点，换量程要同时改驱动与业务，未用的常量还容易误导维护者以为驱动提供物理量接口。**建议**：二选一——删除未用 API 收窄维护面，或把 LSB→物理量与倾角换算搬进 `mpu6050.c`。
3. **POST 只给 DHT11 一次机会**：等待循环条件是"已尝试过且不再忙"（`main.c:410`、`main.c:419`），第一帧一旦失败（`error=1..5`），POST 立刻判 `ERR` 并显示 `E<n>`，尽管 6.5s 预算还够再等一帧（下一次发起在 2.5s 后，`dht11.c:82`）。**影响**：上电瞬间的偶发失败被误报成器件故障。**建议**：条件改为 `dht->valid && !dht->error`。
4. **DHT11 失败后无快速重试**：所有失败路径都只等固定的 2.5s 周期（`dht11.c:82`），`error=4`（帧内时序失败，最可能瞬时恢复）也不例外。**影响**：CLIMATE 页停在 `OLD` 最长 2.5s。**建议**：失败时置 `next_read=now+1200`（仍满足手册 ≥1s 采样间隔），代价是一次额外总线事务。
5. **`DHT11_Busy()` 的让路纪律没有强制机制**：`state==1` 的 20ms 起始脉冲依赖"主循环 20ms 内至少调用一次 `DHT11_Task`"，当前所有长阻塞点只靠自觉查标志（4.9 节表）。新增不查标志的长阻塞调用时，起始脉冲被拖长、收帧窗口被错过，表现为 `error=3/5` 的间歇性失败，而非编译期或运行期告警。**建议**：在 `DHT11_Task` 里记录实际起始脉冲宽度并给出诊断字段。
6. **两处低危项**：其一，`attempts` 是 `uint16_t`（`dht11.h:10`）、每 2.5s 自增（`dht11.c:79`），65535 × 2.5s ≈ 45.5 小时后回绕；它目前只被 POST 当作"是否已尝试过"的哨兵（`main.c:410/419`，窗口 6.5s 内），故不影响本工程，但做长期统计需改 `uint32_t`。其二，`delay_micros16` 每 65.536ms 回绕一次（TIM2 1MHz + `ARR=0xFFFF`，`delay.c:12-13/19`），ISR 的 `(uint16_t)(stamp-edge_time)`（`dht11.c:60`）靠无符号差天然处理，前提是相邻边沿间隔远小于 65.536ms（帧内实际 76~210µs，安全）；只有真实间隔落在 65.536~65.686ms 这 150µs 窄带才会误判，实际不可能出现——风险已闭合，欠缺的只是把隐含前提写进注释。

第 5 章　SPI NOR Flash 驱动：W25QXX

本章要点

- 驱动极小：`w25qxx.c`（82 行）+ `w25qxx.h`（12 行），对外 6 个函数（`w25qxx.h:5-11`）；文件头一行写全硬件前提与工程立场：`Standard 3.3V W25Q20..128, SPI1 PA5/6/7, CS PA4. No implicit unprotect/format.`（`w25qxx.h:4`）。
- CS 是软件控制的推挽输出 PA4（`w25qxx.c:7`、`w25qxx.c:34`）；SPI1 配成 `SPI_Mode_Master` + `SPI_NSS_Soft` +预分频 16（`w25qxx.c:38-39`），PCLK2=72MHz 有工程自检断言背书（`main.c:240`），故 SCLK=72÷16=**4.5MHz**（222ns/位、1.778μs/字节）。
- 只认 Winbond：厂商码 `0xEF`、型别码白名单 `{0x40,0x70}`、容量码 `0x12..0x18`（`w25qxx.c:44`）；`capacity=1UL<<size`（`w25qxx.c:45`）给出**字节数**，即 256KiB(2Mbit) … 16MiB(128Mbit)。
- 唯一在线守护是 `transfer()` 的 1000μs 轮询超时（`w25qxx.c:13-17`）；超时置 `bus_error=1` 后所有传输在首行短路（`w25qxx.c:12`）。驱动内**没有任何重试循环**，重试全在上层。
- `bus_error` 全文只有**一个清零点**：`w25qxx.c:31`（在 `W25Q_Init()` 内）。上层重试链是 `main.c:624-632`，退避 5→10→20→40→60s 封顶，且必须等 `!DHT11_Busy()`。
- 容量只有一个出口 `W25Q_Capacity()`（`w25qxx.c:49`）；三块持久化区域全部**从容量顶端向下**分配：会话事件日志 8KiB 紧贴在传感器日志（末 64KiB）正前方（`eventlog.c:92`、`logger.c:66`），低地址侧约 7.87MiB 完全不用。
- 一切擦除与格式化都由上层显式发起，驱动绝不隐式整片擦除（`w25qxx.h:4`、`logger.c:6`）；遇到未知的原有字节就锁死该区（`logger.c:98`、`eventlog.c:156`），宁可停用也不破坏。

5.1 引脚、总线与时钟

5.1.1 片选与静态状态

`select_chip()` 全文只有一行（`w25qxx.c:6-8`）：

```
static void select_chip(uint8_t select) {
    if(select) GPIO_ResetBits(GPIOA,GPIO_Pin_4); else GPIO_SetBits(GPIOA,GPIO_Pin_4);
}
```

`select(1)` 拉低 PA4 选中，`select(0)` 释放。命名有个小陷阱：参数叫 `select` 而 **0 表示"不选中"**。调用点全部成对（`w25qxx.c:21,41,42,59-61,66,74-76,81`），漏一次 `select_chip(0)` 会让 SPI 上其它从机被一起寻址。

`jedec`、`capacity`、`bus_error` 是文件级静态量，无任何保护（`w25qxx.c:4-5`）。这决定三点：驱动是**单实例**的；状态只在主循环上下文读写（全工程无中断内调用）；以及 5.4 节的锁存语义。

5.1.2 初始化顺序与 GPIO 模式

```
    capacity=jedec=0; bus_error=0;
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA|RCC_APB2Periph_SPI1,ENABLE);
    GPIO_SetBits(GPIOA,GPIO_Pin_4);
    gpio.GPIO_Pin=GPIO_Pin_4; gpio.GPIO_Mode=GPIO_Mode_Out_PP; gpio.GPIO_Speed=GPIO_Speed_50MHz;
    GPIO_Init(GPIOA,&gpio);
    gpio.GPIO_Pin=GPIO_Pin_5|GPIO_Pin_7; gpio.GPIO_Mode=GPIO_Mode_AF_PP; GPIO_Init(GPIOA,&gpio);
    gpio.GPIO_Pin=GPIO_Pin_6; gpio.GPIO_Mode=GPIO_Mode_IN_FLOATING; GPIO_Init(GPIOA,&gpio);
```

—— `w25qxx.c:31-37`

引脚	角色	模式	证据
PA4	CS	`GPIO_Mode_Out_PP` 推挽输出，50MHz	`w25qxx.c:34`
PA5	SCK	`GPIO_Mode_AF_PP` 复用推挽	`w25qxx.c:36`
PA6	MISO	`GPIO_Mode_IN_FLOATING` 浮空输入	`w25qxx.c:37`
PA7	MOSI	`GPIO_Mode_AF_PP` 复用推挽	`w25qxx.c:36`

三处用心：① 第 33 行**先把 CS 置高、第 34 行才配成输出**，避免 PA4 在配置瞬间输出复位默认电平 0 而发出毛刺片选；② MISO 用浮空而非上拉，因为片选无效时从机处于高阻；③ `SPI_StructInit(&spi)`（`w25qxx.c:38`）不可省—— `SPI_CPOL` / `SPI_CPHA` / `SPI_DataSize` / `SPI_FirstBit` 全由它填默认值，手工填充结构体会留下垃圾模式位导致读回全 `0xFF`；默认即 **Mode 0、8 位、MSB first**，正是 W25Q 要求的模式。

5.1.3 SCLK = 4.5MHz 的算式

```
    SPI_StructInit(&spi); spi.SPI_Mode=SPI_Mode_Master;
    spi.SPI_NSS=SPI_NSS_Soft; spi.SPI_BaudRatePrescaler=SPI_BaudRatePrescaler_16;
    SPI_Init(SPI1,&spi); SPI_Cmd(SPI1,ENABLE);
```

—— `w25qxx.c:38-40`

- SPI1 挂在 **APB2**，时钟源是 `PCLK2`；`post_clock_check()` 断言 `clocks.PCLK2_Frequency==72000000UL`（`main.c:240`），故 **PCLK2=72MHz** 是被运行期校验过的量，不是假设；
- `SPI_BaudRatePrescaler_16` → **SCLK = 72MHz ÷ 16 = 4.5MHz**；
- 位周期 = 1 ÷ 4.5MHz = **222ns**；字节周期 = 8 × 222ns = **1.778μs**；
- `SPI_NSS_Soft` 是必须的：CS 由 PA4 手工翻转，不能让硬件 NSS 同时占用 PA4。

需要明确：**驱动本身不做任何时钟计算。** `RCC_GetClocksFreq()` 在 `w25qxx.c` 中一次都没出现（全工程只有 `delay.c:7`、`controls.c:53`、`main.c:237`、`main.c:260` 四处），预分频 16 是硬编码常量。这留下一个隐含耦合：系统时钟若从 72MHz 改掉，SPI 速率随之变化而**无任何断言会报警**（见问题 6）。

5.2 上电识别：唤醒 + JEDEC ID + 三重校验

5.2.1 0xAB 唤醒与 0x9F 读 ID

```
    select_chip(1); transfer(0xAB); select_chip(0); delay_us(10);
    select_chip(1); transfer(0x9F); vendor=transfer(0xFF); type=transfer(0xFF); size=transfer(0xFF); select_chip(0);
    jedec=((uint32_t)vendor<<16)|((uint32_t)type<<8)|size;
```

——`w25qxx.c:41-43`

`0xAB` 是 *Release Power-Down*，上电无条件叫醒，随后 `delay_us(10)` 给醒转时间——这是**驱动里唯一的显式上电等待。** `0x9F` 读 JEDEC ID，三个字节用 `transfer(0xFF)` 驱动：`0xFF` 是哑元发送，主机必须给 CLK 才能收到数据。拼好的 24 位值直接上屏：`sprintf(text,"ID %06lX V%s",(unsigned long)W25Q_ID(),...)`（`dashboard.c:185`）——用户看到的 `ID EF4017` 就是这里的原始值。

5.2.2 三重校验

```
    if(bus_error || vendor!=0xEF || (type!=0x40 && type!=0x70) || size<0x12 || size>0x18) return 1;
    capacity=1UL<<size;
```

——`w25qxx.c:44-45`

#	条件	拒绝的对象
1	`bus_error`	悬空/冲突总线、器件未焊
2	`vendor != 0xEF`	非 Winbond 器件，或空总线全 `0xFF`
3	`type != 0x40 && type != 0x70`	非 W25Q 系列
4	`size < 0x12 || size > 0x18`	容量超出驱动假设

`bus_error` 排在最前是对的：熔断后三个字节必然是 `0xFF`，此时报"厂商码不对"是**误导性诊断。**

容量码的物理含义是 2 的幂次（`capacity = 1UL<<size` 给的是字节数），区间 `0x12..0x18` 恰好对应 `w25qxx.h:4` 注释里的 `W25Q20..128`：

容量码	`1UL<<size` 字节	KiB	位	型号
`0x12`	262144	256	2 Mbit	W25Q20
`0x13`	524288	512	4 Mbit	W25Q40
`0x14`	1048576	1024	8 Mbit	W25Q80
`0x15`	2097152	2048	16 Mbit	W25Q16
`0x16`	4194304	4096	32 Mbit	W25Q32
`0x17`	8388608	8192	64 Mbit	W25Q64
`0x18`	16777216	16384	128 Mbit	W25Q128

即白名单覆盖 **2 Mbit … 128 Mbit**（256KiB … 16MiB）共 7 档；屏幕上显示的 `%luKB` 就是 `cap/1024`（`dashboard.c:183`）。

型别码 `0x40` / `0x70`：源码只做白名单判定，**没有记录型别码到具体料号的映射表**（`w25qxx.c:44` 是唯一的判断点）。两者都是 Winbond W25Q 系列的标识（`0x40` 用于早期 FV/JV 料号，`0x70` 见于后续 JV/DV 料号），因此这个白名单比只认 `0x40` 更能兼容不同批次采购的模块。这是**有意的宽松**而非疏漏；代价是驱动无法区分同容量不同型别的时序差异，而这对本工程无害——所有等待都交给 `W25Q_Busy()` 轮询。

5.3 命令集与地址宽度

5.3.1 命令总表

命令码	名称	地址字节	源码位置	用途
`0xAB`	Release Power-Down	0	`w25qxx.c:41`	上电唤醒（后接 10μs）
`0x9F`	Read JEDEC ID	0（读 3 字节）	`w25qxx.c:42`	识别厂商/型别/容量
`0x05`	Read Status Register-1	0（读 1 字节）	`w25qxx.c:21`	读 WIP/WEL 位
`0x06`	Write Enable	0	`w25qxx.c:66`	置 WEL，擦/写前置动作
`0x03`	Read Data	3（24 位）	`w25qxx.c:59`	任意长度顺序读
`0x02`	Page Program	3（24 位）	`w25qxx.c:74`	页内编程，≤256 字节
`0x20`	Sector Erase (4KiB)	3（24 位）	`w25qxx.c:81`	擦除 4096 字节扇区

地址统一由 `address24()` 按 MSB first 发出（`w25qxx.c:23-25`）：

```
static void address24(uint32_t address) {
    transfer((uint8_t)(address>>16)); transfer((uint8_t)(address>>8)); transfer((uint8_t)address);
}
```

——`w25qxx.c:23-25`

这里有一个硬边界值得点明：**容量白名单的上界 `0x18`（16MiB）与 24 位寻址空间精确重合。**若将来放开到 W25Q256（32MiB），`address24()` 会静默截断高 8 位，必须同时切到 4 字节地址模式（`0xB7`）。这是本驱动最容易被忽视的约束。

5.3.2 读路径

```
uint8_t W25Q_Read(uint32_t address,uint8_t *data,uint16_t size)
{
    uint16_t i;
    if(!capacity || !size || address>=capacity || size>capacity-address || W25Q_Busy()!=0) return 1;
    select_chip(1); transfer(0x03); address24(address);
    for(i=0;i<size;i++) data[i]=transfer(0xFF);
    select_chip(0); return bus_error;
}
```

——`w25qxx.c:55-62`

守卫条件按 `||` 短路递进：`!capacity`（未初始化）→ `!size`（零长度）→ `address>=capacity`（起点越界）→ `size>capacity-address`（**终点越界，写成减法而非 `address+size>capacity`**，在边界上无歧义）→ `W25Q_Busy()!=0`（唯一会**主动发起 SPI 事务**的检查项，故置于最后，避免参数明显非法时白跑一趟总线）。

返回语义是二元的：`0` 成功、`1` 任何失败（参数错、器件忙、通信错共用同一个值），调用方无法区分——见问题 3。

5.3.3 `W25Q_Busy()` 的三态语义

```
uint8_t W25Q_Busy(void) {
    uint8_t v;
    if(!capacity || bus_error) return 2;
    v=status(); return bus_error || v==0xFF ? 2 : (v&1);
}
```

——`w25qxx.c:50-54`

契约写在头文件：`/* 0 ready, 1 busy, 2 communication error */`（`w25qxx.h:11`）。`!capacity || bus_error` 直接返 `2`，**不对硬件发任何命令**；`status()` 读回 `0xFF` 也判 `2`——这是对"空总线的 MISO 恒为高"这一物理事实的针对性防御（若天真地取 `v&1` 会得到"忙"）。代价是"真读出 `0xFF`"会被误判，而 W25Q 的 SR1 正常值只有 `0x00`–`0x03`，故对本工程无害——属于"用魔数当哨兵"的典型代价。否则返回 `v&1`，即状态寄存器 bit0 = **WIP**。

要注意**每次 `W25Q_Busy()` 都是一次完整的 `0x05` 事务**（`status()`，`w25qxx.c:20-22`，2 次 `transfer()` ≈ 3.56µs 加两次片选翻转），而 `W25Q_Read`（`w25qxx.c:58`）与 `write_enable()`（`w25qxx.c:65`）各调一次、`W25Q_ProgramStart` 经 `write_enable()` 再调一次。所以"编程一页"至少含 2 次状态查询 + 1 次写使能确认 + 1 次页编程。对每 60s 写一条记录的应用完全可接受。

5.3.4 写路径：页约束是"拒绝"而不是"拆分"

```
uint8_t W25Q_ProgramStart(uint32_t address,const uint8_t *data,uint16_t size)
{
    uint16_t i;
    if(!capacity || !size || size>256 || (address&255)+size>256 || address>=capacity || size>capacity-address) return 1;
    if(write_enable()) return 1;
    select_chip(1); transfer(0x02); address24(address);
    for(i=0;i<size;i++) transfer(data[i]);
    select_chip(0); return bus_error;
}
```

——`w25qxx.c:69-77`

两个页条件缺一不可：`size>256` 限单次长度；`(address&255)+size>256` 才是关键——它用 `address&255` 取出页内偏移，把"跨页"直接拒掉。为什么必须拒：NOR 的页编程在**跨页时会回卷到本页开头覆盖已写数据**（W25Q 手册明确规定），若放行就会静默破坏刚写好的内容。

选择**拒绝（返回 1）而不是拆成多次编程**，理由很实在：拆分需要多次 `0x02` 且每次都要重新 `write_enable`，会把"一次原子编程"变成"N 次非原子编程"，掉电窗口从 1 个变成 N 个。在掉电安全优先于吞吐的存储子系统里，让调用方自己保证不跨页是更稳的选择。代价是**调用方必须自己对齐**（5.6.2 节验证）。

5.3.5 擦除：扇区对齐是硬前置

```
uint8_t W25Q_EraseStart(uint32_t address)
{
    if(!capacity || address>=capacity || (address&4095) || write_enable()) return 1;
    select_chip(1); transfer(0x20); address24(address); select_chip(0); return bus_error;
}
```

——`w25qxx.c:78-82`

`(address & 4095)` 断言地址必须 **4096 字节对齐**，非对齐直接拒绝，**不做"向下取整到扇区起点"的容错**——静默取整等于静默的数据破坏。命名里带 `Start` 是因为它**只发起、不等待**：发完 `0x20` 就释放 CS，擦除时间由芯片承担；页编程同理。擦 4KiB 扇区典型 45ms、页编程典型 0.7ms，都远超任何合理的阻塞等待，必须靠 `W25Q_Busy()` 轮询。**整个驱动没有任何 `delay_ms` 级的阻塞等待**（唯一的时间等待是 `w25qxx.c:41` 的 `delay_us(10)`），这是本模块"绝不占用主循环"的硬证据。

5.4 超时熔断：`transfer()` 与 `bus_error` 锁存器

```
static uint8_t transfer(uint8_t value)
{
    uint16_t start=delay_micros16();
    if(bus_error) return 0xFF;
    while(!(SPI1->SR & SPI_I2S_FLAG_TXE))
        if((uint16_t)(delay_micros16()-start)>1000) {bus_error=1;return 0xFF;}
    SPI1->DR=value;
    while(!(SPI1->SR & SPI_I2S_FLAG_RXNE))
        if((uint16_t)(delay_micros16()-start)>1000) {bus_error=1;return 0xFF;}
    return (uint8_t)SPI1->DR;
}
```

——`w25qxx.c:9-19`

为什么必须存在：硬 SPI 外设没有"从机应答"概念。器件虚焊、CS 走线断开或供电缺失时，`while(!(SPI1->SR & TXE))` 会永远自旋，把十页仪表盘、DHT 采样、按键响应全部拖死。1000µs 判断是唯一的逃生通道。

1000µs 的算术。计时源 `delay_micros16()` 是 `(uint16_t)TIM2->CNT`（`delay.c:19`），TIM2 被配成 1MHz 自由运行（`delay.c:10-13`：`PSC = timer_clock/1000000-1`，`timer_clock` 在 PCLK1≠HCLK 时乘 2 得 72MHz，故 `PSC=71`），`ARR=0xFFFF`，**每 65.536ms 回绕一次**（`delay.h:10` 注释写明）。阈值只有 1000，故 `(uint16_t)(now-start)` 的模 65536 减法不会产生歧义。代入 SPI 速率：

- 一字节 = 8 bit ÷ 4.5MHz = **1.778µs**；
- 1000µs ÷ 1.778µs ≈ **562 个字节时间。**

也就是说，判定"卡死"的门槛是**正常传输 562 字节所需的时间。**作为对照，软件 I2C 用的是**同一个常量**：`i2c.c:19` 也是 `>=1000U`，而 I2C 名义速率是 250kHz（`i2c.h:31`），1000µs 在那里只有约 **250 个位时间。**两条总线速率相差 **约 16–18 倍**却复用同一个阈值，是明显的**常量搬运**痕迹——超时值按软件 I2C 的位时间定，搬到硬 SPI 后没有重新缩放。

> **I2C 速率的两种口径**（本书统一并列，不强行归一）：`i2c.h:31` 注释的 **250kHz 是名义值**，等于按位周期模型 N=8 解析出的结果（T ≈ 0.52µs + 0.435µs × 8 = 4.00µs/位）；而按实测"每字节 28.9µs"反推得到 **≈277–312kHz 的实测值**——该口径含每字节 1 个 ACK 位与函数调用开销，故高于名义值。两者不矛盾，只是量的是不同的东西：名义值描述位时序，实测值描述端到端吞吐。取名义值 250kHz 得比值 **18×**，取实测口径得 **16.2×**。本节的"常量搬运"结论对两者都成立，因为无论 16 还是 18 倍，1000µs 这个阈值都是从 I2C 侧整体搬过来的量级，与 4.5MHz 的 SPI 字节时间（1.778µs）都不成比例。

不过要公平地说，这个"松"在后果上被两件事抵消：① **超时只可能发生一次**——首次超时即置位（`w25qxx.c:14` 或 `17`），此后每次都在第 12 行短路返回，所以一次总线故障的**总阻塞上限就是 1000µs**，而不是"每字节 1000µs"地累积；② 两个等待循环**共享同一个** `start`（`w25qxx.c:11` 只取一次时间戳），TXE 与 RXNE 合计只有 1000µs 预算，不会各给 1000µs。结论：这是**上限可控（≤1ms）但灵敏度不足**的看门狗——对"器件完全不在"工作得很好，对"器件在但响应变慢"几乎不会触发。合理收紧值应在 10–50µs（约 6–28 个字节时间）量级，并保留一次性锁存语义。

`bus_error` 是锁存器，且只有一个清零点：

行	语句	作用
`w25qxx.c:5`	`static uint8_t bus_error;`	定义，初值 0
`w25qxx.c:12`	`if(bus_error) return 0xFF;`	熔断后所有传输直接失败
`w25qxx.c:14`	`bus_error=1`（TXE 超时）	置位
`w25qxx.c:17`	`bus_error=1`（RXNE 超时）	置位
`w25qxx.c:31`	`capacity=jedec=0; bus_error=0;`	**唯一的清零**，在 `W25Q_Init()` 内
`w25qxx.c:44`	`if(bus_error || ...) return 1;`	ID 校验的第一道条件
`w25qxx.c:52`	`if(!capacity || bus_error) return 2;`	`W25Q_Busy()` 直接报通信错
`w25qxx.c:53`	`bus_error || v==0xFF ? 2 : (v&1)`	二次确认：超时也可发生在 `status()` 内部的两次 `transfer()` 里
`w25qxx.c:61/67/76/81`	`return bus_error;`	各 API 的失败回传

两条结论：① **驱动内部没有任何清除路径**，`bus_error` 一旦为 1，只能靠外部再次调用 `W25Q_Init()`（`w25qxx.c:31`）复位。这是**故障闭锁**设计——宁可彻底停用，也不在总线上反复试探，对"共享总线 + 无 DMA + 无中断"的架构是正确的取舍。② **上层确实有自动恢复路径，但仅限传感器日志**：`W25Q_Init()` 在运行期只有一条调用链可达，`main.c:629` 的 `Logger_Init(now)` → `logger.c:65`。会话事件日志**没有对应路径**，见问题 1。

5.5 重试全部在上层：退避链

驱动不重试，"谁重试、隔多久、退避到多少"就完全决定了系统的恢复能力。全部逻辑在 `main.c:624-633`：

```
        if(Logger_State()==LOG_ERROR&&!flash_retry){
            flash_retry=1;flash_backoff=5000;flash_retry_next=now+flash_backoff;
        }
        if(flash_retry&&(Logger_State()==LOG_ERROR||Logger_State()==LOG_OFFLINE)&&
            due(now,flash_retry_next)&&!DHT11_Busy()){
            Logger_Init(now);flash_retry_next=delay_millis()+flash_backoff;
            if(flash_backoff<60000UL){flash_backoff*=2; if(flash_backoff>60000UL)flash_backoff=60000UL;}
            if(Logger_State()==LOG_SCANNING)flash_retry=0;
        }
        if(Logger_State()==LOG_READY||Logger_State()==LOG_FOREIGN)flash_retry=0;
```

——`main.c:624-633`

机制	实现	证据
触发条件	`Logger_State()==LOG_ERROR` 且尚未进入重试态	`main.c:624`
首次退避	`flash_backoff=5000`，下次尝试 `now+5000`	`main.c:625`
重试窗口	状态仍为 `LOG_ERROR` 或 `LOG_OFFLINE`	`main.c:627`
跨模块互锁	必须 `!DHT11_Busy()` 才能重试	`main.c:628`
退避倍增	每次重试后 `flash_backoff *= 2`，`>60000` 时钳到 **60000**	`main.c:630`
成功退出	进入 `LOG_SCANNING` 即清 `flash_retry`	`main.c:631`
旁路退出	状态为 `LOG_READY` / `LOG_FOREIGN` 时清 `flash_retry`	`main.c:633`

注意赋值顺序：第 629 行用**当前的** `flash_backoff` 设定下一个期限，第 630 行才把它翻倍。这个顺序决定了退避曲线与直觉不符——**"5s 起步、每次翻倍"并不意味着第一次间隔就是 10s。**

5.5.1 六次尝试的绝对时刻表（逐次代入）

以首次进入 `LOG_ERROR` 的时刻为 `t = 0`，把 `main.c:624-633` 逐次代入（`flash_backoff` 是毫秒，绝对时刻已换算为秒）：

迭代	进入时的 `flash_backoff`	`main.c:629` 设定期限	`main.c:630` 更新 `flash_backoff`	本次尝试绝对时刻	与上次的间隔
触发（首次报错）	—	`flash_retry_next = 0 + 5000`	`5000`（`main.c:625`）	尚未尝试	—
第 1 次	5000	`5000 + 5000 = 10000`	5000 → **10000**	**t = 5s**	5s（自触发）
第 2 次	10000	`10000 + 10000 = 20000`	10000 → **20000**	**t = 10s**	5s
第 3 次	20000	`20000 + 20000 = 40000`	20000 → **40000**	**t = 20s**	10s
第 4 次	40000	`40000 + 40000 = 80000`	40000 → 80000 → 钳位 **60000**	**t = 40s**	20s
第 5 次	60000	`80000 + 60000 = 140000`	60000 已满，**不变**	**t = 80s**	40s
第 6 次	60000	`140000 + 60000 = 200000`	60000 **不变**	**t = 140s**	60s
第 7 次起	60000	每次 `+60000`	60000 **不变**	**t = 200s、260s、…**	60s

可观测推论：第 5 次尝试恰落在**第 80 秒**，即**前 5 次尝试全部发生在 80 秒之内**；从第 6 次（140s）起转为每 60s 一次的稳态。注意第 2 次的间隔仍是 5s（不是 10s）——因为第 1 次执行时第 629 行读到的 `flash_backoff` 还只是 5000，翻倍发生在设定期限之后。这个"慢一档"的错位让前两分钟内的重试密度比直觉高一倍，对"瞬时干扰"这类可自愈故障是有利的。

这个曲线对"器件虚焊"（永远失败）会稳定在 60s 一次的低负载——按 60s 周期计，一次重试的实际代价约为一次 2048 槽整区重扫（≈131ms 总线时间，见下文第 2 点），占空比约 0.22%；对"瞬时干扰"则在 5s 内就能恢复。

三点值得注意：

1. **重试被 DHT 的时序窗口门控**（`main.c:628` 的 `!DHT11_Busy()`），与本工程其它 `!DHT11_Busy()` 互锁一致（`main.c:600` 的 MPU 重试、`main.c:661` 的 `OLED_Service`）。另有一类不同的门控用"数据有效性"而非忙碌态：60s 检查点的写入门槛要求 `DHT11_Get()->valid && !DHT11_Get()->error`（`main.c:616-617`），即环境数据无效时干脆不写检查点，宁可少存也不存坏值。一个"存储子系统"的重试节奏受"传感器子系统"的时序支配，这是单主循环架构下不得不做的耦合。
2. **重试的代价是整区重扫，不是定点恢复。** `main.c:629` 调的是 `Logger_Init(now)`，而 `logger.c:63` 会把 `state` 重置为 `LOG_OFFLINE` 并清空 `write_phase` / `recent_count` / `scan_slot` / `count` / `latest_seq` 与 `per_sector[]`，再进入 `LOG_SCANNING`（`logger.c:66`）。也就是每次重试都要**重新扫描全部 2048 个槽**（`LOGGER_SLOTS`，`logger.h:4`），即 2048 次 32 字节读 ≈ 2048 × 36 次 `transfer()` ≈ **131ms 纯总线时间**（4.5MHz 下）。好消息是 `Logger_Task` 每次调用只处理一个槽（`logger.c:96-105`），扫描被摊到 2048 次主循环迭代里，**不阻塞。**
3. **`flash_retry` 有两条退出路径**（`main.c:631` 和 `main.c:633`），后者覆盖 `LOG_FOREIGN`——即"Flash 里有未知数据"这一**正常且持久**的状态也必须停止重试，否则会每 60s 白白重扫一次。这一条容易被漏掉，写得很细。

`EventLog_Init` 则由 `Logger_State()==LOG_READY` 触发，且用 `event_init_attempted` 保证只尝试一次：

```
        if(!event_init_attempted&&Logger_State()==LOG_READY){
            EventLog_Init(now,(uint8_t)(reset_flags>>24));event_init_attempted=1;
        }
        if(event_init_attempted)EventLog_Task(now,uptime_seconds);
```

——`main.c:612-615`

`EventLog_Init` 内部还要再验一次容量（`eventlog.c:91-92`）：

```
    if(Logger_State()!=LOG_READY||W25Q_Capacity()<SENSOR_BYTES+EVENT_BYTES)return;
    base=W25Q_Capacity()-SENSOR_BYTES-EVENT_BYTES;
```

——`eventlog.c:91-92`

即要求 `capacity ≥ 65536 + 8192 = 73728`（`SENSOR_BYTES`=65536、`EVENT_BYTES`=8192，`eventlog.c:7-8`）。因为 `capacity` 是 2 的幂且最小档为 262144，这个条件在 7 个合法档位上**恒成立**——它防的是"未初始化"（`capacity==0`）而不是"容量不足"。`logger.c:65` 的 `W25Q_Capacity()<REGION_BYTES` 是同一手法。

5.6 容量语义与三块区域的地址布局

`W25Q_Capacity()` 是容量语义的唯一出口（`w25qxx.c:49`），全工程 6 个调用点全部是"从顶端向下"的分配：

调用点	用途	算出的值（以 W25Q64 为例）
`logger.c:65`	门控：容量不足则不进 `LOG_SCANNING`	`capacity < 65536` → 停用
`logger.c:66`	**传感器日志基址** `capacity - REGION_BYTES`	8388608 − 65536 = **8323072** (0x7F0000)
`eventlog.c:91`	门控：`capacity < 65536 + 8192`	恒真
`eventlog.c:92`	**事件日志基址** `capacity - SENSOR_BYTES - EVENT_BYTES`	8388608 − 73728 = **8314880** (0x7EE000)
`main.c:430`	POST 步 6 的判定 `capacity >= 65536`	打印 `DONE` / `MISS`
`dashboard.c:175,183`	SYSTEM 页显示 `%luKB` = `cap/1024`	`8192KB`

`W25Q_ID()` 则用于两处显示与一处自检：`dashboard.c:50`（`LOG_FOREIGN` 时区分 `UNSUP` / `ABSENT`）、`dashboard.c:185`（`ID %06lX`）、`main.c:429`（POST 记录 `flash_id`）。

5.6.1 三块区域示意图

```
W25Q64（容量 8MiB = 8388608B），从容量顶端向下分配：

0x000000  ┌──────────────────────────────────────┐
          │     未使用（≈7.87 MiB）              │
0x7EE000  ├──────────────────────────────────────┤ ← capacity - 65536 - 8192
          │  会话事件日志  EVENT_BYTES = 8192B   │
          │  256 槽 × 32B = 2 个 4KiB 扇区       │
0x7F0000  ├──────────────────────────────────────┤ ← capacity - 65536
          │  传感器记录日志  REGION_BYTES= 65536B │
          │  2048 槽 × 32B = 16 个 4KiB 扇区     │
0x7FFFFF  └──────────────────────────────────────┘ ← capacity - 1
```

区域	大小	起始地址表达式	结束地址表达式	证据
未使用	`capacity - 73728`	`0`	`capacity - 73729`	无代码引用
会话事件日志	`EVENT_BYTES` = 8192B	`W25Q_Capacity() - SENSOR_BYTES - EVENT_BYTES`（`eventlog.c:92`）	`W25Q_Capacity() - SENSOR_BYTES - 1`	`eventlog.c:7`、`eventlog.c:9`
传感器记录日志	`REGION_BYTES` = 65536B	`W25Q_Capacity() - REGION_BYTES`（`logger.c:66`）	`W25Q_Capacity() - 1`	`logger.c:8`、`logger.h:4`

两块区域**首尾相接、无一字节重叠或空隙**：事件日志的末地址 `capacity-65537` 的下一个字节就是传感器日志的起点 `capacity-65536`。槽地址由 `address(slot) = base + slot*32` 计算（`logger.c:27`），因此两块都是 32 字节对齐的连续槽数组。

5.6.2 为什么这个布局天然满足 256 字节页约束

5.3.4 节说明 `W25Q_ProgramStart` 会拒绝跨页写入、把责任推给调用方。现在验证调用方是否真的做到了。

基址对齐性：`capacity = 2^n` 且 `n ∈ [18,24]`，故 `capacity` 必为 8192 的整数倍（2^18 = 262144 = 32 × 8192）；`73728 = 9 × 8192`；因此 `base = capacity - 73728 ≡ 0 (mod 8192)`，即事件日志基址是 **8192 字节对齐**的，自然也是 32 字节对齐。传感器日志基址 `capacity - 65536` 同理（65536 = 8 × 8192），且对齐到 **65536** 字节。

尺寸关系：每槽 **32 字节**，Flash 页是 **256 字节**。因为 `32 | 256`，任何 `base + slot*32` 起点的 32 字节窗口都**完整落在同一个 256 字节页内**。所以：

- `W25Q_ProgramStart(address, raw, 31)`（`logger.c:123`）写槽内偏移 0–30，不跨页；
- `W25Q_ProgramStart(address(write_slot)+31, b, 1)`（`logger.c:133`）写偏移 31，与前一次同页；
- `eventlog.c:171`、`eventlog.c:178` 是完全相同的 31 + 1 两步写。

即**调用方确实每页独立，页约束天然满足**，而且这个性质来自"槽尺寸整除页尺寸"这一算术事实，不依赖任何运行期检查——**用尺寸关系代替边界检查**，是很漂亮的一手。两处擦除也对齐到 128 槽边界（128 × 32 = 4096 = 1 扇区）：`logger.c:113` 的 `if(!(write_slot%128))`、`eventlog.c:122` 的 `end=(latest_slot/128U+1U)*128U`。

不过要提醒：**这个性质是隐式的、无断言的。**若把记录尺寸改成 40 或 33 字节，`W25Q_ProgramStart` 会在运行期返回 1（拒绝），表现为存储子系统静默进入 `LOG_ERROR`，而不是编译报错。这是问题 4 的来源。

5.7 工程立场：不做隐式破坏

本章最有分量的结论不是某个算法，而是一组**拒绝**。

驱动层的立场写在文件头（`w25qxx.h:4`）：`No implicit unprotect/format.` 落到代码上是：没有 `0x01`（Write Status Register）调用，即**从不改写块保护位**；没有任何"空白检测→整片擦除"的自动动作；没有任何 Chip Erase（`0xC7` / `0x60`）命令，擦除的最小单位就是 **4KiB 扇区**（`w25qxx.c:81`）。

存储层的立场写在 `logger.c:5-6`：

```
/* Last 64KiB only. 32-byte CRC records, commit byte programmed LAST.
   Unknown pre-existing bytes lock this region; no implicit chip erase. */
```

—— `logger.c:5-6`

实现证据是 `logger.c:97-98`：

```
        if(!blank(b)) {
            if(get32(b)!=MAGIC) {state=LOG_FOREIGN;return;}
```

—— `logger.c:97-98`

即扫描到一个非空白、但开头 4 字节不是 `MAGIC`（`0x31474F4C`，ASCII `"LOG1"`，`logger.c:7`）的槽时，**整个区域被判定为 `LOG_FOREIGN` 并停用**，不擦、不写、不覆盖。事件日志用同构的立场：`eventlog.c:153` 累积 `foreign=1`，扫描结束在 `eventlog.c:156` 锁死该区（`EVENT_MAGIC` = `0x31545645`，ASCII `"EVT1"`，`eventlog.c:10`）。

这个立场的价值在于**把"未知数据"当成一等公民。Flash 里究竟装的是什么内容，无法从代码确定**——驱动只读 4 个字节的 `MAGIC` 做判定（`logger.c:98`），既不解析也不上报内容。但正因为无法确定，代码才选择了"停用而非清除"：自动格式化会**静默销毁**一块来源不明的数据，而 `LOG_FOREIGN` / `EVENT_LOCKED` 只是让功能停用，把判断权交还给人——屏幕显示 `LOCKED`（`dashboard.c:50`）即可让人知道"这块 Flash 上有不属于本固件的既有数据"，而不是在毫不知情的情况下把它擦掉。注释用 `no implicit chip erase` 而不是 `no erase`，措辞也很准确：擦**扇区**（在确认该槽属于自己之后，`logger.c:114`）允许，擦**整片**永远不允许。

与之配套的是"记录原子性"：每槽 32 字节，前 31 字节先写（`logger.c:123`），回读 `memcmp` 校验（`logger.c:131`），最后才写第 31 字节的提交标志 `0xA5`（`logger.c:132-133`）。有效性判定要求提交字节与 CRC32 同时对（`logger.c:30`：`get32(b)!=MAGIC || b[31]!=0xA5 || get32(b+24)!=crc32(b,24) || get32(b+20)>3`），也就是说**"半写"的记录永远不会被认成有效**。这是驱动只提供"页内 ≤256 字节编程"这一原始能力、而把原子性语义**构造在上层**的分工结果（两条日志的掉电安全细节属于存储服务章节的主题，此处只到接口契约为限）。

本章发现的问题与建议

问题 1（高）：事件日志没有重试路径，一次通信错误即永久停用

- **现象**：`eventlog.c:141` 在 `W25Q_Busy()==2` 时置 `state=EVENT_ERROR`；而 `eventlog.c:138` 的 `if(state==EVENT_OFFLINE||state==EVENT_LOCKED||state==EVENT_ERROR)return;` 使 `EventLog_Task` 此后**永远空转**。唯一能复位它的 `EventLog_Init` 由 `main.c:612-613` 调用，被 `event_init_attempted` 门控，而该标志在 `main.c:492` 定义、`main.c:522` 赋值、`main.c:613` 置 1，**全文没有任何复位点**。
- **对比**：传感器日志有完整的退避重试链（`main.c:624-632`），可达 `Logger_Init` → `W25Q_Init()` → `w25qxx.c:31` 清 `bus_error`。
- **影响**：一次 SPI 干扰（哪怕是 1000µs 级的偶发超时）就会让复位溯源、会话计数、`ALIVE` 心跳（`HEARTBEAT_MS=60000`，`eventlog.c:12`）在本上电周期内**全部失效**，必须重启才能恢复；而 `bus_error` 是锁存的，重启前也不可能自愈。
- **建议**：在 `main.c` 增设与 Flash 退避同构的重试路径（如 `event_retry` / `event_backoff`），条件是 `EventLog_State()==EVENT_ERROR && !DHT11_Busy()` 时重试 `EventLog_Init(now, reset_code)`；更彻底的做法是把 `event_init_attempted` 改成一个可复位的重试计数器。

问题 2（中）：1000µs 超时是按软件 I2C 定的，未随 4.5MHz SPI 重新缩放

- **现象**：`w25qxx.c:14` 与 `17` 用 `>1000`（µs），与软件 I2C 的 `i2c.c:19` 同值；而 I2C 约 250kHz（`i2c.h:31`），SPI 为 4.5MHz（`w25qxx.c:39` + `main.c:240`），速率差 18 倍。
- **影响**：门限相当于 562 个 SPI 字节时间。对"器件完全不在"能触发，对"器件在线但响应退化"（供电跌落、走线容性过大）几乎不可能触发。
- **建议**：把超时常量提取为 `#define W25Q_IO_TIMEOUT_US`，按 6–28 个字节时间取 **10–50µs**；同时把 I2C 的常量独立命名，避免两条总线继续共用同一个魔数。保留"超时即锁存、不重试"的语义不变。

问题 3（中）：所有失败共用返回值 `1`，调用方无法区分故障类型

- **现象**：`W25Q_Read`（`w25qxx.c:58`）、`W25Q_ProgramStart`（`w25qxx.c:72`）、`W25Q_EraseStart`（`w25qxx.c:80`）都把"参数非法""器件忙""通信错误"压缩成返回值 `1`；只有 `W25Q_Busy()` 提供了三态（`w25qxx.h:11`）。而调用点只做 `if(W25Q_Read(...)){state=LOG_ERROR;return;}`（如 `logger.c:96`、`eventlog.c:144`）。
- **影响**：`LOG_ERROR` 同时代表"我传错了参数"（编程 bug）和"硬件坏了"（现场故障），两者在屏幕上都显示为同一个 `ERR`。上层重试链会为一个永远不可能成功的参数错误反复重扫整区，每 60s 浪费约 131ms 总线时间。
- **建议**：让驱动返回可区分的错误码（`0` 成功 / `1` 参数错 / `2` 忙 / `3` 通信错）。至少要把 `W25Q_ProgramStart` 的页约束拒绝（`w25qxx.c:72` 的 `size>256 || (address&255)+size>256`）单独报出来：那是全驱动里唯一一条"改一个字节就会静默损坏数据"的约束。

问题 4（中）：256 字节页约束完全依赖隐式尺寸关系，没有静态断言

- **现象**：`W25Q_ProgramStart` 把"不跨页"的责任推给调用方（`w25qxx.c:72`），调用方靠"记录 32 字节整除页 256 字节"这一算术才成立（5.6.2 节）；全工程**没有一处编译期断言**保护这个关系，头文件 `w25qxx.h:9` 的声明旁也未注明该契约。
- **影响**：若把记录尺寸改成 33 或 40 字节，跨页会在运行期被驱动拒绝，表现为存储子系统无预警地进入 `LOG_ERROR`；若有人误把页检查"优化"掉（例如认为"上层会保证"），则跨页时会**静默覆盖本页开头**（W25Q 页编程的回卷行为），造成数据损坏而非报错。
- **建议**：① 加编译期断言 `sizeof(record)==32` 与 `256 % 32 == 0`；② 在 `w25qxx.h:9` 旁注明"调用方必须保证不跨 256 字节页"；③ 保留驱动层的拒绝式检查作为最后一道防线，不要删。

问题 5（低）：`select_chip()` 是裸片选，没有总线互斥

- **现象**：`select_chip()` 直接读写 PA4（`w25qxx.c:7`），无锁、无"总线占用"标志、无嵌套保护。当前工程里 SPI1 只有 W25Q 一个从机，且所有调用都在主循环上下文（无中断内调用），所以**今天不会出错**。
- **影响**：一旦在 PA4 之外再接第二个 SPI 从机，典型错误是"长事务中途被另一个模块插入一次 `select_chip(1)`"，导致两个从机同时被选中、MISO 打架。这不是当前缺陷，而是**扩展时的隐患**：`w25qxx` 对外的 7 个函数没有一个是"可被抢占安全"的。
- **建议**：给驱动加一个最轻量的占用计数与断言（在 `select_chip(1)` 时自增并在为 1 前断言为 0），或在头文件明确写下"本驱动假设 SPI1 独占、单上下文调用"的前提——把隐式假设变成可检查的契约。

问题 6（低）：SPI 速率没有运行期校验，与工程其它部分的自检风格不一致

- **现象**：预分频 16 是硬编码（`w25qxx.c:39`），驱动不调用 `RCC_GetClocksFreq()`，也不验证 `PCLK2`。而同一工程在 `main.c:234-241` 用 `post_clock_check()` 显式断言了 `PCLK2==72000000UL`，在 `main.c:254-268` 用 `post_tim2_check()` 断言了 TIM2 的 `PSC` 与计数频率。
- **影响**：若系统时钟被改动（例如为降功耗把 SYSCLK 降到 48MHz），SPI 速率变成 3MHz，**不会触发任何告警**，只会表现为存储操作的绝对时间变长。本工程对时基与 TIM2 都有严格自检，唯独 SPI 速率没有任何校验，风格不统一。
- **建议**：在 `W25Q_Init()` 里用 `RCC_GetClocksFreq()` 复算 `PCLK2 ÷ 16` 并与期望的 4.5MHz 比对，作为 POST 的一个子项；这同时能把 5.1.3 节提到的隐含耦合变成显式契约。

问题 7（说明，非缺陷）：擦写寿命与坏块策略——算术表明不需要管理

- **现象**：驱动与存储层都**没有**磨损均衡、没有坏块表、没有擦写计数。
- **算式的输入**（源码可查）：传感器日志每 60s 一个检查点（`main.c:618`），2048 槽填满一轮触发 16 次扇区擦除，故**单个扇区每轮只擦一次**，一轮 = 2048 × 60s = 122,880s ≈ **34.13 小时**；事件日志 `HEARTBEAT_MS=60000`（`eventlog.c:12`）即每 60s 一条 `ALIVE`，256 槽分居 2 个扇区，每扇区每 256 × 60s = 15,360s ≈ **4.267 小时**擦一次（轮换逻辑见 `eventlog.c:122-124`）。
- **以 NOR 常见的 10 万次/扇区擦写寿命为假设**（源码中没有这个常数，属外部假设，特此声明）：

区域	单扇区擦除周期	×100,000 次	折合
传感器记录日志	122,880s（34.13h）	3,413,333 小时	**≈ 389 年**
会话事件日志	15,360s（4.267h）	426,667 小时	**≈ 48.7 年**

- **结论**：以"24 小时连续运行"计，事件日志约 48.7 年、传感器日志约 389 年才耗尽一个扇区，两者都远超任何合理的设备寿命；且在 8MiB 器件上只用了 72KiB（0.86%）容量。所以"没有磨损均衡"在这里**不是缺陷，而是正确的省事**——为 48 年的余量引入擦写计数与动态映射表，只会增加代码体积与失效模式。但这个结论对"写入更频繁的将来"不成立：若把检查点间隔从 60s 缩短到 1s，事件日志寿命会线性跌到约 **10 个月**，那时就必须引入管理。
- **建议**：无需改代码；建议把"检查点间隔 × 寿命"的关系补进存储层的文件头注释（`logger.c:5-6` 已写了布局与立场，补一行寿命边界会更有指导性），以便后人调整间隔时能立刻看到代价。

第 6 章　业务服务：主循环调度器与十页仪表盘

本章要点：

- 全部业务逻辑跑在一个没有 RTOS 的 `while(1)`（`main.c:531-663`）里：所谓"并发"只是**截止时间比较 + busy 标志**，没有任务、没有调度器、没有互斥锁。
- `main()` 的启动顺序是刻意排的（`main.c:505-530`）：`Buzzer_Init()` 必须排在最前（理由见 `buzzer.h:12-14`），`delay_init()` 必须早于任何 `delay_*` 调用，11 项自检在屏幕上逐项显示。
- 时间判定有两种原语并存：`due()`（`main.c:49`，11 处调用）能安全处理"未来截止"，裸 `(uint32_t)(now-x)>=Y`（`main.c:596`、`main.c:606` 等 11 处）只适用于"过去起点"。
- 十页仪表盘只有一个绘制入口 `Dashboard_Draw()`（`dashboard.c:319-359`），主循环里唯一的调用点是 `main.c:658`；每帧 `OLED_Clear()` 后重新排版，页间切换不需要清理残留状态。
- 跨模块契约 `DashboardView`（`dashboard.h:12-25`）只覆盖姿态、环境与统计字段；HISTORY / SYSTEM / DEBUG / SESSIONS 页在渲染函数里直接调用 `Logger_*`、`W25Q_*`、`EventLog_*`、`DHT11_Get()`，甚至直读 `TIM1->CCR1/2/3` 与 `GPIOB->IDR`（`dashboard.c:249-250`）。渲染层不是纯函数，这是本章最重要的一条结构性发现。

6.1 main() 启动序列：把最坏的等待放在最前

`main()` 从 `main.c:488` 开始，运行态变量全部是它的**栈上局部变量**（`main.c:490-504`），随后进入初始化：

```
uint32_t reset_flags=RCC->CSR;
memset(&view,0,sizeof(view));Dashboard_Init();
view.reset_reason=reset_reason(reset_flags);RCC_ClearFlag();
Buzzer_Init();LED_Init();Controls_Init();delay_init();Buzzer_Beep(BOOT_BEEP_MS);
SoftI2C_Init();SoftI2C_BusRecover();delay_ms(300);
oled_ok=SoftI2C_Probe(OLED_ADDR7);mpu_ok=SoftI2C_Probe(MPU6050_ADDR7);
while(!oled_ok){uint8_t i;for(i=0;i<(mpu_ok?3:5);i++)
    {LED_On();delay_ms(80);LED_Off();delay_ms(120);}
    delay_ms(1200);SoftI2C_BusRecover();oled_ok=SoftI2C_Probe(OLED_ADDR7);
    mpu_ok=SoftI2C_Probe(MPU6050_ADDR7);
}
OLED_Init();
dht_ready=post_startup(&sensor_ready,(uint8_t)(reset_flags>>24));
```

（`main.c:505-517`）

顺序不是随手写的，每一行都有"失败后果"：

#	动作	失败/漏做的后果	行号
1	读 `RCC->CSR` 保存复位标志	只影响显示；`RCC_ClearFlag()` 早于读取会丢掉复位来源	`main.c:505`
2	`memset(&view,0,…)` + `Dashboard_Init()`	TREND 环形缓冲与气泡状态未复位，首帧画出上一轮的残留	`main.c:506`
3	`view.reset_reason=reset_reason(…)`	无；这是唯一一处把复位标志转成 3 字母代号的地方（`main.c:50-58`）	`main.c:507`
4	`Buzzer_Init();LED_Init();Controls_Init();delay_init();Buzzer_Beep(BOOT_BEEP_MS)`	蜂鸣器晚初始化，PB8 悬空期间会"嘀"一声（`buzzer.h:12-14`）；`delay_init()` 晚于任何 `delay_*` 会死循环（第 2 章）	`main.c:508`
5	`SoftI2C_Init();SoftI2C_BusRecover();delay_ms(300)`	从机若停在半字节状态，后面所有探测都会失败	`main.c:509`
6	一次性探测 0x3C 与 0x68	两个标志决定后面全部分支（含 LED 闪码次数）	`main.c:510`
7	`while(!oled_ok)`：闪灯 3 或 5 次（80ms 亮 / 120ms 灭）+ `delay_ms(1200)` + 重探	**没有重试上限**，OLED 永久失联就永久卡在这个循环里；MPU 不在线只让闪灯从 5 次变 3 次	`main.c:511-515`
8	`OLED_Init()`	没有屏幕，后面所有诊断都看不见	`main.c:516`
9	`post_startup(&sensor_ready, reset_flags>>24)`	返回 `dht_ready`；内部把时基三项结果合成 `post_timebase_ok`（`main.c:390`），时基通过才 `DHT11_Init()`（`main.c:392`）	`main.c:517`
10	`if(!post_timebase_ok)` 死循环闪灯	刻意不再跑业务：时基不可信时所有超时语义都是错的	`main.c:518-521`
11	播种 uptime 与各 deadline	用当前 `delay_millis()` 一次推导，启动耗时不会污染运行时长统计；`view.page/browsing/rank` 清零、`strobe_mode=STROBE_DOUBLE`	`main.c:522-530`

第 4 行的两个理由值得展开。蜂鸣器是 **3 针低电平触发**模块：MCU 把 PB8 拉高之前引脚悬空，上电瞬间可能自响，所以固件"第一件事就是 `Buzzer_Init()`，把这个窗口压到最短"（`buzzer.h:14`）。而 `delay_init()` 必须排在 `delay_ms(300)`（`main.c:509`）与 `Buzzer_Beep(60)`（第 4 行内）之前——后者内部就是 `delay_ms()` 忙等（`buzzer.c:33-39`）。

自检阶段还有一处刻意的时间重叠：时基三项一通过就立刻 `DHT11_Init()`（`main.c:392`），随后才继续 OLED、MPU 自检，源码注释写明这是为了让 DHT11 的 2 秒上电间隔与后面的检查并行（`main.c:391`）。自检本身的分步推进靠 `post_present()` 反复泵 `post_pump()`（`main.c:354-361`），这样屏幕上能逐项看到 `TEST → DONE/ERR`，而不是先黑屏几秒再一次性出结果。

6.2 主循环：一个 5ms 节拍的相位式调度器
主循环每轮从取时间戳开始，把到期项依次判一遍：

```
while(1){
    now=delay_millis();if(dht_ready)DHT11_Task(now);
    elapsed=now-uptime_last;uptime_last=now;uptime_fraction+=elapsed;
    if(uptime_fraction>=1000UL){uptime_seconds+=uptime_fraction/1000UL;
        uptime_fraction%=1000UL;}
    elapsed=now-last_loop_start;last_loop_start=now;
    if(elapsed>view.max_loop_ms)view.max_loop_ms=(uint16_t)(elapsed>999?999:elapsed);
    if(beep_active&&due(now,beep_until)){Buzzer_Off();beep_active=0;}
    if(led_active&&due(now,led_until)){LED_Off();led_active=0;}
    if(due(now,heartbeat_next)){LED_On();led_active=1;led_until=now+30;heartbeat_next=now+1000;}
    turn=Controls_TakeTurn();button=Controls_TakeButton();
```

（`main.c:531-541`）

相位全表（按执行顺序）：

相位	触发条件	动作	行号
取时 + DHT 推进	每轮	`now=delay_millis()`； `DHT11_Task(now)`	`main.c:532`
运行时长累积	每轮	毫秒余数累加到 `uptime_seconds`，不用除法换算	`main.c:533-535`
循环耗时统计	每轮	`max_loop_ms` 记录最坏一轮，超过 999 截断	`main.c:536-537`
蜂鸣 / LED 到期	`due()`	`Buzzer_Off()`； `LED_Off()`	`main.c:538-539`
心跳	每 1s	`LED_On()` 30ms，兼作"还活着"指示	`main.c:540`
输入采集	每轮	`Controls_TakeTurn()` / `Controls_TakeButton()`（临界区内取走旋转与按键事件）	`main.c:541`
HOME / 翻页 / 点击	有输入	复位到第 0 页并 toast `"HOME"`； `navigate_turn()`；点击分支：STROBE 切模式、SESSIONS 需有会话否则 `"NO SESSIONS"`、HISTORY 同理 `"NO RECORDS"`、其余页立即存档	`main.c:542-566`
三页驻留自动进浏览	停在同一页 ≥800ms	`history_pause` / `strobe_pause` / `event_pause` 把 `browsing` 置 1（`HISTORY_PAUSE_MS=800UL`，`main.c:25`）	`main.c:567-569`
5ms 采样	`due(now,sample_next)`	记录采样间隔峰值 → `MPU6050_ReadAll()` → 存原始值 → 滤波/校准/姿态解算	`main.c:570-595`
读超时与 MPU 重初始化	裸 elapsed ≥100ms	标记 `stale`，且满足 `due(now,retry_next)&&!DHT11_Busy()` 时软恢复总线并重跑 `MPU6050_Init()`	`main.c:596-603`
重校准	未校准且已就绪且到期	`cal_begin(now)`	`main.c:604`
趋势喂点	每轮	`Dashboard_TrendFeed(now,pitch,roll,valid)`	`main.c:605`
1Hz 速率统计	裸 elapsed ≥1000ms	重算 `sample_hz` / `frame_hz` 并清零计数	`main.c:606-610`
日志服务	每轮	`Logger_Task(now)`	`main.c:611`
事件日志启动/服务	`Logger_State()==LOG_READY`	`EventLog_Init()` 一次，之后每轮 `EventLog_Task(now,uptime_seconds)`	`main.c:612-615`
60s 检查点存档	`due(now,log_next)` 且日志就绪、事件日志空闲、DHT 有效	`save_snapshot()`，成功才把 `log_next` 推后 60s	`main.c:616-619`
存档结果 toast	`save_pending`	序号前进→ `"SAVED"`，日志报错→ `"SAVE ERROR"`	`main.c:620-623`
Flash 退避重试	状态为 ERROR/OFFLINE 且到期且 DHT 不忙	`Logger_Init()`，退避 5s→10s→…→封顶 60s	`main.c:624-633`
空闲调光	每轮	`screen_activity()` 更新最近活动时刻并驱动 `OLED_SetIdle()`	`main.c:635-639`
页内序号回卷	每轮	记录数变少时把 `rank` 归零	`main.c:640-641`
RGB 灯仲裁	每轮	5 级优先级链：STROBE > SAVING > ERROR > OFFSET > LEVEL/IDLE	`main.c:642-648`
唯一重绘点	`!OLED_RefreshBusy() && due(now,frame_next)`	组装 `view` 全部字段 → `Dashboard_Draw()` → `OLED_StartRefresh()`	`main.c:649-659`
屏幕服务	每轮 / 每轮	`if(!DHT11_Busy())OLED_Service(now);` 与 `OLED_RefreshStep();`	`main.c:661-662`

存档门的写法值得逐字读，它把四个条件压在一个 `if` 里：

```
    if(due(now,log_next)&&Logger_State()==LOG_READY&&!EventLog_Busy()&&
       DHT11_Get()->valid&&!DHT11_Get()->error){
        if(!save_snapshot(uptime_seconds,pitch,roll,calibrated&&!stale&&pose_valid))log_next=now+60000;
    }
```

（`main.c:616-619`）

相位式的代价是把所有逻辑压进一条**单线程时间线**：一轮循环的耗时等于所有到期动作之和，任何一项拖长都会顺延后面的项。`max_loop_ms` 因此是上表里最有价值的观测量——但它截断在 999ms（`main.c:537`），超过 1 秒的卡顿在 DEBUG 页上只显示 999，无法区分"卡了 1 秒"和"卡了 10 秒"。相位式也换来两个好处：执行顺序即调试顺序（上表从上到下就是观测顺序），并且不存在优先级反转、优先级饥饿与栈溢出这类 RTOS 风险。

6.3 没有 RTOS 的并发：截止时间 + busy 门控

这个循环是**相位式**而不是优先级式：没有谁抢占谁，所有互斥都由 `busy` 标志表达。门控依赖链如下：

门控条件	被门控的动作	行号	设计理由
`!DHT11_Busy()`	主循环 `OLED_Service(now)`	`main.c:661`	软件 I2C 是共用总线，DHT 抓帧期间不能插 I2C
`!DHT11_Busy()`	自检期的 `OLED_Service(now)`	`main.c:230`	同上，自检泵函数 `post_pump()` 复用同一规则
`!DHT11_Busy()`	`MPU6050_Init()` 重试	`main.c:600`	源码注释写明：**避免 120ms MPU 复位跨越 DHT 的 20ms 起始脉冲**（`main.c:599`）
`!DHT11_Busy()`	Flash 初始化重试	`main.c:628`	Flash 重试本身很快，但要避免与 DHT 抢总线时序
`!EventLog_Busy()`	点击存档与 60s 检查点	`main.c:559`、`main.c:616-617`	`save_snapshot()` 内部还会再判一次，尽早返回（`main.c:148`）
`Logger_State()==LOG_READY`	事件日志启动、60s 检查点	`main.c:612`、`main.c:616`	日志区未就绪时不写事件区，避免两条 Flash 日志互相踩
`!OLED_RefreshBusy()`	整帧重绘	`main.c:649`	32 字节短块刷新没传完就不再开始新一帧（`oled.c:183-192`）
`calibrated && !stale && pose_valid`	角度写入 `view`、角度有效标志	`main.c:650`、`main.c:618`	姿态不可信时宁可显示 `--` 也不写坏记录
`flash_retry` 状态机	`Logger_Init()` 重试	`main.c:624-633`	只有 ERROR/OFFLINE 才重试；重新进入 SCANNING 后关闭退避

关键点：**没有一个门控是为了"公平"**，全部是为了避免物理冲突（总线、Flash 扇区、屏幕传输）。也因为门控条件是单线程里的普通标志，全部天然无锁；`main.c:16` 的注释「中断只计时；共用 I2C 的采样和短块刷新均在主循环执行」就是这条架构约束的书面形式。

6.4 due() 与时间预算

`due()` 是全工程唯一的"到期"原语：

```
static uint8_t due(uint32_t now, uint32_t deadline) { return (int32_t)(now - deadline) >= 0; }
```

（`main.c:49`）

11 处调用点：`main.c:117/538/539/540/570/600/604/616/628/634/649`。另有 12 处是裸无符号 elapsed 比较：`main.c:92/98/163/202/208/214/266-267/411/596/606/657`。两者混用是**有意的**：`due()` 用于"未来截止时刻"（`now - deadline` 为负表示未到期，回绕后仍正确），裸 elapsed 用于"过去的起点"（只关心"过了多久"）。第 2 章已给出边界：裸写法在参照点被误设为未来值时会把"还没到"当成"早已过期"。

时间预算由三个常量与三个统计量描述：

- `SAMPLE_INTERVAL_MS 5`（`main.c:21`）：采样节拍；`sample_next` 每次只加 5ms（`main.c:574`），若某轮超时不会追补，`max_sample_gap_ms`（`main.c:571-573`）就是给这种情况记账的。
- `LEVEL_DISPLAY_INTERVAL_MS 33` / `DISPLAY_INTERVAL_MS 50`（`main.c:22-23`）：LEVEL 页以 30fps 刷新（气泡动画需要），其余页 20fps；切换点在 `frame_next=now+(view.page==0?LEVEL_DISPLAY_INTERVAL_MS:DISPLAY_INTERVAL_MS)`（`main.c:659`）。
- 统计口径：`sample_hz` / `frame_hz` 用"计数 × 1000 ÷ 真实经过毫秒"（`main.c:607-608`），所以它们对超时有自我修正；`max_loop_ms` 与 `max_sample_gap_ms` 都截断到 999（`main.c:537`、`main.c:573`）。这三个值只被 DEBUG 页消费（`dashboard.c:250-252`），属于"自诊断数据"，不参与控制。

6.5 状态放在哪儿：file-static、栈帧与直读寄存器

`main.c` 的持久状态是 17 条 `file-static` 声明（共 35 个变量，`main.c:43-46/104-107/156-157/165-170/222`），主循环自己的工作变量则是 `main()` 的栈帧（`main.c:498-504`，15 条声明共 47 个变量，含整个 `DashboardView view`）。链接映射里 `main.o` 只贡献一个全局符号 `main`（`MDK-ARM\List\level.map:2271`），其余全是文件内符号——这带来两个直接后果：**单体不可测**（没有任何注入点，测试只能整体重编译），以及**天然单实例**（不存在第二份状态被误用）。仪表盘自己也有一份静态状态：`trend_pitch/trend_roll/trend_valid`（`dashboard.c:12-13`）与气泡的 `bubble_x/bubble_y/bubble_time`（`dashboard.c:15-18`），必须靠 `Dashboard_Init()`（`dashboard.c:20-24`）清理。

主要状态变量：

变量	类型	语义	行号
`lpf[3]`	`int32_t[3]`	Q8 定点三轴低通输出（注释：乘法避免左移负数）	`main.c:43`
`base_ax/base_ay/base_az`	`int32_t`	开机基准零点与 Z 轴量程参照	`main.c:44`
`filter_valid/level_latched/inside_active/outside_active/beep_active`	`uint8_t`	滤波有效标志 + 水平仪迟滞状态机 + 鸣叫中标志	`main.c:45`
`inside_since/outside_since/beep_until`	`uint32_t`	三个时刻：进入容差起点、离开容差起点、短鸣截止	`main.c:46`
`cal_active/calibrated/cal_error/cal_attempts/cal_good`	`uint8_t`	校准状态机与进度	`main.c:104`
`cal_sum[3]/cal_lo[3]/cal_hi[3]/cal_next/cal_retry`	混合	校准累加、上下界、下次采样、失败重试	`main.c:105-107`
`last_activity/idle_pitch/idle_roll`	`uint32_t/int32_t`	调光判据：最近输入时刻与当时姿态	`main.c:156-157`
`history_since/history_armed`	`uint32_t/uint8_t`	HISTORY 页驻留计时与武装标志	`main.c:165-166`

变量	类型	语义	行号
`strobe_since/strobe_armed`	同上	STROBE 页同上	`main.c:167-168`
`event_since/event_armed`	同上	SESSIONS 页同上	`main.c:169-170`
`post_dht_started/post_scan_started/post_event_started/post_timebase_ok`	`uint8_t`	自检期间"哪条服务线已启动"的泵标志与总判据	`main.c:222`

`main.c` 里还大量直读寄存器，这不是风格问题，而是**分层现实**：

直读	位置	用途与理由
`RCC->CSR`	`main.c:505`	复位来源，喂给 `reset_reason()` 与事件日志的 `reset_flags>>24`
`SysTick->CTRL` / `SysTick->LOAD`	`main.c:247`	自检断言：`(CTRL&7)==7`（时钟源、中断、使能全开）且 `LOAD+1==SystemCoreClock/1000`
`TIM2->CR1` / `TIM2->PSC`	`main.c:263-264`	自检断言：CEN 置位、`PSC+1==timer_clock/1000000`
`GPIOB->CRL` / `GPIOB->ODR`	`main.c:272`	编码器两位必须是输入上拉（`(GPIOB->CRL&0xFFUL)==0x88UL`）且 ODR 两位为 1
`GPIOA->CRH`	`main.c:278`	TIM1 三通道 + 蜂鸣器引脚配置必须是 `0xBBB`
`TIM1->CR1/BDTR/CCER/ARR/CCMR1/CCMR2`	`main.c:279-283`	PWM 时基与输出使能位级断言，`ARR+1==3600`
`TIM1->CCR1/CCR2/CCR3`	`dashboard.c:249`	DEBUG 页直接显示三路真实占空比，而不是影子变量
`GPIOB->IDR`	`dashboard.c:250`	DEBUG 页显示编码器 A/B 原始电平

POST 的断言都是**掩码/偏移比较**（例如 `TIM1->ARR+1UL==3600UL`），因为它们要校验的是"配置后的位模式"，而不是读回一个逻辑值。

一处**死配置**：`BUBBLE_X_DIR`、`BUBBLE_Y_DIR`、`HUD_MAX_OFFSET`、`HUD_FS_X10`（`main.c:38-41`）。在 28 个源文件里 grep，这四个宏只命中定义本身，全工程零引用；气泡方向与半径实际硬编码在渲染函数里（`dashboard.c:82` 的 `roll*20L*256/d` 与 `-pitch*20L*256/d`）。它们是早期"可视化 HUD"（十字准星/飞行指引）方案的残留，留在头文件里会让人误以为可调。

6.6 DashboardView：单结构体契约，但不是唯一数据通路

契约本体（`dashboard.h:12-25`）：

字段	类型	语义	生产者	消费者
`page`	`uint8_t`	当前页 0-9；`>=DASHBOARD_PAGE_COUNT` 时绘制直接返回	`main.c:529/542/546/191-193`	`dashboard.c:323-353`
`browsing`	`uint8_t`	是否处于页内浏览（决定标题显示 `BROWSE`）	`main.c:542/551/553/556/203/209/215`	`dashboard.c:259/265/326`
`rank`	`uint8_t`	页内序号（HISTORY 记录号 / SESSIONS 会话号）	`main.c:542/178/183/189/193/640-641`	`dashboard.c:159/295/298`
`mpu_valid`	`uint8_t`	姿态是否可信（=已校准且不陈旧且姿态正常）	`main.c:650`	`dashboard.c:114/232/236`
`calibrating` / `cal_error` / `cal_percent`	`uint8_t`	校准进行中 / 失败 / 进度百分比	`main.c:650/653`	`dashboard.c:117-122`
`level`	`uint8_t`	水平锁定标志（迟滞状态机输出）	`main.c:653`	`dashboard.c:135`
`pitch_x10` / `roll_x10`	`int32_t`	姿态角 ×10（气泡与趋势用）	`main.c:654`	`dashboard.c:91/232/236`
`pitch_text` / `roll_text`	`int32_t`	抖动抑制后的显示值（稳定文本）	`main.c:654`	`dashboard.c:136-139`
`uptime_s`	`uint32_t`	本次运行秒数	`main.c:655`	`dashboard.c:178/302/340-347`
`toast`	`const char *`	顶部黑底白字提示，NULL 表示不画	`main.c:655`	`dashboard.c:354-358`
`accel_raw[3]` / `gyro_raw[3]` / `temperature_raw`	`int16_t`	RAW DATA 页原始读数	`main.c:578-579`	`dashboard.c:193-200`
`raw_valid`	`uint8_t`	原始读数是否新鲜	`main.c:652`	`dashboard.c:180/193/196`
`pose_error`	`uint8_t`	姿态异常码 `POSE_FLIPPED` / `POSE_RANGE`	`main.c:651`	`dashboard.c:118`
`strobe_mode`	`uint8_t`	当前频闪图案序号	`main.c:530/177`	`dashboard.c:257-263`
`sample_hz` / `frame_hz`	`uint16_t`	实测采样率与刷新率	`main.c:607-608`	`dashboard.c:250`
`max_sample_gap_ms` / `max_loop_ms`	`uint16_t`	最坏采样间隔、最坏循环耗时（各截断 999）	`main.c:537/573`	`dashboard.c:252`
`strobe_hint`	`uint8_t`	STROBE 页进入 2s 内的操作提示	`main.c:657`	`dashboard.c:265`
`reset_reason`	`const char *`	复位来源 3 字母代号	`main.c:507`	`dashboard.c:176/343`
`mpu_errors`	`uint32_t`	采样失败累计（显示截断 9999）	`main.c:594`	`dashboard.c:251`
`now_ms`	`uint32_t`	传到渲染层的时间基准	`main.c:656`	`dashboard.c:96/104`（气泡动画）

结构体把"渲染需要什么"集中成一页清单，好处直白：业务侧只负责填值（`main.c:650-657` 一段连续赋值），渲染侧只读 `const DashboardView *`，两者之间没有第二套全局变量。代价同样直白：**每次都是全量快照 + 全量重绘**，没有脏标记，也无法只更新一个数字。

但这个契约并**不完整**。`dashboard.c:1-9` 直接包含 7 个模块头，各页在渲染时按需取数：CLIMATE 调 `DHT11_Get()`（`dashboard.c:143`），HISTORY 调 `Logger_RecentCount()` / `Logger_ReadRecent()`（`dashboard.c:155/159`），SYSTEM 调 `W25Q_Capacity()` / `W25Q_ID()` / `Logger_Count()`（`dashboard.c:175/182-185`），DEBUG 调 `DHT11_Get()` / `Controls_ButtonDown()` 并直读寄存器（`dashboard.c:244/250`），SESSIONS 与 RUNTIME 调 `EventLog_SessionCount()` / `EventLog_ReadSession()` / `EventLog_BootCount()`（`dashboard.c:290/295/302/338/345`）。结论：`DashboardView` 承载的是"每帧都在变的姿态与统计"，而"偶尔才变或按需读取"的数据走模块直调。这解释了一个现实约束——**渲染函数里会做 Flash 同步读**（`dashboard.c:159`），整帧时间因此包含一次记录读取。

6.7 十页渲染

绘制入口把页码映射到页函数（`dashboard.c:319-336`）：

```
static const char *titles[DASHBOARD_PAGE_COUNT]={"LEVEL","CLIMATE","HISTORY","SYSTEM","RAW DATA","TREND","DEBUG","STROBE","RUNTIME","SESSIONS"};
uint8_t width;
if(v->page>=DASHBOARD_PAGE_COUNT)return;
OLED_Clear();
if(v->page) {bubble_ready=0;if(v->page!=3)
    OLED_ShowSmallString(4,1,(v->page==2||v->page==EVENT_PAGE)&&v->browsing?
        "BROWSE":titles[v->page]);}
if(v->page==0) level_page(v);
else if(v->page==1) climate_page();
else if(v->page==2) history_page(v);
else if(v->page==3)system_page(v);
else if(v->page==4)raw_page(v);
else if(v->page==5)trend_page(v);
else if(v->page==6)debug_page(v);
else if(v->page==STROBE_PAGE)strobe_page(v);
```

（`dashboard.c:321-336`）

页	标题	内容	渲染函数 / 行号
0	LEVEL	圆形气泡 + 四角括号 + PITCH/ROLL 大字号角度	`level_page`，`dashboard.c:111-140`
1	CLIMATE	温度、湿度大字号，DHT 失败显示 `--` 与 `OLD` / `WAIT`	`climate_page`，`dashboard.c:141-151`
2	HISTORY	最近记录：温湿度、姿态、序号与运行秒数	`history_page`，`dashboard.c:152-172`
3	SYSTEM	复位来源、运行时长、MPU/DHT/FLASH 状态、容量与固件版本（此页不画标题）	`system_page`，`dashboard.c:173-187`
4	RAW DATA	三轴加速度/陀螺原始值与温度原始值	`raw_page`，`dashboard.c:188-203`
5	TREND	120 点双通道趋势（12s 窗口，自动量程）	`trend_page`，`dashboard.c:204-241`
6	DEBUG	DHT 原始字节、边沿/错误计数、三路 PWM 占空比、编码器电平、速率	`debug_page`，`dashboard.c:242-254`
7	STROBE	频闪图案名 + 5 个模式圆点 + 操作提示	`strobe_page`，`dashboard.c:255-266`
8	RUNTIME	`BOOT #`、`HH:MM:SS`、`DAYS`、复位来源、上一会话时长	内联在 `Dashboard_Draw`，`dashboard.c:336-352`
9	SESSIONS	会话列表：启动序号、运行时长下界、复位来源、CURRENT/SAVED	`event_page`，`dashboard.c:286-318`（兜底分支 `dashboard.c:353`）

其余六页的画法各有取舍。CLIMATE 只画温度与湿度两个大字号数值，DHT 出错时退化为 `--` 并在右上角标 `OLD` / `WAIT`（`dashboard.c:145-150`），把"数值"和"是否新鲜"分开表达。HISTORY 每条记录画温湿度、带符号姿态与 `#序号 运行秒数`（`dashboard.c:159-170`），并显式注释采集时间是**开机运行时长而非墙上时钟**（`dashboard.c:171`）。SYSTEM 把复位来源、`UP xxS`、MPU/DHT/FLASH 三态、`W25Q` 容量、记录条数与芯片 ID + 固件版本压进 6 行（`dashboard.c:173-187`），是现场排障先看的一页。DEBUG 页最"裸"：DHT 五字节原始帧、边沿/错误/重试计数、三路 PWM 占空比、编码器电平与实测速率（`dashboard.c:242-254`）。STROBE 借 `Controls_StrobeName()` 取图案名并按 12 像素/字符居中（`dashboard.c:257-260`），下方 5 个圆点表示模式序号。RUNTIME 页内联在入口函数里（`dashboard.c:336-352`），`HH:MM:SS` 用大字号，底部显示上一会话时长或会话总数。

页号是双轨制：`dashboard.h:5-7` 只给了 `STROBE_PAGE 7`、`RUNTIME_PAGE 8`、`EVENT_PAGE 9` 三个宏，0-6 在分派链里是裸数字（`dashboard.c:328-334`），`main.c` 里同样混用——`view.page==2`（`main.c:555`、`main.c:640`）、`view.page==1`（`main.c:637`）、`view.page==3`（`main.c:638`）、`selection<0?6:RUNTIME_PAGE`（`main.c:178`，长按进入 STROBE 浏览后左旋跳到第 6 页）。兜底是 `else event_page(v);`（`dashboard.c:353`）：任何既不是 0-6 也不是 7、8 的合法页（即 9）都落到 SESSIONS。插入一张新页需要同时改 `dashboard.h`、`dashboard.c` 的分派链与 `main.c` 的裸数字。

四个深读页：

- **LEVEL**（`dashboard.c:88-140`）：气泡目标是姿态向量的整数归一化——`bubble_target()` 用整数平方根（`dashboard.c:53-59`）算出模长，除数下限 `150`（`dashboard.c:81`）保证小倾角下方向仍然可见，坐标按 `roll*20*256/d`、`-pitch*20*256/d` 映射到 20 像素半径（`dashboard.c:82`）。平滑是**指数逼近**：`bubble_x+=(x-bubble_x)*elapsed/(elapsed+45)`（`dashboard.c:97`），`elapsed` 截断 100ms 防止卡顿后瞬移；只有当整数位置偏离已画位置超过 160/256 像素才重绘圆盘（`dashboard.c:99-103`），幅度显示超过 ±999 时改用 `" OVER"`（`dashboard.c:137/139`）。
- **TREND**：环形缓冲 120 点、100ms 一点（`TREND_POINTS`，`dashboard.c:11-13`；窗口 12s 与页面标注 `12S` 一致，`dashboard.c:239`）。喂点函数按**真实时间差**补空槽：

```
if((int32_t)(now-trend_next)<0)return;
steps=(now-trend_next)/100+1;
/* Preserve time gaps without drawing a fictitious line through missing data. */
for(k=1;k<steps&&k<TREND_POINTS;k++)trend_store(0,0,0);
trend_store(pitch,roll,valid);trend_next+=steps*100;
```

（`dashboard.c:38-42`）

缺失的数据点以 `valid=0` 存进去（`trend_store` 的第三个参数，`dashboard.c:29-30`），绘图时断开折线而不是拉直线（`dashboard.c:223`）；量程从 `{20,50,100,200,500,1000,3000}` 里按当前缓冲峰值向上取（`dashboard.c:206-213`），所以一次大动作会让整段历史在视觉上变平——这是量程共用的必然结果。

- **RAW DATA**（`dashboard.c:188-203`）：三轴加速度与陀螺各用 `"%c%6d"` 排一列（`dashboard.c:193-198`），右侧对齐靠格式宽度而不是坐标运算；底部一行打印 `TEMP RAW`，无数据时整页提示 `MPU DATA UNAVAILABLE`。
- **SESSIONS**（`dashboard.c:286-318`）：会话号与总数取右上角，`BOOT #` 取中间，时长一行有三种形态：

```
if(event.boot_count==EventLog_BootCount())
    sprintf(text,"RUN %luD %02lu:%02lu:%02lu",(unsigned long)(up/86400UL),
    (unsigned long)(up/3600UL%24UL),(unsigned long)(up/60UL%60UL),
    (unsigned long)(up%60UL));
else if(up)
    sprintf(text,">= %luD %02lu:%02lu:%02lu",(unsigned long)(up/86400UL),
        (unsigned long)(up/3600UL%24UL),(unsigned long)(up/60UL%60UL),
        (unsigned long)(up%60UL));
else sprintf(text,"NO CHECKPOINT");
```

（`dashboard.c:304-312`）

当条目是**当前**会话时直接把 `view.uptime_s` 覆盖进去（`dashboard.c:302`），所以这一页显示的时长是本机实时值；其余会话是断电前最后一个检查点，因此前缀是 `>=`（下界），页脚写 `SAVED MINIMUM`（`dashboard.c:317`）。`RESET` 行：

```
sprintf(text,"RESET %s  #%lu",event_reset_name(event.reset_code),
    (unsigned long)(event.sequence>999999UL?999999UL:event.sequence));
```

（`dashboard.c:314-315`）

`event_reset_name()` 自己有一份复位位表（`dashboard.c:276-285`），与 `main.c:50-58` 的实现**重复但不等价**：前者按 RCC 位逐个判（`0x20`→IWD、`0x40`→WWD、`0x10`→SW、`0x08`→POR、`0x80`→LPW、`0x04`→PIN），后者用的是 SPL 的 `RCC_CSR_*` 宏。序号用 `999999UL` 截断后才进 `%lu`，这是该页唯一的溢出防护手段。

最后，toast 覆盖层在**所有页之后**统一画（`dashboard.c:354-359`），黑底白字从右往左贴，宽度超 124 像素截断；它是唯一一个"跨页复用"的绘制部件。

6.8 sprintf 审计

`dashboard.c` 共 41 处 `sprintf`，**0 处 `snprintf`**。缓冲区尺寸与最坏长度：

缓冲区	声明处	最宽的一条格式	估算最坏长度	风险
`text[16]`	`dashboard.c:113/143`	`level_page` 的固定文案	15	无，文案全是常量
`a[16]` / `b[16]`	`dashboard.c:154`	`fixed()` 输出 `"+214748364.7"`	12+1	无（实际姿态值远小于此；`fixed()` 里 `-value` 对 `INT32_MIN` 是未定义，但该值不可达）
`number[16]`	`dashboard.c:68/231`	同上	13	无
`text[24]`	`dashboard.c:175/190/231`	`"ID %06lX V%s"` → `ID FFFFFF V20261010E`	20	低；再加字段就会溢出
`text[32]`	`dashboard.c:337`	`"%02lu:%02lu:%02lu"`	9	无
`text[40]`	`dashboard.c:154/244/289`	`"D:%02X "` ×5 = 16；`"RESET %s #%lu"` ≤21；`"RUN %luD %02lu:%02lu:%02lu"` ≤24	24	低

审计结论：**当前没有可证的溢出路径**，原因是所有宽字段在进格式串之前都被手工截断或格式宽度约束——`boot_count` / `sequence` 截断到 `999999`（`dashboard.c:300/315/338`）、`mpu_errors` 截断到 `9999`（`dashboard.c:251`）、`PREV` 用 `%lu MIN`（`dashboard.c:347`）。真正的问题是**风格脆弱**：缓冲区尺寸、截断逻辑、格式串三者分散在 41 个调用点上，任何一处加入新的 `%lu` 或 `%s` 都可能越界且没有任何编译期或运行期保护。最低成本加固是把 `text[40]` 这类局部缓冲换成 `snprintf(text,sizeof(text),…)` 并在超出时显示 `"…"`。

本章发现的问题与建议

1. **`DashboardView` 不是唯一数据通路，契约名不副实。** 现象：CLIMATE/HISTORY/SYSTEM/DEBUG/SESSIONS/RUNTIME 页绕过 `view` 直接调用 `DHT11_Get()`（`dashboard.c:143/175/244`）、`Logger_*`（`dashboard.c:155/159/182/184`）、`W25Q_*`（`dashboard.c:175/185`）、`EventLog_*`（`dashboard.c:290/295/302/338/345`）、`Controls_*`（`dashboard.c:250/257`），甚至直读 `TIM1->CCR1/2/3`、`GPIOB->IDR`（`dashboard.c:249-250`）。影响：`dashboard.c` 硬依赖 7 个模块头（`dashboard.c:1-9`），渲染层无法在无硬件环境独立验证，页间行为差异全凭约定。建议：把"哪些数据必须进结构体"写成头文件注释，至少让 SESSIONS/RUNTIME 的会话数据走结构体快照，渲染层不再直接触 `EventLog_*`。
2. **绘制路径里有同步 Flash 读。** 现象：`history_page()` 在 `Dashboard_Draw()` 内部调用 `Logger_ReadRecent(v->rank,&r)`（`dashboard.c:159`），这是整帧时间的一部分，而 OLED 短块刷新在它之后才开始（`main.c:658`）。影响：HISTORY 页每帧都读同一条记录，等于用 Flash 读延迟换刷新率；若正巧与主循环的存档写冲突（写扇区时 SPI 忙），该帧会被拉长。建议：按 `rank` 缓存最近一条记录，只在序号变化时重读；或在主循环里预取到 `view`。
3. **页号双轨制，插入新页要改三处。** 现象：只有 3 个页号宏（`dashboard.h:5-7`），分派链用裸 0-6（`dashboard.c:328-334`），`main.c` 里还有裸 1/2/3/6（`main.c:178/555/637/638/640`）。影响：页序调整时极易漏改，且 `dashboard.c:353` 的 `else event_page(v)` 兜底会把任何未匹配页静默画成 SESSIONS。建议：在 `dashboard.h` 补齐 0-6 的宏或建一张"页码→函数指针"表，并让兜底分支显示明确错误页。
4. **死配置常量误导调参。** 现象：`BUBBLE_X_DIR`、`BUBBLE_Y_DIR`、`HUD_MAX_OFFSET`、`HUD_FS_X10`（`main.c:38-41`）全工程零引用（grep 仅命中定义），气泡方向与半径硬编码在 `dashboard.c:82`。影响：想反转气泡方向或改满量程的人会改 main.c 的宏，然后发现毫无效果。建议：删除这四个宏，或在 `bubble_target()` 里真正接入。
5. **`main()` 不可返回、状态不可观测、逻辑不可单测。** 现象：全部运行态在 `main()` 栈帧（`main.c:490-504`），持久态为 17 条 `file-static` 声明（`main.c:43-46/104-107/156-157/165-170/222`），并且存在两个"永不退出"的循环（`main.c:511-515`、`main.c:518-521`）。影响：调度逻辑无法在 PC 上做单元测试（工作区已明确删除仿

真框架），任何回归只能整体重编译 + 上板观察。建议：把主循环体抽成 `App_Step(now)`，状态收进一个结构体，`main()` 只负责初始化与循环调用——这不改变运行时行为，却让关键判定（如 60s 存档门槛）可被单独测试。

6. **自检期与运行期有两条绘制路径。** 现象：自检画面由 `post_draw()`（`main.c:292`）/ `post_present()`（`main.c:354-361`）直接画并自行泵 `post_pump()`，运行期走 `Dashboard_Draw()`（`dashboard.c:319`，唯一调用点 `main.c:658`）。影响：坐标与字体风格靠人工保持一致，自检页的 `1/3`、`2/3`、`3/3` 进度（`main.c:302`）与主界面无复用关系。建议：让自检复用 `Dashboard_Draw()` 的一个"诊断页"，或至少共享同一套 `center_small` 风格助手。

第 7 章　存储服务：传感器记录日志与开机基准标定

> 本章覆盖 `logger.c`（140 行）、`logger.h`（25 行）的全部逻辑，以及 `main.c` 里的开机基准标定、倾角换算与姿态判据。W25Q 的 SPI 驱动只在"日志的可靠性、寿命与失败面"直接相关处引用。

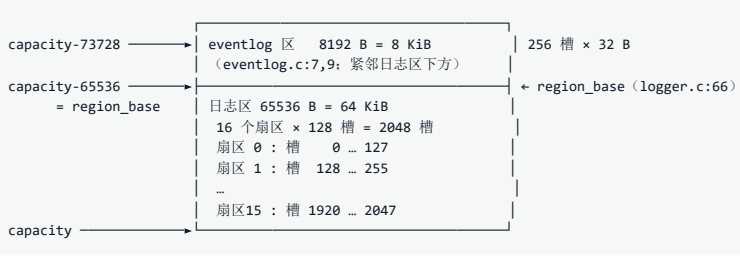
本章要点：

- 日志区固定占用 Flash 的**最后 64 KiB**：`LOGGER_SLOTS 2048`（`logger.h:4`）× 32 字节 = 65536 字节，`region_base=W25Q_Capacity()-REGION_BYTES`（`logger.c:66`），内部是 16 个 4 KiB 扇区 × 128 槽。
- 每条记录用 **CRC32 + 提交字节 0xA5 双重保护**，提交字节在最后一个写相位单独编程（`logger.c:130-134`）。
- 上电扫描遇到"不是本模块写的字节"就进入 `LOG_FOREIGN` 并锁死该区域（`logger.c:98`），工程里**没有任何整片擦除/格式化接口**（`w25qxx.h:4`）。
- 写入是四相位状态机（`logger.c:9,111-139`），**每个相位都写后回读**，任一步失败即 `LOG_ERROR`；恢复靠主循环的 5 s→60 s 指数退避重初始化（`main.c:624-633`）。
- 开机标定在后台串行采 100 点，要求 ≥90 点有效、单轴峰峰 ≤1200 LSB、Z 均值 ≥8000 LSB（`main.c:17-20,114-132`），成功后同时写入三轴基准与 Q8 滤波种子。
- 倾角由 `(lpf[k]/256-base_ak)*5730L/base_az` 得到（`main.c:586-587`）：5730 = 180/π×100，本质是"相对开机姿态"的小角度线性化。

7.1 记录区：容量算式与 Flash 地图

容量算式是本章的地基，结论先给：**2048 槽 × 32 字节/槽 = 65536 字节 = 64 KiB**，与 `REGION_BYTES 65536UL`（`logger.c:8`）一致。槽号到物理地址的映射只有一行：`address(slot)=region_base+slot*32`（`logger.c:27`），而 `region_base=W25Q_Capacity()-REGION_BYTES`（`logger.c:66`）——即永远取容量最高的那 64 KiB。

扇区/槽的换算是第二层：`per_sector[16]`（`logger.c:12`）说明作者按 16 个扇区记账，16 × 128 = 2048；128 这个数来自 W25Q 的扇区擦除粒度是 4096 字节（`w25qxx.c:80` 用 `address&4095` 校验对齐，`w25qxx.c:81` 发 0x20 扇区擦除命令），4096/32 = 128。页编程粒度是 256 字节（`w25qxx.c:72` 校验 `(address&255)+size>256`），所以一条 32 字节记录永远落在同一页内，31 字节主体与 1 字节提交字节是两次独立的页编程。



两个隐含的不变量值得点明。其一，**地址对齐是构造保证的**：`W25Q_Capacity()` 返回 `1UL<<size`（`w25qxx.c:45`），而 `size` 被限制在 0x12…0x18（`w25qxx.c:44`），即容量是 256 KiB…16 MiB 的 2 的幂；再减去 65536 之后，`region_base` 必然是 4096 的整数倍，`W25Q_EraseStart` 的对齐校验（`w25qxx.c:80`）永远通过。其二，**区域大小与容量无关**：换成 16 MiB 的 W25Q128，日志仍然只占最后 64 KiB，多出来的空间不会被本模块使用。

寿命可以据此估出来。擦除只发生在扇区起点（`logger.c:113-116`），因此**每写满 128 条记录才消耗一次擦除周期**。按主循环 60 s 一条的节奏（`main.c:527,618`）算：

- 每天 86400/60 = 1440 条 → 1440/128 = **11.25 次扇区擦除/天**，但这是全 16 个扇区的合计值，**均摊到每个扇区只有 11.25/16 ≈ 0.703 次/天**（一次环形回卷里每个扇区恰好被擦一次，而回卷一轮要 2048 × 60 s ≈ 34.1 小时）；
- W25Q 的典型擦写寿命 10 万次 → 100000/0.703 ≈ 142000 天 ≈ **389 年**；
- 反过来，2048 槽在 60 s 节奏下只能装 2048×60 s ≈ 34.1 小时，之后进环形覆盖。

一个可观测的副作用：环形回卷时擦除的是整个扇区，`drop_sector()`（`logger.c:52-60`）会立刻把 `count` 减去该扇区的记录数、并清掉 `recent` 里属于该扇区的条目——所以 `Logger_Count()` 会**一次掉 128 条**，而不是一条一条地退化。

7.2 32 字节记录格式：CRC 与提交字节的分工

偏移	长度	字段	编码（logger.c:36-42）	解码与校验（logger.c:28-35）
0..3	4	魔术字 `MAGIC 0x31474F4CUL`	`put32(raw,MAGIC)`（`logger.c:38`）	小端即 `4C 4F 47 31` = "LOG1"；不等则判坏（`logger.c:30`）
4..7	4	`sequence` 全局递增序号	`put32(raw+4,...)`（`logger.c:38`）	`get32(b+4)`（`logger.c:31`）
8..11	4	`uptime_s` 本次上电秒数	`put32(raw+8,...)`（`logger.c:38`）	`get32(b+8)`（`logger.c:31`）
12..13	2	`temperature_x10` 温度×10	`put16`（`logger.c:39`）	`(int16_t)get16`（`logger.c:32`）
14..15	2	`humidity_x10` 湿度×10	`put16`（`logger.c:39`）	`get16`（无符号，`logger.c:32`）
16..17	2	`pitch_x10` 倾角×10	`put16`（`logger.c:40`）	`(int16_t)get16`（`logger.c:33`）
18..19	2	`roll_x10` 横滚×10	`put16`（`logger.c:40`）	`(int16_t)get16`（`logger.c:33`）
20..23	4	`flags`（只用低 2 位）	`put32(raw+20,r->flags&3)`（`logger.c:41`）	`get32(b+20)>3` 即判坏（`logger.c:30`）
24..27	4	`crc32(b,24)`	`put32(raw+24,crc32(raw,24))`（`logger.c:41`）	重算并逐位比较（`logger.c:30`）
28..30	3	保留，恒为 `0x00`	由 `memset(raw,0,...)` 决定（`logger.c:38`）	不校验
31	1	**提交字节 `0xA5`**	先填 `0xFF`（`logger.c:41`），最后单独改写	`b[31]!=0xA5` 即判坏（`logger.c:30`）

后端的读写辅助函数把字节序钉死在小端：`get32`/`put32`（`logger.c:16-17`）与 `get16`/`put16`（`logger.c:18-19`）都是显式移位拼装，不依赖编译器或 CPU 的对齐行为。CRC 用的是标准反射式 CRC-32：初值 `0xFFFFFFFF`，多项式 `0xEDB88320UL`，末尾按位取反（`logger.c:20-25`），即 zlib/PNG 用的那一套，所以只要拿到 24 字节就

能用 PC 侧工具独立复算。

编码有一个容易被忽略的细节：`encode()` 第一件事是 `memset(raw,0,sizeof(raw))`（`logger.c:38`），**把 32 字节全部清零**，然后才逐字段填值，最后把 `raw[31]` 改成 `0xFF`（`logger.c:41`）。这带来两个后果：字节 28..30 恒为 `0x00`，而提交字节在"未提交"状态是 `0xFF`。

于是提交字节的写入是一次"只能把 1 变成 0"的编程：`0xA5` 的每一位都是 `0xFF` 的子集，所以对同一字节再编程一次是合法操作，且**方向唯一**（NOR Flash 的页编程只能 1→0，见 `w25qxx.c:69-77`）。这一步在 BODY 相位完成，写完 31 字节主体、回读比对通过之后才执行：

```
    } else if(write_phase==WRITE_BODY) {
        if(W25Q_Read(address(write_slot),b,32) || memcmp(b,raw,31)) {state=LOG_ERROR;return;}
        b[0]=0xA5;
        if(W25Q_ProgramStart(address(write_slot)+31,b,1)) {state=LOG_ERROR;return;}
        write_phase=WRITE_COMMIT;timeout=now+50;
```

（`logger.c:130-134`，逐字摘录）

为什么 CRC 之外还要一个提交字节？两者回答的是不同问题：**CRC 回答"内容对不对"，提交字节回答"这条记录写完了没有"**。留意 CRC 覆盖范围是 `crc32(raw,24)`，即字节 0..23——它既不覆盖 CRC 自身，也不覆盖 28..31。假设掉电发生在主体页编程的中途：内容被部分写入，重算的 CRC 与残留的 CRC 字段不符，记录被拒（这一层由 CRC 兜住）。再假设更坏的情况——主体写完后掉电，CRC 恰好落在"与部分内容自洽"的状态，或者今后有人把 CRC 挪出记录、改成整区校验——那么"缺少完成标记"就成了唯一能区分"写了一半"与"写完了"的信息。提交字节把"记录完成"这一事件压缩成**一次 1 字节的页编程**，它要么发生要么不发生，不存在中间态：掉电在提交之前，字节停在 `0xFF`，解码第一关就被拒（`logger.c:30`）。

`flags` 的 4 字节写法（`logger.c:41`）与解码侧的 `get32(b+20)>3`（`logger.c:30`）还构成第三道"合理性金丝雀"：结构体里 `flags` 是 `uint8_t`（`logger.h:13`），编码时被提升成 32 位写入，所以合法记录的字节 21..23 恒为 0；一旦某条记录的这几字节不是 0，说明它不该被当作有效记录。

7.3 上电扫描：三种终止态与"绝不隐式擦除"

`Logger_Init()`（`logger.c:61-67`）先无条件把所有状态清零并置 `LOG_OFFLINE`，再要求两件事同时成立才继续：`W25Q_Init()` 返回 0（`logger.c:65`，其内部校验 JEDEC ID 必须 `vendor==0xEF`、容量代码 0x12…0x18，`w25qxx.c:44`），以及 `W25Q_Capacity()>=REGION_BYTES`（`logger.c:65`）。通过后算 `region_base`，进入 `LOG_SCANNING`，并设一个绝对截止时间 `timeout=now+2000`（`logger.c:66`）。

扫描在 `Logger_Task()` 里逐槽推进（`logger.c:94-105`），每槽只读 32 字节，判定顺序是：

1. `blank(b)`（`logger.c:26`，全部 32 字节都是 `0xFF`）→ 跳过，不计数；
2. `get32(b)!=MAGIC` → **`state=LOG_FOREIGN;return;`**（`logger.c:98`），扫描立即终止；
3. `Logger_Decode` 失败 → 静默跳过（`logger.c:99`），既不报错也不计数；
4. 解码成功 → `count++`、`per_sector[scan_slot/128]++`、按序号更新 `latest_seq/latest_slot`、`add_recent()`（`logger.c:100-101`）；
5. `++scan_slot==LOGGER_SLOTS` → `LOG_READY`（`logger.c:104`）。

这条判定链的设计哲学写在文件头的注释里："Unknown pre-existing bytes lock this region; no implicit chip erase."（`logger.c:5-6`）。它意味着各终止态有明确语义：

终止态	触发条件	语义	行号
`LOG_READY`	扫满 2048 槽	区域属于本模块，可读可写	`logger.c:104`
`LOG_FOREIGN`	出现非空且魔术字不符的字节	**拒绝接管**：这块区域里有别人的数据，本模块只读不写	`logger.c:98`
`LOG_ERROR`	读失败、`busy==2`（通信错误，`w25qxx.c:52-53`）或超时	通信/介质层面异常	`logger.c:93,95,96`
`LOG_OFFLINE`	芯片未初始化或容量不足	Flash 不可用，全部功能降级	`logger.c:63,65`

"宁可不可用，也不破坏用户数据"是这里的核心取舍：`LOG_FOREIGN` 是**永久终止态**——`Logger_Task()` 的第一行短路（`logger.c:91`）只放行 `SCANNING`/`WRITING`，`Logger_Save()` 又要求 `state==LOG_READY`（`logger.c:82`），所以一旦进入 `LOG_FOREIGN`，本模块再也不会向该区域写一个字节；`W25Q_Init()` 也明确不做解保护与格式化（`w25qxx.h:4`）。代价同样清楚：区域里只要有一个字节被别的程序（或半途而废的旧固件）写坏，日志功能就永久停摆，唯一出路是人工整片擦除。

扫描的实际开销可以估出来。每槽的 SPI 流量 = 1 字节命令 + 3 字节地址 + 32 字节数据 = 36 字节，另加 2 次状态寄存器读（`logger.c:92` 与 `w25qxx.c:58` 各一次，每次 2 字节）。SPI1 挂在 72 MHz 的 APB2 上、预分频 16（`w25qxx.c:39`），即 4.5 MHz、每字节 1.78 µs；2048 槽的数据流量 2048 × 36 B × 1.78 µs ≈ **131 ms**，再加每槽 2 次状态寄存器读（4 字节）后合计 2048 × 40 B × 1.78 µs ≈ **146 ms**。这是纯 SPI 时间的下界，加上 `transfer()` 每次调用都读一次 TIM2 微秒计数（`w25qxx.c:11`）与主循环里 OLED 刷新的占用，实机扫描量级在数百毫秒。

启动自检把这段过程显式呈现给用户：POST 的第 7 项先显示进度百分比（`main.c:443-444`，`Logger_ScanProgress()` 就是 `scan_slot*100/LOGGER_SLOTS`，`logger.c:72`），并用一个 120000 次的上限保护死等（`main.c:440-445`）；结果映射为 `DONE`/`LOCK`（对应 `LOG_FOREIGN`）/`TIME`（超时仍在扫描）/`ERR`/`SKIP`（`main.c:446-453`）。第 6 项则负责更早的可用性判定：`Logger_Init` 之后看状态是否离开 `LOG_OFFLINE`、JEDEC ID 是否非零、容量是否 ≥65536（`main.c:429-432`）。还有一处顺序依赖：eventlog 的初始化要求 `Logger_State()==LOG_READY`（`main.c:612-614`），也就是说**日志模块是这块 Flash 的第一个使用者**，它没准备好，会话记录功能也不会启动。

7.4 写入状态机：四相位、写后回读、单点失败即终止

`Logger_Save()`（`logger.c:80-86`）只做"受理"不做实际写入：前置条件是 `state==LOG_READY` 且 `record->flags&3` 非零（`logger.c:82`）；通过后把记录拷进 `pending`、把序号改成 `latest_seq+1`（`logger.c:83`）、算出目标槽 `write_slot=(count||latest_seq)?(latest_slot+1)%LOGGER_SLOTS:0`（`logger.c:84`，空区域从 0 开始，否则从"序号最大槽"的下一个开始）、编码进 `raw`、置 `state=LOG_WRITING` 并立即返回 0（`logger.c:85`）。真正的时序由主循环每轮调用一次的 `Logger_Task()`（`main.c:611`）推进——**保存是一个跨越多个主循环周期的后台过程**，不阻塞显示与采样。

相位	前条件	动作	失败处理	超时	行号
`WRITE_SELECT=1`	目标槽是扇区起点（`%128==0`）	发起扇区擦除 → 相位转 ERASE	擦除命令失败 → `LOG_ERROR`	转 ERASE 后 1500 ms	`logger.c:113-116`
同上	目标槽不是扇区起点	回读 32 字节；非空则槽号 +1、`skipped++`，空则写 31 字节主体 → 相位转 BODY	`skipped>=2048` → `LOG_ERROR`	转 BODY 后 50 ms	`logger.c:117-124`
`WRITE_ERASE=2`	擦除已完成	回读必须**全为 0xFF**，否则 `LOG_ERROR`；然后 `drop_sector(write_slot/128)` 清账，再写 31 字节主体 → 转 BODY	回读非空或编程失败 → `LOG_ERROR`	转 BODY 后 50 ms	`logger.c:125-129`
`WRITE_BODY=3`	主体已编程	回读并 `memcmp(b,raw,31)` 必须完全相等；然后单独编程 1 字节 `0xA5` 到 `address(write_slot)+31` → 转 COMMIT	回读不符或编程失败 → `LOG_ERROR`	转 COMMIT 后 50 ms	`logger.c:130-134`
`WRITE_COMMIT=4`	提交字节已写	回读 32 字节并 **`Logger_Decode` 必须成功**；然后更新 `latest_slot/latest_seq/count/per_sector/add_recent` 并回 `LOG_READY`	回读或解码失败 → `LOG_ERROR`	无独立超时	`logger.c:135-139`

三点结构性说明。第一，**"写后回读"贯穿每一个相位**：擦除后必须全 `0xFF`（`logger.c:126`）、主体写完必须逐字节相等（`logger.c:131`）、提交后必须能通过完整解码（`logger.c:136`）。这不是冗余——SPI Flash 的失败模式（写保护、掉电、通信丢字节）大多不会通过命令返回值暴露，只能靠回读发现。

第二，`busy` 的处理分两段：相位 1 只要求"芯片不忙"就动手（`logger.c:112`），相位 2 之后则进入轮询分支——`busy` 非零时只检查超时然后返回（`logger.c:107-110`）。`W25Q_Busy()` 返回 2 表示通信错误（`w25qxx.h:11`），在主循环入口被单独拦下并直接判 `LOG_ERROR`（`logger.c:92-93`）。

第三，**`skipped>=LOGGER_SLOTS` 这条分支在稳态下不可达**：槽号从任意位置起每自增 1 都会检查扇区起点（`logger.c:113`），所以在最多 127 次跳过之后一定会遇到一个 `%128==0` 的槽并走擦除分支，不可能攒满 2048 次。它是防御性代码，不是正常路径。

失败面是这里最需要警惕的地方：`LOG_ERROR` 一旦置位，`Logger_Task()` 首行短路（`logger.c:91`）、`Logger_Save()` 拒绝受理（`logger.c:82`）、`Logger_ReadRecent()` 也拒绝读取（`logger.c:76`），**pending 里那条记录被静默丢弃**，没有任何重试。整个模块不会自愈，恢复责任在主循环（`main.c:624-633`）：状态一旦进入 `LOG_ERROR` 就武装 `flash_retry`，退避初值 5000 ms，每次重试翻倍直到封顶 60000 ms（`main.c:625,630`）；到点后若仍未恢复且 DHT 不忙，调用 `Logger_Init(now)` 重新走一遍"初始化 → 扫描"（`main.c:629`），若重新进入 `LOG_SCANNING` 就撤掉重试标志（`main.c:631`）；状态回到 `LOG_READY` 或 `LOG_FOREIGN` 时也清标志（`main.c:633`）。注意最后这一条：**`LOG_FOREIGN` 不会触发重初始化**，因为它本来就是设计上的终止态，重试没有意义。

7.5 读取路径与 `recent` 环形缓冲

界面翻页不会去读 Flash。扫描阶段每发现一条有效记录就把它塞进 RAM 里的 `recent` 列表（`logger.c:101`），列表容量固定 `LOGGER_RECENT 32`（`logger.h:5`）：

```
static void add_recent(uint16_t slot,uint32_t seq)
{
    uint8_t pos=0,k;
    while(pos<recent_count && (int32_t)(seq-recent_seq[pos])<=0) pos++;
    if(pos==LOGGER_RECENT) return;
    if(recent_count<LOGGER_RECENT) recent_count++;
    for(k=recent_count-1;k>pos;k--) {recent_seq[k]=recent_seq[k-1];recent_slot[k]=recent_slot[k-1];}
    recent_seq[pos]=seq;recent_slot[pos]=slot;
}
```

（`logger.c:43-51`，逐字摘录）

这是一个"最新在前"的插入排序：`(int32_t)(seq-recent_seq[pos])<=0` 用**有符号差值**比较序号（同一个技巧也用在扫描里找最大序号，`logger.c:100`），因此序号 `uint32_t` 回绕不会破坏顺序；一旦列表满了（`pos==LOGGER_RECENT`）新记录就直接丢弃（`logger.c:47`），保证内存里始终是最新的 32 条。RAM 开销是 `recent_seq` 32×4 + `recent_slot` 32×2 = **192 字节**，加上 `raw[32]`（`logger.c:10`）与 `pending` 结构（`logger.c:14`），整个模块的静态占用在 300 字节以内。

配套访问器都很薄：`Logger_RecentCount` / `Logger_LastSequence` / `Logger_ScanProgress`（`logger.c:68-72`）。真正读 Flash 的只有 `Logger_ReadRecent()`（`logger.c:73-79`）：它先检查 `rank<recent_count` 与状态（`logger.c:76`），按 `address(recent_slot[rank])` 读回 32 字节（`logger.c:77`），再交给 `Logger_Decode` 校验（`logger.c:78`）。这里有一个刻意的例外——**`LOG_FOREIGN` 状态仍然允许读取**（`logger.c:76` 只排斥 `SCANNING` / `WRITING` / `ERROR`），因为在别人的区域被锁死之前，扫描已经捡回来一批可用记录，界面还能继续看。

代价是"读失败"的后果被放大了：任何一次 `W25Q_Read` 失败都会把整个模块打成 `LOG_ERROR`（`logger.c:77`），而这条路径只是界面翻页。主循环里的用法与之匹配：翻页时把 `Logger_RecentCount()` 传给导航模块（`main.c:546`），有记录才允许进入浏览态（`main.c:556`），越界时把 `rank` 归零（`main.c:640`）——列表为空时界面直接提示 `NO RECORDS`（`main.c:557`）。

7.6 `flags` 位域与"没有有效数据就不记"

有效位只有两位：`LOG_ENV_VALID 1`（温湿度有效）与 `LOG_LEVEL_VALID 2`（姿态有效）（`logger.h:6-7`）。它们在 `main.c` 的采集入口处组装：

```
static uint8_t save_snapshot(uint32_t uptime_seconds,int32_t pitch,int32_t roll,uint8_t angle_valid)
{
    const DHT11_Data *d=DHT11_Get();LogRecord r;
    if(EventLog_Busy())return 1;
    r.sequence=0;r.uptime_s=uptime_seconds;r.flags=0;
    r.temperature_x10=d->temperature_x10;r.humidity_x10=d->humidity_x10;
    r.pitch_x10=(int16_t)pitch;r.roll_x10=(int16_t)roll;
    if(d->valid&&!d->error)r.flags|=LOG_ENV_VALID;
    if(angle_valid)r.flags|=LOG_LEVEL_VALID;
    return Logger_Save(&r);
}
```

（`main.c:145-155`，逐字摘录）

注意 `r.sequence=0`：序号由 `Logger_Save` 自己重写（`logger.c:83`），调用方不需要知道当前进度。第三个入参 `angle_valid` 在两个调用点分别是手动保存的 `calibrated&&!stale&&pose_valid`（`main.c:559`）与周期保存的同名表达式（`main.c:618`），都要求"标定完成 + 姿态可信"。

`Logger_Save` 的受理条件里，"状态就绪"与"有有效数据"被合并成了一个返回值：`if(state!=LOG_READY || !(record->flags&3)) return 1;`（`logger.c:82`）。两个调用点的行为差别值得对照：周期保存有前置条件 `DHT11_Get()->valid&&!DHT11_Get()->error`（`main.c:616-617`），所以那时 `LOG_ENV_VALID` 必为 1，"两位全零被拒"只可能出现在手动保存路径（`main.c:559`）——若当时温湿度帧无效且姿态也不可信，这条记录会被**整条丢弃**，而界面只能报 `NOT READY`（`main.c:561-562`），用户看不出真实原因（详见章末问题 1）。

7.7 开机基准标定：后台步进的 100 点

标定解决的问题是"每个模块装到板子上的姿态都不一样"，所以角度必须有参照系。四个阈值宏（`main.c:17-20`）：

宏	值	含义
`CAL_SAMPLES`	100	目标采样点数（`main.c:17`）
`CAL_MIN_GOOD`	90	其中至少要有 90 点 `valid`（`main.c:18`）
`CAL_MOTION_LSB`	1200	单轴峰峰值上限，判定"期间没有在动"（`main.c:19`）
`MIN_CAL_Z`	8000	Z 轴均值下限，判定"板子的 Z 轴大致朝上"（`main.c:20`）

`cal_begin()`（`main.c:108-113`）只做重置：置 `cal_active=1`、清零错误/计数、把 `cal_next` 设为"立刻到期"，并给三个轴的极值寄存器铺上初值（`cal_lo=32767`、`cal_hi=-32768`）。推进全在 `cal_feed()`（`main.c:114-132`）里，它被采样循环在每次拿到 MPU 数据后调用（`main.c:582,594`），但内部有节流：`if(!cal_active || !due(now,cal_next))return;`（`main.c:117`）配合 `cal_next=now+10`（`main.c:118`），实际是**每 10 ms 取一点**，100 点约 1 s。

判定段是这样的：

```
    if(cal_attempts<CAL_SAMPLES)return;
    ok=cal_good>=CAL_MIN_GOOD;
    if(ok){for(k=0;k<3;k++)if((int32_t)cal_hi[k]-cal_lo[k]>CAL_MOTION_LSB)ok=0;
        if(cal_sum[2]/cal_good<MIN_CAL_Z)ok=0;}
    cal_active=0;
    if(ok){
        base_ax=cal_sum[0]/cal_good;base_ay=cal_sum[1]/cal_good;base_az=cal_sum[2]/cal_good;
        for(k=0;k<3;k++)lpf[k]=cal_sum[k]/cal_good*256;
        filter_valid=calibrated=1;level_reset();
    }else{cal_error=1;cal_retry=now+1000;}
```

（`main.c:122-131`，逐字摘录）

三个条件的物理含义值得逐条换算（按 MPU6050 标称 16384 LSB/g）：

- **100 点 @10 ms ≈ 1 s 的窗口**：足够长到滤掉单次采样噪声，又短到用户能忍受静止不动。
- **≥90 有效点**：允许 10% 的采样被丢弃（I2C 读失败等原因），但不允许"大部分时间读不到数据"。
- **峰峰 ≤1200 LSB**：1200/16384 = 0.0732 g 的峰峰，即 ±0.0366 g。在水平姿态附近 $a_x \approx g\cdot\sin\theta$，反解得 $\theta = \arcsin(0.0366) = 2.10°$——所以这个条件等价于"标定期间板子相对基准的抖动不超过约 ±2.1°"（在实测灵敏度为 1 g≈18400 LSB 的克隆件上，同一阈值反解为 ±1.87°：LSB 门槛是绝对的，物理门槛随芯片灵敏度漂移）。
- **Z 均值 ≥8000 LSB**：8000/16384 = 0.4883 g，而 $\cos(60.77°)=0.4883$，所以它允许板子在"Z 轴朝上"的姿态附近**最多倾斜约 60.8°**（克隆件上 8000/18400 = 0.4348 → 64.2°）。这个条件只保证"不是竖着放的"，不保证水平；真正的水平校正由三轴均值本身完成。反过来说，如果开机时板子是倒扣的（Z 为负），标定永远不会成功——这一条与运行时的 `POSE_FLIPPED` 判据互补。

成功后写入两组基准：`base_ax/ay/az` 是原始 LSB 均值，`lpf[k]=均值*256` 是 Q8 定点形式的显示滤波种子（`main.c:128-129`），并置 `filter_valid=calibrated=1`（`main.c:130`）让第一帧角度立即可用。失败则置 `cal_error=1` 并把重试时间推到 1 s 后（`main.c:131`），主循环在下一轮满足"未标定、无标定进行中、传感器就绪、数据不陈旧"时重新开始（`main.c:604`）。开机时若传感器就绪则立刻启动一次（`main.c:525`）。整个标定是**后台串行**的：注释明确写了"后台校准：页面切换和温湿度读取继续工作"（`main.c:103`），采集在 10 ms 节拍上让位给显示任务。

7.8 倾角换算：5730 的来历、误差与除零审计

角度计算在采样分支里，前置条件是"已标定"与"姿态可信"，核心只有四行：

```
        px100=PITCH_DIR*((lpf[0]/256-base_ax)*5730L/base_az);
        rx100=ROLL_DIR*((lpf[1]/256-base_ay)*5730L/base_az);
        pitch=px100/10;roll=rx100/10;
        pitch_text=stable_text(px100,pitch_text);roll_text=stable_text(rx100,roll_text);
```

（`main.c:586-589`，逐字摘录）

公式的形状是"轴偏差 / Z 基准"，配上方向宏 `PITCH_DIR (-1)` 与 `ROLL_DIR (1)`（`main.c:36-37`）：方向宏吸收的是"哪个轴朝前、装反了要乘 -1"这类装配差异，与数值换算无关。

5730 的来历是纯单位换算：小角度看，绕 Y 轴倾斜 Δθ（弧度）会让 X 轴读数变化 $\Delta a_x \approx g\cdot\cos\theta_0\cdot\Delta\theta$，而开机基准记录的 $base_az = g\cdot\cos\theta_0$ 正好是那个比例因子。于是

$\Delta\theta_{rad} \approx (a_x - base_ax) / base_az$，再乘 $180/\pi = 57.2958$ 得到度，乘 100 得到"度的百分之一"（代码里的 `_x100` 后缀与 `pitch=px100/10` 的整数除法都印证了这一点）：$57.2958 \times 100 = 5729.58$，取整即 **5730**（相对误差 +0.007%）。

这个近似在多大范围内成立？下表按第一行的定义给出相对误差（负号表示读数偏小），第二列假设开机基准本身是水平的（$\theta_0=0$），第三列假设基准已经歪了 10°：

相对基准的倾角	读数相对误差（$\theta_0=0$）	读数相对误差（$\theta_0=10°$）
5°	−0.13%	−0.90%
10°	−0.51%	−2.04%

相对基准的倾角	读数相对误差（$\theta_0=0$）	读数相对误差（$\theta_0=10°$）
15°	−1.14%	−3.43%
20°	−2.02%	−5.06%
30°	−4.51%	−9.02%

误差来源有两项：`sinΔθ` 展开的三次项（$\theta_0=0$ 时即 `-Δθ²/6`），以及基准倾斜引入的 `-tanθ₀·(1-cosΔθ)` 项。第三种偏差来自分母：`base_az = g·cosθ₀` 小于重力模长 `g`，用它做分母会让读数整体**偏大约 1/cosθ₀**（$\theta_0=10°$ 时 +1.54%），方向与上表的负误差相反，两者部分抵消。如果一定要更准，分母应换成 `sqrt(ax²+ay²+az²)` 或改用 `base_az/cosθ₀`——但工程选了最省事的一种。

除零风险审计的结论是"当前不可达，但依赖一个隐式不变量"。`base_az` 是 `static int32_t`（`main.c:44`），未标定时为 0，因此所有除法点都要查：

除法点	行号	保护
`cal_sum[2]/cal_good`	`main.c:125`	在 `if(ok)` 内，此时 `cal_good>=90`
`cal_sum[k]/cal_good`	`main.c:128-129`	同上，且在 `if(ok)` 分支内
`base_az/4`	`main.c:142`	除数是常量 4
`…*5730L/base_az`	`main.c:586-587`	在 `if(calibrated)`（`main.c:583`）内，而 `calibrated` 只在 `main.c:130` 置位——同一分支已把 `base_az` 赋成 ≥8000 的均值

也就是说：**`calibrated==1` 蕴含 `base_az>=8000`**，运行期不会除零。这个不变量是隐式的（只由 `main.c:125-130` 的语句顺序保证），一旦有人把角度计算挪到 `if(calibrated)` 之外，或在校准失败时复用 `base_az`，就会踩到除零。届时会发生什么也值得记下：本工程用 ARM Compiler 5、目标 Cortex-M3（`level.uvprojx:21,177`），map 文件里**没有任何 `__aeabi_idiv` 引用**（只有 printf 用到的 `__aeabi_uidivmod` / `__aeabi_uldivmod`，`level/MDK-ARM/List/level.map:724,780`），说明这些除法被编译成了 Cortex-M3 的硬件除法指令而不是 AEABI 帮助函数；按 Cortex-M3 架构定义，除数为 0 时结果返回 0 且不触发异常，除非置位 `CCR.DIV_0_TRP`，而 USER 目录里检索不到任何 `CCR` / `DIV_0_TRP` 引用。所以真实后果是"角度悄悄变成 0 或异常值"，而不是立刻停机——**静默错误比硬故障更难发现**，更值得加一句显式防护。

7.9 `pose_check`：翻转与超范围

姿态判据只有两行，却是角度显示的总闸门：

```
static uint8_t pose_check(int16_t raw_z)
{
    /* X/Y alone cannot distinguish a level board from an upside-down board. */
    if(raw_z<0)return POSE_FLIPPED;
    if(raw_z<base_az/4||lpf[2]/256<base_az/4)return POSE_RANGE;
    return POSE_OK;
}
```

（`main.c:138-144`，逐字摘录）

注释点明了设计动机：只看 X/Y 无法区分"水平"与"倒扣"，因为两者的 X/Y 都接近 0，所以必须直接看 Z 的符号。两条判据的物理含义是：

- `raw_z<0` → `POSE_FLIPPED`：瞬时 Z 为负，板子翻过来了；
- `raw_z < base_az/4` **或** 滤波值 `lpf[2]/256 < base_az/4` → `POSE_RANGE`：Z 塌缩到基准的四分之一以下，即 `a_z/g < 0.25`、`θ > acos(0.25) = 75.5°`，板子已经接近竖立。

用"原始值或滤波值任一越界"是保守写法：瞬时值负责快速响应，滤波值负责在抖动时不被单点尖峰骗过，两者都不满足才判定姿态可信。

判据的后果链很长，而且全部汇到一处：`pose_valid=0` 时跳过角度更新并调用 `level_reset()`（`main.c:583-592`），平台仪的锁定态、蜂鸣与 HUD 也随之复位（`main.c:644-647` 决定指示灯状态、`main.c:636` 决定 HUD 是否绘制）；更关键的是**记录里的姿态位会一起降级**——两个保存点传的都是 `calibrated&&!stale&&pose_valid`（`main.c:559,618`），于是翻转期间写下的记录只有温湿度、没有角度（`main.c:152-153`）。这个取舍是对的：宁可在记录里标注"这次没有有效姿态"，也不要写入一个翻转后的假角度；但它也意味着**翻过来放置的板子会持续产生 `LOG_LEVEL_VALID` 为 0 的记录**，而扫描/解码路径不区分这一点（解码只校验标志位形式合法，不解释语义）。

还有一处细节值得注意：界面发布的是 `view.pose_error = stale?POSE_OK:pose_error`（`main.c:651`）——数据陈旧时**故意上报 OK**，把"传感器没数据"与"姿态不对"两种状态分开呈现（前者由 `view.raw_valid`、`view.mpu_valid` 表达，`main.c:650-652`）。

7.10 `lpf` 的更新率与滤波强度

角度用的是滤波后的加速度，滤波强度由一个"快慢双速"的一阶 IIR 决定：

```
    if (filter_valid)
        for (k = 0; k < 3; k++)
            if (abs32((int32_t)acc[k] * 256 - lpf[k]) > LPF_FAST_LSB * 256L)
                shift = LPF_FAST_SHIFT;
    for (k = 0; k < 3; k++) {
        int32_t target = (int32_t)acc[k] * 256;
        if (!filter_valid) lpf[k] = target;
        else lpf[k] += (target - lpf[k]) / (1L << shift);
    }
```

（`main.c:64-72`，逐字摘录）

三个参数决定了手感：`LPF_SHIFT 4`（`main.c:26`）是常态，等效 `α = 1/16`；`LPF_FAST_SHIFT 2`（`main.c:27`）是快速档，`α = 1/4`；触发条件是任一轴偏差超过 `LPF_FAST_LSB 600`（`main.c:28`）个 LSB。600/16384 = 0.0366 g ≈ 2.1°（与标定的抖动阈值同量级，这不是巧合——两个数字都在回答"多小的变化算噪声"）。

更新率不是自由参数：`filter_feed()` 只在一次 MPU 读取**成功**之后被调用（`main.c:575-581`），而采样节拍是 `SAMPLE_INTERVAL_MS 5`（`main.c:21`，`sample_next=now+SAMPLE_INTERVAL_MS`，`main.c:574`），所以标称更新率是 200 Hz。据此可以给出等效时间常数与带宽：

档位	α	采样周期 T	等效时间常数 τ≈T/α	−3 dB 带宽 α/(2πT)	触发
常态（`LPF_SHIFT 4`）	1/16	5 ms	80 ms	≈2.0 Hz	默认
快速（`LPF_FAST_SHIFT 2`）	1/4	5 ms	20 ms	≈8.0 Hz	任一轴偏差 >600 LSB（`main.c:66`）

有两个让"标称 200 Hz"打折的现实因素：主循环里 OLED 刷新、DHT 读取、Flash 任务都排在同一线程里，采样节拍会被拉长；代码自己也知道这一点，所以 `elapsed=now-last_sample_try` 被记进 `view.max_sample_gap_ms`（`main.c:571-573`），实测值可以在 DEBUG 页看到。若实际周期翻倍到 10 ms，上表的 τ 与带宽随之整体变差一倍。

两个工程细节保证了滤波器的健壮性：一是 Q8 定点（`main.c:43` 注释"Q8，乘法避免左移负数"），用乘 256 代替左移，避开负数右移的实现定义行为；二是数据陈旧时 `filter_valid=0`（`main.c:580`），下一次有效采样会让 `lpf` 直接跳到目标值，而不是从一个跨越了几秒的旧值慢慢爬回来。标定成功时把 `lpf[k]` 预置成均值（`main.c:129`）同理——让第一帧就落在合理位置。

本章发现的问题与建议

1. "状态未就绪"与"没有有效数据"共用一个返回值，界面无法区分。 现象：`Logger_Save` 把两类拒绝合并成 `return 1`（`logger.c:82`：状态非 `LOG_READY` **或** `flags&3==0`），手动保存的失败提示只有 `NO FLASH` / `LOCKED` / `NOT READY` 三种（`main.c:561-562`）。当温湿度帧无效且姿态不可信时（`main.c:559` 传入的 `angle_valid` 可能为 0），记录被整条丢弃，界面却显示 `NOT READY`。影响：排障时无法区分"Flash 坏了"和"这会儿没有可记录的数据"。建议：让 `Logger_Save` 返回分类错误码（如"无有效数据"与"未就绪"分开），`main.c` 据此分别提示。

2. 写路径单点失败即 `LOG_ERROR`，pending 记录静默丢失。 现象：SELECT/ERASE/BODY/COMMIT 任一相位失败都直接置错返回（`logger.c:114,117,123,126,128,131,133,136`），已编码的 `pending` 既不重试也不保留；恢复完全依赖主循环的退避重初始化（`main.c:624-633`）。影响：一次瞬时通信抖动就会丢掉这条记录，并让日志功能停摆到下一次重试（最长 60 s）。建议：在 `LOG_ERROR` 时保留 `pending` 与相位信息，退避重试时从断点继续（或至少重放一次），把"瞬时失败"与"介质损坏"区分开。

3. 一次读失败就会让整个模块停摆。 现象：`Logger_ReadRecent` 中 `W25Q_Read` 失败即 `state=LOG_ERROR`（`logger.c:77`），而这条路径只服务于界面翻页。影响：用户翻页时的一次 SPI 抖动可能让后台日志停写。建议：读取路径不改变全局状态，改为向调用方返回错误，让界面降级显示。

4. 扫描的 2000 ms 截止时间只在"忙轮询"里检查，语义偏窄。 现象：`timeout=now+2000` 在 `Logger_Init` 设定（`logger.c:66`），但只在 `busy` 非零的分支里被比较（`logger.c:95`）；扫描按估算只需约 146 ms 的纯 SPI 时间，而 POST 期间它与 OLED 刷新共用主循环（`main.c:441-445`）。影响：慢速显示或长线缆导致单次 `transfer()` 重试时，扫描可能被判 `LOG_ERROR`，且它与"真失败"在界面上无法区分（`main.c:449`）。建议：改成按槽推进的耗时预算（例如每 N 槽检查一次总耗时），或在 POST 阶段放宽该阈值。

5. `flags` 的前向兼容是断的。 现象：编码把 `flags` 提升成 4 字节写入（`logger.c:41`），解码用 `get32(b+20)>3` 判定记录损坏（`logger.c:30`）。影响：今后若增加第 3 个有效位（比如"陀螺仪有效"），新版固件写下的记录会被所有旧版固件的扫描判为坏记录（`logger.c:99` 静默跳过），而旧记录在新固件里仍能读——单向不兼容。建议：解码只校验已知位，或把 flags 留在 1 字节并单独校验该字节。

6. `MIN_CAL_Z` 是绝对 LSB 阈值，随芯片灵敏度漂移。 现象：判据是 `cal_sum[2]/cal_good<MIN_CAL_Z`（`main.c:125`），阈值 8000 LSB（`main.c:20`）。在标称 16384 LSB/g 的正品上等价于 0.488 g / 60.8°，在本机克隆件（工作区记录的实测 1 g≈18400 LSB）上等价于 0.435 g / 64.2°。影响：换芯片就要重新评估"什么样的姿态能被接受标定"，而这一条没有任何文档提示。建议：改成与重力模长成比例的无量纲判据，例如要求 Z 均值 ≥ 模长的一半。

7. 保留字节 28..30 既不参与 CRC 也不校验。 现象：`encode()` 先把 32 字节清零（`logger.c:38`），CRC 只覆盖 0..23（`logger.c:41`），28..30 三个字节永远为 `0x00` 且解码时不检查。影响：今后若把它们当字段使用，旧固件写下的记录（这三字节也是 0）与新固件的语义会混淆，而且改动它们不会破坏 CRC。建议：要么纳入 CRC 覆盖范围，要么在 `logger.h` 里显式注释为"保留，必须为 0"。

第 8 章　会话事件日志与复位溯源

本章要点

- 事件日志把两个 4 KiB 私有扇区放在既有 64 KiB 传感器日志的**正前方**，容量算式是 `base = W25Q_Capacity() - 65536 - 8192`（`eventlog.c:92`），共 256 槽 × 32 B。
- 它不需要 VBAT、也不需要时钟：开机时全量扫描 256 个槽（`eventlog.c:142-161`），从 Flash 内容反推开机次数，随即排队一条 BOOT 记录。
- 每条记录以 `EVT1` 魔数开头、由 `crc32` 校验，并把提交字节 `0xA5` **最后单独写入**（`eventlog.c:176-178`）——"写了一半"与"写完了"在字节层面可区分。
- 记录只有两类：开机一条 BOOT（时长强制写 0，`eventlog.c:78`），运行期每 60 s 一条 ALIVE 心跳（`eventlog.c:191-193`）。断电后屏幕给出的是时长的**下界**，误差上限为一个心跳周期。
- 它硬依赖传感器日志先进入 `LOG_READY`（`eventlog.c:91`）；传感器区被判为外来数据时，事件日志连同复位溯源一起失效。
- `EVENT_ERROR` 与 `EVENT_LOCKED` 是终态：`eventlog.c:138` 命中即返回，本章找不到回到 READY 的路径，只能掉电重启。

8.1 区域位置与容量算式

事件日志不引入新的 Flash 管理，它只是在既有传感器日志之前挤出两个扇区。规模常量集中在 `eventlog.c:7-12`：

```
#define EVENT_BYTES 8192UL       /* 2 × 4 KiB 私有扇区 */
#define SENSOR_BYTES 65536UL     /* 既有传感器日志区 */
#define EVENT_SLOTS 256U         /* 8192 / 32 */
#define EVENT_MAGIC 0x31545645UL /* EVT1 */
#define EVENT_COMMITTED 0xA5U
#define HEARTBEAT_MS 60000UL
```

（出处 `eventlog.c:7-12`）

基址从芯片容量反算，不写死地址，因此换用不同容量的 W25Q 时布局自动跟随：

```
event_base = W25Q_Capacity() - SENSOR_BYTES - EVENT_BYTES
           = W25Q_Capacity() - 65536 - 8192            （eventlog.c:92）
地址(槽)    = event_base + 槽号 × 32                    （eventlog.c:34）

低地址                                                  高地址
   ……  本固件未使用  ……
       +-----------------------------------------+
       | 事件扇区 A   槽 0..127     4096 B = 128×32  |
       +-----------------------------------------+ <- event_base + 4096
       | 事件扇区 B   槽 128..255   4096 B = 128×32  |
       +-----------------------------------------+ <- W25Q_Capacity() - 65536
       | 传感器日志   槽 0..2047    65536 B = 16 扇区 |
       +-----------------------------------------+ <- W25Q_Capacity()
       ^ event_base = W25Q_Capacity() - 73728
```

（出处 `eventlog.c:34`、`eventlog.c:92`、`logger.c:66`）

两处对齐不是巧合：`73728 = 4096 × 18`，所以 `event_base` 天然满足 `W25Q_EraseStart` 的 4 KiB 对齐要求（`w25qxx.c:80`）；槽地址恒为 32 B 对齐，而驱动要求一次编程不跨 256 B 页边界（`w25qxx.c:72`），因此一条记录（含最后那 1 字节）永不跨界，也永不把一页拆成两次编程。传感器区基址在 `logger.c:66` 另算为 `W25Q_Capacity() - 65536`，与事件区上界严格相接，两者既不重叠也不留缝。

8.2 硬依赖：传感器日志不 READY，事件日志就不启动

`EventLog_Init` 在算出基址之前有一道门：

```
    if(Logger_State()!=LOG_READY||W25Q_Capacity()<SENSOR_BYTES+EVENT_BYTES)return;
    base=W25Q_Capacity()-SENSOR_BYTES-EVENT_BYTES;
    state=EVENT_SCANNING;
```

（出处 `eventlog.c:91-93`）

条件不满足时状态停在 `EVENT_OFFLINE`（`eventlog.c:85-88` 只清状态，不置扫描位）。`EventLog_Task` 每轮还会复查同样的条件（`eventlog.c:139`），并对 OFFLINE/LOCKED/ERROR 直接返回（`eventlog.c:138`）。这条依赖把两个模块的故障绑在了一起：传感器区一旦被判为外来数据，`logger.c:98` 会把状态置成 `LOG_FOREIGN` 并立即返回，本会话不再回到扫描流程；于是 `Logger_State()` 永远不等于 `LOG_READY`，事件日志既不扫描也不写 BOOT，`boot_count` 保持 0。屏幕上的表现是 SESSIONS 页的 `NO EVENTS` 加状态文本 `NA`（`dashboard.c:291-293`、`dashboard.c:273`）、RUNTIME 页的 `BOOT #0`（`dashboard.c:338`）——**复位溯源能力静默消失**，而用户看到的最强提示只是 HISTORY/SYSTEM 页的红灯（`main.c:644-646`），那是传感器日志的故障指示，并没有告诉他事件日志也停了。

还有一条更隐蔽的路径：主循环只允许**重试一次**事件日志初始化（`main.c:612-614`，条件是 `!event_init_attempted`），而传感器日志允许按 5 s 起、上限 60 s 的指数退避反复重试（`main.c:624-633`）。若首个机会被花在传感器日志正处于 `LOG_ERROR` 的时刻，此后即使传感器日志自我修复成功，事件日志在本会话内也不会再启动。

8.3 记录格式：32 字节逐字段

`prepare`（`eventlog.c:73-81`）负责填充，`decode`（`eventlog.c:35-45`）负责验收，验收条件要全部通过才把字段交给上层：

```
static uint8_t decode(const uint8_t *b,EventLast *event)
{
    if(get32(b)!=EVENT_MAGIC||b[31]!=EVENT_COMMITTED||
       get32(b+24)!=crc32(b,24)||
       (b[20]!=EVENT_BOOT&&b[20]!=EVENT_ALIVE&&b[20]!=3)||
       b[21]>1)return 0;
    event->sequence=get32(b+4);
    event->uptime_seconds=get32(b+12);event->boot_count=get32(b+16);
    event->kind=b[20];event->reset_code=b[22];
    return 1;
}
```

（出处 `eventlog.c:35-45`）

偏移	字节	prepare 写入值	decode 校验	EventLast 字段与含义
[0..3]	4	`EVENT_MAGIC` = `0x31545645`（`EVT1`）	必须相等	— 区分本固件记录与外来数据
[4..7]	4	`latest_seq + 1`	只取最大值，不校验	`sequence` 单调序号，决定会话排序
[8..11]	4	恒 `0xFFFFFFFF`	不校验	— 保留字，写死
[12..15]	4	`uptime_seconds`，BOOT 强制 0	不校验	`uptime_seconds` 本会话已运行秒数
[16..19]	4	`boot_count`	不校验	`boot_count` 开机序号，恢复时的锚点
[20]	1	`kind`（1=BOOT，2=ALIVE）	必须是 1、2 或 3	`kind` 记录类型
[21]	1	恒 0	必须 ≤ 1	— 预留开关位
[22]	1	`reset_reason`	不校验	`reset_code`，`RCC->CSR` 高 8 位
[23]	1	恒 0	不校验	— 保留字
[24..27]	4	`crc32(b,24)`	必须等于重算值	— 校验覆盖 [0..23]
[28..30]	3	保持擦除态 `0xFF`	不校验	— 未使用
[31]	1	`EVENT_COMMITTED` = `0xA5`，最后单独写	必须等于 `0xA5`	— 提交门

`decode` 还接受 `kind == 3`（`eventlog.c:39`），但全工程没有任何一处会写入这个值：它是一条**未实现**的预留记录类型，只保证了校验器不会被将来的新类型卡住。保留字段的取值是有信息量的：`[8..11]` 写 `0xFFFFFFFF` 而非 0，让"未使用"与"字段值为 0"在字节图样上可以区分；`[21]` 被限制为"必须 ≤ 1"，说明它是一个将来只打算取 0/1 的开关位，当前恒 0；`[28..30]` 保持擦除态 `0xFF`。`crc32`（`eventlog.c:26-31`）覆盖 `[0..23]` 这 24 字节，**不含提交字节**，因此提交字节可以最后补写而不必重算校验；唯一的校验盲区是 `[28..30]`——若将来启用它们，必须同步扩大 `crc32(b,24)` 的覆盖长度。

与磁盘字段对照，`EventLast`（`eventlog.h:8-11`）只暴露 5 项：`sequence`、`uptime_seconds`、`boot_count`、`kind`、`reset_code`，正好是需要参与上层逻辑的全部字段；魔数、保留字、CRC 与提交字节留在驱动内部，上层既看不到也无从误用。

8.4 扫描与恢复：开机第一次读 Flash

```
    if(state==EVENT_SCANNING){
        if(busy)return;
        if(W25Q_Read(address(scan_slot),b,32)){state=EVENT_ERROR;return;}
        if(!blank(b)){
            if(decode(b,&event)){
                uint32_t seq=get32(b+4);
                if(event.kind==EVENT_BOOT)add_session(scan_slot,seq);
                update_duration(&event);
                if(!have_latest||(int32_t)(seq-latest_seq)>0){
                    have_latest=1;latest_seq=seq;latest_slot=scan_slot;latest=event;
                }
            }else if(get32(b)!=EVENT_MAGIC)foreign=1;
        }
        if(++scan_slot==EVENT_SLOTS){
            if(foreign){state=EVENT_LOCKED;return;}
            if(have_latest)boot_count=latest.boot_count;
            search_slot=have_latest?(uint16_t)(latest_slot+1U):0;
            boot_count++;pending=EVENT_BOOT;state=EVENT_READY;
        }
        return;
    }
```

（出处 `eventlog.c:142-161`）

逐句读：一次 `EventLog_Task` 只处理一个槽（`eventlog.c:144`），`scan_slot` 从 0 走到 256 即扫描完成；节流靠 Flash 驱动的忙标志，上一笔操作没结束就整轮跳过（`eventlog.c:143`），不阻塞主循环。只有"非全 `0xFF` "的槽才继续判断（`eventlog.c:145`），而 `blank()` 要求 32 字节**全部**为 `0xFF`（`eventlog.c:32-33`），所以写到一半的记录不会被误认为空槽。解码成功才更新内存视图：按 `sequence` 降序插入会话表（`eventlog.c:148`、`eventlog.c:47-55`）、聚合时长（`eventlog.c:149`）、维护最大序号的 `latest`（`eventlog.c:150-152`）。解码失败**且魔数也不是** `EVT1` 才判定为外来数据（`eventlog.c:153`）——魔数相同但提交字节或 CRC 不对的情形，是本固件自己写坏的记录，不能拿它去锁住整个区域。扫满 256 槽后：确实发现过外来数据就进入 `EVENT_LOCKED` 且什么都不写（`eventlog.c:156`），保住对方的数据；否则用最新记录的开机序号作基准（`eventlog.c:157`），把写指针放到最新记录的下一槽（`eventlog.c:158`），然后 `boot_count++` 并排队一条 BOOT（`eventlog.c:159`）。

`eventlog.c:159` 这一行值得单独说：**这里的"开机次数"不是硬件计数器，而是从 Flash 内容推导出来的量。**凡是能在事件区落下记录的运行，序号就加一；若 BOOT 记录最终没写成功（进入 `EVENT_ERROR`），RAM 里加过的值不会被持久化，下次开机仍从 Flash 推导出同一个数字。这种推导式计数是幂等的，也是本仪表不需要 VBAT 与校时就能维持开机序号的原因。全新（全 `0xFF`）区域首次扫描得到 `have_latest = 0`，于是 `search_slot = 0`、`boot_count = 1`，第一条 BOOT 落在槽 0（`eventlog.c:158-159`）。

8.5 追加写状态机：四个相位、三次回读

写一条记录要经过 4 个内部状态；需要换扇区时还要先擦除。状态推进由"回读确认"驱动，`EventLog_Busy()` 只在这些相位里为真（`eventlog.c:96-98`）：

相位	动作	回读校验	超时与失败	行号
READY（发起）	`choose_slot()` → `prepare()` → 写前 31 字节；需换扇区则先发起擦除	无	驱动返回非 0 → `EVENT_ERROR`	`eventlog.c:194-205`
`EVENT_ERASING`	擦除目标扇区	交给下一相位	1500 ms 未完成 → `EVENT_ERROR`	`eventlog.c:163-166`、`eventlog.c:199-200`
`EVENT_VERIFY`	逐槽回读整扇区 128 槽确认为空 → `drop_sector()` → 写前 31 字节	每槽 `blank()`	任一槽非空 → `EVENT_ERROR`	`eventlog.c:167-173`
`EVENT_BODY`	回读前 31 字节与 `raw` 比较 → 写提交字节	`memcmp(b,raw,31)`	不等 → `EVENT_ERROR`；50 ms 超时	`eventlog.c:174-180`
`EVENT_COMMIT`	整条回读并解码，通过后才更新内存视图	完整 `decode()`（含 CRC 与提交字节）	失败 → `EVENT_ERROR`；50 ms 超时	`eventlog.c:181-190`

提交字节的写法是这套设计里最巧的一处：

```
        if(W25Q_Read(address(write_slot),b,32)||memcmp(b,raw,31)){state=EVENT_ERROR;return;}
        b[0]=EVENT_COMMITTED;
        if(W25Q_ProgramStart(address(write_slot)+31UL,b,1)){state=EVENT_ERROR;return;}
```

（出处 `eventlog.c:176-178`）

先写 `[0..30]`，回读确认后再**只写第 31 个字节**。掉电语义因此清晰：断电在第一笔之后，记录表现为"非空但解码失败"，扫描会跳过它（`eventlog.c:145-153`）；断电在第二笔之后，记录完整，扫描能正常还原。因为 CRC 不覆盖提交字节，第二笔不必重算校验。代价是一条记录要发两次页编程、做两次回读，总线时间约翻倍——在 60 s 一条的节奏下无关紧要。

三次回读防的是三种不同故障，缺一不可：`VERIFY` 防"擦了但没擦净"（NOR 擦除必须回读确认全 `0xFF`），`BODY` 防"编程丢字节"，`COMMIT` 防"提交字节没写进去"。加上 CRC，**内存里的会话视图只会落后 Flash 最多一条记录，不会出现"以为写了其实没写"**。

8.6 槽位选择与双扇区轮换

```
    if(!have_latest){
        if(search_slot==EVENT_SLOTS){state=EVENT_LOCKED;return 0;}
        end=EVENT_SLOTS;
    }else{
        end=(uint16_t)((latest_slot/128U+1U)*128U);
        if(search_slot>=end){
            write_slot=(uint16_t)(end%EVENT_SLOTS);
            return 2; /* Rotate to the other sector; keep the current sector intact. */
        }
    }
```

（出处 `eventlog.c:118-126`，注释为源码原文）

返回值有三种语义：`1` 表示找到空槽可以直接写，`2` 表示必须先擦除目标扇区，`0` 表示本轮不动作（`eventlog.c:128-132`）。搜索范围只有两种：全新区域从槽 0 找到 255，搜满即 `EVENT_LOCKED`（`eventlog.c:119`）；一旦有 `latest`，就**只搜当前扇区的剩余槽**——`end` 被算成"当前扇区末尾"（`eventlog.c:122`），到扇区尾就轮换到另一个扇区（`eventlog.c:124`，注释写明"保留当前扇区完整"）。

这种 A/B 交替相对单扇区擦写，换来的是掉电安全：擦除只发生在"当前扇区已写满、准备换到另一扇区"的时刻，而此刻被擦的那个扇区已经在上一次扫描中被完整读过——它的会话记录早已进了内存表，`drop_sector()`（`eventlog.c:170`）只是把内存视图与实际内容对齐。任何时刻掉电，最多损失"正在写的那一条"和"即将被覆盖的旧会话"，不会出现"擦到一半连自己刚写的数据一起丢"的局面。单扇区方案则必须在写到一半时回头擦自己：擦除需要数十毫秒到数秒，这段时间内的任何掉电都会让整个扇区的历史消失，且没有第二份记录能自证。代价是任一时刻只有半个区域是活跃写区，但这一点不影响寿命（见 8.10）。

8.7 心跳与时长聚合：断电只能给出下界

心跳是事件日志唯一周期性写盘的动力：

```
    if((int32_t)(now-next_heartbeat)>=0&&pending==0){
        pending=EVENT_ALIVE;next_heartbeat=now+HEARTBEAT_MS;
    }
```

（出处 `eventlog.c:191-193`）

它每 60 s 排队一条 `EVENT_ALIVE`，且只在没有待写记录时排队（`pending==0`），写入过程中不会叠加。`prepare` 用的 `uptime_seconds` 是**发起时刻**由调用方传入的值（`eventlog.c:197`），到 COMMIT 阶段不再刷新，因此记录里的秒数与真正落盘时刻最多差一个擦除超时（1500 ms，`eventlog.c:200`），相对 60 s 粒度可忽略。

由于心跳是周期性的，同一会话会留下多条记录，取哪一条由 `update_duration` 决定：

```
static void update_duration(const EventLast *event)
{
    uint16_t index=(uint16_t)(event->boot_count%EVENT_SLOTS);
    if(session_boot_key[index]==event->boot_count){
        if(event->uptime_seconds>session_uptime[index])session_uptime[index]=event->uptime_seconds;
    }else if(session_boot_key[index]==0xFFFFFFFFUL||newer(event->boot_count,session_boot_key[index])){
        session_boot_key[index]=event->boot_count;session_uptime[index]=event->uptime_seconds;
    }
}
```

（出处 `eventlog.c:56-64`）

同一开机序号保留最大 `uptime_seconds`，不同序号则用 `newer()` 判断是否替换，而 `newer(a,b)` 是有符号差比较（`eventlog.c:46`），32 位回绕后依然正确。

断电之后屏幕能给出什么，因此可以完整推导：`EventLog_ReadSession` 回读该会话的 BOOT 记录（`eventlog.c:109` 要求 `kind==EVENT_BOOT`），再用内存聚合值覆盖时长（`eventlog.c:110-111`）。若该次开机只留下 BOOT（时长恒 0），页面显示 `NO CHECKPOINT`（`dashboard.c:312`）；若留下过至少一条 ALIVE，则显示 `>= D HH:MM:SS`（`dashboard.c:308-311`），页脚标为 `SAVED MINIMUM`（`dashboard.c:317`）。当前会话不受此限，它用 RAM 里的实时秒数显示 `RUN ...`（`dashboard.c:302-307`）。误差有多大？掉电时最后一条 ALIVE 与真实运行时长之差小于 60 s，所以 `>=` 是一个误差不超过约 1 分钟的下界。这不是近似，而是**结构性下界**：事件日志从未记录"关机时刻"，因为掉电不可预知，没有任何机制能在断电瞬间完成一次写盘。

8.8 复位溯源链：从 `RCC->CSR` 到屏幕文本

复位原因的采集只有一次机会——寄存器读后即清：

```
uint32_t reset_flags=RCC->CSR;
memset(&view,0,sizeof(view));Dashboard_Init();
view.reset_reason=reset_reason(reset_flags);RCC_ClearFlag();
```

（出处 `main.c:505-507`）

`main.c:505` 在读 CSR 的同时把高 8 位保留在 RAM 里；`RCC_ClearFlag()`（`main.c:507`）之后，复位标志不会再被任何后续代码看到。这个 RAM 值随后走两条路：一条给显示（`main.c:507` → RUNTIME 页 `dashboard.c:343-344`），另一条给持久化（`main.c:517` 传入 `(uint8_t)(reset_flags>>24)` → `main.c:467` 的 `EventLog_Init` → `eventlog.c:90` 存进 `reset_reason` → `eventlog.c:79` 写进 BOOT 记录的 `[22]`），最终由 SESSIONS 页解出文本（`dashboard.c:314`）。

`>>24` 不是随意取的位移：CSR 的复位标志集中在 bit31..bit26，右移 24 位后它们正好落在 bit7..bit2，`event_reset_name`（`dashboard.c:276-285`）逐位测试：

标志位	位号	右移后掩码	显示名	SESSIONS 页判定次序	行号
`LPWRRSTF`	bit31	`0x80`	`LPW`	第 5	`dashboard.c:282`、`main.c:56`
`WWDGRSTF`	bit30	`0x40`	`WWD`	第 2	`dashboard.c:279`、`main.c:53`
`IWDGRSTF`	bit29	`0x20`	`IWD`	第 1	`dashboard.c:278`、`main.c:52`
`SFTRSTF`	bit28	`0x10`	`SW`	第 3	`dashboard.c:280`、`main.c:54`
`PORRSTF`	bit27	`0x08`	`POR`	第 4	`dashboard.c:281`、`main.c:55`
`PINRSTF`	bit26	`0x04`	`PIN`	第 6	`dashboard.c:283`、`main.c:57`
无位命中	—	—	`UNK`	兜底	`dashboard.c:284`、`main.c:58`

判定次序就是优先级：看门狗复位排在最前（`dashboard.c:278-279`），多位同时置位时只显示先命中的那一个。上电时 `PORRSTF` 与 `PINRSTF` 常常同时置位，此时显示 `POR` 而不是 `PIN`（`dashboard.c:281` 先于 `dashboard.c:283`）。右移后 bit0 是 CSR 的 `RMVF`（清零命令位，bit24），`event_reset_name` 不测试它，所以清标志这一动作本身不会被误报成一次复位。

值得注意的是，同一张映射表在本工程写了两遍：`main.c:50-58` 用 `RCC_CSR_*` 宏吃原始位掩码（供 RUNTIME 页），`dashboard.c:276-285` 用字面量吃右移后的字节（供 SESSIONS 页）。两处的判定次序与 `UNK` 兜底目前一致，但没有任何机制保证它们继续一致。

8.9 互锁与调用点

`EventLog_Busy()` 只覆盖四个写入相位（`eventlog.c:96-98`），它不是"出故障"的意思，而是"正在动 Flash"。工程里全部调用点如下：

调用点	作用	行号
`save_snapshot` 开头	事件日志在写时不落传感器快照，直接返回 1	`main.c:148`
SESSIONS 页进入浏览	有会话且不忙才允许进入浏览态	`main.c:213-215`
POST 等待循环	等扫描与首条 BOOT 写完	`main.c:469-470`
手动保存失败的提示	显示 `BUSY` 而不是 `NOT READY`	`main.c:561-562`
60 s 传感器存档门控	不忙才写传感器记录	`main.c:616-617`

驱动侧则由主循环和 POST 泵两处推动：POST 期间用 `now/1000` 充当运行秒数（`main.c:229`），进入主循环后改用真实 `uptime_seconds`（`main.c:615`）。初始化同理有两条路径：POST 里的 `main.c:467`，以及主循环里的一次性补救 `main.c:612-614`。

两个辅助函数值得单独记一笔：`EventLog_ScanProgress()`（`eventlog.c:102-103`）把 `scan_slot` 换算成 0..100 的百分比，POST 阶段用它刷进度条（`main.c:346-348`、`main.c:471-472`），主循环里不用；而 `W25Q_Busy()` 返回 2 表示 SPI **通信错误**而不是忙（`w25qxx.h:11` 注释 `0 ready, 1 busy, 2 communication error`），`EventLog_Task` 对它的处理是直接判 `EVENT_ERROR`（`eventlog.c:141`）——这是工程里唯一一处把驱动级通信故障提升为模块终态的地方。

这套互锁有一个值得点明的**不对称**：事件日志忙会挡住传感器快照，但事件日志坏了（`EVENT_ERROR` / `EVENT_LOCKED`）不会挡——因为这两个状态不在 `EventLog_Busy()` 的判定范围里（`eventlog.c:96-98`）。换言之"正在写"是共享资源冲突，"已损坏"只是自认失效，不会连累传感器日志继续记录。反面则是 8.2 描述的那条：传感器日志不 READY，事件日志连启动的机会都没有。

8.10 写入寿命与计数上限

心跳把 Flash 写入变成常态，寿命需要按算式核一遍：

```
记录速率      60 s 一条 ⇒ 1440 条/天 ⇒ 525 600 条/年（外加每次开机 1 条 BOOT，可忽略）
扇区容量      128 槽 × 32 B
轮换周期      256 条记录 ⇒ 每个扇区各擦除 1 次
扇区擦除率    525 600 ÷ 256 ≈ 2053 次/年（两扇区合计约 11.25 次/天，每扇区约 5.6 次/天）
寿命结论      100 000 次 ÷ 2053 次/年 ≈ 49 年
```

结论：按 W25Q 系列 10 万次擦写计，事件区可支撑约 **49 年**的连续心跳，远长于设备可能的使用年限。若改成单扇区（128 槽）且不做轮换，擦除次数翻倍到约 4106 次/年，寿命约 24 年——也就是说 A/B 轮换在寿命上并不吃亏，它买到的是掉电安全。同一颗芯片上传感器日志也按 60 s 写一条（`main.c:616-618`），但它摊在 16 个扇区：每年 525 600 条覆盖整个 64 KiB 约 257 次，每个扇区每年擦除约 257 次，约 389 年才耗尽 10 万次，同样不是瓶颈。

计数上限方面，`boot_count` 每次开机加一（`eventlog.c:159`），uint32 上限 $2^{32} \approx 4.29\times10^9$ 次开机：即使每秒重启一次也要 2^{32} 秒 ≈ 136 年，实际每天重启 3 次则约 392 万年，溢出不是现实约束。`sequence` 每条记录加一（`eventlog.c:76`），52.56 万条/年下 $2^{32} \div 525\,600 \approx 8171$ 年，同样安全。真正先到上限的是**内存会话表**：`session_count` 封顶 256（`eventlog.c:51-52`），而 Flash 里同时存在的记录也只有 256 槽——一次运行 10 分钟会消耗 1 条 BOOT 加 10 条 ALIVE，于是 256 个槽只记得下约 23 次开机。**决定"能回看多少次开机"的是槽位总数与每次会话产生的记录数，而不是 `boot_count` 的位宽**（与 `logger` 的环形覆盖是同一类取舍，见第 7 章）。

本章发现的问题与建议

1. **`EVENT_ERROR` / `EVENT_LOCKED` 无自愈**：状态赋值点共 16 处（`eventlog.c:108/119/128/131/141/144/156/164/168/171/176/178/182/183/199/202`），`eventlog.c:138` 命中终态即返回；唯一回到扫描的入口是 `EventLog_Init`（`eventlog.c:82-93`），而主循环只允许一次性补救（`main.c:612-614`）。任何一次 SPI 读失败或擦除超时都会让本次开机彻底没有事件日志，却没有任何可见告警。建议把 `EVENT_ERROR` 纳入 `main.c:624-633` 那样的退避重试：重新 `EventLog_Init` 并清除 `event_init_attempted`，至少每分钟允许重扫一次。
2. **显示路径的读失败会毒化写入路径**：`EventLog_ReadSession` 在屏幕刷新中读 Flash 失败就把状态置成 `EVENT_ERROR`（`eventlog.c:108`）——在 SESSIONS 页翻页，就可能让事件日志从此停写。建议读路径只返回错误码（该函数已有返回值），不改状态机状态。
3. **两条恢复路径不对称**：传感器日志能从 `LOG_ERROR` 退避重扫（`main.c:624-633`），事件日志却不能（见 8.2）。建议把 `event_init_attempted` 改成"每当 `Logger_State()` 从非 READY 转为 READY 时清零"，让事件日志跟随传感器日志的恢复一起重试。
4. **撕裂记录会永久占槽**：写到一半的记录既不空、也不 `foreign`、又无法解码（`eventlog.c:145-153`），而 `choose_slot` 只承认全 `0xFF` 的空槽（`eventlog.c:129`），因此这个槽要到整个扇区被擦除才能回收。建议扫描时把"魔数正确但未提交"的槽记录下来，允许 `choose_slot` 直接复用。
5. **同一张复位映射表写了两遍**：`main.c:50-58` 与 `dashboard.c:276-285`，一个吃原始位掩码、一个吃右移后的字节。目前结果一致，但没有同步机制；一旦只改一处，RUNTIME 页与 SESSIONS 页会对同一次复位给出不同名字。建议只保留一份实现，由显示层调用。
6. **POST 的失败文本不足以定位**：EVENT 步骤只给出 `DONE/LOCK/ERR/SKIP/TIME`（`main.c:476-480`），而 `ERR` 背后有 13 种互不相同的故障点（见第 1 条的行号列表），`LOCK` 也无法区分"检测到外来数据"与"256 槽耗尽"。建议在驱动里保留一个"最后失败相位"字节，POST 直接显示它。
7. **心跳把 Flash 擦写变成常态**：每扇区每天约 5.6 次擦除（见 8.10）。它换来的是断电后 1 分钟精度的时长下界。这是一个可调取舍：把 `HEARTBEAT_MS`（`eventlog.c:12`）放宽到 5 分钟，寿命提升到约 244 年，代价是下界误差变成 5 分钟。建议在文档里明确这个旋钮，而不是把它留在常量里无人知晓。

第 9 章　人机交互：OLED 显示子系统

本章要点

- 面板是 SSD1306 0.96 寸 128×64 单色屏，走软件 I2C（PB6/PB7），I2C 层统一用 7 位地址 `0x3C`（`oled.h:7`、`oled.h:21`）；模块丝印的 `0x78` 是 8 位写地址，两者是同一个从机（`oled.h:17-18`）。
- 所有绘图都落在 1024 字节的 `static uint8_t OLED_GRAM[8][128]`（`oled.c:11`）上，**不调用刷新函数屏幕上什么都不会变**（`oled.h:25`）；字节 bit0 是该页最上一行，位序必须与字库一致，否则整屏上下镜像（`oled.c:5-10`）。
- 初始化是 16 次独立短事务、共送出 41 字节命令（`oled.c:22-31`、`oled.c:111-115`），最后 `OLED_Clear()+OLED_Refresh()` 清屏并整帧上屏（`oled.c:116`）。
- 上屏走**差分短块刷新**：整屏切成 32 个 32 字节块（8 页 × 4 块），与影子缓存 `sent_gram` 逐字节比较，未变化的块整块跳过（`oled.c:15`、`oled.c:185-192`）。
- 每次 `OLED_RefreshStep()` 最多只发**一个**变化块（`oled.c:203`），因此最坏只会占用一次约 1.5 ms 的阻塞事务（`oled.c:196-197`），主循环在块间仍可取样（`oled.c:132-134`）。
- 每字节都检查从机 ACK；任一字节失败即 `need_recovery()`：丢弃影子缓存、补 9 个时钟复位总线、随后按每调用一条命令的节奏重放 16 条初始化表，结束再强制整帧重发（`oled.c:33-38`、`oled.c:196-201`、`oled.c:155-159`）。
- 调光是**改对比度寄存器**而不是关屏：`OLED_Service()` 每次最多改动 8 个对比度计数、间隔 40 ms（`oled.c:165-171`），`0xCF→0x28` 共 21 步约 840 ms；唤醒方向则一次跳到目标值（`oled.c:165-166`）。
- 主循环里唯一的重绘点在 `main.c:649-659`，并且 `OLED_Service()` 被 `!DHT11_Busy()` 门控（`main.c:661`、`main.c:230`）——这道门只管对比度/恢复事务，**不管** `OLED_RefreshStep()`（`main.c:662`）。

9.1 显存模型：8 页 × 128 列与位序约定

SSD1306 的显存是 128×64 位，物理组织为 8 页（page）× 128 列（column），每页 8 行像素。本驱动用完全一致的结构做软件镜像：

```
/*------------------------------------------------------------------------
  显存：8 页 x 128 列。
  OLED_GRAM[页][列]，字节的 bit0 是那一页最上面一行，bit7 是最下面一行。
  这个位序必须和字库一致（字库里每个字节的 bit0 也是最上面一行），
  否则整屏的字会上下镜像。
------------------------------------------------------------------------*/
static uint8_t OLED_GRAM[8][128];
```

（`oled.c:5-11`）

模型要点：

项	取值	证据
显存容量	8 × 128 = 1024 字节	`oled.c:11`
坐标约定	像素坐标 `x = 0~127`、`y = 0~63`，不用页号	`oled.h:12-13`
页/位换算	`page = y >> 3`、`bit = y & 0x07`	`oled.c:219-220`
置位/清零	`|=(1<<bit)` / `&=~(1<<bit)`	`oled.c:222-225`
越界策略	`x>127 || y>63` 直接丢弃	`oled.c:217`

`OLED_DrawPoint()` 是全套绘图的唯一写入口（圆、线、矩形、字符最终都调它，`oled.c:223`/`225`）。bit0 在页的最上面正是 SSD1306 的原生顺序（第 0 页 bit0 对应 COM0/SEG0），所以显存可以整块搬给屏幕而不需要任何位变换（`oled.c:8-9`）。

`oled.c:217` 的越界处理是**丢弃**而不是钳位，注释写明原因："越界直接丢，画圆时会有负坐标绕回来"（`oled.c:217`）：圆的对称点里会有 `xc-r` 这种负值，转成 `uint8_t` 后会回绕成 200 多的数，丢弃可避免在屏上画出错误边线。

9.2 从 7 位地址说起：0x3C 与丝印 0x78

I2C 层所有函数都只接受 7 位地址，内部再左移一位拼读/写位（`i2c.h:18`、`i2c.c:190`）。因此：

- 驱动里用 `#define OLED_ADDR7 0x3C`（`oled.h:21`）；
- 模块丝印的 `0x78` 是 8 位**写**地址，`0x78 >> 1 = 0x3C`（`oled.h:17-18`）。

这不是风格问题：误把 `0x78` 传进 `SoftI2C_WriteBytes()` 会被左移成 `0xF0`，从机不应答，全屏黑且 POST 里 OLED 那一步报 `ERR`（`main.c:396`）。规则与 MPU6050 的 `0x68`（本身就是 7 位，见 `i2c.h:20-21`）并列写出 —— OLED 模块标 8 位、MPU6050 模块标 7 位，不能互相套用。

9.3 初始化：16 条短事务与 41 字节命令

初始化表按"每条命令一次事务"组织，最多 3 字节/条，前导的 `0x00` 是 SSD1306 的控制字节（Co=0、D/C#=0，表示后续都是命令；数据流则用 `0x40`，见 `oled.c:194`）：

```
static const uint8_t init_commands[][3]={
    {0x00,0xAE,0}, {0x00,0xD5,0x80}, {0x00,0xA8,0x3F},
    {0x00,0xD3,0x00}, {0x00,0x40,0}, {0x00,0x20,0x00},
    {0x00,0xA1,0}, {0x00,0xC8,0}, {0x00,0xDA,0x12},
    {0x00,0x81,0xCF}, {0x00,0xD9,0xF1}, {0x00,0xDB,0x30},
    {0x00,0xA4,0}, {0x00,0xA6,0}, {0x00,0x8D,0x14},
    {0x00,0xAF,0}
};
static const uint8_t init_sizes[]={2,3,3,3,2,3,2,2,3,3,3,3,2,2,3,2};
#define INIT_COMMAND_COUNT (sizeof(init_sizes)/sizeof(init_sizes[0]))
```

（`oled.c:22-31`）

`init_sizes[]` 把每条命令的实际长度和 3 字节的行宽解耦（`oled.c:30`），总长度 `2+3+3+3+2+3+2+2+3+3+3+3+2+2+3+2 = 41` 字节，条数由 `sizeof` 推导而不是硬编码（`oled.c:31`），增删命令只需同步改 `init_sizes`。各命令含义（命令语义取自 SSD1306 数据手册，取值逐字来自 `oled.c:23-28`）：

命令	参数	物理含义	备注
`0xAE`	—	显示关闭（进入睡眠）	先关后配，避免配一半被看到
`0xD5`	`0x80`	显示时钟分频/振荡频率	数据手册复位默认值
`0xA8`	`0x3F`	多路复用比 = 0x3F+1 = 64 行	与 64 行面板匹配
`0xD3`	`0x00`	显示垂直偏移 = 0	无偏移
`0x40`	—	显示起始行 = 0	低 6 位即行号
`0x20`	`0x00`	存储寻址模式 = 水平寻址	**关键**：`0x21/0x22` 窗口命令只有在水平/垂直寻址下有效，复位默认是页寻址
`0xA1`	—	段重映射（列 127 映射到 SEG0）	非复位默认（默认 `0xA0`），决定左右镜像
`0xC8`	—	COM 扫描方向反向（COM63→COM0）	非复位默认（默认 `0xC0`），决定上下镜像
`0xDA`	`0x12`	COM 引脚硬件配置 = 交替式、无左右反转	**易错项**：128×64 必须 `0x12`，128×32 才是 `0x02`
`0x81`	`0xCF`	对比度 = 207/256	非复位默认（默认 `0x7F`），接近驱动能给出的最亮档
`0xD9`	`0xF1`	预充电周期	非复位默认（默认 `0x22`），内部电荷泵常用值
`0xDB`	`0x30`	VCOMH 撤销电平	非复位默认（默认 `0x20`）
`0xA4`	—	全局显示跟随显存内容	与 `0xA5`（强制全亮）区分
`0xA6`	—	正常显示（非反色）	与 `0xA7` 区分
`0x8D`	`0x14`	电荷泵使能	**最易错项**：3.3 V 单电源模块不开电荷泵就整屏不亮
`0xAF`	—	显示开	最后一条

三条最容易配错的是 `0x8D 0x14`（漏了则黑屏）、`0xDA 0x12`（配成 `0x02` 会出现隔行/半屏异常）、`0xA1` + `0xC8`（配反则整屏镜像或上下颠倒）。表里没有 `0xB0/0x00/0x10` 这类页/列设置命令，因为水平寻址模式下位置完全由 `0x21`（列窗口）和 `0x22`（页窗口）决定（`oled.c:193`）。

初始化函数本身很短：

```
    for(k=0;k<INIT_COMMAND_COUNT;k++) {
        if(SoftI2C_WriteBytes(OLED_ADDR7,init_commands[k],init_sizes[k])) {
            need_recovery();break;
        }
    }
    OLED_Clear();OLED_Refresh();
```

（`oled.c:111-116`）

三个设计细节：

1. **失败不致命**。任一条命令失败就 `break` 出去，但流程继续走到 `OLED_Clear()+OLED_Refresh()`（`oled.c:116`）。此时 `recovery_active=1`（`oled.c:36`），`OLED_StartRefresh()` 会因 `if (refresh_busy||recovery_active) return;` 直接返回（`oled.c:175`），所以这次"同步整帧"实际是空操作、不会在坏总线上再撞一次；真正的复位交给运行期的 `OLED_Service()` 分步重放。
2. **顺序是"关显示 → 配置 → 开显示"**，`0xAE` 在第 1 条、`0xAF` 在最后一条（`oled.c:23`、`oled.c:28`）。
3. `OLED_Clear()` 只清显存（`oled.c:119-130`），可见性完全靠紧随其后的 `OLED_Refresh()`（`oled.c:207-211`）。

9.4 差分短块刷新（核心机制）

整屏 1024 字节被切成 32 个块：`refresh_block` 从 0 数到 31，`page = block/4`、`col = (block%4)*32`，即每页 4 个 32 字节块、每块覆盖 32 列 × 8 行（`oled.c:185-187`）。

```
    while (refresh_block < 32) {
        page = refresh_block / 4;
        col = (refresh_block % 4) * 32;
        changed = !sent_valid;
        for (i = 0; i < 32; i++)
            if (sent_gram[page][col + i] != OLED_GRAM[page][col + i]) changed = 1;
        refresh_block++;
        if (!changed) continue;
        cmd[2] = col; cmd[3] = col + 31; cmd[5] = cmd[6] = page;
        data[0] = 0x40;
        for (i = 0; i < 32; i++) data[i + 1] = OLED_GRAM[page][col + i];
```

（`oled.c:185-195`）

```
        for (i = 0; i < 32; i++) sent_gram[page][col + i] = data[i + 1];
        break;
    }
    if (refresh_block == 32) { refresh_busy = 0; sent_valid = 1; }
```

（`oled.c:202-205`）

逐点解释：

- **影子缓存**：`sent_gram[8][128]`（`oled.c:15`）记录屏幕上当前呈现的内容，`OLED_GRAM` 是下一帧要呈现的内容，两者相减即需要上屏的块 —— 纯软件差分，"已发送缓存仅用于比较，不是 OLED 的硬件双缓冲"（`oled.c:132`）。
- **窗口命令**：`cmd[7] = {0x00, 0x21, col, col+31, 0x22, page, page}`（`oled.c:182`、`oled.c:193`）—— `0x21` 设列窗口 `col..col+31`、`0x22` 设页窗口 `page..page`，数据流自然落在这 32 字节矩形里，下次事务重新设窗。
- **数据事务**：`data[33]`（`oled.c:183`），首字节 `0x40` 是数据控制字节，后 32 字节是像素（`oled.c:194-195`）。
- **跳过逻辑**：`changed` 初始值 `!sent_valid`（`oled.c:188`）——影子缓存尚未建立时（上电、恢复后）所有块都算变过、强制整帧重发一次；否则逐字节比较 32 次，全同则 `continue`（`oled.c:189-192`）。
- **一次一块**：发完一个块就 `break`（`oled.c:203`），把"本轮最多发一个块"作为时间上界；下一块留到下一次 `OLED_RefreshStep()`。
- **终止条件**：`refresh_block == 32` 时清 `refresh_busy`、置 `sent_valid=1`（`oled.c:205`）；一轮无变化也会走完 32 次比较后正常结束 —— 静态画面每帧只花 1024 次字节比较、零总线流量。
- **失败即恢复**：两次 `SoftI2C_WriteBytes()` 任一返回非 0（0 = 成功、1 = 失败，`i2c.c:189-206`）就 `need_recovery()` 并 `return`（`oled.c:196-201`）；`refresh_busy` 已被清零（`oled.c:35`），不会卡在"永远刷新中"。

为什么必须分块，源码给了设计约束原文：

```
/* 差分短块刷新：已发送缓存仅用于比较，不是 OLED 的硬件双缓冲。
   每次发一个 32 字节块，主循环可在块间读取 MPU6050。
   刷新期间不要改变 OLED_GRAM：完成后才允许绘制下一帧。 */
```

（`oled.c:132-134`）

配套约束写在头文件里：绘制只改显存，忙时不要重绘显存，运行时用 `StartRefresh/RefreshStep` 分步、初始化与校准才用同步 `OLED_Refresh`（`oled.h:10-15`）。主循环确实按这个约定调度（每轮最多走一步）：

```
            Dashboard_Draw(&view);OLED_StartRefresh();rate_frames++;
            frame_next=now+(view.page==0?LEVEL_DISPLAY_INTERVAL_MS:DISPLAY_INTERVAL_MS);
        }
        if(!DHT11_Busy())OLED_Service(now);
        OLED_RefreshStep();
```

（`main.c:658-662`）

代价也清楚：整帧时间被摊到多轮主循环上，`OLED_RefreshStep()` 通过 `OLED_RefreshBusy()` 反向节流了重绘（`main.c:649` 的 `if(!OLED_RefreshBusy()&&due(now,frame_next))`），所以帧周期只是**上限**而不是承诺值（详见 9.9）。

9.5 失败与恢复：need_recovery() 与 16 步重放

```
static void need_recovery(void)
{
    refresh_busy=sent_valid=0;
    recovery_active=1;recovery_step=recovery_wait=0;
    SoftI2C_BusRecover();
}
```

（`oled.c:33-38`）

触发点有三处：初始化某条命令失败（`oled.c:113`）、刷新块写入失败（`oled.c:199`）、对比度事务失败（`oled.c:169`）。动作可归纳为"三重否认"：

1. `refresh_busy=0` —— 解除刷新忙状态，允许上层重新 `OLED_StartRefresh()`；
2. `sent_valid=0` —— 影子缓存的**可信性**作废，下一轮必须整帧重发（`oled.c:188`）；
3. `SoftI2C_BusRecover()` —— 补 9 个时钟 + 一个 STOP，解开"从机数到一半把 SDA 拉死"的死锁（`i2c.c:49-69`）。

随后由运行期的 `OLED_Service()` 按状态机重放整张初始化表（`oled.c:146-161`）：

阶段	条件	动作	证据
等待	`recovery_wait==0`	置 `recovery_next=now+500`、`recovery_wait=1` 并返回	`oled.c:147-149`
退避	`(int32_t)(now-recovery_next)<0`	直接返回（最多等 500 ms）	`oled.c:150`
发一条	退避到点	发 `init_commands[recovery_step]`，检查 ACK	`oled.c:151`
失败重来	该条失败	`recovery_step=0`、再等 500 ms、再次 `SoftI2C_BusRecover()`	`oled.c:152`
完成	`++recovery_step==16`	清 `recovery_active/recovery_wait`、`contrast_now=0xCF`、`sent_valid=0`、`OLED_StartRefresh()`	`oled.c:155-159`

几个容易忽略的点：

- 成功发出一条后 `recovery_next=now`（`oled.c:154`），所以 16 条命令是**连续**发出的（每次调用一条，无额外间隔）；只有在进入恢复或某条失败时才插入 500 ms 退避。
- 恢复序列包含 `0x81 0xCF`（`oled.c:26`，第 10 条），与 `contrast_now=0xCF`（`oled.c:157`）保持一致 —— 影子值和硬件寄存器都被拉回同一状态，不会出现"软件以为还是暗的、寄存器已经被复位成 0x7F"的错位。
- 恢复期间禁止启动刷新：`OLED_StartRefresh()` 首行就判 `recovery_active`（`oled.c:175`）。

9.6 调光与现实中的"空闲"定义

调光事务在 `OLED_Service()` 的后半段：

```
if(refresh_busy||contrast_now==contrast_target||
    (contrast_wait&&(int32_t)(now-contrast_next)<0))return;
/* Wake in one transaction; dim gently without filling or blanking the screen. */
next=contrast_target>contrast_now?contrast_target:
    (uint8_t)(contrast_now-contrast_target>8?contrast_now-8:contrast_target);
cmd[2]=next;
if(SoftI2C_WriteBytes(OLED_ADDR7,cmd,3)){
    need_recovery();recovery_next=now+500;recovery_wait=1;return;
}
contrast_now=next;contrast_next=now+40;contrast_wait=1;
```

（`oled.c:162-171`）

- 事务是 3 字节：控制字节 `0x00` + `0x81`（设置对比度）+ 新值（`oled.c:145`、`oled.c:167`）。
- **两个方向不对称**（`oled.c:165-166`）：`target>now`（唤醒）时 `next=target` 一次到位；`target<now`（变暗）时每次最多降 8 个计数。
- 节奏由 `contrast_next=now+40`（`oled.c:171`）与 `(int32_t)(now-contrast_next)<0`（`oled.c:163`）共同决定，即**最多 25 次/秒**的对比度写入。`0xCF(207) → 0x28(40)` 差 167，按每次 8 计需 21 次，约 `21×40 ms = 840 ms` 完成一次变暗；反向唤醒是 1 次事务（几十微秒）。
- 刷新期间不发对比度：`refresh_busy` 直接返回（`oled.c:162`），避免与短块数据争总线。
- 语义在头文件里写清了：`OLED_SetIdle(uint8_t idle); /* Request gradual dimming; does not turn off. */`（`oled.h:29`）——`idle` 非 0 时目标是 `0x28`，否则 `0xCF`（`oled.c:138`）。**它是"变暗"不是"熄屏"**，像素仍在点亮，只是驱动电流更低。

上游的"空闲"判据是姿态 + 输入，而不是计时器单打独斗（下列为逻辑摘要，逐字定义见 `main.c:158-164`）：

```
if(input||!allow_idle||abs32(pitch-idle_pitch)>=3||abs32(roll-idle_roll)>=3){
    last_activity=now;idle_pitch=pitch;idle_roll=roll;
}
OLED_SetIdle((uint32_t)(now-last_activity)>=IDLE_DIM_MS);
```

（`main.c:160-163`；阈值 `IDLE_DIM_MS 45000UL` 在 `main.c:24`，`abs32` 在 `main.c:48`）

三个刷新条件：有按键/编码器输入（`input`，来自 `main.c:635` 的 `turn||button||Controls_ButtonDown()`）、该页面不允许变暗（`allow_idle`）、或俯仰/横滚相对上次记录变化 ≥ 3 LSB（`abs32(...)>=3`）。判据是**相对量**：手上没动、姿态漂了 3 LSB 也算有人在看。`allow_idle` 把 STROBE 页、未标定、标定中、有 toast、DHT 报错、历史/系统页遇 `LOG_ERROR/LOG_FOREIGN` 都排除在外（`main.c:635-639`）——错误页永不暗屏，是有意的可读性保障；45 s 无动作才开始变暗（`main.c:24`）。

`now` **回绕是否安全**：安全，用的是与全工程调度原语同一套写法 —— `main.c:49` 定义 `static uint8_t due(uint32_t now, uint32_t deadline) { return (int32_t)(now - deadline) >= 0; }`，而 `oled.c:163` 内联了同样的有符号差比较（`(int32_t)(now-contrast_next)<0`），恢复路径沿用（`oled.c:150`）。只要间隔小于 `2^31 ms ≈ 24.8 天` 就恒正确，40 ms 与 500 ms 远低于此。

9.7 刷新调度与"主循环唯一重绘点"

四个刷新原语分工明确（`oled.c:135`、`173-178`、`179-206`、`207-211`）：

原语	作用	证据
`OLED_RefreshBusy()`	返回 `refresh_busy`，供上层决定能否重绘	`oled.c:135`
`OLED_StartRefresh()`	忙或恢复中直接返回；否则 `refresh_block=0`、`refresh_busy=1`	`oled.c:173-178`
`OLED_RefreshStep()`	推进一个块；`!refresh_busy` 时立即返回	`oled.c:179-184`
`OLED_Refresh()`	同步整帧：`StartRefresh()` + `while(busy) Step()`	`oled.c:207-211`

运行期用异步分步：`main.c:649` 的 `if(!OLED_RefreshBusy()&&due(now,frame_next))` 是主循环里**唯一**的重绘入口，里面 `Dashboard_Draw(&view)` 后立刻 `OLED_StartRefresh()`（`main.c:658`），再由 `OLED_RefreshStep()` 每轮推一块（`main.c:662`）。`Dashboard_Draw()` 自身第一件事是 `OLED_Clear()`（`dashboard.c:324`），即每帧全清 + 全画，差分完全交给 `sent_gram`。

同步整帧只出现在启动/自检阶段：`post_present()` 先等忙清、画 POST 页、启动刷新、再等忙清（`main.c:357-360`），这样 POST 页在进入主循环前一定是完整上屏的。

`!DHT11_Busy()` 门控的实际范围要精确理解：`main.c:661` 与 `main.c:230` 门控的是 `OLED_Service()`（对比度/恢复事务），`OLED_RefreshStep()` 在两道门之外（`main.c:662`、`main.c:231`）。唯一有注释的理由在 MPU6050 那一侧：`main.c:599` 原文"避免 120ms MPU 复位跨越 DHT 的 20ms 起始脉冲"，实际阻塞量在 `mpu6050.c:23-28` 与 `mpu6050.c:46`（`delay_ms(100)` + 两次 `delay_ms(10)` = 120 ms）。OLED 侧同一门控源码无注释，属同类推断（推断）：DHT11 的 20 ms 起始脉冲由主循环在 `deadline=now+20` 处翻状态（`dht11.c:82-89`），响应阶段的脉宽判据窗口很窄（`span<120||span>210`、`span<55||span>150`，`dht11.c:63`、`dht11.c:67`），把可长可短的 I2C 事务（恢复期最长 16 条命令重放）挡在约 30 ms 窗口外是稳妥做法；刷新块的 1.5 ms 是硬上界且显示流畅性要求它不能被推迟，故留在门外。代价可量化：DHT11 每 2500 ms 触发一次（`dht11.c:82`），`DHT11_Busy()` 窗口最长 20+10 ms（`dht11.c:83`、`dht11.c:89`、`dht11.c:105`），即约 1.2% 的时间不允许对比度/恢复事务（推断：30/2500）。

9.8 绘图与字体 API 一览

本驱动的文字有三套字模：自绘 5×7（`small_font`，`oled.c:41-55`）、5×7 放大 2 倍（`OLED_ShowLargeString`，`oled.c:92-103`）、8×16 点阵（`OLED_F8x16`，取用见 `oled.c:334` / `340`，表定义在同目录 `oledfont.h`，该文件未随阅读副本提供故不引行号）。绘图 API：

API	算法/行为	越界与边界	链接状态（`level.map`）
`OLED_DrawPoint(x,y,dot)`	唯一写入口：`page=y>>3`、`bit=y&7`	`x>127||y>63` 丢弃	保留，76 B（`:2219`）
`OLED_DrawRect(x0,y0,x1,y1,dot)`	四条边，循环变量用 `uint16_t` 防死循环	`x1=127` 时不会回绕	保留，84 B（`:2220`）
`OLED_FillRect(x0,y0,x1,y1,dot)`	逐点填充	同 DrawPoint	保留，54 B（`:2221`）
`OLED_DrawLine(x0,y0,x1,y1,dot)`	Bresenham（`oled.c:257`）	端点相等时画 1 点	保留，170 B（`:2218`）
`OLED_DrawCircle(xc,yc,r,dot)`	中点画圆，8 对称点	负坐标由 DrawPoint 丢弃	保留，184 B（`:2216`）
`OLED_DrawDisc(xc,yc,r,dot)`	实心圆：`dx²+dy² ≤ r²+r/2`	同上	保留，80 B（`:2217`）

文本 API：

API	字形	关键行为	链接状态（`level.map`）
`OLED_ShowSmallString(x,y,str)`	5×7，步进 6 px	`x<=122 && y<=57` 才继续（`oled.c:81`）	保留，22 B（`:2229`）
`OLED_ShowSmallStringInverse`	5×7 反白	同一实现 + `inverse=1` 异或（`oled.c:85`）	保留，22 B（`:2230`）
`OLED_ShowLargeString(x,y,str)`	5×7 的 2×2 放大	步进 12 px，`x<=116 && y<=50`（`oled.c:95`）	保留，140 B（`:2228`）
`OLED_ShowChar(x,y,ch)`	8×16	只接受 `' '~'~'`；上半 8 字节 + 下半 8 字节	**已被剔除**，144 B（`level.map:937`）
`OLED_ShowString(x,y,str)`	8×16	`y>48` 截断、`x>120` 换行 `y+=16`（`oled.c:352-353`）	**已被剔除**，60 B（`level.map:941`）
`OLED_ShowNum(x,y,num,digits)`	8×16 定宽	缓冲区 `char buf[12]`，`digits` 限 1~10（`oled.c:368`）	**已被剔除**，80 B（`level.map:939`）
`OLED_ShowSignedNum(x,y,num,digits)`	8×16 定宽带符号	占 `digits+1` 位，负号在前（`oled.c:400-401`）	**已被剔除**，66 B（`level.map:940`）
`OLED_ShowFixed1(x,y,val_x10,int_digits)`	8×16 定点一位小数	纯整数运算，`int_digits` 限 1~6（`oled.c:412`）	**已被剔除**，130 B（`level.map:938`）

刷新与调光 API：

API	作用	链接状态（`level.map`）
`OLED_Init()`	复位软件状态 + 16 条命令 + 清屏 + 整帧	保留，96 B（`:2222`）
`OLED_Clear()`	显存全 0	保留，36 B（`:2215`）
`OLED_RefreshBusy()`	查询忙	保留，6 B（`:2224`）
`OLED_StartRefresh()`	开始一轮差分	保留，30 B（`:2231`）
`OLED_RefreshStep()`	推一个变化块	保留，270 B（`:2225`）
`OLED_Refresh()`	同步整帧	保留，22 B（`:2223`）
`OLED_Service(now)`	对比度渐变 / 恢复重放	保留，290 B（`:2226`）
`OLED_SetIdle(idle)`	设置目标对比度	保留，30 B（`:2227`）

字符映射见 `small_bits()`（`oled.c:57-77`），支持集是"26 个大写字母 + 10 个数字 + 7 个符号"，其余字符返回 0（空白）：

```
    if(col>=5) return 0;
    if(ch>='A' && ch<='Z') return small_font[ch-'A'][col];
    if(ch>='0' && ch<='9') return digits[ch-'0'][col];
    if(ch=='-') return 0x08;
    if(ch=='+') return col==2 ? 0x3E : 0x08;
    if(ch=='.') return col==2 ? 0x60 : 0;
    if(ch==':') return col==2 ? 0x36 : 0;
    if(ch=='/') return (uint8_t)(0x40>>(col*1));
    if(ch=='%') {static const uint8_t b[5]={0x63,0x13,8,0x64,0x63};return b[col];}
    if(ch=='#') {static const uint8_t b[5]={0x14,0x7F,0x14,0x7F,0x14};return b[col];}
    return 0;
```

（`oled.c:66-76`）

也就是说：**小写字母没有映射，会被静默画成空白**（落到 `oled.c:76` 的 `return 0`），空格同理。数字另有 `digits[10][5]` 表（`oled.c:59-65`）。仪表盘与 POST 页文案全用大写（如 `"STM32 SYSTEM POST"`、`"WAIT"`，`main.c:301`、`dashboard.c:150`），当前界面没有踩到这个坑。

9.9 一个 32 字节块的时间预算

软件 I2C 的 SCL 由半周期空转循环决定，`I2C_DELAY_LOOPS=8` 在 72 MHz、`-O0` 下对应"约 250 kHz"，并记录了实测参考："整屏刷新从 85 ms 降到约 37 ms"（`i2c.h:29-36`）。按本项目自己的算式核对：

- 一个变化块的线上流量 = 命令事务 `1(地址)+7(载荷)` + 数据事务 `1(地址)+33(载荷)` = 42 字节；
- I2C 每字节 8 位 + 1 个 ACK = 9 位 → 42×9 = 378 位（START/STOP 另有约 8 位，忽略或按 386 位计）；
- 250 kHz 下 = `378 / 250000 ≈ 1.51 ms` 每块；
- 32 块全推 = `32×378 = 12096` 位 ≈ **48.4 ms**（250 kHz）—— 而 `i2c.h:33` 记录的实测整屏约 37 ms，反推 SCL ≈ `12096/0.037 ≈ 327 kHz`，说明 `i2c.h:30-32` 的"约 250 kHz"是按档位估算的保守值，实测更快（推断）。
- 反过来算每帧能推多少块：33 ms（LEVEL 页，`main.c:23`）内最多 `33ms×250kHz/9 ≈ 916` 字节 ≈ 22.9 块；50 ms（其余页，`main.c:22`）内约 34.7 块。

两条结论：其一，**只有靠差分跳过才能守住帧率** —— 若一帧 32 块全变，上屏约 48 ms，超过 LEVEL 页的 33 ms 周期，此时 `OLED_RefreshBusy()` 会挡掉下一次 `StartRefresh()`（`main.c:649` 的门），帧率自动降到实际能达到的值，而不是丢帧或撕裂；其二，单块 1.51 ms 与 5 ms 采样周期（`main.c:21`）同量级，是主循环时序抖动的主要来源 —— 这正是"一次一块"（`oled.c:203`）的价值。

9.10 本章发现的问题与建议

1. **显存无任何保护，靠调用者纪律维持"刷新期间不改 GRAM"**
   - 现象：`OLED_GRAM` 是裸 `static` 数组（`oled.c:11`），没有互斥、没有忙标志校验、没有双缓冲；`OLED_RefreshStep()` 在 195 行拷贝 `OLED_GRAM`、在 202 行回写 `sent_gram`，期间若有人改 GRAM 就会撕裂。
   - 证据：`oled.c:11`、`oled.c:195`、`oled.c:202`、约束注释 `oled.c:132-134`、`oled.h:10-11`（"屏幕本身没有原子整帧交换"）。
   - 风险：当前唯一绘制者在 `if(!OLED_RefreshBusy()&&due(now,frame_next))` 之内（`main.c:649`），纪律成立；但任何新增的中断内绘制（或未来把 `post_draw` 挂在别处调用）都会静默撕裂，且差分缓存会把撕裂内容当成"已发送"保留下来。
   - 建议：至少在 `OLED_RefreshStep()` 里加"忙时拒绝写入"的软开关（例如 `oled_gram_lock` 计数），或把绘制入口统一收敛到 `Dashboard_Draw()` 并加断言注释；若将来要真双缓冲，把 `sent_gram` 换成"前台缓冲"即可，接口已经具备形态。
2. **8×16 文本 API 整栈是死代码，但 1520 字节字库仍占 Flash**
   - 现象：`OLED_ShowChar/ShowString/ShowNum/ShowSignedNum/ShowFixed1`（`oled.c:325-420`）在本工程中没有任何调用者（全树只在 `oled.c` 内部互相调用，仪表盘统一用 `OLED_ShowSmallString` / `OLED_ShowLargeString`）。
   - 证据：链接器已把它们全部剔除 —— `level.map:937-941` 依次"Removing oled.o(i.OLED_ShowChar), (144 bytes)"…"OLED_ShowString), (60 bytes)"，共 480 字节代码；但 `OLED_F8x16` 仍留在映像里：`level.map:2272 OLED_F8x16 0x080071dc Data 1520`（1520 = 95 字符 × 16 字节）。
   - 风险：1520 字节 Flash 被无用字库占用（该型号 Flash 余量通常宽松，属浪费而非故障）；同时"声明齐全却没被链接"会让读者误判驱动能力。
   - 建议：若要保留 8×16 能力，就把 `oledfont.h` 里的数组改为在 `oled.c` 内 `static`（未被引用时编译器/链接器可整体丢弃），或干脆删掉这 5 个函数与字库；若要启用，至少在一个页面里真实调用一次并验证上半/下半字节的排布（`oled.c:334`、`oled.c:340`）。
3. **整帧重推没有快速通道，全变化帧必然超过 LEVEL 页帧周期**
   - 现象：`sent_valid=0`（上电、恢复完成）后的第一轮刷新是整帧重发（`oled.c:188`），32 块 × 1.51 ms ≈ 48 ms 阻塞量（算式见 9.9）。
   - 证据：`oled.c:188`、`oled.c:185-205`、`main.c:23`（33 ms）、`i2c.h:29-36`（速率与实测参考）。
   - 风险：并非功能故障（差分 + 忙门控保证最终一致），但"恢复完成后第一帧"的呈现延迟约 48 ms，且这段时间内每轮主循环都被占用约 1.5 ms。
   - 建议：`i2c.h:45-50` 已经提供了整帧流式接口（`SoftI2C_Begin/SendByte/End`，注释写明"适合一次事务里连发很多字节"），但它在本构建中同样被剔除（`level.map:933-936`）。若要让恢复后的首帧更快，可用它把 8 页各发一次（8 次事务而不是 32 次），把每块 1 字节地址开销和事务间隔省掉，理论可省 `32-8 = 24` 个地址字节与 24 组 START/STOP。
4. **没有任何屏幕内容自检，只能检测 ACK**
   - 现象：POST 里 OLED 那一步只是一个 ACK 探测：`results[step]=(char *)(SoftI2C_Probe(OLED_ADDR7)?"DONE":"ERR");`（`main.c:396`）；启动重试也只是探测 + `BusRecover` + 1.2 s 等待（`main.c:510-515`）。
   - 证据：`main.c:396`、`main.c:510-516`；驱动从不回读屏幕：`SoftI2C_ReadBytes` 声明存在（`i2c.h:43`）但被链接器剔除（`level.map:935`）。
   - 风险：半屏坏（部分 COM/SEG 驱动失效）、对比度寄存器被复位、电荷泵退化等故障都能通过 POST，表现为"自检全绿但屏幕几乎看不见"；此时软件仍会照常刷新，白耗总线。
   - 建议：SSD1306 支持通过 I2C 回读 GDDRAM（首个字节为哑字节），可考虑在 POST 里做一次"写图案 → 回读比较"的最小自检；若不打算引进读路径，至少在 POST 页明显位置显示当前对比度影子值（`contrast_now`）与最近一次刷新的失败计数，让"看不见"的故障可被读出。
5. **小字体只有 43 个字形，小写字母静默变空白**
   - 现象：`small_bits()` 只处理 `'A'-'Z'`、`'0'-'9'`、`- + . : / % #`，其余（含所有小写字母）统一 `return 0`（`oled.c:67-76`）。
   - 证据：`oled.c:66-76`、字形表 `oled.c:41-55`、数字表 `oled.c:59-65`。
   - 风险：任何新文案写了小写字母或未支持符号（如 `(`、`,`、`_`），会在屏上出现"缺失字符"而不是报错，且大字号用的是同一套映射（`oled.c:97`），放大后更显眼。
   - 建议：在 `small_bits()` 里加一段小写→大写回落（`if(ch>='a'&&ch<='z') ch=(char)(ch-32);`），成本几乎为零；同时在 `OLED_ShowSmallString` 的注释里写明"仅大写 + 数字 + 7 符号"，避免后续踩坑。
6. **一次 3 字节的对比度写失败就判定"设备可能已复位"，代价放大到整帧重推**
   - 现象：对比度事务失败（ACK 丢失或 `i2c_fault`）时直接 `need_recovery()`（`oled.c:168-170`），于是走完 16 条命令重放、`sent_valid=0` 强制整帧重发（`oled.c:155-159`），并让 840 ms 的变暗过程从头再来一次。

- 证据：`oled.c:168-170`、`oled.c:196-201`（同样的处理）、`oled.c:33-38`、`oled.c:155-159`、`i2c.c:185-206`（失败返回 1 的判定面）。
- 风险：一次偶发 ACK 丢失的成本是"约 48 ms 全屏重推 + 复位总线 + 对比度重置"，在高噪声长线场景下会明显放大大屏闪烁。
- 建议：区分两类失败 —— 先用 `SoftI2C_Probe(OLED_ADDR7)` 确认从机是否还在（是否真的掉电/复位），只有探测失败才升级为整套恢复；探测成功则仅重试本次对比度写入。这样设备复位仍能恢复，总线毛刺不会引发全屏重推。

7. **刷新块的 1.5 ms 阻塞仍会落在 DHT11 的握手窗口内**
   - 现象：`OLED_Service()` 被 `!DHT11_Busy()` 门控（`main.c:661`、`main.c:230`），但 `OLED_RefreshStep()` 在同一位置**不在门内**（`main.c:662`、`main.c:231`），其单块事务约 1.51 ms（9.9 算式）。
   - 证据：`main.c:661-662`、`main.c:230-231`、`oled.c:196-197`；DHT 侧窗口与判据 `dht11.c:82-89`（20 ms 起始脉冲 + 10 ms 响应窗口）、`dht11.c:63`/`67`（脉宽容差仅 ±几十微秒）、`dht11.c:105`。
   - 风险：DHT 采样每 2500 ms 一次（`dht11.c:82`），刷新块的阻塞会推迟 `DHT11_Task()` 的状态推进与 `EXTI1_IRQHandler` 读取时间戳的及时性，理论上增加 `rx_done=2`（脉宽判据失败）的概率；实测未见专门统计（`data.edges`/`error` 只显示在 DEBUG 页，`dashboard.c:248`）。
   - 建议：把 `OLED_RefreshStep()` 也纳入 `!DHT11_Busy()` 门控（代价是空闲窗口 30 ms 摊到更多轮，视觉上无感），或在 DHT 窗口内改推"更小的块"（例如 16 字节）以缩短单次占用；同时把 `data.error==2` 的计数打到 DEBUG 页，用数据判断是否值得处理。
8. **API 的坐标加法没有饱和保护，回绕会画到屏上的错误位置**
   - 现象：`OLED_ShowFixed1()` 里 `x + 16 + int_digits * 8` 与 `x + 8 + int_digits * 8` 都先做整型加法再截成 `uint8_t`（`oled.c:417-419`），`OLED_ShowSignedNum()` 的 `x+8` 同理（`oled.c:401`）。例如 `x=120, int_digits=6` 时最后一位算出的列号已超过 255 而回绕，`OLED_DrawPoint()` 的"越界丢弃"（`oled.c:217`）拦不住回绕后的合法小坐标。
   - 证据：`oled.c:401`、`oled.c:417-419`、`oled.c:217`。
   - 风险：低（当前无调用者，属第 2 条的死代码），但同一模式在保留 API 里也存在：`OLED_ShowSmallString()` 的 `x += 6`（`oled.c:87`）靠 `x<=122` 的循环条件保护（`oled.c:81`），一旦有人改动循环条件就会失效。
   - 建议：在 `OLED_ShowFixed1/ShowSignedNum/ShowNum` 里对起始 `x` 加一次上界检查（例如 `if(x>100) return;`），把"回绕后误画"变成"直接不画"，与 `oled.c:217` 的策略保持一致。
9. **"空闲变暗"没有真正的熄屏档，夜间与低功耗场景无出路**
   - 现象：`OLED_SetIdle()` 只有两个目标值 `0x28`/`0xCF`（`oled.c:138`），且头文件明确"does not turn off"（`oled.h:29`），即最暗档仍在持续点亮全部像素；面板也没有运行期可用的关显示态（`0xAE` 只在初始化表首条出现，`oled.c:23`）。
   - 证据：`oled.c:136-142`、`oled.h:29`、`oled.c:23`；上游判定 `main.c:24`、`main.c:158-164`。
   - 风险：45 s 无操作后仅降低对比度（`main.c:24`），功耗下降有限，OLED 老化仍在继续；作为"手电筒式"仪表在暗环境下仍偏亮。
   - 建议：在 `OLED_SetIdle()` 的暗档之外增加一个"关显示"档（发 `0xAE`，唤醒时发 `0xAF` + 恢复对比度），或让长时空闲（例如 ≥ 5 min，`main.c` 里再加一个阈值）直接进入 `0xAE`；注意恢复路径完成时会把 `contrast_now` 拉回 `0xCF`（`oled.c:157`），需要在恢复结束后重新套用空闲目标值，否则会出现"被唤醒的亮屏"。

本章共发现 9 个问题，其中第 2、3、5、6、8 条有明确的改动落点（删/改/加回退），第 1、4、7、9 条属于架构性建议。

第 10 章　人机交互：编码器、按键、RGB、蜂鸣器与板载灯

本章要点

- 编码器 A/B（PB0/PB1）、按钮（PB10）、蜂鸣器（PB8）、板载 LED（PC13）全部低有效；RGB（PA8/PA9/PA10）由 TIM1 以高有效 PWM 驱动（`controls.h:4`、`led.c:3-8`）。
- 编码器不用外部中断，每 1ms 采样 A/B 两比特查 16 项格雷表累加四分之一步，满 4 步记 1 格（`controls.c:170-182`、`controls.h:5`）。
- 按键 20ms 去抖、800ms 长按，即 20 与 800 个 SysTick 计数；短按在**松手**时发出（`controls.c:184-194`、`delay.c:15`）。
- TIM1 用 7.2MHz 计数器时钟与 3600 计数周期产生 2.000kHz 载波，占空比 3600 级；高级定时器必须置 MOE 才有输出（`controls.c:60-71`）。
- 软件只做包络：SysTick 里每 5ms 用 smoothstep 与 gamma≈2 的 Q8 校正算一次三路亮度再写 CCR（`controls.c:85-90`、`controls.c:111`）。
- 蜂鸣器在切换为输出前必须先把 PB8 预置高，避免上电毛刺（`buzzer.c:16-18`、`buzzer.h:6-17`）。

硬件映射与有效电平

五个交互器件分布在三个端口，有效电平并不统一。

信号	引脚	有效电平	GPIO 方向 / 模式	配置源码行
编码器 A	PB0	低有效	输入上拉 `GPIO_Mode_IPU`	`controls.c:150-152`
编码器 B	PB1	低有效	输入上拉 `GPIO_Mode_IPU`	`controls.c:150-152`
按钮	PB10	低有效（按下读 0）	输入上拉 `GPIO_Mode_IPU`	`controls.c:150-152`
R	PA8	高有效 PWM	复用推挽 `GPIO_Mode_AF_PP`	`controls.c:161-162`
G	PA9	高有效 PWM	复用推挽 `GPIO_Mode_AF_PP`	`controls.c:161-162`
B	PA10	高有效 PWM	复用推挽 `GPIO_Mode_AF_PP`	`controls.c:161-162`
蜂鸣器	PB8	低电平触发	推挽输出 50MHz	`buzzer.c:13-18`
板载 LED	PC13	低电平点亮	推挽输出 50MHz	`led.c:16-19`

三个输入引脚只用一次 `GPIO_Init` 配好，代码里**没有**单独写 ODR 开上拉：

```
    gpio.GPIO_Pin = GPIO_Pin_0 | GPIO_Pin_1 | GPIO_Pin_10;
    gpio.GPIO_Mode = GPIO_Mode_IPU;
    gpio.GPIO_Speed = GPIO_Speed_2MHz;
```

（`controls.c:150-151`，`GPIO_Init(GPIOB,&gpio);` 在 `controls.c:152`）

上拉由标准外设库代劳：`stm32f10x_gpio.c:216-218` 对 `GPIO_Mode_IPU` 主动写 `GPIOx->BSRR` 置 ODR 位。这也解释了自检断言 `(GPIOB->ODR&3UL)==3UL` 为什么能成立（`main.c:272`）。

编码器解码：1ms 轮询加一张格雷表

解码全部发生在 `Controls_Tick()`（`controls.c:168-196`）里，它由 SysTick 中断每 1ms 调用一次（`stm32f10x_it.c:136-140`、`delay.c:15`）：

```
    uint8_t ab = (uint8_t)(GPIOB->IDR & 3), key;
    if (ab != old_ab) {
        if ((ab ^ old_ab) == 3) quarter_steps = 0; /* missed/illegal edge */
        else quarter_steps += gray[(old_ab << 2) | ab];
        old_ab = ab;
        if (quarter_steps >= ENCODER_EDGES_PER_STEP) {
            if (turns < 20) turns++;
            quarter_steps = 0;
        } else if (quarter_steps <= -ENCODER_EDGES_PER_STEP) {
            if (turns > -20) turns--;
            quarter_steps = 0;
        }
    }
```

（`controls.c:171-183`；查表常量在 `controls.c:170`：`static const int8_t gray[16] = {0,-1,1,0,1,0,0,-1,-1,0,0,1,0,1,-1,0};`）

这张表以 `(old_ab << 2) | ab` 为下标，把 16 种状态迁移映射成三类结果：单比特变化取 ±1，双比特变化与状态不变都取 0。标准正交编码器顺时针依次经过 0→1→3→2→0，每步得 −1；逆时针经过 0→2→3→1→0，每步得 +1。四个 ±1 累加满 `ENCODER_EDGES_PER_STEP`（`controls.h:5` 定义为 4）才增减一格 `turns`，这就是"4 个四分之一步 = 1 格（一个机械档位）"：一个档位内 A/B 会经历两个边沿共四种组合，按单边沿计数会把同一转速的分辨率放大四倍。

`(ab ^ old_ab) == 3` 这条分支注释写"漏边或非法跳变就清零"，实际作用比注释微妙：双比特跳变时 `gray[]` 本来就返回 0，所以它唯一的效果是**丢弃已累积但不足一格的进度**（`controls.c:172-174`）。1ms 内跨过两个状态必然表现为双比特跳变，漏边必被检出，代价是丢掉 0 到 3 个四分之一步，方向是宁少不多。1ms 采样的不漏边上限是 250 格/秒，远高于人手，纯轮询够用且免去 EXTI 被机械抖动打成中断风暴的风险。

`turns` 被限制在 ±20（`controls.c:176-181`）。这个上限不对应任何"某页的选择范围"：仪表盘共 10 页（`dashboard.h:4`）且翻页取模（`main.c:192-193`），STROBE 只有 5 档（`controls.h:8`），HISTORY 与 SESSIONS 的游标上限是记录条数（`main.c:180-181`）。它的真实作用是限制两次主循环读取之间可能累积的格数，并使 `int8_t` 的 `turns` 值域自洽。

按键：20ms 去抖与 800ms 长按

同一个 1ms 节拍还驱动按键状态机：

```
    key = (GPIOB->IDR & GPIO_Pin_10) != 0;
    if (key != key_last) { key_last = key; key_ticks = 0; }
    else if (key_ticks < 20) key_ticks++;
    if (key_ticks == 20 && key != key_stable) {
        key_stable = key;
        if (!key) { hold_ticks = 0; long_sent = 0; }
        else if (!long_sent) button_event |= CONTROL_CLICK;
    }
    if (!key_stable && !long_sent && ++hold_ticks >= 800) {
        button_event |= CONTROL_HOME; long_sent = 1;
    }
```

（`controls.c:184-194`）

三点必须讲清楚。`key_ticks` 只在电平与上次采样不同时归零且封顶在 20，所以 `key_ticks == 20` 表达的是"该电平已连续保持 20ms 或更久"（`controls.c:185-186`）。按键读电平而非边沿，`key` 为 1 表示松开（低有效），故 `controls.c:189` 的 `if (!key)` 是按下瞬间复位长按计时，`controls.c:190` 的 `else if (!long_sent)` 是松手瞬间发出短按事件——**短按在松手时产生**，按下不放不会立即翻页。长按计时只在当前稳定态是按下且未发过长按时递增，达到 800 发出 `CONTROL_HOME` 并置 `long_sent`（`controls.c:192-194`），保证一次按压只发一次。

于是 20 个计数就是 20ms 去抖、800 个计数就是 800ms，从按下到 `CONTROL_HOME` 是 20 + 800 = 820ms。这依赖 SysTick 被配成整 1ms：

```
    SysTick_Config(SystemCoreClock / 1000UL);
```

（`delay.c:15`；72MHz 下每 72000 个时钟周期一次中断，`SysTick_Handler` 先调 `delay_tick()` 再调 `Controls_Tick()`，`stm32f10x_it.c:136-140`）

`core_cm3.h:1699` 在 `SysTick_Config` 内部把 SysTick 优先级设为最低，其它中断可抢占它。tick 计数不丢，所以 20 与 800 这两个计数的语义精确，最长按压的**墙钟**时长可能略长于 820ms。

跨中断传递：为什么这里必须关中断

`turns` 与 `button_event` 的写者是 SysTick 中断，读者是主循环，两者用一对取值函数交接：

```
    uint32_t mask = __get_PRIMASK(); int8_t result;
    __disable_irq(); result = turns; turns = 0; __set_PRIMASK(mask); return result;
```

（`controls.c:200-201`，`Controls_TakeTurn()`（`controls.c:198-202`）的函数体；`Controls_TakeButton()`（`controls.c:203-207`）在 `controls.c:205-206` 同构，操作对象是 `button_event`）

被保护的不是读或写单个动作，而是"读并且清零"这个**复合操作**。Cortex-M3 上单字节与单字的读写在总线层面原子，所以 `volatile` 足以保证主循环看到新值而非寄存器缓存；但读与清之间若插入一次 SysTick 中断，中断里刚置上的 `CONTROL_CLICK` 会被随后的清零无声吞掉——用户按了一次，界面毫无反应。这才是必须关中断的原因。恢复用 `__set_PRIMASK(mask)` 而非 `__enable_irq()`，调用者自身的屏蔽状态不会被意外打开。

另两个接口只有单次访问，不需要临界区：

```
void Controls_SetLight(uint8_t mode) { light_mode = mode; }
uint8_t Controls_ButtonDown(void) { return !key_stable; }
```

（`controls.c:208`、`controls.c:209`）

后者返回**去抖后的实时按住状态**，无副作用，可直接用于界面提示（`main.c:655`、`dashboard.c:250`）。

TIM1 硬件 PWM：2kHz 载波与 3600 级分辨率

RGB 亮度由 TIM1 的三个比较通道硬件产生（`controls.c:49-72`），初始化每一行都对应一个必须正确的寄存器位：

```
    /* PA8/PA9/PA10 = TIM1 CH1/2/3; PWM1, active high, shadowed duties. */
    TIM1->PSC=timer_clock/(LED_PWM_HZ*LED_PWM_PERIOD)-1;
    TIM1->ARR=LED_PWM_PERIOD-1;
    TIM1->CCR1=TIM1->CCR2=TIM1->CCR3=0;
    TIM1->CCMR1=TIM_CCMR1_OC1PE|TIM_CCMR1_OC1M_1|TIM_CCMR1_OC1M_2|
                TIM_CCMR1_OC2PE|TIM_CCMR1_OC2M_1|TIM_CCMR1_OC2M_2;
    TIM1->CCMR2=TIM_CCMR2_OC3PE|TIM_CCMR2_OC3M_1|TIM_CCMR2_OC3M_2;
    TIM1->CCER=TIM_CCER_CC1E|TIM_CCER_CC2E|TIM_CCER_CC3E;
    TIM1->CR1=TIM_CR1_ARPE;
    TIM1->EGR=TIM_EGR_UG;
    TIM1->SR=0;
    TIM1->BDTR=TIM_BDTR_MOE; /* Advanced timer needs main output enable. */
    TIM1->CR1|=TIM_CR1_CEN;
```

（`controls.c:59-71`）

定时器时钟。 `controls.c:53-55` 取 `RCC_GetClocksFreq()` 后把 `PCLK2_Frequency` 赋给 `timer_clock`，只在 APB2 被分频时才乘 2。本机 APB2 不分频，故 `timer_clock` 就是 72MHz；`controls.c:56-58` 完成 TIM1 时钟使能与复位释放。

预分频与重装载。 常数在 `controls.c:16-17`：`LED_PWM_PERIOD 3600U`、`LED_PWM_HZ 2000U`。代入 `controls.c:60`：

```
PSC = 72,000,000 / (2000 × 3600) − 1 = 72,000,000 / 7,200,000 − 1 = 10 − 1 = 9
```

分频系数 PSC+1 = 10，计数器时钟 = 72MHz / 10 = **7.2MHz**；`ARR = 3600 − 1 = 3599`（`controls.c:61`），一周期 3600 个计数：

```
f_PWM = 7.2MHz / 3600 = 2.000kHz
```

"2kHz 载波、3600 级分辨率"都成立，而且 3600 选得刚好整除（无余数）。反过来说这个常数隐含依赖 72MHz（推断）。

为什么必须有 MOE。 `controls.c:70` 的 `TIM1->BDTR=TIM_BDTR_MOE;` 是最容易漏掉的一行。`BDTR` 的 MOE 位是高级定时器输出链的总闸：MOE 为 0 时即使通道已使能、计数器在跑，OCx 输出也一直停在无效电平，不写就没有波形。

位值与极性。 这些宏不在 `stm32f10x_tim.h`，而在 CMSIS 设备头 `stm32f10x.h` 的各寄存器位定义区（`stm32f10x.h:4191-4456` 之间，如 `TIM_CCMR1_OC1PE` 在 `stm32f10x.h:4299`、`TIM_BDTR_MOE` 在 `stm32f10x.h:4456`）。手算 `controls.c:63-66` 的写入值：CCMR1 为 0x6868、CCMR2 为 0x0068、CCER 为 0x0111，稍后会被启动自检原样比对（`main.c:280`、`main.c:281`、`main.c:279`）。四个 `OCxPE` 位让 CCR 在下个更新事件整体生效；`ARPE`（`controls.c:67`）同样保护 ARR。`EGR = TIM_EGR_UG`（`controls.c:68`）手动产生一次更新事件把预装载值立刻装入影子寄存器。`CCER` 只写三个使能位、`CCxP` 保持 0（`controls.c:66`），配合 PWM 模式 1 即注释所说 `active high`：计数值小于 CCR 时输出高；`controls.c:62` 先把三个 CCR 清零，此时三路全低，CCR=3600 等于 ARR+1 则恒高。配置全部在计数器停止时完成，最后才置 CEN 启动（`controls.c:71`）。三个引脚在 `controls.c:161-162` 被复用推挽接管，这里有个与自检有关的细节留到章末讨论。

软件包络：200Hz 的呼吸与 gamma 校正

硬件负责载波，软件只负责这一瞬间三路各多亮（`controls.c:91-144`）。亮度用 Q8 定点表示（`controls.c:10` 注释 `/* Perceived intensity, Q8. */`），满量程 255×256 = 65280；到占空比的换算做了一次平方映射：

```
uint32_t q=(uint32_t)perceived*1024U/(255U*256U); return q*q*LED_PWM_PERIOD/1048576UL;
```

（`controls.c:85-90` 的 `pwm_duty()`，上方注释是 `/* Perceived (Q8) -> CCR, gamma ~2. */`）

`q` 是归一化到 0..1024 的感知亮度，输出正比于 `q²` 即 gamma≈2，用来补偿人眼接近平方律的感知。呼吸与淡变两条曲线也都是纯整数运算：

```
uint32_t q=(uint32_t)value*1024/span; return q*q*(3072-2*q)>>20;
```

（`controls.c:75-79` 的 `ease()`，smoothstep，输出 0..1024，展开即 1024×(3q²−2q³)）

```
half=period/2,position=tick<half?tick:period-tick; return 14*256+(peak-14)*256*ease(position,half)/1024;
```

（`controls.c:80-84` 的 `breath()`）

`breath()` 取三角波位置再套 smoothstep，两端慢中间快；底数 14×256 表示最暗也有 14/255 的感知亮度，灯在谷底不完全熄灭，避免视觉上的闪断。

这三条计算在 SysTick 上下文里，但不是每个 tick 都算——`light_divider` 每 5 次中断归零一次，包络计算与 CCR 写入因此是 200Hz：

```
if(++light_divider<5)return; /* 200Hz envelope, PWM itself runs in hardware. */
```

（`controls.c:111`，下一行 `controls.c:112` 是 `light_divider=0;`）

要注意 `controls.c:104-110` 的 `fade_ticks`、`light_ticks`、`error_ticks`、`strobe_ticks` 仍按 **1ms** 递增（单位就是毫秒，`light_ticks` 的 %3000 是 3 秒呼吸周期，`error_ticks` 的 %1600 是 1.6 秒）。模式切换后 300ms 内用 `ease()` 做淡入，之后 OFFSET 模式改走 60ms 一档的平滑逼近（`controls.c:132-140`，其中 `controls.c:137` 是 OFFSET 分支）。

六种灯光模式与五档频闪

`light_mode` 由主循环每轮按下述优先级唯一确定（`main.c:642-648`），SysTick 只负责渲染。

模式（controls.h:7）	触发条件	表现	源码行
`LIGHT_STROBE`	当前页是 STROBE	按图案表闪红/蓝	`main.c:642`
`LIGHT_SAVE`	`Logger_State()==LOG_WRITING`	静止的绿+蓝	`main.c:643`
`LIGHT_ERROR`	水平页未标定/数据陈旧、CLIMATE 页 DHT11 报错、或存储页处于 ERROR/FOREIGN/OFFLINE	红色呼吸（1.6 秒）	`main.c:644-645`
`LIGHT_OFFSET`	水平页标定有效	按倾角在红/绿间变色	`main.c:646`
`LIGHT_LEVEL`	水平已锁定	绿色呼吸（3 秒）	`main.c:647`
`LIGHT_IDLE`	其余情况	蓝色呼吸（3 秒）	`main.c:647`

`LIGHT_OFFSET` 是唯一用颜色编码数值的模式：径向角距离由 `Controls_SetLevelOffset()`（`controls.h:15`，全工程唯一调用点在 `main.c:590`）算出，单位 0.1°、上限 150 即 15.0°（`controls.h:16`），`controls.c:129-130` 让距离越大越红（径向角用逐位整数平方根求，全程无浮点，`controls.c:210-219`）。

频闪是唯一绕过包络直接写 CCR 的模式：

```
        color=strobe_color(style,strobe_ticks);
        light_value[0]=color==1?255U*256U:0;
        light_value[1]=0;
        light_value[2]=color==2?255U*256U:0;
        TIM1->CCR1=color==1?LED_PWM_PERIOD:0;
        TIM1->CCR2=0;
        TIM1->CCR3=color==2?LED_PWM_PERIOD:0;
        return; /* Do not soften individual flashes with the normal color fade. */
```

（`controls.c:113-121`）

`controls.c:121` 的注释说明了动机：单次闪光只有 40 到 70ms（图案表 `controls.c:21-38`，取色函数 `controls.c:39-47`），若走 300ms 淡变，每次闪光都会被拉成缓坡，看起来是渐变而不是闪。所以这里直接给 CCR 写满值或 0，同时把 `light_value[]` 也设成对应满值，避免退出频闪时从错误起点插值。

档位（controls.h:8）	周期	步数	图案特征	源码行
STROBE_OFF	1	0	全灭（count=0，查表恒返回 0）	controls.c:36
STROBE_ALT	400ms	4	红蓝交替单闪	controls.c:23
STROBE_DOUBLE	800ms	8	每个颜色连打两下	controls.c:24-26
STROBE_TRIPLE	960ms	12	每个颜色连打三下	controls.c:27-30
STROBE_CHASE	1000ms	17	红蓝快速交替后停顿	controls.c:31-34

周期与步数可由图案表逐项相加验证（ALT 为 160+40+160+40 = 400ms，DOUBLE 的两组 70+70+70+190 合计 800ms），它们同时是 controls.c:110 的重启条件：++strobe_ticks >= period 时计时器归零，图案从头播。切换档位时 strobe_active 被置为 255（controls.c:157、controls.c:102），下一 tick 必定命中他人分支把 strobe_ticks 清零。档位只存在 RAM 里，掉电不保留，默认 STROBE_DOUBLE（controls.c:157）；它在 STROBE 页用编码器选择（main.c:176-181），图案名由 Controls_StrobeName()（controls.c:222-225）取出并显示（dashboard.c:257）。

蜂鸣器：低电平触发与上电毛刺

有源蜂鸣器接 PB8，低电平触发（buzzer.c:4-5），初始化（buzzer.c:7-21）的顺序藏着这个模块唯一的关键点：

```
    /* 先预置高电平，再切换输出，避免低电平毛刺。 */
    GPIO_SetBits(BUZZER_PORT, BUZZER_PIN);
    GPIO_Init(BUZZER_PORT, &GPIO_InitStructure);
```

（buzzer.c:16-18，GPIO_InitStructure 在 buzzer.c:13-15 配成推挽输出 50MHz）

先写 ODR 再切输出。GPIO 复位后 ODR 为 0 即低电平，若先执行 GPIO_Init 把引脚切成推挽输出，引脚会立刻被驱动到低电平，蜂鸣器随即响一声。buzzer.h:6-17 从硬件角度讲得更完整：低电平触发的模块在 MCU 把 PB8 拉高之前引脚是悬空的，上电瞬间可能响一声，所以 main() 里第一件事就是 Buzzer_Init()（main.c:508）。

发声有两套接口：

```
void Buzzer_Beep(uint16_t ms) {
    if (ms == 0) return;
    Buzzer_On();
    delay_ms(ms);
    Buzzer_Off();
}
```

（buzzer.c:33-39，其中 delay_ms(ms); 在 buzzer.c:37）

Buzzer_Beep() 内部用 delay_ms() **阻塞**，是全工程罕见的一处阻塞调用：delay_ms() 本身忙等（delay.c:29-33），调用期间主循环完全停摆。它唯一的调用点是开机提示音 Buzzer_Beep(BOOT_BEEP_MS)（main.c:508，BOOT_BEEP_MS 为 60，main.c:35），此时 OLED 还没点亮，阻塞 60ms 没有代价。进入主循环后再无阻塞调用，水平锁定提示音走非阻塞接口 beep_start()（main.c:76-80）：只开声并记下截止时刻，关声由主循环每轮检查 if(beep_active&&due(now,beep_until)){Buzzer_Off();beep_active=0;}（main.c:538）。水平进入窗口时调用它响 100ms（BEEP_MS 为 100，main.c:34，调用点在 main.c:95 附近）。

板载 LED：心跳、报码与三方仲裁

板载 LED 接 PC13、低电平点亮（led.c:3-8），接口只有三个原型（led.h），实现为 LED_Init()（led.c:10-22，推挽 50MHz 且第一件事就熄灭，led.c:21）、LED_On() 拉低（led.c:26）与 LED_Off() 拉高（led.c:31）。

运行期它承担心跳：每 1000ms 亮 30ms，3% 占空比，由 main.c:540 的开灯与 main.c:539 的关灯两个到期检查收尾，与蜂鸣器共用 due() 截止时间模式（main.c:49）。

led.c:3-8 的注释还声称它把总线扫描的结果用闪灯次数报出来。核对实现后需要修正：**报码机制确实存在，但触发条件与报告对象同注释描述并不一致**，它只出现在 OLED 重试循环里：

```
            while(!oled_ok){
                uint8_t i;
                for(i=0;i<(mpu_ok?3:5);i++){LED_On();delay_ms(80);LED_Off();delay_ms(120);}
                delay_ms(1200);
```

（main.c:511-513）

即只有当 OLED 探测失败、屏幕无法显示任何信息时，程序才退回到用 LED 报码：闪 3 次表示 MPU6050 探测成功、闪 5 次表示失败，每闪 80ms 亮加 120ms 灭，然后静默 1200ms 重试。它不报告 W25QXX 或 DHT11 的结果——那些走 OLED 上的启动自检页面（main.c:461）。工程里还有第三处闪灯报码：时基自检失败时进入不返回的死循环交替闪灯，亮灭时长由空循环计数决定（main.c:518-521）。

主循环如何消费这些输入

主循环每轮取值，取值本身清零，一轮内不会重复处理同一批输入（main.c:541）：

```
        turn=Controls_TakeTurn();button=Controls_TakeButton();
```

- turn 非零时交给 navigate_turn() 翻页或移动浏览游标，并强制刷新一帧（main.c:545-548）。
- button & CONTROL_HOME 时复位到首页、退出所有浏览态并弹出 HOME 提示 1.5 秒（main.c:542-544）。
- button & CONTROL_CLICK 时按当前页分派：STROBE / SESSIONS / HISTORY 页进入或切换浏览态（无记录时提示 NO SESSIONS、NO RECORDS），其它页调用 save_snapshot() 并按结果提示 SAVING / BUSY / NO FLASH / LOCKED / NOT READY（main.c:549-566）。

启动自检期间也有一次取值，意义相反——**丢弃**：`main.c:484` 的 `(void)Controls_TakeTurn();(void)Controls_TakeButton();`，位于 `post_startup()` 末尾（该函数体覆盖 `main.c:363-486`）。自检期间用户可能已在拧编码器，把这段输入留在队列里会让仪表一进主循环就疯狂翻页，所以主动清空。

主循环还每轮重定灯光模式（`main.c:642-648`），并把按键状态反映到界面：`view.toast=Controls_ButtonDown()?"KEY DOWN":toast;`（`main.c:655`）。DEBUG 页则把两个编码器比特和去抖后的按键状态直接显示出来（`dashboard.c:250`）。

本章发现的问题与建议

（一）RGB 引脚的启动自检是确定性假阴性，POST 第 10 项必然显示 ERR。

`controls.c:151` 把 GPIO 结构体的速度设为 `GPIO_Speed_2MHz`（值为 2，`stm32f10x_gpio.h:59-61`），随后 `controls.c:161-162` 复用同一结构体配置 RGB，只改了 `GPIO_Pin` 与 `GPIO_Mode`：

```
gpio.GPIO_Pin = GPIO_Pin_8 | GPIO_Pin_9 | GPIO_Pin_10; gpio.GPIO_Mode = GPIO_Mode_AF_PP; GPIO_Init(GPIOA, &gpio);
```

标准外设库在输出模式下把 `GPIO_Speed` 写进 CRH 的 MODE 位（`stm32f10x_gpio.c:182-189`），`GPIO_Mode_AF_PP` 基值是 0x08（`stm32f10x_gpio.h:78`），所以 PA8/PA9/PA10 三个半字节实际是 `0x08 | 0x02 = 0xA`，`GPIOA->CRH & 0x0FFF` 等于 **0xAAA**；而自检要求 **0xBBB**（`main.c:278`），于是 POST 第 10 项（名字表下标 9，`main.c:289`）显示 **ERR**（`main.c:461`）。RGB 功能本身不受影响——2MHz 翻转率对 2kHz 方波（半周期 250µs）绰绰有余。同一工程里还有反向证据：蜂鸣器明确设了 `GPIO_Speed_50MHz`（`buzzer.c:15`），所以 `(GPIOB->CRH&0x0FUL)==0x03UL` 的校验（`main.c:283`）能通过。建议在 `controls.c:162` 前补一行 `gpio.GPIO_Speed = GPIO_Speed_50MHz;`，或把 `main.c:278` 的期望值改成 0xAAA；前者与蜂鸣器写法一致。

（二）编码器快速旋转时的漏边靠清零兜底，代价是丢进度且不报错。

`controls.c:173` 能检出漏边，但处理方式是丢弃已累积的 0 到 3 个四分之一步，用户感受是转快了少翻一页。上限 250 格/秒远高于手速，所以不是现实故障，但它是一个静默失效：界面不会有任何提示。若要改进，可在清零分支累加一个溢出计数器并在 DEBUG 页显示（`dashboard.c:250` 是现成落点）。

（三）长按只触发一次，无法用于连续调参。

`controls.c:192-193` 发过一次 `CONTROL_HOME` 并置 `long_sent = 1` 后，即使一直按着也不再发；`controls.c:190` 又让长按后的松手不补发短按。这杜绝了长按之后又触发一次翻页的双触发，但长按只能表达一个瞬时命令。若将来需要按住连续加减，要在 `long_sent` 置位后引入重复触发间隔（例如每 200ms 一次），并明确松手时是否补发短按。

（四）整个包络运算都在中断上下文里，但中断占用不是瓶颈。

`Controls_Tick()` 在 SysTick 中断里调用（`stm32f10x_it.c:139`），其中 `light_tick()` 每次做两个 16 位取模（`controls.c:105-106`）与频闪计时比较，每第 5 次还算 `ease()`、`breath()` 与三路 `pwm_duty()`（`controls.c:141-143`）。按指令数估算（推断）：1ms 公共路径约 40 到 80 个周期，200Hz 包络路径再加约 150 到 250 个周期，平均约 100 到 130 个周期/ms；72MHz 下每毫秒有 72,000 个周期，占比约 **0.14% 到 0.18%**。结论是不需要优化：把浮点全换成整数定点（`controls.c:75-90`）已经解决了真正的问题。

（五）蜂鸣器同时存在阻塞与非阻塞两套 API，边界靠约定维持。

`Buzzer_Beep()`（`buzzer.c:33-39`）阻塞，`beep_start()`（`main.c:76-80`）非阻塞，二者的分工没写在任何头文件里，只能靠当前是否处于启动阶段这一隐含约定判断。`buzzer.h` 只暴露前者，后者是 `main.c` 里的静态函数，其它模块物理上无法误用非阻塞版本，也无法复用它。若要收敛，可把 `beep_start()` 的模式提到 `buzzer.c`，让 `Buzzer_Beep()` 只留给启动阶段并加注释说明。

（六）三路 PWM 没有互斥，但也不需要——真正的共享变量是模式字。

`TIM1->CCR1/CCR2/CCR3` 的写者只有两处：频闪分支（`controls.c:118-120`）与包络输出（`controls.c:141-143`），都在 `light_tick()` 内、都在 SysTick 上下文，不存在两个写者并发覆盖。真正跨上下文共享的是 `light_mode`（`controls.c:208` 写、`controls.c:93` 读）与 `strobe_mode`（`controls.c:221` 写、`controls.c:93` 读）。单字节读写在中断边界上原子，最坏是某一个 5ms 包络周期用了切换前的旧模式，视觉上完全不可辨；`controls.c:97-103` 在模式变化时立即记录 `fade_from[]`，保证下一帧的过渡也从当前亮度出发，不会跳变。

（七）两处接口健壮性不一致，另有一个无人调用的死接口。

`Controls_SetLight()`（`controls.c:208`）对模式值不做任何校验，越界值会落到 `controls.c:131` 的 `else` 分支被当作 `LIGHT_IDLE` 渲染成蓝呼吸；同族的 `Controls_SetStrobe()` 有明确兜底（`controls.c:221`）。此外 `Controls_LevelDistance()` 是**未实现调用方的死接口**：它在 `controls.c:220` 定义、在 `controls.h:16` 声明，但全工程没有任何调用点——径向角距离目前只在 `Controls_SetLevelOffset()` 内部使用。建议给 `Controls_SetLight()` 补同样的范围判断，并决定 `Controls_LevelDistance()` 是删除还是接入界面。

第 11 章　专题一：总线共享与 32 字节差分块刷新

本章要点

- 整条 I2C 总线**不存在真并发**：全工程只有主循环一个执行流会碰 `SoftI2C_*`，`stm32f10x_it.c` 全文没有一处 `SoftI2C`、`GPIO_SetBits`/`GPIO_ResetBits`（已逐行核对），唯一的 I2C 中断源不存在。总线共享因此退化为"同一轮主循环内的调用顺序"问题（`main.c:531-663`）。
- 三个参与者：MPU6050 每 **5ms** 做一次 14 字节突发读（`main.c:21`、`mpu6050.c:62`）；OLED 每次 `OLED_RefreshStep()` **最多发出一个 32 字节块**（`oled.c:179-206`，`oled.c:203` 的 `break` 是硬证据）；DHT11 稳态每 **2.5s** 触发一次（**不是 2s**，见 11.2 的纠正），独占总线最长 **30ms**（`dht11.c:82`、`dht11.c:89`、`dht11.c:105`）。
- 全工程共 **4 处** `!DHT11_Busy()` 互锁：`main.c:661`、`main.c:230`、`main.c:600`、`main.c:628`。
- 按源码自带的两个实测锚点（110kHz→85ms、250kHz→37ms，`i2c.h:29-33`）反推：当前 `I2C_DELAY_LOOPS=8` 下**每字节约 28.9μs、每个 32 字节块约 1.16ms、整屏 32 块约 37ms**，等效位速率约 312kHz。
- 一个 32 字节块 = **2 次事务共 40 字节**（7 字节命令 + 33 字节数据，`oled.c:182-197`）；块宽减半到 16 字节会让总字节数从 1280 涨到 1536（**+20%**）、事务数翻倍，这是"为什么是 32 而不是 16"的量化答案。
- 互锁装错了对象：被 `!DHT11_Busy()` 保护的是**不超过 3 字节**的对比度事务，而真正长占用（1.16ms/块）的 `OLED_RefreshStep()` 在 `main.c:662` **完全没有门控**。
- `i2c_fault` 是"单次事务内短路"而非永久锁存：`i2c.c:85` 在每次 `i2c_start()` 里无条件清零，`i2c.c:115`/`i2c.c:146` 只在本事务内提前返回。

11.1 拓扑与电气前提

总线的电气与位级实现属于第 3 章，这里只固定本章要用到的四个事实：

项目	取值	出处
引脚	SCL = PB6，SDA = PB7	`i2c.h:24-26`
输出模式	开漏 `GPIO_Mode_Out_OD`	`i2c.c:41`
上拉	两块模块各带 4.7k，并联后约 2.35k	`i2c.h:13-14`
半周期延时	`I2C_DELAY_LOOPS = 8`	`i2c.h:36`
从机 7 位地址	OLED `0x3C`、MPU6050 `0x68`	`i2c.h:18-21`

开漏模式下 STM32 的输入通道仍然有效，所以读 SDA 不需要切换方向寄存器，直接读 `IDR`：

```
#define SDA_READ()      ((I2C_PORT->IDR & I2C_SDA_PIN) != 0)
```

（`i2c.c:8`）这条宏是后面所有"边发边收"逻辑的基础：MPU6050 的 14 字节突发读、OLED 块写失败后的回读，都依赖它。

11.2 三个参与者的真实周期

把三个参与者的周期与单次总线占用放在一张表里，是本章后面所有量化的骨架：

参与者	触发点	周期	单次总线占用
MPU6050 采样	`main.c:570` `if(due(now,sample_next))`	5ms（`main.c:21`）	1 次事务：地址 + 寄存器 + 14 字节数据 = 16 字节
OLED 短块刷新	`main.c:662`，每轮主循环	无固定周期	最多 1 块 = 2 次事务共 40 字节
DHT11	`dht11.c:76` `if(!state && ...)`	**2.5s**	独占 20ms 起始脉冲 + ≤10ms 应答窗

纠正一处常见误述：DHT11 不是每 2s 一次。 首次读确实排在 2s 之后（`dht11.c:50` `next_read=now+2000;`），但每次触发后重新装载的是 2500ms：

```
        deadline=now+20; next_read=now+2500; state=1;
```

（`dht11.c:82`）所以稳态周期是 **2.5s**，2s 只出现在上电后的第一次。这一点会直接影响第 12 章"一条 ALIVE 心跳对应几次环境采样"的换算，故在此固定。

同样是这条语句给出了 20ms：`deadline=now+20`，即 state 1 期间把 PA1 拉低 20ms。但**独占期不止 20ms**：

```
        deadline=now+10; state=2;
```

（`dht11.c:89`）state 2 要等 `rx_done`（42 个边沿收齐）或最多 10ms 才退出。而 `DHT11_Busy()` 的判据是：

```
uint8_t DHT11_Busy(void) { return state != 0; }
```

（`dht11.c:105`）state 1 与 state 2 都非 0，所以 **`DHT11_Busy()` 的实际时长 = 20ms + 帧接收时间（典型约 4ms，上限 10ms）= 典型约 24ms、最坏 30ms**。全工程所有互锁判的都是这 30ms 上界。

11.3 三道（实为四道）互锁

#	位置	被保护的操作	门控条件
1	`main.c:661`	`OLED_Service(now)`（对比度/恢复，≤3 字节）	`!DHT11_Busy()`
2	`main.c:230`	`post_pump()` 内的 `OLED_Service(now)`	`!DHT11_Busy()`
3	`main.c:600`	`SoftI2C_BusRecover()` + `MPU6050_Init()`	`due(now,retry_next) && !DHT11_Busy()`

#	位置	被保护的操作	门控条件
4	`main.c:628`	`Logger_Init(now)`（Flash 重新初始化）	`due(now,flash_retry_next) && !DHT11_Busy()`

第 3 道的理由是源码自己写明的，也是四道里**唯一机制清晰**的一道：

```
            /* 避免 120ms MPU 复位跨越 DHT 的 20ms 起始脉冲。 */
            if(due(now,retry_next)&&!DHT11_Busy()){
                SoftI2C_BusRecover();sensor_ready=MPU6050_Init()==0;retry_next=delay_millis()+1000;
            }
```

（`main.c:599-602`）"120ms"可以逐条对上 `MPU6050_Init()` 内部的三个阻塞延时：复位后 `delay_ms(100)`（`mpu6050.c:24`）、唤醒后 `delay_ms(10)`（`mpu6050.c:28`）、配置完成后 `delay_ms(10)`（`mpu6050.c:46`），合计 120ms。

机制是：`DHT11_Task(now)` 只在 `main.c:532` 每轮调用一次。如果 state 1 刚把 PA1 拉低，紧接着主循环进入一个 120ms 的阻塞调用，那么 state 1 → state 2 的迁移就会晚 120ms 才执行，**PA1 会被低电平压住 120ms 而不是 20ms**，DHT11 因此复位并放弃本次应答。第 4 道的 Flash 重新初始化同理（`Logger_Init` 内部会擦除与扫描）。

问题在于第 1、2 道。 它们保护的 `OLED_Service()` 最长只发 3 字节（恢复态发 `init_commands[step]`，`init_sizes[step]` 只有 2 或 3，`oled.c:151`；对比度态发 `cmd[3]`，`oled.c:168`），按 11.4 的换算不到 90µs，内部也没有任何阻塞延时，**不可能吞掉 20ms 起始脉冲**。与此同时，真正会连续占用总线 1.16ms 的 `OLED_RefreshStep()` 却是无条件执行的：

```
        if(!DHT11_Busy())OLED_Service(now);
        OLED_RefreshStep();
```

（`main.c:661-662`）`OLED_RefreshStep()` 在 `oled.c:184` 只有 `if (!refresh_busy) return;` 一道自检，不看 DHT11 状态。也就是说：**门控加在了短事务上，长事务没有被门控。** 这条不对称列入章末问题第 2 条。

11.4 量化：从两个实测锚点到每个块

`i2c.h` 给了三个实测速度点，并额外给了两个整屏刷新耗时，这五个数字互相校验：

I2C_DELAY_LOOPS	SCL 实测	整屏刷新
20	约 110kHz	85ms
8（当前）	约 250kHz	约 37ms
5	约 380kHz	—

（`i2c.h:28-35`）整屏的字节数是可数的：一个块 = 7 字节命令事务 + 33 字节数据事务 = **40 字节**，共 **8 页 × 4 块 = 32 块**（`oled.c:185-187` 的 `refresh_block/4` 与 `%4`），所以

```
整屏字节数 = 32 块 × (7 + 33) 字节 = 1280 字节
```

用两个锚点反推每字节耗时：

```
110kHz: 85ms   / 1280 B = 66.4 µs/字节
250kHz: 37ms   / 1280 B = 28.9 µs/字节   → 每块 40 B × 28.9 µs ≈ 1.16 ms
```

自洽性校验：`66.4 / 28.9 = 2.30`，而标称频率之比 `250 / 110 = 2.27`，两者差 1.3%，在"约"的精度内吻合，说明这两个耗时确实是同一条总线、同一种刷屏路径测出来的，可以直接用。

由此得到本章需要的四个数：

量	推导	结果
位周期	28.9µs ÷ 9（8 数据位 + 1 应答位）	≈ 3.21µs，等效 ≈ 312kHz
一个 32 字节块	40 B × 28.9µs	≈ **1.16ms**
一次 MPU6050 突发读	16 B × 28.9µs	≈ 0.46ms
整屏 32 块	32 × 1.16ms	≈ **37ms**（与 `i2c.h:33` 一致）

两个必须说明的偏差：

1. 标称 250kHz 推得的位周期应是 4.0µs，但由 37ms 反推是 3.21µs。原因是"250kHz"是按"每减小 1 约快 15kHz"的线性经验外推出来的**标称值**，而 37ms 是比值缩放的实测值；两者相差约 25%，本章一律以上表反推的 28.9µs/字节为准（推断）。
2. 32 块里只有内容发生变化的块才会真正发送（`oled.c:188-192` 先比较 `sent_gram` 再 `continue`），所以 37ms 是**全屏都变**的上界；静止画面下每轮主循环常常只发 0 块。

5ms 采样周期内能塞下几块?

```
5ms 窗口预算                  5000 µs
减去一次 MPU6050 读           5000 - 460 = 4540 µs
除以每块耗时                  4540 / 1160 ≈ 3.9 块
```

即**理论上限约 3 到 4 块/5ms**。但每轮主循环只发一块（`oled.c:203` 的 `break`），所以这个上限在真实固件里达不到，实际吞吐由主循环周期决定，而不是由总线速度决定——这一点在第 6 章的主循环预算里已经量化。

DHT11 独占期挤掉多少?

```
20ms 起始脉冲  →  20000 / 1160 ≈ 17 块
30ms 独占上界  →  30000 / 1160 ≈ 25 块
```

每 2.5s 一次，占空比 `30ms / 2500ms ≈ 1.2%`，对刷屏的影响可以忽略；真正需要注意的是这 30ms 里 `OLED_Service()` 被完全跳过，对比度渐变（步进间隔 40ms，`oled.c:171`）最多被推迟 30ms——不构成可见问题，但它是"45s 息屏"计时被拉长的一个微小来源。

11.5 为什么是 32 字节而不是 16 字节

块的大小由 SSD1306 的列地址范围命令决定。命令模板是：

```
    uint8_t cmd[7] = {0x00, 0x21, 0, 0, 0x22, 0, 0};
    uint8_t data[33];
```

（`oled.c:182-183`） `0x00` 是控制字节（后面跟命令流）， `0x21` 设置列地址范围、 `0x22` 设置页地址范围。每次发送前填三个字节：

```
        cmd[2] = col; cmd[3] = col + 31; cmd[5] = cmd[6] = page;
```

（`oled.c:193`） `col` 与 `col+31` 正好跨 32 列；数据侧的 `0x40` 是"后面跟数据流"的控制字节：

```
        data[0] = 0x40;
        for (i = 0; i < 32; i++) data[i + 1] = OLED_GRAM[page][col + i];
```

（`oled.c:194-195`）SSD1306 在收到列地址范围后会**自动递增列指针**，所以一次事务连发 32 字节就等价于 32 次单字节写，命令开销被摊薄了 32 倍。

把块宽改成 16 字节，代价可以直接算出来：

块宽	块数（8 页 × 128/块宽）	每块字节	总字节	相对 32 字节
32（当前）	32	7 + 33 = 40	1280	基准
16	64	7 + 17 = 24	1536	**+20%**
64	16	7 + 65 = 72	1152	−10%

16 字节块的 7 字节命令要发 64 次，占总量 `64 × 7 / 1536 = 29%`；32 字节块只占 `32 × 7 / 1280 = 17.5%`。同时 16 字节方案的事务数从 64 次（32 命令 + 32 数据）涨到 128 次，每次事务都要付出起始/停止条件的固定代价（每次约 4 个位周期）。**这就是选 32 的量化理由：在"块间要能被 5ms 采样打断"这一约束下（`oled.c:132-133` 的注释写明了这个意图），32 字节是命令开销与块粒度的折中点，而 8 字节或 16 字节会让开销显著上升。**反过来，64 字节块虽然总字节更少，但单块耗时涨到约 2.1ms，块间响应变粗，且 128/64 = 2 不能整除页内 128 列之外没有别的限制，实际上仍然可行——所以 32 的最终选择也包含了"块宽整除 128 且块间延迟不大于半个采样周期"的工程权衡（推断）。

块编号到页/列的映射是无除法的位运算：

```
        page = refresh_block / 4;
        col = (refresh_block % 4) * 32;
```

（`oled.c:186-187`） `/4` 与 `%4` 编译器会各自折成一次移位和一次掩码。

11.6 故障路径： `need_recovery()` 、 `SoftI2C_BusRecover()` 与 `i2c_fault`

触发点共三处，全部汇聚到 `need_recovery()`：

触发场景	调用点	后果
上电初始化命令失败	`oled.c:113`	`break` 跳出初始化循环，剩余命令不再发
对比度命令失败	`oled.c:169`	进入恢复态，并额外设置 `recovery_next=now+500`
32 字节块写失败	`oled.c:199`	立刻返回，当前块不写入 `sent_gram`

`need_recovery()` 本体只有四行：

```
static void need_recovery(void)
{
    refresh_busy=sent_valid=0;
    recovery_active=1;recovery_step=recovery_wait=0;
    SoftI2C_BusRecover();
}
```

（`oled.c:33-38`）它做三件事：中止当前刷新、把"已发送缓存"标记为无效（`sent_valid=0` 会让下一次刷新把 32 块全部重发， `oled.c:188`）、启动恢复状态机。

`SoftI2C_BusRecover()`（`i2c.c:55-81`）能救回什么、不能救回什么：

它先释放两条线并读一次 SDA。只有检测到 SDA 被从机拉低才补时钟：

```
        if (SDA_READ() == 0)              /* SDA 被拉死了才需要补时钟 */
        {
                for (i = 0; i < 9; i++)
                {
                        SCL_LOW();
                        i2c_delay();
                        SCL_HIGH();
                        i2c_delay();
                }
        }
```

（`i2c.c:63-72`）九个时钟刚好把从机"没发完的一个字节"走完，让它进入等待应答的位置；随后 `i2c.c:74-80` 手工构造一个停止条件（`SDA_LOW` → `SCL_HIGH` → `SDA_HIGH`），把总线交回空闲态。**能救回的是**：复位或掉电恰好打断在从机发送中途、从机正把 SDA 拉低的情况——这也是"接对了却扫不到器件"最常见的原因。

不能救回的是：SDA 被硬件性拉死（从机损坏、短路、上拉缺失）。这时 `SDA_READ()==0` 恒成立，九个时钟改变不了任何东西；更关键的是**本函数没有返回值、也没有恢复后的验证**——它把停止条件盲发一遍就返回 `void`（`i2c.c:55`），调用者无法知道总线上是否真的恢复了。唯一的间接证据是后续 `SoftI2C_Probe()` 的失败（`main.c:513` 的重试循环正是靠这个）。

`i2c_fault` **的真实作用域**（第 3 章已纠正过一次，这里给出本章需要的形式化描述）：

动作	位置	说明
清零	`i2c.c:85`	`i2c_start()` 的第一条语句，**无条件**
置位	`i2c.c:20`	`scl_released()` 等 SCL 超过 1000µs 仍不释放
置位	`i2c.c:91`	起始条件前 SDA 已被从机拉低
置位	`i2c.c:108`	停止条件后 SDA 没有被释放
短路	`i2c.c:115`	`i2c_send_byte()` 入口直接 `return 1`
短路	`i2c.c:146`	`i2c_read_byte()` 入口直接 `return 0xFF`

因为每次都清零，**它只在一个事务内部有效**，不会跨事务保持，也不是"总线熔断器"。它的价值是让一次已经判定失败的事务尽快返回，避免继续占用总线时间；事务之间的事务级失败靠返回值向上传递（`SoftI2C_WriteBytes` 返回 `i2c_fault`，`i2c.c:206`）。

`i2c_fault` 唯一的时间守护在 `scl_released()`：

```
    start=delay_micros16();
    while(!SCL_READ() && guard--)
        if((uint16_t)(delay_micros16()-start)>=1000U)break;
    if(!SCL_READ()){i2c_fault=1;return 0;}
```

（`i2c.c:17-20`）超时上限是 **1000µs**，用 `delay_micros16()` 的 16 位差值判断，所以最坏卡死时间是 1ms 而不是无限。这 1ms 会串在执行流里：一次块写里每个字节的每位都要过 `scl_released()`（`i2c.c:122`、`i2c.c:133`），理论上一个块最坏可以拖到 `40 B × 9 bit × 1ms` 量级——当然实际只在从机做时钟拉伸时才会命中，正常情况下这条分支只做一次 `IDR` 读就返回（`i2c.c:16`）。

11.7 为什么全工程没有 I2C 互斥原语

工程里**有**临界区惯用法，但只用在控制输入上：

```
int8_t Controls_TakeTurn(void)
{
    uint32_t mask = __get_PRIMASK(); int8_t result;
    __disable_irq(); result = turns; turns = 0; __set_PRIMASK(mask); return result;
}
```

（`controls.c:198-202`）`__disable_irq()/__set_PRIMASK()` 这一对在这里是必需的，因为 `turns` 由定时器中断写、主循环读。

I2C **从不用**它，这个选择是可以辩护的，理由有两条且都已核实：

1. **中断里不碰 I2C。** `stm32f10x_it.c` 全文没有任何 `SoftI2C`、`I2C_PORT`、`GPIO_SetBits` / `GPIO_ResetBits`（逐行 grep 确认无匹配）；`dht11.c:53-71` 的 `EXTI1_IRQHandler` 只写 `rx[]`、`edge_count`、`rx_done`、`edge_time`、`receiving`，并用 `delay_micros16()` 打时间戳，全是纯内存操作。
2. **主循环是单线程串行。** 所有 `SoftI2C_*` 调用点都在 `main()` 的执行流或它调用的函数里，没有第二个执行流。

因此"总线共享"在软件层面完全退化成**调用顺序**问题：只要主循环不主动把两个总线使用者叠在一起，就不会冲突。11.3 的四道 `!DHT11_Busy()` 门就是"用调用顺序代替互斥锁"的全部实现。

代价是这套约定**没有编译期或运行期保护**：任何一次新增的 I2C 调用，只要落在 DHT11 的 20ms 窗口里且带有阻塞延时，就会重现 11.3 描述的"脉冲被拉长导致读数失败"，而编译器不会有任何提示。换句话说，这个工程的安全性来自**约定 + 注释**（`main.c:599` 那条注释就是全部文档），而不是来自机制。

本章发现的问题与建议

1. 互锁加在了短事务上，长事务完全没门控（互锁对象不对称）

- 现象：`main.c:661` 用 `!DHT11_Busy()` 保护 `OLED_Service()`（最长 3 字节、不到 90µs、内部无阻塞延时），而 `main.c:662` 的 `OLED_RefreshStep()`（每块 40 字节、约 1.16ms）在 DHT11 独占期内照常发送。
- 证据：`main.c:661-662`；`OLED_Service` 的最大事务见 `oled.c:151`（`init_sizes` 仅 2/3 字节）与 `oled.c:168`（`cmd[3]`）；`OLED_RefreshStep` 的自检只有 `oled.c:184` 的 `if (!refresh_busy) return;`。

- 风险：门控形同虚设。若将来把对比度步进改成多字节或加上延时，保护仍然在，但真正需要保护的长事务依旧裸奔；反之如果哪天误删 `main.c:661` 的门控，也不会出现可观测故障——这类"无观测差异的保护"会在重构中被最先删掉。
- 建议：把门控移到真正的耗时点，即改成 `if(!DHT11_Busy() && !recovery_active)` 之类的条件只用于**恢复态**（恢复态会连发 16 条命令、可能触发 500ms 重试），并删除对比度路径上的门控；或者在 `OLED_RefreshStep()` 入口统一加 `if(DHT11_Busy())return;`，把约定从调用者收回到模块内部。

2. `main.c:661` 的门控没有对应的机制性理由

- 现象：四道门控里，`main.c:600`（120ms 阻塞）与 `main.c:628`（Flash 擦除扫描）都有明确的"阻塞调用会吞掉 20ms 起始脉冲"机制，`main.c:661` / `main.c:230` 没有。
- 证据：`main.c:599` 的注释只为第 3 道门解释了理由，第 1、2 道没有注释；`OLED_Service()` 两次总线写（`oled.c:151`、`oled.c:168`）都是 `SoftI2C_WriteBytes` 的直接调用，不含 `delay_ms`。
- 风险：读者会误以为"任何 I2C 访问都必须避开 DHT 忙期"，从而在新增驱动时过度加锁，或反过来在别的路径上遗漏真正需要的锁。
- 建议：给 `main.c:230` / `main.c:661` 补注释说明它们防的是"对比度命令与 DHT 起始脉冲同时翻转电平引起的可听/可见干扰"（若确有此考虑），否则删除；把门控规则写成一句可执行的判据："**任何含 `delay_ms` 的 I2C 路径都必须加 `!DHT11_Busy()`**"。

3. `OLED_GRAM` 的撕裂保护只在调用点，模块内部不强制、不检测

- 现象：`oled.c:132-134` 的注释要求"刷新期间不要改变 `OLED_GRAM`"，但 `OLED_DrawPoint`（`oled.c:213`）等绘图 API 没有任何忙碌检查。
- 证据：保护实际存在于两个调用点——`main.c:649` `if(!OLED_RefreshBusy()&&due(now,frame_next))`，绘制与 `OLED_StartRefresh()` 都在其内部（`main.c:658`）；`main.c:357` `while(OLED_RefreshBusy())post_pump();` 之后才 `post_draw()`（`main.c:358`）。grep 确认 `Dashboard_Draw` 与 `post_draw` 各自只有这一个调用点，因此**当前固件不会撕裂**。
- 风险：这是一条"靠纪律维持"的不变式。任何新增的绘制路径（例如在中断或任务里直接调 `OLED_DrawPoint`）都会静默破坏正在发送的那一块，表现为偶发花屏，且无任何断言可查。
- 建议：在 `OLED_DrawPoint()` 入口加 `if(refresh_busy) return;`（或在调试构建下 `assert`），把不变式从注释升级为代码；正式构建可保留该判断，代价是一次字节读。

4. `SoftI2C_BusRecover()` 无返回值、无恢复验证

- 现象：函数签名是 `void SoftI2C_BusRecover(void)`（`i2c.h:39`），补完 9 个时钟与停止条件后直接返回，不确认总线是否真的空闲。
- 证据：`i2c.c:55-81` 全函数无返回值、无回读校验；`SoftI2C_Init()` 之后立刻调用它（`main.c:509`），立即失败时唯一的补救是 `main.c:511-515` 的探测重试循环。
- 风险：RTC 级"接对了却扫不到器件"的问题会被掩盖成需要反复重试；现场排查时无法区分"总线已恢复"与"总线仍被拉死"。
- 建议：改为 `uint8_t SoftI2C_BusRecover(void)`，返回恢复后 `SDA_READ() && SCL_READ()` 的实测结果，并让 `OLED_Init`（`oled.c:112-113`）把它计入失败判据。

5. `scl_released()` 的 1ms 超时在同一事务内会被反复付出

- 现象：每次 `SCL_HIGH()` 之后都要过一遍 `scl_released()`，单次超时 1000µs，且点位在每一字节的每一位上（`i2c.c:122`、`i2c.c:133`）。
- 证据：`i2c.c:17-20` 的 `>=1000U` 判据与 `i2c.c:122` / `i2c.c:133` 的调用点。
- 风险：一旦从机持续做时钟拉伸，一次 32 字节块写的最坏时间不再是 1.16ms，而是量级上升，DHT11 起始脉冲的 20ms 窗口会被击穿（因为 `OLED_RefreshStep()` 没有门控，见问题 1）。
- 建议：把 `i2c_fault` 置位后改为在 `i2c_start()` 之前就拒绝新事务（当前已经是这个效果，但只在事务内）；同时在 `OLED_RefreshStep()` 里检查 `i2c_fault`（或让 `SoftI2C_WriteBytes` 区分"超时失败"与"无应答失败"的返回码），以便对超时类失败执行"跳过若干块"而不是立即整屏重发。

6. 恢复态期间的 500ms 重试会连发 16 条初始化命令，且过程中屏幕不可用

- 现象：`need_recovery()` 把 `sent_valid=0`（`oled.c:35`），恢复完成后 `OLED_StartRefresh()`（`oled.c:158`）会触发整屏 32 块重发（约 37ms）；而恢复过程本身一次只发一条命令、失败则退回 `recovery_step=0` 并再等 500ms（`oled.c:152`）。
- 证据：`oled.c:146-161` 的状态机；`oled.c:155` 的 `++recovery_step==INIT_COMMAND_COUNT` 判据；`oled.c:148` / `oled.c:152` / `oled.c:154` 的三处 500ms。
- 风险：OLED 掉电重上电（`oled.c:198` 的注释明确提到这种场景）后最长可能停留在恢复态数秒，期间画面停在旧内容或全黑，而上层无法查询 `recovery_active`（没有对应的查询 API，只有 `OLED_RefreshBusy()` 暴露 `refresh_busy`，`oled.c:135`）。
- 建议：导出 `uint8_t OLED_Recovering(void)` 供仪表盘显示"OLED 重连中"，并把 500ms 固定间隔改为与 DHT11 忙期互锁的短间隔（例如 50ms），缩短恢复窗口。

第 12 章　专题二：两条 Flash 日志的掉电安全与区域布局

本章要点

- 两块日志共用一片 SPI NOR，靠**容量减法**自下而上分三块：事件日志 `base = C-65536-8192`（`eventlog.c:92`）、传感器日志 `region_base = C-65536`（`logger.c:66`），两者都必须 4KiB 对齐，否则 `W25Q_EraseStart` 直接拒绝（`w25qxx.c:80`）。
- 两套记录都是 **32 字节**，格式不同但提交协议相同：先写前 31 字节、回读 `memcmp` 确认、**最后单独写第 31 字节 = 0xA5** 作为原子提交标志（`logger.c:123`+`133`、`eventlog.c:171`+`178`）。
- 提交字节的原子性可以严格论证：页编程只能把 1 清成 0，而 0xA5 相对 0xFF 清了 bit1/3/4/6，**任何部分完成都只可能得到"多于 0xA5 的 1"的值**，永远不可能恰好等于 0xA5，所以掉电中断只会让记录"不存在"，不会让它"半真"。
- 两套"陌生数据"判据不等价：`logger` 在第一个陌生槽**立即** `return` 并锁成 `LOG_FOREIGN`（`logger.c:98`），`eventlog` 则**扫完 256 槽**才锁成 `EVENT_LOCKED`（`eventlog.c:153`、`156`）。
- `eventlog` 有硬前置：`EventLog_Init` 要求 `Logger_State()==LOG_READY`（`eventlog.c:91`），所以传感器日志一旦被外来数据锁死，事件日志连 `base` 都算不出来，整个模块停在 `EVENT_OFFLINE`。这是真实的**级联失效**。
- 扇区策略两者相反：传感器日志"**先擦当前扇区再写**"（`logger.c:113-116`），事件日志"**本扇区用满才去擦另一扇区**"（`eventlog.c:122-125`）——后者因此总能保住一个完整的上一个扇区。
- 会话时长的可证下界 = **最近一次成功提交的 ALIVE 心跳**（60s 一条，`eventlog.c:12`、`191-193`），报告值由 `update_duration` 的聚合覆盖（`eventlog.c:56-64`、`111`），**误差上限 60s**；BOOT 记录的 uptime 被强制写 0（`eventlog.c:78`），所以"一次会话"本身不携带时长。
- 最坏保存时长：事件日志 **128 条 = 2.13 小时**（不是 256 条），传感器日志 **1920 条 = 32.0 小时**（不是 2048 条）——因为轮换前必须先擦掉一个满扇区。

12.1 物理前提：4.5MHz、容量探测与页/扇区约束

理解这两套日志之前，必须先弄清 Flash 驱动给上层留下的**约束边界**，因为协议的大部分复杂度都是这些约束逼出来的。

总线速率。 SPI1 用 PA5/6/7 + CS PA4（`w25qxx.h:4`），主频由预分频器决定：

```
spi.SPI_NSS=SPI_NSS_Soft; spi.SPI_BaudRatePrescaler=SPI_BaudRatePrescaler_16;
```

—— `w25qxx.c:39`

SPI1 挂在 APB2 上，PCLK2 = 72MHz，所以 **SCK = 72MHz / 16 = 4.5MHz**，位周期 222.2ns，字节线上时间 1.78µs。下文所有 Flash 耗时都按这个数推导。

逐字节轮询带超时。 `transfer()` 不用 DMA 也不用中断，而是轮询 TXE/RXNE 并带 1000µs 兜底：

```
    uint16_t start=delay_micros16();
    if(bus_error) return 0xFF;
    while(!(SPI1->SR & SPI_I2S_FLAG_TXE))
        if((uint16_t)(delay_micros16()-start)>1000) {bus_error=1;return 0xFF;}
    SPI1->DR=value;
    while(!(SPI1->SR & SPI_I2S_FLAG_RXNE))
        if((uint16_t)(delay_micros16()-start)>1000) {bus_error=1;return 0xFF;}
    return (uint8_t)SPI1->DR;
```

—— `w25qxx.c:11-18`

`bus_error` 是**锁存**的：一旦置位，`W25Q_Busy()` 恒返回 2（`w25qxx.c:52`），而所有操作都把 2 当作致命错误（`logger.c:93`、`eventlog.c:141`）。这与软件 I2C 的 `i2c_fault`（每次事务清零）完全不同——**Flash 总线错误是不可自愈的，必须重新 `W25Q_Init()`**，这正是 `main.c:624-633` 退避重试存在的理由。

容量是运行时探测的，不是编译期常量。 `W25Q_Init` 读 JEDEC ID 后做三重校验再定容量：

```
    if(bus_error || vendor!=0xEF || (type!=0x40 && type!=0x70) || size<0x12 || size>0x18) return 1;
    capacity=1UL<<size;
```

—— `w25qxx.c:44-45`

所以 `capacity = 2^size`，`size ∈ [0x12, 0x18]` → **容量在 256KiB ~ 16MiB 之间浮动。**这意味着两块日志的绝对地址在编译期是未知的，`region_base` / `base` 必须每次上电重算——这是"区域布局"问题的根源。

三条硬性几何约束（这是本章协议复杂度的直接来源）：

约束	源码	违反后果
页编程不得跨 256 字节页：`(address&255)+size>256` 即拒绝	`w25qxx.c:72`	返回 1，调用方转 `LOG_ERROR` / `EVENT_ERROR`
扇区擦除必须 4KiB 对齐：`(address&4095)` 非零即拒绝	`w25qxx.c:80`	同上
读操作在器件忙时直接失败：`W25Q_Busy()!=0` 即返回 1	`w25qxx.c:58`	调用方必须"下次再试"

第三条是理解两套状态机的钥匙：**因为"读回校验"不能在编程后就地做**（那时器件的 BUSY 位还没落），所以"写—读回—提交"必须被拆成三次独立的 `_Task()` 调用，每次都要重新轮询 BUSY。这不是风格选择，是硬件约束。

至于第一条，`31 字节 + 1 字节` 的拆分恰好落在页内：记录地址 `A` 是 32 的倍数，`(A&255)` 最大 224，`224+31 = 255 ≤ 256`；提交地址 `A+31` 的页内偏移最大 255，`255+1 = 256 ≤ 256`——**刚好不越界，一个字节的余量都没有。**

12.2 区域布局：三块区域与它们之间的依赖

两块日志都锚定在**容量的顶端**，用减法自下而上生长：

```
W25Q 物理地址空间（C = W25Q_Capacity()，运行时探测，256KiB ~ 16MiB）

0x000000  ┌──────────────────────────────────────────┐
          │                                          │
          │   未使用区  C - 73728 字节               │
          │   （预留，无任何代码引用）               │
C-73728   ├──────────────────────────────────────────┤ ← base        (eventlog.c:92)
          │   事件日志区  8192 B = 2 扇区 × 128 槽   │   EVENT_BYTES 8192 (eventlog.c:7)
          │   256 槽 × 32 B (EVENT_SLOTS, eventlog.c:9) │
C-65536   ├──────────────────────────────────────────┤ ← region_base (logger.c:66)
          │   传感器日志区 65536 B = 16 扇区 × 128 槽 │   REGION_BYTES 65536 (logger.c:8)
          │   2048 槽 × 32 B (LOGGER_SLOTS, logger.h:4) │
C         └──────────────────────────────────────────┘ ← 容量顶端
```

两个基址的表达式：

```
    region_base=W25Q_Capacity()-REGION_BYTES;state=LOG_SCANNING;timeout=now+2000;
```

—— `logger.c:66`

```
    base=W25Q_Capacity()-SENSOR_BYTES-EVENT_BYTES;
```

—— `eventlog.c:92`

4KiB 对齐是巧合还是设计？ 两者都是 4096 的整数倍（65536 = 16×4096，8192 = 2×4096），而 `W25Q_Capacity()` 是 2 的幂（≥256KiB），所以 `C-65536` 与 `C-73728` 都天然 4KiB 对齐，`W25Q_EraseStart` 的 `(address&4095)` 检查才通得过。**但代码里没有任何断言保证这一点**：如果将来有人把 `EVENT_BYTES` 改成 9000，事件日志会在第一次扇区轮换时静默地永久失败（每次都 `EVENT_ERROR`）。这是一个应当加静态断言的位置。

依赖顺序（级联失效的根源）。

```
    if(Logger_State()!=LOG_READY||W25Q_Capacity()<SENSOR_BYTES+EVENT_BYTES)return;
```

—— `eventlog.c:91`

事件日志把"传感器日志已就绪"当作自己的启动前提。调度侧同样如此：

```
        Logger_Task(now);
        if(!event_init_attempted&&Logger_State()==LOG_READY){
            EventLog_Init(now,(uint8_t)(reset_flags>>24));event_init_attempted=1;
        }
        if(event_init_attempted)EventLog_Task(now,uptime_seconds);
```

—— `main.c:611-615`

这里有三个可用状态：`LOG_OFFLINE`（容量不够或 `W25Q_Init` 失败）、`LOG_FOREIGN`（扫到陌生数据）、`LOG_ERROR`。**只有 `LOG_READY` 能放行事件日志**，所以 `LOG_FOREIGN` 会让事件日志永远停在 `EVENT_OFFLINE`，连 `base` 都没赋值。后果不是"事件日志也锁死"，而是**"事件日志从未启动"**—— `EventLog_State()` 返回 `EVENT_OFFLINE`，UI 上表现为"没有会话记录"，与"事件区被外来数据占用"完全无法区分。

注意 `main.c:522` 有一步补偿：`event_init_attempted=post_event_started;`——如果 POST 阶段已经成功启动了事件日志，主循环就不再重复初始化。

12.3 记录格式：两套 32 字节格式逐字段对照

两套格式都定长 32 字节，都用 **CRC32（多项式 `0xEDB88320`，初值 `0xFFFFFFFF`，结果取反）覆盖前 24 字节**，都把第 31 字节留给提交标志。但字段语义差别很大。

CRC 的实现两份完全一样（`logger.c:20-25`、`eventlog.c:26-31`），值得注意的是最后 `return ~crc;`——标准 CRC-32 的收尾取反。

传感器记录 `LogRecord`（`logger.c:36-42` 编码，`logger.c:28-35` 解码，结构体 `logger.h:9-14`）：

偏移	长度	字段	写入	解码校验
`[0..3]`	4	MAGIC `0x31474F4C`（ASCII "LOG1"）	`put32(raw,MAGIC)`（`logger.c:38`）	`get32(b)!=MAGIC` → 无效（`logger.c:30`）
`[4..7]`	4	`sequence`（全局递增序号）	`put32(raw+4,r->sequence)`（`logger.c:38`）	不校验
`[8..11]`	4	`uptime_s` 开机秒数	`put32(raw+8,r->uptime_s)`（`logger.c:38`）	不校验
`[12..13]`	2	`temperature_x10`（int16，℃×10）	`put16(raw+12,…)`（`logger.c:39`）	不校验
`[14..15]`	2	`humidity_x10`（uint16，%×10）	`put16(raw+14,…)`（`logger.c:39`）	不校验
`[16..17]`	2	`pitch_x10`（int16，度×10）	`put16(raw+16,…)`（`logger.c:40`）	不校验
`[18..19]`	2	`roll_x10`（int16，度×10）	`put16(raw+18,…)`（`logger.c:40`）	不校验
`[20..23]`	4	`flags`，**只用低 2 位**（`LOG_ENV_VALID`=1、`LOG_LEVEL_VALID`=2，`logger.h:6-7`）	`put32(raw+20,r->flags&3)`（`logger.c:41`）	`get32(b+20)>3` → 无效（`logger.c:30`）
`[24..27]`	4	CRC32，覆盖 `[0..23]`	`put32(raw+24,crc32(raw,24))`（`logger.c:41`）	`get32(b+24)!=crc32(b,24)` → 无效（`logger.c:30`）

偏移	长度	字段	写入	解码校验
[28..30]	3	恒 0x00（memset 后未再写）	memset(raw,0,sizeof(raw))（logger.c:38）	不参与判定
[31]	1	**提交字节**：编码时置 0xFF，提交阶段写 0xA5	raw[31]=0xFF（logger.c:41）→ b[0]=0xA5（logger.c:132-133）	b[31]!=0xA5 → 无效（logger.c:30）

一个容易忽略的细节：encode() 里 raw[31]=0xFF 是**显式**写的，而 [28..30] 靠 memset(…,0,…) 兜底为 0x00。但 WRITE_BODY 只编程前 31 字节（logger.c:123），**第 31 字节根本没被写**——它保持擦除后的 0xFF。所以 raw[31]=0xFF 这行赋值严格来说是死代码（写不写都一样），真正的语义是"提交字节必须先保持 0xFF 才合法"。

事件记录 EventLast（eventlog.c:73-81 准备，eventlog.c:35-45 解码，结构体 eventlog.h:8-11）：

偏移	长度	字段	写入	解码校验
[0..3]	4	MAGIC 0x31545645（ASCII "EVT1", eventlog.c:10）	put32(raw,EVENT_MAGIC)（eventlog.c:76）	get32(b)!=EVENT_MAGIC → 无效（eventlog.c:37）
[4..7]	4	sequence = latest_seq+1	put32(raw+4,latest_seq+1UL)（eventlog.c:76）	不校验
[8..11]	4	恒 0xFFFFFFFF（保留位）	put32(raw+8,0xFFFFFFFFUL)（eventlog.c:77）	**完全不校验**
[12..15]	4	uptime_seconds；**BOOT 记录被强制写 0**	put32(raw+12,kind==EVENT_BOOT?0:uptime_seconds)（eventlog.c:78）	不校验
[16..19]	4	boot_count（开机次数）	put32(raw+16,boot_count)（eventlog.c:78）	不校验
[20]	1	kind：1=BOOT、2=ALIVE、3=保留	raw[20]=kind（eventlog.c:79）	b[20] 必须 ∈{1,2,3}（eventlog.c:39）
[21]	1	恒 0	raw[21]=0（eventlog.c:79）	b[21]>1 → 无效（eventlog.c:40）
[22]	1	reset_code（复位原因，RCC->CSR 高字节）	raw[22]=reset_reason（eventlog.c:79）	不校验
[23]	1	恒 0	raw[23]=0（eventlog.c:79）	不校验
[24..27]	4	CRC32，覆盖 [0..23]	put32(raw+24,crc32(raw,24))（eventlog.c:80）	get32(b+24)!=crc32(b,24) → 无效（eventlog.c:38）
[28..30]	3	恒 0xFF	memset(raw,0xFF,sizeof(raw))（eventlog.c:75）	不参与判定
[31]	1	**提交字节** 0xA5	准备阶段保持 0xFF；EVENT_BODY 写 b[0]=EVENT_COMMITTED（eventlog.c:177-178）	b[31]!=EVENT_COMMITTED → 无效（eventlog.c:37）

两份 memset 的取值相反（logger.c:38 用 0，eventlog.c:75 用 0xFF），这是一个**未记录的不一致**：传感器记录在"半写"状态下 [28..30] 是 0x00，事件记录是 0xFF。好在这 3 个字节两边都不参与校验，所以不构成功能差异，但会让"用 hexdump 人工判读"的人误以为溢出。

两份格式共同的弱点：blank() 要求**全部 32 字节都是 0xFF**（logger.c:26、eventlog.c:33）。于是一条"写了一半的记录"既不 blank、也解不出，落在一个尴尬的第三态。两套日志对此的处理不同（下一节）。

12.4 提交协议：四阶段状态机与原子性论证

为什么需要四阶段。 朴素写法是"擦除 → 写 32 字节 → 写序号"。问题在于：**掉电可能发生在任意字节之间**，而 NOR Flash 的页编程在一次事务内可能只完成了一部分字节。恢复时你无法区分"这条记录是这次的"和"这条记录是上次的残骸"。协议用三个手段解决：

1. **提交字节放在最后**，且是**独立的一次编程事务**；
2. 每条记录自带 **MAGIC + CRC32**，可独立判定自洽；
3. **提交字节的合法值只有一个 0xA5**。

传感器日志的四阶段（状态枚举 logger.c:9：WRITE_SELECT=1, WRITE_ERASE, WRITE_BODY, WRITE_COMMIT）：

阶段	进入动作	关键源码	超时
WRITE_SELECT	若 !(write_slot%128) → 先 W25Q_EraseStart 并转 ERASE；否则读 32B 验空，非空则 write_slot+1 重试（skipped 累计到 2048 报错）；空则 W25Q_ProgramStart(address,raw,31) 并转 BODY	logger.c:111-124	无
WRITE_ERASE	等 BUSY 落 → 读回验 blank → drop_sector() 扣减统计 → ProgramStart(raw,31) 转 BODY	logger.c:125-129	**1500ms**（logger.c:115）
WRITE_BODY	等 BUSY 落 → 读回并 memcmp(b,raw,31) → 单独写第 31 字节 0xA5 → 转 COMMIT	logger.c:130-134	**50ms**（logger.c:124）
WRITE_COMMIT	等 BUSY 落 → 读回并 Logger_Decode 复验 → 更新 latest_slot/latest_seq/count/per_sector/recent → 回 LOG_READY	logger.c:135-139	**50ms**（logger.c:134）

关键的三行——**"只写前 31 字节" 与 "只写第 31 字节" 是两次独立事务**：

```
        if(W25Q_ProgramStart(address(write_slot),raw,31)) {state=LOG_ERROR;return;}
        write_phase=WRITE_BODY;timeout=now+50;
```

—— logger.c:123-124

```
        b[0]=0xA5;
        if(W25Q_ProgramStart(address(write_slot)+31,b,1)) {state=LOG_ERROR;return;}
```

—— `logger.c:132-133`

事件日志的四阶段（状态枚举 `eventlog.h:5-6`，用状态名而非数字）：

阶段	进入动作	关键源码	超时
`EVENT_ERASING`	`W25Q_EraseStart` 已发出，等 BUSY 落 → 转 VERIFY	`eventlog.c:163-166`	**1500ms**（`eventlog.c:200`）
`EVENT_VERIFY`	**逐槽**读回验空，直到 `verify_slot >= write_slot+128`（整个 128 槽扇区）→ `drop_sector()` → `ProgramStart(raw,31)` 转 BODY	`eventlog.c:167-173`	无
`EVENT_BODY`	等 BUSY 落 → 读回 `memcmp(b,raw,31)` → 写第 31 字节 `0xA5` → 转 COMMIT	`eventlog.c:174-180`	**50ms**（`eventlog.c:203`）
`EVENT_COMMIT`	等 BUSY 落 → 读回 `decode` 复验 → 更新 `latest`、`search_slot=latest_slot+1` → 回 `EVENT_READY`	`eventlog.c:181-190`	**50ms**（`eventlog.c:179`）

事件日志多了一个 `EVENT_VERIFY` 阶段，而且它**每个 `_Task()` 调用只校验一个槽**：

```
        if(W25Q_Read(address(verify_slot),b,32)||!blank(b)){state=EVENT_ERROR;return;}
        if(++verify_slot<write_slot+128U)return;
```

—— `eventlog.c:168-169`

也就是说，**一次扇区擦除后的全扇区校验要消耗 128 次 `EventLog_Task()` 调用。**这是 12.6 节要讨论的性能代价。

原子性论证：为什么"最后写第 31 字节"能给出"要么完整要么不存在"

这不是一句口号，可以严格证明。设擦除后第 31 字节为 `0xFF`，目标是 `0xA5`：

```
0xFF = 1111 1111
0xA5 = 1010 0101
被清掉的位：bit6(0x40)、bit4(0x10)、bit3(0x08)、bit1(0x02)
0xFF - 0x40 - 0x10 - 0x08 - 0x02 = 0xA5    ✓
```

NOR Flash 页编程的物理语义是**只能把 1 清成 0，不能把 0 置回 1**（所以 `W25Q_ProgramStart` 永远不需要先擦除，只要求目标是"可清"的）。因此任一时刻第 31 字节的取值 `x` 必满足 `x & 0xA5 == 0xA5`（0xA5 的所有 1 位在 x 中仍是 1）：

- **若编程完整完成** → `x = 0xA5`，记录被判**有效**；
- **若编程只完成了一部分** → 被清零的位是 `{bit6,bit4,bit3,bit1}` 的某个真子集 → `x` 至少还保留其中 1 位为 1 → `x ≠ 0xA5`，记录被判**无效**；
- **若完全没开始** → `x = 0xFF ≠ 0xA5`，记录被判**无效。**

不存在第三种结果。 这是整个掉电安全设计的支点，而且它有一个很容易被忽略的优点：**它不依赖 CRC32 去区分"新旧"。**CRC 在这里只负责抵御位翻转和逻辑错误，掉电语义完全由提交字节承担——两者职责分离得很干净。

需要诚实指出的是，这个论证**只保证单条记录的自洽性，不保证记录之间的有序性和完整性。**具体地：

- 它保证"读到的记录要么完整要么被忽略"，**不保证"已提交的记录都在 Flash 里"**；
- 它**不处理序号回绕**：`pending.sequence=latest_seq+1`（`logger.c:83`）与 `put32(raw+4,latest_seq+1UL)`（`eventlog.c:76`）都是 32 位无符号加一，`latest_seq` 到 `0xFFFFFFFF` 后回绕到 0，而扫描时的比较用的是有符号差值 `(int32_t)(record.sequence-latest_seq)>0`（`logger.c:100`）——回绕点附近这个比较会给出错误方向，把最新的记录判成最旧；
- 它**不保证写指针与最新记录一致**：`write_slot` 由 `latest_slot+1` 推出（`logger.c:84`），而 `latest_slot` 是扫描时按 `sequence` 最大值选出来的（`logger.c:100`）。如果扫描只走到一半（FOREIGN 或超时），`latest_slot` 就是错的，后续写入会从错误位置开始。

12.5 两套"陌生数据"判据的差异

"陌生数据"指的是：这块区域里出现了**不是本固件写的**内容。两块日志都必须对此保守处理——因为 NOR 里可能是别人的固件配置、可能是上一版固件的日志布局、也可能只是未初始化数据，任何"自动格式化"都会毁掉证据。所以判据是"停"而不是"清"。

但两套判据的**触发点、作用范围、以及停止时机**完全不同：

维度	传感器日志	事件日志
判据	扫到**非空**槽，且 `get32(b)!=MAGIC`	扫到**非空**槽，且 `decode()` 失败，且 `get32(b)!=EVENT_MAGIC`
源码	`logger.c:97-98`	`eventlog.c:145`、`eventlog.c:153`
判定时机	**命中即停**：`if(get32(b)!=MAGIC) {state=LOG_FOREIGN;return;}`	**扫完才停**：先置 `foreign=1`，扫到 `scan_slot==EVENT_SLOTS` 时才 `state=EVENT_LOCKED`
源码	`logger.c:98`	`eventlog.c:153`、`eventlog.c:155-156`
已收集的统计	只包含陌生槽**之前**的记录	包含全部 256 槽的记录（`session_count` 完整）
对外可见性	`Logger_State()==LOG_FOREIGN`；`Logger_ReadRecent` 在 FOREIGN 下**仍可读**（`logger.c:76` 只排除 SCANNING/WRITING/ERROR）	`EventLog_State()==EVENT_LOCKED`；`EventLog_ReadSession` 要求 `state==EVENT_READY`（`eventlog.c:107`），**LOCKED 下读不到**
可恢复	否	否

自己写坏的半条记录不会触发"陌生"。 这是设计上的正确取舍：因为 `WRITE_BODY` 已经把 MAGIC 写进去了，所以一条"有 MAGIC 但没提交字节"的记录在两个判据下都**不是陌生的**——

- 传感器日志：`get32(b)==MAGIC` 成立，跳过 `LOG_FOREIGN` 分支，随后 `Logger_Decode` 因缺 `0xA5` 返回 1，于是 `if(!Logger_Decode(b,&record))` 不进入，**该槽被静默忽略**（`logger.c:99-102`）；
- 事件日志：`decode` 失败但 `get32(b)==EVENT_MAGIC`，所以 `foreign` **不置位**（`eventlog.c:153`），该槽同样被忽略。

换句话说，**判据只对"连 MAGIC 都不是"的数据报警。**这是正确的：只有 MAGIC 不匹配才真正说明"这块区域不是我的"。

下游影响更值得警惕。 由于 `EventLog_Init` 要求 `Logger_State()==LOG_READY`（`eventlog.c:91`），而 `LOG_FOREIGN ≠ LOG_READY`，一旦传感器日志区被外来数据锁死：

```
LOG_FOREIGN
   ↓  main.c:612 的 if(Logger_State()==LOG_READY) 不成立
event_init_attempted 永远为 0
   ↓  main.c:615 的 if(event_init_attempted) 不成立
EventLog_Task() 从不被调用；EventLog_Init() 从未执行
   ↓  eventlog.c:92 的 base=... 从未赋值，base 保持 0
EventLog_State() 恒为 EVENT_OFFLINE
```

结果是：**用户的传感器日志区里有一片陌生数据，会导致事件日志功能完全消失**——而不是"事件日志也报 FOREIGN"。这两个故障在 UI 上是同一种表现（无会话记录），但根因完全不同，现场无法区分。`main.c:633` 的 `if(Logger_State()==LOG_READY||Logger_State()==LOG_FOREIGN)flash_retry=0;` 更把 `LOG_FOREIGN` 明确归入"不必重试"——这个判断是对的（重试也没用），但它同时意味着**没有任何路径能让事件日志从这个状态恢复。**

12.6 扇区轮换策略：两种相反的取舍

两套日志的槽数量差 8 倍，扇区数差 8 倍，但真正重要的是它们的**轮换策略正好相反。**

传感器日志：先擦当前扇区，再往里写。 写指针严格 `+1` 前进：

```
    write_slot=count || latest_seq ? (latest_slot+1)%LOGGER_SLOTS : 0;
```

—— `logger.c:84`

每次进入 `WRITE_SELECT` 时，只要落在扇区首槽就先擦：

```
        if(!(write_slot%128)) {
            if(W25Q_EraseStart(address(write_slot))) {state=LOG_ERROR;return;}
            write_phase=WRITE_ERASE;timeout=now+1500;return;
        }
```

—— `logger.c:113-116`

擦除完成后立即**扣减统计**（不是等写成功）：

```
        if(W25Q_Read(address(write_slot),b,32) || !blank(b)) {state=LOG_ERROR;return;}
        drop_sector((uint8_t)(write_slot/128));
```

—— `logger.c:126-127`

`drop_sector` 做两件事：从 `count` 里减去该扇区的记录数（`logger.c:55`），并从 `recent_slot[]` 里剔除该扇区的条目（`logger.c:56-58`）。`per_sector[16]` 是 16 个扇区各自的记录计数（`logger.c:12`、`logger.c:101`、`logger.c:137`）。

所以传感器日志是**纯环形覆盖**：2048 槽 = 16 扇区，每 128 条记录触发一次擦除，绕一圈正好把 16 个扇区各擦一次。**没有坏块跳过，没有磨损均衡，擦除均匀地轮转在所有 16 个扇区上**——就这一点而言，均匀性是自然获得的。

事件日志：本扇区用满，才去擦另一个扇区。 逻辑在 `choose_slot()`：

```
    }else{
        end=(uint16_t)((latest_slot/128U+1U)*128U);
        if(search_slot>=end){
            write_slot=(uint16_t)(end%EVENT_SLOTS);
            return 2; /* Rotate to the other sector; keep the current sector intact. */
        }
    }
```

—— `eventlog.c:121-126`

注释写得很明白："**keep the current sector intact**"。返回值 `2` 是一个三态信号：`0`=本次没找到（下次再试）、`1`=找到空槽可直接写、`2`=需要先擦另一个扇区。调用点据此分派：

```
    if(slot_state==2){
        if(W25Q_EraseStart(address(write_slot))){state=EVENT_ERROR;return;}
        state=EVENT_ERASING;deadline=now+1500;
    }else{
        if(W25Q_ProgramStart(address(write_slot),raw,31)){state=EVENT_ERROR;return;}
        state=EVENT_BODY;deadline=now+50;
    }
```

—— `eventlog.c:198-204`

这个差异的收益是"永远有一个完整的上一个扇区"。 因为当前扇区在写满之前不会被擦，而那 128 条记录是连续的、自洽的；另一个扇区则在被重写。任意时刻可读的历史 = 当前扇区已写的记录 + 另一扇区完整的 128 条。这比"边写边擦当前扇区"要多一条证据，代价是**磁盘利用率只有 50%**（8KiB 有效区实际只保住约一半）。

代价：128 次调用的校验窗口。 `EVENT_VERIFY` 逐槽验空（`eventlog.c:168-169`），128 槽需要 128 次 `EventLog_Task()`。这期间 `EventLog_Busy()` 一直为真：

```
uint8_t EventLog_Busy(void)
{return (uint8_t)(state==EVENT_ERASING||state==EVENT_VERIFY||
    state==EVENT_BODY||state==EVENT_COMMIT);}
```

—— `eventlog.c:96-98`

而传感器日志的 60s 存档门控里恰好有 `!EventLog_Busy()`（`main.c:616`），`save_snapshot` 内部还再查一次（`main.c:148`）。于是**一次事件日志扇区轮换会推迟最多一次传感器存档**。这不丢数据（`log_next` 只在成功时前进，`main.c:618`），但会让存档间隔偶尔从 60s 变成 60s+轮换时长。

12.7 断电损失边界：一条会话能证明什么

事件日志的语义目标是"复位溯源"——回答"上次为什么重启、跑了多久"。这里有一个**必须先说清的边界**：这条日志只能证明**最近一次 60s 检查点**，不能证明断电瞬间的状态。

根因一：BOOT 记录的 uptime 被强制写 0。

```
    put32(raw+12,kind==EVENT_BOOT?0:uptime_seconds);put32(raw+16,boot_count);
```

—— `eventlog.c:78`

开机时写 BOOT 记录，那一刻 `uptime_seconds` 本来就是接近 0 的小值，但代码**不用它**，而是无条件写 0。所以 BOOT 记录**本身不携带任何时长信息**——它只是一个"这次开机的编号（`boot_count`）+ 复位原因（`reset_reason`）"的锚点。

根因二：时长来自 ALIVE 心跳的聚合，而心跳每 60s 才一条。

```
#define HEARTBEAT_MS 60000UL
```

—— `eventlog.c:12`

```
    if((int32_t)(now-next_heartbeat)>=0&&pending==0){
        pending=EVENT_ALIVE;next_heartbeat=now+HEARTBEAT_MS;
    }
```

—— `eventlog.c:191-193`

每次扫描和每次提交都会把该 `boot_count` 的**最大 uptime** 记进聚合表：

```
static void update_duration(const EventLast *event)
{
    uint16_t index=(uint16_t)(event->boot_count%EVENT_SLOTS);
    if(session_boot_key[index]==event->boot_count){
        if(event->uptime_seconds>session_uptime[index])session_uptime[index]=event->uptime_seconds;
    }else if(session_boot_key[index]==0xFFFFFFFFUL||newer(event->boot_count,session_boot_key[index])){
        session_boot_key[index]=event->boot_count;session_uptime[index]=event->uptime_seconds;
    }
}
```

—— `eventlog.c:56-64`

读取时用聚合值**覆盖**记录里的值：

```
    if(!decode(b,event)||event->kind!=EVENT_BOOT)return 1;
    index=(uint16_t)(event->boot_count%EVENT_SLOTS);
    if(session_boot_key[index]==event->boot_count)event->uptime_seconds=session_uptime[index];
```

—— `eventlog.c:109-111`

注意 `eventlog.c:109` 的第二半：`event->kind!=EVENT_BOOT` 就返回 1—— **`EventLog_ReadSession` 只报告 BOOT 会话**，ALIVE 记录虽然占着槽，但从不出现在会话列表里。这是"一条记录 ≠ 一条会话"的关键：ALIVE 是**为会话服务的元数据**，不是会话本身。

于是可证的时间下界是：

```
设断电时刻的 uptime 为 T，最近一次成功提交的 ALIVE 的 uptime 为 A。

A = 60s × floor(T / 60s)      （实际还要减去写入延迟）
报告值 = session_uptime[boot_count]
       = max(所有该 boot_count 记录的 uptime)
       = A（因为 BOOT 贡献 0，ALIVE 贡献 60/120/180…）

可证下界：会话至少持续了 A 秒
不可证部分：T - A ≤ 60s + 提交延迟
```

结论：报告时长是真实时长的一个下界，误差上限 60s。 这个 60s 不是实现缺陷，而是"心跳周期"与"证据粒度"的必然换算——想让误差更小就必须提高心跳频率，代价是事件日志槽位消耗速度同比上升（见 12.8 的寿命换算）。**这是一个显式的工程折衷，应当写进设计说明。**

还有一个边界值得指出：**BOOT 记录的提交本身也可能失败。** `boot_recorded` 只在 `EVENT_COMMIT` 成功后被置位（`eventlog.c:185`），而 POST 阶段会等待它。如果 BOOT 记录因为 Flash 错误没能提交，`boot_count` 已经加过（`eventlog.c:159` 在扫描结束时 `boot_count++`），但 Flash 里没有对应记录——**下一条 ALIVE 会带一个"没有 BOOT 记录"的 `boot_count`**。这种情况下扫描时 `session_boot_key[index]` 会被 ALIVE 建立，而 `add_session` 只在 `kind==EVENT_BOOT` 时调用（`eventlog.c:148`、`eventlog.c:186`），所以**这条会话在列表里彻底不可见**，但它占掉了一个槽。这是一个真实的数据一致性缺口。

12.8 容量换算与寿命

槽位与扇区的换算关系：

项目	传感器日志	事件日志
区域字节	`REGION_BYTES = 65536`（`logger.c:8`）	`EVENT_BYTES = 8192`（`eventlog.c:7`）
槽数	`LOGGER_SLOTS = 2048`（`logger.h:4`）	`EVENT_SLOTS = 256`（`eventlog.c:9`）
扇区数	2048 / 128 = **16**	256 / 128 = **2**
单扇区槽数	128（`32 B × 128 = 4096 B`）	128（同）
写入周期	60s 一条快照（`main.c:616-619`）	60s 一条 ALIVE（`eventlog.c:12`）+ 每次开机一条 BOOT
满环名义条数	2048	256

最坏保存时长（必须扣掉轮换时被擦掉的那一个扇区）：

传感器日志：写指针走到扇区首槽（`write_slot%128==0`）时会先擦该扇区，**该扇区的 128 条记录在写入新记录之前就消失了。**所以稳态下条数在 `2048-128=1920` 与 `2048` 之间波动：

```
最坏条数 = 2048 - 128 = 1920 条
最坏时长 = 1920 × 60s = 115200s = 32.0 小时
最好条数 = 2048 条
最好时长 = 2048 × 60s = 122880s = 34.1 小时
```

事件日志：`choose_slot` 在本扇区用满时 `return 2`，随后擦掉**另一个**扇区（`eventlog.c:122-125`、`eventlog.c:199`）。所以刚完成轮换的那一刻，新扇区是空的，另一扇区是满的：

```
最坏条数 = 128 条（另一扇区满，当前扇区刚擦完）
最坏时长 = 128 × 60s = 7680s = 2.13 小时
最好条数 = 256 条
最好时长 = 256 × 60s = 15360s = 4.27 小时
```

"事件日志只存 2 小时"这个结论出乎意料，但它是 8KiB 区域与 60s 心跳直接相乘的结果。 对于"复位溯源"这个用途，2 小时窗口可能不够——如果设备每天开关机一次，你只能看到最近两小时内的会话。这是一个应当在设计说明里明确写出的功能边界。

上电扫描耗时。 每次读到 32 字节数据都要先发 4 字节命令（`w25qxx.c:59` 的 `transfer(0x03)` + `address24()`），所以单槽线上开销是 36 字节：

```
传感器日志扫描 = 2048 槽 × 36 B = 73728 B
               × 8 bit/B = 589824 bit
               ÷ 4.5×10⁶ bit/s = 131.1 ms

事件日志扫描   = 256 槽 × 36 B = 9216 B
               × 8 = 73728 bit
               ÷ 4.5×10⁶ = 16.4 ms
```

但 131ms 只是**纯总线时间。**真正的墙上时间由调度决定：`Logger_Task` 在扫描态**每个调用只推进一个槽**：

```
        if(W25Q_Read(address(scan_slot),b,32)) {state=LOG_ERROR;return;}
        if(!blank(b)) { /* 此处省略 :98-103 — 判 MAGIC、置 LOG_FOREIGN、解码与计数 */ }
        if(++scan_slot==LOGGER_SLOTS) state=LOG_READY;
        return;
```

—— `logger.c:96-105`

而它每轮主循环只被调一次（`main.c:611`），**所以 2048 槽需要 2048 个主循环周期。**主循环里最重的是刷屏（一个完整帧 = 32 次 `OLED_RefreshStep()`，`main.c:662`），因此到 `LOG_READY` 的墙上时间量级是**数十帧、约 1.5-3 秒**——比 131ms 大一个数量级以上。POST 阶段为此专门用计数循环等待扫描完成。

这也让 `logger.c:66` 的超时值失去了意义：

```
    region_base=W25Q_Capacity()-REGION_BYTES;state=LOG_SCANNING;timeout=now+2000;
```

—— `logger.c:66`

`timeout = now+2000` 是**从 `Logger_Init` 时刻起算的绝对截止时间**，但扫描态只在器件忙时才检查它：

```
        if(busy) {if((int32_t)(now-timeout)>=0)state=LOG_ERROR;return;}
```

—— `logger.c:95`

扫描过程中器件基本不忙（读操作是即时的），所以**这个 2s 看门狗实际上从不触发**；如果 Flash 真的在扫描中途卡住（BUSY 一直不落），那么 2s 之后**第一次**遇到 busy 就会立刻报 `LOG_ERROR`——此时扫描可能已经进行到第 1500 槽，全部结果被丢弃。**正确做法是每个槽重置 `timeout`，或改成"距上次进展超过 N ms"的看门狗。**

事件日志侧的扫描**完全没有超时检查**（`eventlog.c:142-162`），所以如果 Flash 卡在 busy，`EventLog_Task` 会永远停在 `if(busy)return;`（`eventlog.c:143`）而不报错，POST 的 `guard` 计数循环成为唯一的兜底。

擦写寿命。 需要说明 W25Q 的 4KiB 扇区耐受度取数据手册标称的 **100000 次**（该值不在本仓库源码内，属外部器件参数）：

```
传感器日志（60s 一条，1440 条/天）：
  每 128 条触发 1 次扇区擦除
  总擦除次数/天 = 1440 ÷ 128 = 11.25 次/天
  分摊到 16 个扇区 = 11.25 ÷ 16 = 0.703 次/天/扇区
  年化 = 0.703 × 365 = 256.6 次/年/扇区
  单扇区寿命 = 100000 ÷ 256.6 ≈ 390 年

事件日志（60s 一条 ALIVE，1440 条/天）：
  总擦除次数/天 = 1440 ÷ 128 = 11.25 次/天
  分摊到 2 个扇区 = 11.25 ÷ 2 = 5.625 次/天/扇区
  年化 = 5.625 × 365 = 2053 次/年/扇区
  单扇区寿命 = 100000 ÷ 2053 ≈ 48.7 年
```

两者都远超设备寿命，磨损不是这两个日志的限制因素。 值得注意的是事件日志的磨损比传感器日志快 8 倍（因为只有 2 个扇区分担），但 48.7 年仍然够用。真正的问题不是磨损，而是 12.6 节说的**有效利用率**：事件日志为了保住"另一个完整扇区"而放弃了 50% 区域，8KiB 实际只提供 2 小时的 ALIVE 历史。

未实现的机制（明确列出）：

- **无磨损均衡（wear leveling）**：擦除之所以均匀，是因为槽按 0→2047 顺序使用，这属于"天然轮转"，不是算法。一旦某个扇区出现坏块，`drop_sector` 与 `write_slot` 都不会跳过它——`blank()` 检查会失败，`W25Q_EraseStart` 会成功但读回不空，于是 `state=LOG_ERROR`（`logger.c:126`），整块日志**从那一刻起永久不可用**。
- **无坏块管理**：没有坏块表、没有备用扇区、没有 ECC。上述 100000 次以外的失效模式完全未处理。
- **无绝对时间轴**：`LogRecord.uptime_s`（`logger.h:10`）与 `EventLast.uptime_seconds`（`eventlog.h:9`）都只有开机秒数，全工程没有 RTC、没有外部时钟源。**两次开机之间的时间无法从日志推断**——只能靠 `boot_count` 计数推断"开过几次机"。

12.9 两种"事件日志消失"的可区分性，与恢复路径的不对称

上一节提到一个令人不安的可能性：传感器日志区被外来数据锁死后，事件日志会**从未启动**。而事件日志区自己也可能被外来数据锁死。两种情况在用户眼里都是"没有会话记录"。本节回答一个必须用代码而非直觉回答的问题：**能不能把它们分开？**

12.9.1 先说结论：能分开，但功劳全在 `Logger_State()` 一边

`EventLog_State()` **单独看无法区分**"从未初始化"与"初始化后被拒绝"，因为两者的取值**位完全相同**：

```
enum { EVENT_OFFLINE, EVENT_SCANNING, EVENT_READY, EVENT_ERASING,
       EVENT_VERIFY, EVENT_BODY, EVENT_COMMIT, EVENT_LOCKED, EVENT_ERROR };
```

—— `eventlog.h:5-6`

`EVENT_OFFLINE` 是枚举首位，**其值为 0**。而状态变量是零初始化的静态量：

```
static uint8_t state,pending,reset_reason,raw[32],foreign,have_latest;
```

—— `eventlog.c:13`

于是：

- `EventLog_Init` **从未被调用**（`Logger_State()!=LOG_READY`，`main.c:612` 的 `if` 不成立）→ `state` 保持 C 启动值 `0`；
- `EventLog_Init` **被调用但在** `eventlog.c:91` **提前返回** → `state` 在 `eventlog.c:85` 被显式写成 `EVENT_OFFLINE`，也是 `0`。

两条路径产出同一个字节。 这不是缺陷，但它意味着：**任何试图用** `EventLog_State()` **判断"事件日志到底有没有尝试启动过"的代码都会得到错误答案。**

真正承载区分信息的是 `Logger_State()`。完整的配对诊断表：

`Logger_State()`	值	`EventLog_State()`	值	诊断	可自愈
`LOG_READY`	2	`EVENT_READY`	2	正常	—
`LOG_READY`	2	`EVENT_LOCKED`	7	**事件日志区自己的陌生数据**（`eventlog.c:156`）	否
`LOG_FOREIGN`	5	`EVENT_OFFLINE`	0	**级联：传感器区陌生 → 事件日志从未启动**（`logger.c:98` → `main.c:612` 不放行）	否
`LOG_ERROR`	4	`EVENT_OFFLINE`	0	传感器区瞬时错误 → 事件日志同样从未启动	**是**（见 12.9.3）
`LOG_OFFLINE`	0	`EVENT_OFFLINE`	0	`W25Q_Init()` 失败，或容量 < `REGION_BYTES`（`logger.c:65`）	是（同 12.9.3）
`LOG_SCANNING`	1	`EVENT_OFFLINE`	0	开机扫描进行中，正常瞬态	—

枚举值取自 `logger.h:8`（`LOG_OFFLINE=0 … LOG_FOREIGN=5`）与 `eventlog.h:5-6`。

结论：`(LOG_FOREIGN, EVENT_OFFLINE)` 这一对是级联场景独有的，可以与 `(LOG_READY, EVENT_LOCKED)` 区分。 但要清醒地看到两点：

1. **区分能力完全由** `Logger_State()` **提供**，`EventLog_State()` 在这两种情况下都是同一个值，只是充当"这不是 `EVENT_LOCKED` "的排除项；
2. `(LOG_ERROR, EVENT_OFFLINE)` **与** `(LOG_FOREIGN, EVENT_OFFLINE)` **的配对形状一致**，只能靠 `Logger_State()` 的具体值（4 vs 5）分辨，而这两者的**命运完全不同**（前者可自愈，后者不可）。

12.9.2 UI 上能不能看见？能，但被拆到两个页面，且 `EVENT_OFFLINE` 文案是死胡同

`Logger_State()` 与 `EventLog_State()` **都**被渲染到屏幕上，但位置和文案如下：

```
static const char *flash_state(uint8_t state)
{
    switch(state) {
        case LOG_SCANNING:return "SCAN";case LOG_READY:return "READY";
        case LOG_WRITING:return "SAVING";case LOG_ERROR:return "ERROR";
        case LOG_FOREIGN:return "LOCKED";default:return W25Q_ID()!=0 && W25Q_ID()!=0xFFFFFF ? "UNSUP" : "ABSENT";
    }
}
```

—— `dashboard.c:45-52`

```
    OLED_ShowSmallString(4,32,"FLASH");OLED_ShowSmallString(68,32,flash_state(Logger_State()));
```

—— `dashboard.c:182`

```
static const char *event_text(uint8_t state)
{
    if(state==EVENT_READY)return "OK";
    if(state==EVENT_SCANNING)return "SCAN";
    if(state==EVENT_LOCKED)return "LOCK";
    if(state==EVENT_ERROR)return "ERR";
    if(state==EVENT_OFFLINE)return "NA";
    return "BUSY";
}
```

—— `dashboard.c:267-275`

```
    if(!count){
        OLED_ShowLargeString(4,21,"NO EVENTS");
        center_small(46,event_text(EventLog_State()));return;
    }
```

—— `dashboard.c:291-293`

于是级联场景下用户看到的是**两块屏、两个词**：

屏幕上	级联（`LOG_FOREIGN`）	事件区自己锁死（`EVENT_LOCKED`）
FLASH 行（`dashboard.c:182`）	`LOCKED`	`READY`
事件页第二行（`dashboard.c:293`）	`NA`	`LOCK`

这里有三个真实问题：

其一，`EVENT_OFFLINE` 被渲染成 `NA`，这是一个信息量为零的文案。 它在三种完全不同的处境下都会出现——事件日志从未启动（级联）、芯片容量不足、`W25Q_Init()` 失败。从 `NA` 出发没有任何办法回溯到"去看 FLASH 行"。

其二，`LOCKED` 与 `LOCK` 太像。 前者是 `LOG_FOREIGN`（传感器区陌生），后者是 `EVENT_LOCKED`（事件区陌生）。两个词只差一个字母，出现在两个不同页面上，而**它们描述的是两个独立的物理区域。**现场人员极易把"FLASH 行写着 LOCKED"误读成"事件日志锁了"。

其三，作者其实已经想过"区分 `OFFLINE` 的成因"，但没有把这份用心扩展到级联场景。 `dashboard.c:50` 的 `default` 分支是唯一一处**当场发起 SPI 事务**去反查芯片的分支：

```
        case LOG_FOREIGN:return "LOCKED";default:return W25Q_ID()!=0 && W25Q_ID()!=0xFFFFFF ? "UNSUP" : "ABSENT";
```

—— `dashboard.c:50`

`LOG_OFFLINE` 落进 `default`，于是屏幕会给出 `UNSUP`（芯片在，但 JEDEC ID 不被识别，即容量不达标）或 `ABSENT`（芯片没响应，即 `W25Q_Init()` 失败）。**这是一处设计得很好的诊断**——它把 `LOG_OFFLINE` 的两种成因分开显示了。缺口在于：`LOG_FOREIGN`（`LOCKED`）没有享受同等待遇，它没有告诉用户"你的事件日志因此没了"。

12.9.3 "同一根因、两种命运"——恢复路径的不对称，确认成立

传感器日志与事件日志共享同一个底层故障源：`W25Q_Busy()` 在 `bus_error` 锁存后恒返回 2。

```
static uint8_t transfer(uint8_t value)
{
    uint16_t start=delay_micros16();
    if(bus_error) return 0xFF;
    while(!(SPI1->SR & SPI_I2S_FLAG_TXE))
        if((uint16_t)(delay_micros16()-start)>1000) {bus_error=1;return 0xFF;}
    SPI1->DR=value;
    while(!(SPI1->SR & SPI_I2S_FLAG_RXNE))
        if((uint16_t)(delay_micros16()-start)>1000) {bus_error=1;return 0xFF;}
    return (uint8_t)SPI1->DR;
}
```

—— `w25qxx.c:9-19`

`bus_error` **没有清零路径**，只在 `W25Q_Init()` 里被重置。这就决定了：**谁有能力重新调用 `W25Q_Init()`，谁就能自愈。**

传感器日志：有完整的指数退避重试，且重试会重跑驱动初始化。

```
        if(Logger_State()==LOG_ERROR&&!flash_retry){
            flash_retry=1;flash_backoff=5000;flash_retry_next=now+flash_backoff;
        }
        if(flash_retry&&(Logger_State()==LOG_ERROR||Logger_State()==LOG_OFFLINE)&&
            due(now,flash_retry_next)&&!DHT11_Busy()){
            Logger_Init(now);flash_retry_next=delay_millis()+flash_backoff;
            if(flash_backoff<60000UL){flash_backoff*=2; if(flash_backoff>60000UL)flash_backoff=60000UL;}
            if(Logger_State()==LOG_SCANNING)flash_retry=0;
        }
```

—— `main.c:624-631`

而 `Logger_Init` 的第一件事之一就是**重新初始化总线**：

```
void Logger_Init(uint32_t now)
{
    state=LOG_OFFLINE;write_phase=recent_count=0;scan_slot=count=latest_slot=0;latest_seq=0;
    memset(per_sector,0,sizeof(per_sector));
    if(W25Q_Init() || W25Q_Capacity()<REGION_BYTES) return;
    region_base=W25Q_Capacity()-REGION_BYTES;state=LOG_SCANNING;timeout=now+2000;
}
```

—— `logger.c:61-67`

`W25Q_Init()` 清掉 `bus_error`，`state` 重置回 `LOG_SCANNING` 重新扫描。退避序列 **5s → 10s → 20s → 40s → 60s（封顶）**（`main.c:630`），并且**只要扫描启动就把 `flash_retry` 归零**（`main.c:631`），留下 `main.c:633` 作为终态清理。这是一条完整、克制的自愈路径。

事件日志：`_Task` 在错误态直接空转，且 `EventLog_Init` 永远不会被第二次调用。

```
void EventLog_Task(uint32_t now,uint32_t uptime_seconds)
{
    uint8_t b[32],slot_state,busy,kind;
    EventLast event;
    if(state==EVENT_OFFLINE||state==EVENT_LOCKED||state==EVENT_ERROR)return;
    if(Logger_State()!=LOG_READY)return;
```

—— `eventlog.c:134-139`

`EVENT_ERROR` 一旦进入，`EventLog_Task` 就是**永久空操作**。唯一的出路是重新调用 `EventLog_Init`，而调用点只有两处：

```
        EventLog_Init(delay_millis(),reset_code);post_event_started=1;
```

—— `main.c:467`（POST 阶段）

```
        if(!event_init_attempted&&Logger_State()==LOG_READY){
            EventLog_Init(now,(uint8_t)(reset_flags>>24));event_init_attempted=1;
        }
        if(event_init_attempted)EventLog_Task(now,uptime_seconds);
```

—— `main.c:612-615`

`event_init_attempted` 的三种赋值**没有一种是清零**：

```
    uint8_t flash_retry=0,event_init_attempted=0;
```

—— `main.c:492`（初始 0）

```
    event_init_attempted=post_event_started;
```

—— `main.c:522`（继承 POST 结果）

```
            EventLog_Init(now,(uint8_t)(reset_flags>>24));event_init_attempted=1;
```

—— `main.c:613`（置 1，此后永不复位）

所以：**`EventLog_Init` 在整个程序生命周期内最多被调用一次。** 一旦它启动过的事件日志进入 `EVENT_ERROR`，就没有任何代码路径能回到它。

"同一根因、两种命运"确认成立，机制差异可以这样概括：

维度	传感器日志	事件日志
错误源	`bus_error` 锁存 → `W25Q_Busy()` 恒返 2	同一个
出错后 `_Task` 行为	`logger.c:91` 直接 `return`（不再推进）	`eventlog.c:138` 直接 `return`（不再推进）
是否有重试状态机	**有**：`flash_retry` + `flash_backoff`（`main.c:624-631`）	**无**
重试触发条件	`LOG_ERROR || LOG_OFFLINE`，且 `!DHT11_Busy()`	不存在
是否重跑驱动初始化	**是**：`Logger_Init` 内部调 `W25Q_Init()`（`logger.c:65`），清 `bus_error`	不适用（从未重试）
初始化次数上限	每次退避到点各一次，无上限	**1 次**（`event_init_attempted` 只置位不复位）
结果	瞬时干扰后可自愈	**永久失效，直到复位**

最值得指出的一点：`EventLog_Init` 本身完全支持被重复调用。 它的前 6 行（`eventlog.c:85-90`）把 `state`、`pending`、`foreign`、`have_latest`、`boot_recorded`、五个槽指针、`session_count`、`latest_seq`、`boot_count`、`latest` 结构体、两张会话表全部重置。**代码是准备好恢复的，只是没有人再调用它。** 对称地加一条退避重试即可，但它需要与 `boot_count` 的自增语义一起改（见问题 7：`boot_count++` 在 `eventlog.c:159` 无条件执行，重试会导致重复自增）。

本章发现的问题与建议

问题 1：`LOG_FOREIGN` 会级联导致事件日志完全无法启动，且该状态不可恢复、成因在屏幕上被隐藏。

- **现象**：传感器日志区扫到一片非本固件数据即锁成 `LOG_FOREIGN`（`logger.c:98`）。由于 `EventLog_Init` 要求 `Logger_State()==LOG_READY`（`eventlog.c:91`）、调度侧也以同一条件放行（`main.c:612`），事件日志的 `base` 永远不会被赋值，`EventLog_State()` 恒为 `EVENT_OFFLINE`。
- **证据**：`logger.c:98`、`eventlog.c:91`、`main.c:612-615`、`main.c:633`。
- **风险（依据 12.9 修正过一次）**：在**代码层面**这两种故障其实是可分的——`(LOG_FOREIGN, EVENT_OFFLINE)` 是级联独有，`(LOG_READY, EVENT_LOCKED)` 是事件区自己锁死（见 12.9.1 配对表）。真正的风险在**呈现层**：区分信息只存在于 `Logger_State()` 一侧，而屏幕上它被放在 FLASH 行显示为 `LOCKED`（`dashboard.c:50`、`dashboard.c:182`），事件页却只显示 `NA`（`dashboard.c:273`、`dashboard.c:293`）——`NA` 在"从未启动 / 容量不足 / `W25Q_Init` 失败"三种处境下取值相同，无法回溯到上游原因；且 `LOCKED` 与事件页的 `LOCK` 仅差一字母、分处两屏。此外 `main.c:633` 明确把 `LOG_FOREIGN` 归入"不必重试"，使该状态不可恢复。
- **建议**：① `EventLog_Init` 的放行条件改为 `Logger_State()==LOG_READY || Logger_State()==LOG_FOREIGN`——事件日志区的安全性本就由它自己的陌生判据（`eventlog.c:156`）独立保证，不依赖传感器日志的状态；②把 `event_text()` 里 `EVENT_OFFLINE` 的文案从 `NA` 改为能区分成因的字符串（例如"看 FLASH 行"，或直接复用 `flash_state(Logger_State())` 的结果），并把事件页的 `LOCK` 改为 `EV LOCK` 以避免与 `LOCKED` 混淆；③更彻底的做法是让 `EventLog_State()` 对"从未初始化"返回一个**非零**的独立值（当前 `EVENT_OFFLINE` 恰为枚举首位 0，与静态零初始化位相同，见 12.9.1），从根上消除歧义。

问题 2：`base` / `region_base` 的 4KiB 对齐是巧合，无静态断言保护。

- **现象**：`W25Q_EraseStart` 要求 `(address&4095)==0`（`w25qxx.c:80`），而两个基址靠 `EVENT_BYTES=8192`、`SENSOR_BYTES=65536` 恰好是 4096 倍数才对得齐。
- **证据**：`w25qxx.c:80`、`logger.c:8`、`eventlog.c:7-8`、`logger.c:66`、`eventlog.c:92`。
- **风险**：任何人把 `EVENT_BYTES` 改成非 4096 倍数，事件日志会在第一次扇区轮换时**静默地永久失败**（`W25Q_EraseStart` 恒返回 1 → `EVENT_ERROR`）。
- **建议**：加编译期断言 `_Static_assert(EVENT_BYTES % 4096 == 0 && SENSOR_BYTES % 4096 == 0, "flash regions must be 4KiB aligned");`，并在 `Logger_Init` / `EventLog_Init` 里加运行时校验 `if(base & 4095) return;`。

问题 3：`logger.c:66` 的 2s 扫描看门狗在扫描态实际上从不触发，一旦触发就丢弃全部扫描结果。

- **现象**：`timeout` 在 `Logger_Init` 里一次性设定（`logger.c:66`），扫描态只在 `if(busy)` 时检查（`logger.c:95`）。扫描期间器件基本不忙，看门狗形同虚设；若某次读取后 BUSY 异常不落，2s 后第一次检查即报 `LOG_ERROR`，此前已扫出的最多 2047 条记录全部作废。
- **证据**：`logger.c:66`、`logger.c:95`、`logger.c:96-105`。
- **风险**：慢速但健康的 Flash 会让 2048 槽扫描的墙上时间超过 2s，此时只要中途撞上一次 busy 就会误报 `LOG_ERROR`，进而触发 `main.c:624-633` 的 5s→60s 退避重试循环。事件日志侧则完全无超时（`eventlog.c:143`），形成另一种极端。
- **建议**：把 `timeout` 改成"每完成一个槽就重置"（`timeout=now+200`，语义变为单槽进展看门狗）；`eventlog` 的扫描也应补上对称的超时检查。

问题 4：`W25Q_Busy()` 返回 2（通信错误）被两套日志统一当作终结错误，且事件日志没有恢复路径。

- **现象**：`bus_error` 在 `transfer()` 里锁存（`w25qxx.c:12-17`），此后 `W25Q_Busy()` 恒返回 2（`w25qxx.c:52`），两处都在读到 2 时直接转 `LOG_ERROR` / `EVENT_ERROR`（`logger.c:93`、`eventlog.c:141`）。传感器日志有完整退避重试（`main.c:624-633`：5s 起步、×2、上限 60s），事件日志只有一次性标志 `event_init_attempted`（`main.c:492`、`main.c:612-615`），**没有任何代码重新调用 `EventLog_Init`**。
- **证据**：`w25qxx.c:14`、`w25qxx.c:17`、`w25qxx.c:52`、`logger.c:93`、`eventlog.c:141`、`eventlog.c:138`、`main.c:492`、`main.c:612-615`、`main.c:624-633`。
- **风险**：一次瞬时干扰会让两块日志同时进入错误态；传感器日志能自愈，事件日志**永久失效**。这是两块日志之间最重要的健壮性不对称。
- **建议**：在 `main.c:624-633` 旁加一条对称的事件日志退避重试分支，复用同一个 `flash_backoff`（或另置一份）。可行性已由 12.9.3 确认：`EventLog_Init` 的前 6 行（`eventlog.c:85-90`）会把状态与会话表全部重置，**它本身完全支持被重复调用**，缺的只是有人再调用它。两个必须一并修正的前置问题：① `boot_count++` 目前在扫描结束时无条件执行（`eventlog.c:159`），而 `boot_recorded` 要到 `eventlog.c:185` 才置位，重试会导致 `boot_count` 重复自增并留下孤儿会话（见问题 7）；② `event_init_attempted` 当前在 `main.c:492/522/613` 三处赋值且**没有一处是清零**，必须引入可复位的重试标志。

问题 5：序号回绕无处理，`(int32_t)` 差值比较在回绕点给出错误方向，且两套日志的"谁更新"判定缺失对称。

- **现象**：`pending.sequence=latest_seq+1`（`logger.c:83`）、`put32(raw+4,latest_seq+1UL)`（`eventlog.c:76`）都是 32 位无符号自增；扫描判定用 `(int32_t)(record.sequence-latest_seq)>0`（`logger.c:100`）与 `(int32_t)(seq-latest_seq)>0`（`eventlog.c:150`）。`eventlog.c:46` 已有现成的 `newer()` 辅助函数，`logger.c` 却没有。
- **证据**：`logger.c:83`、`logger.c:100`、`eventlog.c:46`、`eventlog.c:76`、`eventlog.c:150`。
- **风险**：按 60s 一条的速率，32 位序号需约 8160 年才回绕，**实际不可达**，风险等级低。真正的隐患是这段逻辑**没有任何注释声明它依赖"差值小于 2^{31}"这一前提**，后续维护者提高写入频率时会踩坑。
- **建议**：把 `logger.c:100` 的比较改为复用 `newer()` 语义，并在两处补注释显式声明前提。

问题 6：`LogRecord.flags` 占 4 字节只用 2 位，且 `flags==0` 的记录被 `Logger_Save` 直接拒绝。

- **现象**：`put32(raw+20,r->flags&3)`（`logger.c:41`）用 4 字节存 2 位；解码时 `get32(b+20)>3` 作为有效性判据（`logger.c:30`）。`Logger_Save` 首条判据是 `if(state!=LOG_READY || !(record->flags&3)) return 1;`（`logger.c:82`）。
- **证据**：`logger.c:41`、`logger.c:30`、`logger.c:82`、`logger.h:6-7`、`main.c:149-153`。
- **风险**：`flags==0`（DHT 无效且姿态无效）时存档被静默拒绝；调用点只在成功时推进 `log_next`（`main.c:618`），所以主循环会每周期重试直到 DHT 恢复——不丢数据，但跳过事件对用户不可见。格式上浪费了 30 位。
- **建议**：把 `flags` 压缩为 1 字节并预留 6 位给状态位（如 `MPU_STALE`、`DHT_ERROR`、`POSE_FLIPPED`），并给被拒绝的存档增加计数器供 UI 显示"因传感器无效跳过 N 次"。

问题 7：BOOT 记录提交失败会造成"孤儿 `boot_count`"，该会话永久不可见却占槽。

- **现象**：`boot_count++` 在扫描结束时就执行（`eventlog.c:159`），BOOT 记录的提交是后续异步完成的。若提交失败，Flash 里不存在该 `boot_count` 的 BOOT 记录，但随后的 ALIVE 心跳仍会带这个 `boot_count` 写入（`eventlog.c:78`）。而 `add_session` 只在 `kind==EVENT_BOOT` 时被调用（`eventlog.c:148`、`eventlog.c:186`），`EventLog_ReadSession` 也要求 `kind==EVENT_BOOT`（`eventlog.c:109`）。
- **证据**：`eventlog.c:159`、`eventlog.c:185`、`eventlog.c:148`、`eventlog.c:186`、`eventlog.c:109`。
- **风险**：该会话在列表中不可见，但其 ALIVE 记录占据槽位并参与 `update_duration` 聚合（`eventlog.c:149`、`eventlog.c:187`），造成"会话数少于实际开机次数"且槽位被无谓消耗。
- **建议**：改为"提交成功后再自增"——把 `boot_count++` 从 `eventlog.c:159` 移到 `eventlog.c:185` 的 `boot_recorded=1` 旁边。

问题 8：`EVENT_VERIFY` 逐槽校验整扇区，一次轮换消耗 128 次 `_Task()` 调用。

- **现象**：`eventlog.c:168-169` 每次调用只校验一个槽，`if(++verify_slot<write_slot+128U)return;` 要 128 次才走完。
- **证据**：`eventlog.c:167-173`、`eventlog.c:96-98`、`main.c:616`。
- **风险**：这期间 `EventLog_Busy()` 恒为真，导致 `main.c:616` 的传感器存档门控被推迟（最坏推迟一整个校验窗口）；128 槽 × 36 B × 8 bit ÷ 4.5MHz ≈ 16.4ms 的纯总线时间被拉长到 128 个主循环周期。
- **建议**：把校验粒度改为批量读——一次 `W25Q_Read` 读 256 字节（8 个槽）再循环比对，可把 128 次调用压缩到 16 次。

问题 9：`[8..11]` 保留字段与 `reset_code` 的校验语义没有文档化，容易被误认为漏检。

- **现象**：`decode()` 的判据（`logger.c:30`、`eventlog.c:37-40`）只覆盖 MAGIC/提交字节/CRC/枚举取值；`[8..11]` 恒 `0xFFFFFFFF`（`eventlog.c:77`）与 `reset_code`（`eventlog.c:79`、`eventlog.c:43`）**既不单独校验也不参与判定**。
- **证据**：`eventlog.c:37-40`、`eventlog.c:77`、`eventlog.c:79`、`eventlog.c:43`、`eventlog.c:80`。
- **风险**：它们**确实在 CRC32 的覆盖范围内**（`[0..23]`），所以功能上是安全的；缺口在于代码里没有任何注释说明这一点，维护者容易反推出"这里漏检了 `reset_code`"的错误结论，进而画蛇添足地加冗余判据。
- **建议**：在 `decode()` 上方补一段注释，逐条列出"每条判据拦住了哪一类损坏"，并显式写明"`[8..11]` 与 `[22]` 由 CRC32 保护，无需独立判据"。

问题 10：两份 `memset` 的填充值相反，hexdump 人工判读时会误导。

- **现象**：`logger.c:38` 用 `memset(raw,0,sizeof(raw))`，`eventlog.c:75` 用 `memset(raw,0xFF,sizeof(raw))`。由于 `WRITE_BODY` 只写前 31 字节（`logger.c:123`、`eventlog.c:171`），传感器记录的 `[28..30]` 落地为 `0x00`，事件记录落地为 `0xFF`。
- **证据**：`logger.c:38`、`eventlog.c:75`、`logger.c:123`、`eventlog.c:171`。
- **风险**：这 3 个字节两边都不参与校验，因此**不构成功能差异**；但现场用 hexdump 排查时，传感器的 `0x00` 会看起来像"被清零的损坏"，与事件日志的 `0xFF` 表现不一致，增加误判概率。
- **建议**：统一为 `0xFF`（与擦除态一致，便于区分"未写"与"写了零"），或把 `[28..30]` 明确定义为保留域并写注释。

本章小结

两块日志在**同一片 SPI NOR 上共享一个容量锚点**，用 32 字节定长记录 + **单字节原子提交**实现了可证明的掉电安全：提交字节"只能清 1 不能置 1"的物理语义保证记录状态非黑即白，CRC32 负责位翻转，MAGIC 负责"这是不是我写的"。区域布局靠容量减法自下而上生长，顺序依赖（`logger` 先 `LOG_READY` 才允许 `eventlog` 启动）保证了事件日志的 `base` 一定落在传感器日志区之下。

代价有三条：**顺序依赖带来级联失效**（问题 1）、**事件日志只有 2 小时历史窗口**（12.8 节）、**两块日志健壮性不对称**（问题 4）。三条里最值得优先修的是问题 1，因为它把一个可诊断的存储故障变成了一个不可区分、不可恢复的功能消失。

第 13 章　专题三：姿态解算与标定的整数数学

本章要点

- 全链路只有一处 ×100（`px100`），下游全部是 ×10；`pitch=px100/10` 之后直接进 `view.pitch_x10`（`main.c:654`），单位链自洽。**`5730` 的 ×100 在同一行就被 `/10` 消费掉了**，这是读这段代码最容易误判的地方（13.1）。
- 定点基准选 Q8 的真实理由写在注释里：`main.c:43` "Q8，乘法避免左移负数"。`acc[k]` 是有符号的，左移负值在 C 里是未定义行为——这不只是风格（13.2）。
- 一阶低通两档可以算出确切时间常数：正常档 τ=80 ms、f_c≈2.0 Hz；快速档 τ=20 ms、f_c≈8.0 Hz（T=5 ms）。整数除法截断给它留下 ±15 Q8 的极限环（13.3）。
- **滤波器的 α 是按"每次采样"定义的，不是按时间**，而主循环的采样周期会被整屏刷新拉长；工程已经把这个间隔记进 `view.max_sample_gap_ms`，却没有用它（13.3）。
- 快速档的触发阈值只有 600 LSB = **0.0366 g**，且"任一轴触发、三轴同速"（`main.c:63` 注释给出理由）。本次量化：硬件 DLPF 44 Hz 不限制软件滤波器（13.3、13.9）。
- 标定的窗口是 100×10 ms = **1.0 s**（不是 100×5 ms），判据是"有效数 ≥90 + 三轴峰峰 ≤1200 LSB + Z 均值 ≥8000"。峰峰是整数友好的替代品，但**没有异常值剔除**：一次有效而异常的采样就能让整轮白跑（13.4）。
- `stable_text` 的迟滞是 ±0.15°，舍入方向"离开零"；`fixed()` 的截断不引入额外误差，因为它的输入已经是四舍五入过的 0.1° 值（13.5）。
- 同一个工程里有两份**逐字节相同**的整数开方：`dashboard.c:53-59` 与 `controls.c:212/216-217`（13.6）。
- `bubble_target` 把 (roll, −pitch) 归一化了，所以**气泡半径恒为 20 px、只表方向不表幅值**；`dashboard.c:77-78` 的 ±3000 钳位是死代码（13.8）。
- 本章最重的一条：`MIN_CAL_Z 8000` 允许在 **60.77°** 倾角下完成标定，而这会让此后**所有**角度读数被放大到 **2.05 倍**，并且上下不对称——屏幕上没有任何提示（13.10）。
- 误差总账（13.10）：常数取整 +0.007%、小角度线性化 −0.51%…−4.51%、基准倾斜 −？…**+104.8%**、Q8 极限环 ≈3.6×10⁻⁶ g、文本迟滞 ±0.15°。**支配项是基准倾斜，不是线性化。**

13.1 从 14 个字节到屏幕上的小数点：单位链

姿态链路的物理输入只有 14 个字节的突发读：

```
    uint8_t buf[14];
    if (SoftI2C_ReadRegs(MPU6050_ADDR7, MPU6050_REG_ACCEL_XOUT_H, buf, 14) != 0)
    {
        return 1;
    }
    acc[0] = (int16_t)((buf[0] << 8) | buf[1]);     /* AX */
    acc[1] = (int16_t)((buf[2] << 8) | buf[3]);     /* AY */
    acc[2] = (int16_t)((buf[4] << 8) | buf[5]);     /* AZ */
    *temp_raw = (int16_t)((buf[6] << 8) | buf[7]);  /* 温度 */
    gyro[0] = (int16_t)((buf[8] << 8) | buf[9]);    /* GX */
```

（`mpu6050.c:60-73`，逐字摘录）

一次事务取回加速度三轴、温度、角速度三轴。**角速度在本工程里不参与姿态解算**：倾角全部由 `lpf[]`（加速度计路径）算出（`main.c:586-587`），`gyro[]` 只被原样搬进 `view.gyro_raw`（`main.c:578`）与日志。这不是缺陷（水平仪不需要陀螺），但读者容易误以为 ±250 °/s 的量程与倾角精度有关。

真正需要小心的是**单位**。同一个物理量在链路上换过五次单位，而 C 的类型系统对此毫无帮助：

量	类型	单位	赋值点
`acc[k]`	`int16_t`	原始 LSB（±2 g 档，16384 LSB/g）	`mpu6050.c:67-69`
`lpf[k]`	`int32_t`	**LSB × 256（Q8）**	`main.c:43`、`main.c:71`
`base_ak`	`int32_t`	原始 LSB（均值）	`main.c:128`
`px100` / `rx100`	`int32_t`	**0.01°**	`main.c:586-587`
`pitch` / `roll`	`int32_t`	**0.1°**	`main.c:588`
`pitch_text` / `roll_text`	`int32_t`	0.1°（四舍五入后的）	`main.c:589`
`view.pitch_x10` / `roll_x10`	`int32_t`	0.1°	`main.c:654`
`Controls_SetLevelOffset(pitch,roll)`	`int32_t`	0.1°	`controls.c:210`、`main.c:590`
`level_distance`	`uint16_t`	0.1°（径向，封顶 15.0°）	`controls.c:218`
`LogRecord.pitch_x10`	`int16_t`	0.1°	`logger.c:40`、`main.c:151`
`DashboardView.temperature_x10`	`int16_t`	0.1 °C	`dashboard.c:146`

这张表能一次解释三处"看起来少了 10 倍"的地方：

1. `px100` 叫 `_x100` 是**字面意思**（0.01°），`pitch=px100/10` 是降到 0.1°，不是降到度。
2. `main.c:654` 是 `view.pitch_x10=pitch;`——字段名与变量名同为"×10"，**没有换算**，所以 `pitch` 必须是 0.1° 而不是度。
3. `controls.h:16` 的注释 `/* Radial degrees x10, capped at 15deg. */` 与 `Controls_SetLevelOffset` 里的 ±150 钳位一致：150 就是 15.0°。

也就是说，**`5730` 里的 ×100 只活了一行**：`px100` 是 0.01°，`main.c:588` 立刻 `/10` 变成 0.1°，之后全链路都是 0.1°。`5730 = 180/π × 100` 的推导与它的小角度误差见第 7 章 7.8 节，本章不重推。

链路的最后一跳是显示。`fixed()` 用整数除法与取模拼小数点：

```
static void fixed(char *buf,int32_t value,uint8_t sign)
{
    uint32_t mag=(uint32_t)(value<0 ? -value : value);
    if(sign) sprintf(buf,"%c%lu.%lu",value<0?'-':'+',(unsigned long)(mag/10),(unsigned long)(mag%10));
    else sprintf(buf,"%s%lu.%lu",value<0?"-":"",(unsigned long)(mag/10),(unsigned long)(mag%10));
}
```

（`dashboard.c:60-65`，逐字摘录）

`sign=1` 时正数也带 `+`（PITCH/ROLL 两栏，`dashboard.c:136/138`），`sign=0` 时负数才带 `-`（温度、湿度那两栏，`dashboard.c:146-147`）。先取绝对值再拆位，所以不存在"-0.5"被写成"+0.5"这类符号丢失。

符号约定本身是装配文档：`PITCH_DIR (-1)`、`ROLL_DIR (1)`（`main.c:36-37`）吸收"哪个轴朝前、模块装反了要乘 −1"，与数值换算无关。

13.2 为什么是 Q8 乘法而不是左移

滤波器的定点基准是 8 位小数，但代码里看不到一个 `<<8`：

```
static int32_t lpf[3]; /* Q8，乘法避免左移负数 */
```

（`main.c:43`）

```
    for (k = 0; k < 3; k++) {
        int32_t target = (int32_t)acc[k] * 256;
        if (!filter_valid) lpf[k] = target;
        else lpf[k] += (target - lpf[k]) / (1L << shift);
    }
```

（`main.c:68-72`）

这条注释值得展开，因为它记的是一个**语言层面的正确性问题**而不是风格问题：

- `acc[k]` 是 `int16_t`，板子倾斜时它是**负值**。`(acc[k] << 8)` 对负值左移在 C 标准里是未定义行为（C99 §6.5.7 只对非负值可表示时定义左移）；ARM Compiler 5 不会报错，只会按"乘法"生成 `LSL`，于是同一个表达式在换编译器或开更高优化等级后行为可能改变。写成 `* 256` 则**完全定义**。
- 右移那边反而是安全的：`(1L << shift)` 的左操作数是正的常量，`shift ∈ {2,4}`。

顺带把这几个定点表达式都做一次溢出审计（32 位 `int`）：

表达式	上界	是否安全
`(int32_t)acc[k] * 256`	32767 × 256 = 8.39×10⁶	✅ 远小于 2^{31}
`target - lpf[k]`	≤ 1.68×10⁷	✅
`LPF_FAST_LSB * 256L`	600 × 256 = 153600	✅ 是常量
`lpf[k] = cal_sum[k]/cal_good*256`	32767 × 256	✅ 先除后乘，中间值更小
`pitch*pitch + roll*roll`（钳位后）	$150^2 \times 2 = 45000$	✅

值得单独记一笔的是 `(int32_t)acc[k] * 256` 里的**强制转换其实是防御性的**：`int16_t` 在 ARM 上会提升为 32 位 `int`，所以即使不写转换，`acc[k] * 256` 也不会溢出。作者的意图是让"这里要按 32 位算"显式化——同一个 `mpu6050.c` 里全是 `(int16_t)((buf[0] << 8) | buf[1])` 这种写法，风格一致。

`lpf[k]/256` 用于取回 LSB（`main.c:586-587`、`main.c:142`）。这是**向零截断**的有符号除法：`-100/256` 得 0 而不是 −1。误差上界是 1 LSB = 1/16384 g ≈ 6.1×10^{-5} g，折算到角度约 0.0035°，比后面所有误差项都小两个数量级，可以忽略——但它确实是一个**在零点附近不对称的量化**，本节把它记下来是为了闭合审计，而不是因为它重要。

13.3 一阶低通的离散化：τ、f_c、极限环

滤波器的全部实现只有 15 行：

```
static void filter_feed(const int16_t *acc)
{
    uint8_t k, shift = LPF_SHIFT;
    /* 三轴使用同一速度，快速转动时保持轨迹方向。 */
    if (filter_valid)
        for (k = 0; k < 3; k++)
            if (abs32((int32_t)acc[k] * 256 - lpf[k]) > LPF_FAST_LSB * 256L)
                shift = LPF_FAST_SHIFT;
    for (k = 0; k < 3; k++) {
        int32_t target = (int32_t)acc[k] * 256;
        if (!filter_valid) lpf[k] = target;
        else lpf[k] += (target - lpf[k]) / (1L << shift);
    }
    filter_valid = 1;
}
```

（`main.c:60-74`，逐字摘录）

它是一个单极点 IIR（指数滑动平均），系数 `α = 2^-shift`：

档位	shift	α	时间常数 τ = T/α	等效截止 f_c = 1/(2πτ)
正常	`LPF_SHIFT` = 4（`main.c:26`）	1/16	5 ms × 16 = **80 ms**	**1.99 Hz**
快速	`LPF_FAST_SHIFT` = 2（`main.c:27`）	1/4	5 ms × 4 = **20 ms**	**7.96 Hz**

（T = `SAMPLE_INTERVAL_MS` = 5 ms，`main.c:21`）

这套时间常数和硬件滤波器是自洽的。 `MPU6050_Init` 把片上数字低通设成 44 Hz：

```
        /* 数字低通 44Hz：水平仪要的是稳，不是快 */
        if (SoftI2C_WriteReg(MPU6050_ADDR7, MPU6050_REG_CONFIG, 0x03)) return 1;
```

（`mpu6050.c:35-36`）

`CONFIG[2:0]=3` 对应加速度计带宽 44 Hz、延迟 4.9 ms；采样率 `SMPLRT_DIV=0x04` 对应 1 kHz/(1+4) = 200 Hz（`mpu6050.c:30-33`）。200 Hz 采样对 44 Hz 抗混叠带宽是安全余量，而软件滤波器的 2 Hz（正常）与 8 Hz（快速）**都远低于 44 Hz**——意味着主导动态特性的是软件滤波器，硬件滤波器不会拖慢快速档。三处数字放在一起看才是完整设计。

触发快速档的阈值只有 0.0366 g。 `LPF_FAST_LSB * 256L = 600 × 256 = 153600`（Q8），除以 256 得 600 LSB，再除以 16384 LSB/g：

```
600 / 16384 = 0.0366 g
```

这个阈值离"静止噪声"很近，所以它实际测的是"**这一拍和滤波器当前值差多少**"，而不是绝对加速度。这个设计选择有正反两面：好处是只要手一动就立刻跟上；代价是**抖动也会触发**，快速档下的噪声抑制只有 1/4 系数（f_c≈8 Hz），而显示屏刷新是 33 ms（`LEVEL_DISPLAY_INTERVAL_MS`，`main.c:23`）——刚好在同一量级，屏幕上看不出抖动。

"任一轴触发、三轴同速"是刻意的，注释给出了理由（`main.c:63`）。这段代码的结构值得注意：`shift` 在更新循环**之前**只算一次，所以：

1. 三个轴共用同一个 α，**本次调用内不会出现某轴已经跟上、另一轴还在滞后**的状态；
2. 触发判据读的是**本拍更新前**的 `lpf[k]`，三轴看到的是同一个时刻的状态；
3. 逐轴独立切换是很容易写出来的"更精确"版本，但它会让三轴的时间常数在同一拍内不同，于是 (x, y) 的**方向**在过渡期被扭曲——气泡指向会甩一下。这就是注释说的"保持轨迹方向"。

整数除法留下一个极限环。 `(target - lpf[k]) / (1L << shift)` 是向零截断的整数除法：当 `|target - lpf[k]| < 2^shift` 时增量正好是 0，`lpf` 停住不动。

档位	死区宽度	折算成 LSB	折算成 g
正常（shift=4）	±15 Q8	±0.0586 LSB	$\pm 3.6\times10^{-6}$ g
快速（shift=2）	±3 Q8	±0.0117 LSB	$\pm 7.1\times10^{-7}$ g

折算到角度是 10^{-4} 度量级，**对读数完全无关紧要**；它的真正意义是：`lpf` 永远不会精确等于 `target`，稳态下一直在真值附近 ±15 Q8 内游走。审计的结论是"存在、但被量化阶淹没"。

这里有一条更重要的结构问题。 α 是**按采样次数**定义的（每拍乘 1/16），而 `SAMPLE_INTERVAL_MS` 只是一个"至少间隔 5 ms"的门（`main.c:570-574`）：整屏刷新时 `OLED_RefreshBusy()` 会把这一拍整体推迟（`main.c:649`），第 9 章量过整屏刷新是 37…48 ms 量级。于是**当主循环被拖慢时，滤波器在墙钟时间上的 τ 会被等比放大**——一次 48 ms 的刷新间隔让这一段的等效 τ 从 80 ms 涨到接近 1 s。

这个效应**工程是看得见的**：`main.c:571-573` 专门统计它。

```
            elapsed=now-last_sample_try;last_sample_try=now;
            if(elapsed>view.max_sample_gap_ms)
                view.max_sample_gap_ms=(uint16_t)(elapsed>999?999:elapsed);
```

（`main.c:571-573`）

它被记录下来、显示出来，却**没有反馈到滤波器**。用 `elapsed` 去缩放 α（或改用 `lpf += (target-lpf)*elapsed/(elapsed+τ)` 的形式）能把 τ 钉在墙上时间上，代价是每次一次除法。这不是"写错了"，而是"测了但没用"——本章把它列为问题 3。

13.4 标定：100×10 ms 的整数统计

标定是后台串行运行的，采样分支每拍喂一次：

```
static void cal_feed(uint32_t now,const int16_t *acc,uint8_t valid)
{
    uint8_t k,ok;
    if(!cal_active || !due(now,cal_next))return;
    cal_next=now+10;cal_attempts++;
    if(valid){cal_good++;for(k=0;k<3;k++){
        cal_sum[k]+=acc[k];if(acc[k]<cal_lo[k])cal_lo[k]=acc[k];if(acc[k]>cal_hi[k])cal_hi[k]=acc[k];
    }}
    if(cal_attempts<CAL_SAMPLES)return;
    ok=cal_good>=CAL_MIN_GOOD;
    if(ok){for(k=0;k<3;k++)if((int32_t)cal_hi[k]-cal_lo[k]>CAL_MOTION_LSB)ok=0;
        if(cal_sum[2]/cal_good<MIN_CAL_Z)ok=0;}
    cal_active=0;
    if(ok){
        base_ax=cal_sum[0]/cal_good;base_ay=cal_sum[1]/cal_good;base_az=cal_sum[2]/cal_good;
        for(k=0;k<3;k++)lpf[k]=cal_sum[k]/cal_good*256;
        filter_valid=calibrated=1;level_reset();
    }else{cal_error=1;cal_retry=now+1000;}
}
```

（`main.c:114-132`，逐字摘录）

第一件容易算错的事是窗口长度。 采样节拍是 5 ms（`SAMPLE_INTERVAL_MS`），但标定自己又加了一道 10 ms 的门（`cal_next=now+10`），所以：

```
100 次尝试 × 10 ms = 1.0 s
```

即**每次 5 ms 采样中有一次会因为 `!due(now,cal_next)` 被丢掉**，实际只用上一半的样本。窗口是 1 s 而不是 0.5 s。（顺带：`cal_attempts` 在 `due()` 过关时就自增，与此拍 `valid` 无关，所以窗口长度不受读数失败影响。）

三道判据的含义各不相同，全用整数：

判据	常量	物理含义	作用
`cal_good >= 90`	`CAL_MIN_GOOD`（`main.c:18`）	100 次里至少 90 次读到有效数据	容忍 ≤10 次 I2C 失败
`cal_hi[k]-cal_lo[k] <= 1200`	`CAL_MOTION_LSB`（`main.c:19`）	三轴**峰峰** ≤ 0.0732 g	"别动"
`cal_sum[2]/cal_good >= 8000`	`MIN_CAL_Z`（`main.c:20`）	Z 均值 ≥ 0.4883 g	"别竖着放"

为什么是峰峰而不是方差/标准差？ 因为峰峰只需要两个 `int16_t` 哨兵和四次比较，**零乘法、零除法**；方差需要先平方求和（`int32` 溢出风险）再除以样本数。在一个连 `float` / `double` 都不许出现的工程里（`src-utf8` 全树检索 `\b(float|double)\b` **零命中**，本章已复核），这是个务实的选择。

代价是**峰峰不抗异常值**：它取的是极差，一个尖峰就撑满整个区间。而 `cal_good>=90` 只统计"读没读到"，读到却异常的单点**照样进 `cal_lo/cal_hi`**。所以一次有效而畸变的采样会让整轮 1 s 白跑，然后再等 1 s 重试（`cal_retry=now+1000`，`main.c:131`）。换成"排序后取 5%…95% 分位"或"去掉最大最小各若干点"能把这一类误判挡掉，代价是几十字节 RAM 与一次插入排序。

两个值得记的数值边界：

- `cal_sum[3]` 是 `int32_t`（`main.c:105`），累加上界 100 × 32767 = 3,276,700，离 2^{31} 还有 **655 倍**余量。也就是说 `CAL_SAMPLES` 就算被抬到六万也不会溢出——这个余量比看上去大得多。
- `cal_lo[k]=32767; cal_hi[k]=-32768;`（`main.c:112`）是哨兵初值。**如果出现"100 次尝试、0 次有效"的极端情况**，`cal_good==0` 会让 `ok=cal_good>=90` 先判假，于是判据短路、除法不执行；但如果有人把 `CAL_MIN_GOOD` 改成 0，`(int32_t)cal_hi[k]-cal_lo[k]` 会算出 **−65535**，而 `-65535 > 1200` 为假——**静止判据就此恒定通过**，同时 `cal_sum[2]/cal_good` 会除零。两个错误都会被同一次改动引入，而它们都不是显式写出来的依赖。这属于"隐式不变量"，本章列为问题 6。

失败路径是一个完整的状态机。 失败置 `cal_error=1` 并把重试时间推到 1 s 后，主循环在四个条件同时成立时重新开始：

```
        if(!calibrated&&!cal_active&&sensor_ready&&!stale&&due(now,cal_retry))cal_begin(now);
```

（`main.c:604`）

`!stale` 这一条很关键：它保证重试不会被"传感器还没回来"的抖动打断。整条链路上，标定失败时用户在 LEVEL 页看到的是 `PLACE FLAT`（`dashboard.c:117-118`），绿灯常亮变成红灯呼吸（`main.c:644`、`controls.c:124`），信息是够的。

但 `calibrated` 一旦置位就再不复位。 全树检索 `calibrated` 的写点只有 `main.c:130` 一处，其余全为读（`main.c:583/604/636/644/647/650`）。这意味着**现场没有重新标定的入口**：板子被挪到别的台面、或者通电时是歪的（标定把歪姿态记成了基准），唯一恢复手段是断电重来。这一条本章列为问题 4——它的影响正好接在 13.10 的"基准倾斜放大"上。

13.5 `stable_text`：0.15° 迟滞与舍入方向

屏幕上的数字不是每拍都变，中间隔了一层迟滞：

```
static int32_t stable_text(int32_t raw_x100,int32_t previous)
{
    if(abs32(raw_x100-previous*10)<15)return previous;
    return raw_x100>=0?(raw_x100+5)/10:(raw_x100-5)/10;
}
```

（`main.c:133-137`，逐字摘录）

单位在这一行里被验证得很干净：`previous` 是 0.1°，`previous*10` 把它放回 0.01° 与 `raw_x100` 比较；返回值 `(raw_x100±5)/10` 回到 0.1°。**如果 `pitch_text` 是"度"而不是 0.1°，这一行会立刻失效**——所以它同时是单位约定的自检点。

两件事值得分开看：

迟滞窗口是 ±15（0.01°）= ±0.15°，围绕上一次**显示过的**值。所以数字不会在阈值附近来回跳：要改变显示，真值必须离开当前显示值 0.15° 以上。在一个 f_c≈2 Hz 的滤波器下游（13.3），这个量级是合适的——它挡的是最后一位的抖动，不是真实运动。

舍入方向是"离开零"：`(raw+5)/10` 与 `(raw-5)/10` 让正负两侧对称，都做四舍五入。这一点比看起来重要：如果写成 `raw/10`，负角度会向零截断、正角度也向零截断，负半轴的显示会**整体偏向零**，在 ±15° 附近造成 0.1° 的符号相关偏差。`(raw-5)/10` 正是为了避免这个。

滞后的代价是**读数有 ±0.05° 的量化 + 0.15° 的迟滞窗**，合计最坏 0.2° 的"显示值落后真值"。与 13.10 的线性化误差（10° 处 −0.51% = 0.05°，30° 处 −4.51% = 1.35°）相比，**在 10° 以内迟滞是主导误差，在 20° 以外线性化是主导误差。**这个交叉点对"水平仪到底准不准"这个问题是决定性的：**在小角度（真正需要精度的区间）里，限制精度的不是数学近似而是显示迟滞。**

13.6 两份逐字节相同的整数开方

`sqrt()` 在这个工程里出现了两次，实现完全相同：

```
static uint32_t root(uint32_t n)
{
    uint32_t result=0,bit=1UL<<30;
    while(bit>n)bit>>=2;
    while(bit){if(n>=result+bit){n-=result+bit;result=(result>>1)+bit;}else result>>=1;bit>>=2;}
    return result;
}
```

（`dashboard.c:53-59`）

```
    uint32_t n,result=0,bit=1UL<<30;
    if(pitch>150)pitch=150;else if(pitch< -150)pitch=-150;
    if(roll>150)roll=150;else if(roll< -150)roll=-150;
    n=(uint32_t)(pitch*pitch+roll*roll);
    while(bit>n)bit>>=2;
    while(bit){if(n>=result+bit){n-=result+bit;result=(result>>1)+bit;}else result>>=1;bit>>=2;}
    level_distance=(uint16_t)(result>150?150:result);
```

（`controls.c:212-218`）

`dashboard.c` 里是 `static` 函数，`controls.c` 里是就地内联——**算法一行不差**，包括 `bit=1UL<<30` 这个"从 2^{30} 开始、每次右移 2 位"的经典逐位复原法（restoring square root）。

两份拷贝等价吗

等价，而且是**强等价**：同一个逐位复原算法、同一组类型（`uint32_t n, bit`）、同一初值（`1UL<<30`）、同一循环结构，没有任何一处用整型提升或舍入差异把两条路径分开。对同样的输入，两边都返回 $\lfloor\sqrt{n}\rfloor$（向下取整），对 $n < 2^{32}$ 全域成立。

真正不同的是**调用点上多做的那一步**：

调用点	得到	舍入
`bubble_target`（`dashboard.c:79-80`）	`root(n)` 之后 `if(d*d<n)d++;`	**向上取整** $\lceil\sqrt{n}\rceil$
`Controls_SetLevelOffset`（`controls.c:216-217`）	直接用循环结果	**向下取整** $\lfloor\sqrt{n}\rfloor$

所以正确的抽法是"共用函数返回 $\lfloor\sqrt{n}\rfloor$，向上取整留在调用点"——因为 `dashboard.c:80` 的注释解释了它为什么必须上取整（`/* Round up so the target remains strictly inside the bowl. */`），那是**气泡的几何需求**，不是开方函数的语义。这也正是"复制粘贴"最典型的代价：**两份拷贝眼下等价，但它们各自被上层的不同需求推着走**；改动其中一份的理由，在另一份那边根本不存在。

省得下多少字节

map 里有实测尺寸（`level\MDK-ARM\List\level.map`）：

符号	地址	Thumb Code
`dashboard.o(i.root)`	`0x08006b94`	**44 字节**（`level.map:1821-1822`）
`controls.o(i.Controls_SetLevelOffset)`	`0x0800049c`	**104 字节**（`level.map:2189`）
`dashboard.o(i.bubble_target)`	`0x080034dc`	118 字节（`level.map:1692-1693`）

两条引用关系把"内联"这件事坐实了：

- `level.map:508` `dashboard.o(i.bubble_target) refers to dashboard.o(i.root) for root` —— `root` 是一段**真实存在的函数**，被 `bubble_target` 调用（1 条 `BL`）；
- `level.map:293` 里 `controls.o(i.Controls_SetLevelOffset)` 的引用列表**只有** `controls.o(.data) for level_distance`，**没有**对 `root` 的引用——因为 `root` 是 `dashboard.c` 的 `static`，`controls.c` 根本调不到它（这也是复制发生的原因）。紧邻的 `level.map:292` 是同一张清单里**已被链接器删掉的** `Controls_LevelDistance` 的同类引用行（见问题 9），两条容易看串行。

于是同一段两级循环在映像里存在了两份：一份是 44 字节的 `root`，另一份被内联在 `Controls_SetLevelOffset` 的 104 字节里。

`Controls_SetLevelOffset` 的 104 字节能拆成：两次 ±150 钳位（约 24 字节）、`pitch*pitch+roll*roll`（约 10）、内联开方、末级 `result>150?150:result` 与写 `level_distance`（约 12）、函数进出（约 8）。**扣掉这些，内联的那份开方与 44 字节的 `root` 同量级（约 36–44 字节）。**（这一步是按指令类型估的，工作区里没有 `.axf`/反汇编——`level\MDK-ARM\List\` 只有 `level.map` 和启动文件的 `.lst`，所以只能给区间，不能给精确值。）

共享的净收益因此是"约 28–40 字节 Flash"，占 64 KB 的 0.05% 以下。这个量级不足以成为重构理由——本章把问题 8 记为问题，靠的不是字节数，而是下面两条：

1. 内联那份**不共享改进**：`root` 将来若补一次 Newton 收敛步或加上界保护，`controls.c` 那边不会有；
2. 两台调用点对**同一函数采用了不同的舍入约定**，而这个差异只写在 `dashboard.c:80` 的注释里。

选这个算法的理由很硬：

- Cortex-M3 **没有 FPU**，`sqrt()` 会从数学库拉进整条软件浮点链；
- 全树 `float` / `double` **零命中**（本章复核），map 文件里也没有 `__aeabi_f*` 系列符号；
- 逐位法只用移位、比较、加减，**零乘法、零除法**；
- 位数固定，最坏 17 次循环（`bit` 从 2^{30} 降到 ≤ n 约 9 次，再 8 次主体），不依赖 n 的分布。

`dashboard.c:80` 还有一行补正：

```
    if(d*d<n)d++; /* Round up so the target remains strictly inside the bowl. */
```

`d*d` 在 32 位下有**翻转风险**：`root(0xFFFFFFFF)` 返回 65535，`d++` 后 `d*d` 溢出为 0、`0 < n` 成立，d 变成 65536。当前不可达——但**兜住它的不是那两行钳位。** `dashboard.c:77-78` 的 ±3000 是一段死代码（见 13.8：`view.pitch_x10` 实际到不了 ±3000，且归一化让钳位不改变输出），真正提供安全性的是**这个量本身的取值范围**：`|pitch_x10| ≤ 2188`（推导见 13.10.2），于是 $n \le 2\times2188^2 = 9.57\times10^6$、$d \le 3094$、$(d+1)^2 \le 9.6\times10^6$，离 2^{32} 还有 447 倍余量。`controls.c` 那边 $n \le 2\times150^2 = 45000$、$d \le 213$，更安全。

这个区别是实质性的：**读者会以为"钳位保护了溢出"，实际上钳位什么也没保护**，安全性来自上游数值范围的巧合。一旦将来出现重标定入口（问题 4 建议的那类），`base_az` 可能被改得更小、量程随之放大，这条隐式契约就会失效，而钳位不会拦住它。所以 `root()` 应当在入口加一句显式上界（问题 8 建议），让安全性不再依赖调用方。

13.7 ±150 径向钳位与它的消费者

`Controls_SetLevelOffset` 做了两级钳位（`controls.c:210-219`，见 13.6 摘录）：

1. **逐轴**钳到 ±150（= ±15.0°）；
2. 算出径向距离 `sqrt(pitch²+roll²)` 后再钳到 150。

第二级才是它真正输出的量：`level_distance` = **径向倾角（0.1°），封顶 15.0°。** `controls.h:16` 的注释 `/* Radial degrees x10, capped at 15deg. */` 与实现一致。

于是有一个角点：`pitch=roll=150` 的径向距离是 212.1，被钳回 150。**灯光无法区分"单轴 15°"与"两轴各 10.6°"**——两者给出同一个 `level_distance=150`。这是有意的（灯只管"有多歪"），但值得写下来，因为它是径向钳位的直接后果而不是 bug。

消费点在灯的配色上：

```
    else if(mode==LIGHT_OFFSET){
        uint16_t distance=level_distance;
        target[0]=(uint16_t)((uint32_t)distance*190U*256U/150U);
        target[1]=(uint16_t)((uint32_t)(150-distance)*140U*256U/150U);
    }
```

（`controls.c:126-130`）

红色 ∝ distance、绿色 ∝ (150−distance)：**水平时最绿、越歪越红**，中间是两者的线性混合。Q8 量纲下 `190×256 = 48640`、`140×256 = 35840`，都远小于 `uint16_t` 上限 65535，没有翻转风险。

这一支还有自己的平滑，不走通用的淡入淡出：

```
        if(mode==LIGHT_OFFSET&&fade_ticks==LIGHT_FADE_MS){
            int32_t delta=(int32_t)target[k]-light_value[k];
            /* 60ms smoothing; rounded steps also converge at very low light. */
            light_value[k]=(uint16_t)(light_value[k]+(delta>=0?(delta+11)/12:(delta-11)/12));
        }
```

（`controls.c:134-137`）

`(delta±11)/12` 与 13.5 的 `stable_text` 是**同一个技巧**：向零截断的整数除法在低亮度时会"卡死"（增量算成 0），加上分母的一半就能保证收敛。`controls.c:136` 的注释把它说明白了："rounded steps also converge at very low light"。全工程用同一套整数收敛技巧，是个好的一致性信号。

顺带记一条 API 层的事实：`Controls_LevelDistance()`（`controls.c:220`）在全树**零调用**——只有声明（`controls.h:16`）与定义，内部用的是模块私有变量 `level_distance`（`controls.c:11`）。它与第 15 章登记的 5 个死宏属于同一类问题（对外暴露了一个没人用的接口），但性质更轻：变量本身在用。

13.8 气泡映射：15.0° 满量程，之后只剩方向

气泡位置的计算只有 8 行：

```
static void bubble_target(int32_t pitch,int32_t roll,int32_t *x,int32_t *y)
{
    uint32_t n,d;
    if(pitch>3000)pitch=3000;else if(pitch< -3000)pitch=-3000;
    if(roll>3000)roll=3000;else if(roll< -3000)roll=-3000;
    n=(uint32_t)(pitch*pitch+roll*roll);d=root(n);
    if(d*d<n)d++; /* Round up so the target remains strictly inside the bowl. */
    if(d<150)d=150;
    *x=roll*20L*256/(int32_t)d;*y=-pitch*20L*256/(int32_t)d;
}
```

（`dashboard.c:74-83`，逐字摘录）

关键在第 81 行：分母不是模长本身，而是 `max([模长], 150)`。写成闭式：

```
|(x, y)| = 20·256·√(pitch² + roll²) / max([√(pitch² + roll²)], 150)
```

`150` 就是 §13.7 里那个 150——**0.1° 单位的 15.0°**。于是映射分两段，转折点是精确的：

径向倾角	分母 d	气泡半径	行为
< 15.0°（真模长 < 150）	被抬到 **150**	`20 × 径向/150`，从 0 线性涨到 20 px	**幅值指示器**，1.333 px/°
≥ 15.0°	真模长	恒为 **20 px**	**只剩方向**：半径锁死，气泡沿 20 px 圆周滑动

临界角是精确的 15.0°，而且它与 `controls.c` 的 ±150 径向钳位是**同一个常量**：`controls.h:16` 的 `/* Radial degrees x10, capped at 15deg. */` 一句话同时管着两处。这一点必须写清，因为最容易得出的错误结论是"分母永远是模长，所以半径恒定"——那会漏掉 `if(d<150)d=150;` 这一行，也会漏掉下面整段用户可见后果。

`d` 的两种角色因此是分开的：

- **`d ≥ 150`（大角度）**：`d` 只剩"把 (pitch, roll) 单位化"的作用，幅值被约掉；
- **`d` 被 150 顶住（小角度）**：分子里的真模长没被约掉，`d` 退化成**满量程常量**，`*x/*y` 就是标准的定点线性映射。

也就是说，"除以固定满量程 150 而不是除以 d"这个改法，代码在 15° 以内**本来就是这个行为**；`d` 只是为了 15° 以外还保留方向。

20 px 是被几何顶出来的

碗画在 (31, 32)、半径 28（`dashboard.c:127` 的 `OLED_DrawCircle(31,32,28,1)`），气泡本体是半径 7/6/4 的三个同心圆盘加一个高光点（`dashboard.c:133-134`）。`20 + 7 = 27 ≤ 28`——气泡贴到圆周时仍严格在碗内。

`dashboard.c:80` 的向上取整正是为这条几何服务的：`d = ⌈√n⌉ ≥ √n` 使半径**严格小于** 20 px，永不越界；若改用 `⌊√n⌋`，半径会**大于** 20 px，在模长小的象限能顶到 28 px 之外。这就是那行注释 `/* Round up so the target remains strictly inside the bowl. */` 的完整含义——它保的不是"数值好看"，是"圆盘不出碗"。

向上取整还有一个副作用：饱和区的半径不是精确常数，而是一个**锯齿**。当 `⌈√n⌉ = k+1` 而真模长落在 `(k, k+1)` 内时，半径在 `20·k/(k+1)` 与 `20` 之间摆动，摆幅 `20/(k+1)` px——`k=150` 时 0.132 px，`k=573`（57.3°）时 0.035 px。**全部低于一个像素**，画不出来；写在这里只是为了让"恒为 20 px"这句话有准确边界。

±3000 那两行是死代码

`dashboard.c:77-78` 把 pitch/roll 钳到 ±3000 才开始算。它在**两个独立意义上**都不生效：

1. **量程上够不到**：`view.pitch_x10` 的实际取值范围是 ±2188（推导见 13.10.2），距 3000 还差 27%；
2. **即使够到也不改变输出**：在饱和区半径本来就被约掉，`n` 变小不改变方向；在线性区（`d` 被 150 顶住）分子分母**同步**缩放，方向与半径都不变。钳位唯一能改变的是**方向**——当两轴同时超过 3000 时，它把合成方向折向 45° 线——而第 1 条说这不可达。

所以这是一段**看起来在保护、实际上什么也不保护**的代码。危险不在于多余，而在于它让读者以为"气泡的输入被限幅了"，从而不会去问"输入的真实范围是多少"——而 `root()` 的溢出安全性恰好依赖那个真实范围（见 13.6 末段）。

同一段钳位在 `trend_store`（`dashboard.c:27-28`）里也出现了一次，同样不可达：那里把值存进 `int16_t`，而 ±3000 距 `int16_t` 的 ±32767 还有 10 倍余量。

屏上的实际后果：两把量程差 6 倍的尺

把映射和 §13.4 的判据叠起来，LEVEL 页上有两个**量程差 6 倍**的视觉线索：

线索	几何	折算到倾角
四个靶括号（`dashboard.c:129-132`）	以 (31,32) 为中心、半宽 **8 px** 的正方形	每轴 **±6.0°**
气泡圆盘本体（`dashboard.c:133`）	半径 **7 px**	每轴 ±5.25°
`LEVEL` / `TILT` 文字（`main.c:88-93`、`dashboard.c:135`）	气泡中心距圆心 ≤ **1.33 px**	每轴 **±1.0°**（`LEVEL_ENTER_X10 10`）

（括号是四个 L 形角标：(23,24)-(26,24)-(23,27) 等，相对中心正好是 (±8, ±8)；折算用 15° 以内的线性段 `20 px / 15.0° = 1.333 px/°`，此时径向 ≤ 8.49°，落在线性段内，自洽。）

于是：

- **靶括号是一把 ±6° 量级的"粗尺"**——气泡盘正好填满括号时约 5–6°，这是设计的可读极限；
- **`LEVEL` 文字是一把 ±1° 量级的"细尺"**——它要求气泡中心几乎压在圆心上，比括号紧 **6 倍**。用户把气泡稳稳放进括号里（"看起来水平了"），文字仍很可能显示 `TILT`；
- **超过 15° 之后气泡不再提供任何幅值信息**。而 `pose_check` 要到 **75.52°**（`base_az/4` ⇒ `cos θ < 0.25`，`main.c:141-142`）才把整页清空成 `TILT RANGE`。也就是说 **15.0°–75.5° 这 5 倍的角度范围全部压在同一个 20 px 圆周上**：气泡只告诉你"往哪边歪"，歪多少只能读右边的 `PITCH` / `ROLL` 数字。

这个 15° 满量程不止管气泡，还管灯：`level_distance`（`controls.c:218`）用同一个 150 封顶，`controls.c:127-129` 的 `LIGHT_OFFSET` 红分量 `distance*190*256/150` 因此在 15.0° 就到达最大值。**模拟量反馈子系统整体是 15° 满量程**——气泡、RGB、`controls.h:16` 的注释三者一致，这是设计上的一处自洽；不一致的是它与 ±3000 那两行所暗示的 300° 量程。

平滑那一支（`dashboard.c:95-99`）在 20 px 的圆周上做插值，插值路径会**穿过圆内**——这正是想要的观感（气泡"滑"过去而不是绕过去）。重绘门限是 160 Q8 = 0.625 px：

```
        delta=bubble_x-(int32_t)bubble_draw_x*256;
        if(delta>160||delta< -160)bubble_draw_x=bubble_pixel(bubble_x);
```

（`dashboard.c:99-100`）

它保证航向没变时不重画（省 OLED 刷新时间），与第 9 章的差分块刷新是同一思路。

13.9 一次没有副作用的初始化：写-回读-校验

`MPU6050_Init` 是本工程里唯一一处"写完立刻回读比对"的驱动代码：

```
        for (i = 0; i < 5; i++)
        {
                if (SoftI2C_ReadReg(MPU6050_ADDR7, regs[i], &value) || value != values[i]) return 1;
        }
```

（`mpu6050.c:50-53`）

```
        /* ACK 只表示收到字节；回读确认量程、滤波和时钟真正生效。 */
```

（`mpu6050.c:49`）

I2C 写事务的 ACK 只证明从机收到了字节，不证明寄存器接受了值（保留位、只读位、上电时序都可能吃掉它）。这 5 次回读把"配置生效"从假设变成了实测，代价是 5 次寄存器读（每次约 2 个总线字节 + 起始/停止）。对一个开机只做一次的函数，这是划算的。

里面有一处**与厂商例程的刻意偏离**，值得完整引用，因为它是全工程唯一被注释记录下来的"原厂给的值是错的"：

```
        /* 加速度 ±2g，不自检。
           注意这里必须是 0x00：原厂 51 例程给的是 0x01，
           那个值会打开加速度的高通滤波，把重力分量滤掉，算倾角就废了。 */
        if (SoftI2C_WriteReg(MPU6050_ADDR7, MPU6050_REG_ACCEL_CONFIG, 0x00)) return 1;
```

（`mpu6050.c:41-44`）

`ACCEL_CONFIG` 的 bit[3] 是 `ACCEL_HPF` 使能路径的一部分（经 `CONFIG` 的 DHPF 模式选择）。一旦高通生效，**重力这个直流分量会被滤掉**，而整套倾角算法（`Δax/base_az`）恰恰只有重力可依——13.1 的整条链路会归零。这条注释的价值在于它挡住了一个"照抄例程就悄悄失效"的坑。

`MPU6050_TempToC` 是同一文件里唯一有**数值缺陷**的函数，但这条缺陷**没有接进任何在跑的通路**——它是死代码：

```
/* 数据手册给的换算：TEMP = TEMP_OUT / 340 + 36.53
   这里只用整数运算，避免把浮点库拉进工程 */
int16_t MPU6050_TempToC(int16_t temp_raw)
{
        return (int16_t)(temp_raw / 340 + 36);
}
```

（`mpu6050.c:94-99`，逐字摘录）

注释引的是数据手册的 `+ 36.53`，代码写的是 `+ 36`：**若这个函数被接通，它会引入单向的 −0.53 °C 偏差**。整数除法的截断误差只有 ±1/340 ≈ 0.003 °C，比 0.53 小两个数量级，所以偏差的几乎全部来自那个被丢掉的 `.53`。

它确实没有接通，而且不是"零调用"这种要靠 grep 推断的结论——链接器已经把它删掉了：

```
    Removing mpu6050.o(i.MPU6050_ReadAccel), (78 bytes).
    Removing mpu6050.o(i.MPU6050_TempToC), (16 bytes).
```

（`level\MDK-ARM\List\level.map:956-957`）

断言的强度差别很重要：`static` 函数零调用只会触发编译告警，而这个函数是**非 static 的公开 API**（`mpu6050.h:43` 有声明），编译器不会因为它没被调用而抱怨——只有链接器会。map 里的 `Removing` 行是链接期实测，等价于"整个映像里不存在这段代码"。

在跑的路径是**原样透传**，从不换算：

- `main.c:575` 读到 14 字节突发里的温度计原值 `temp_raw`；
- `main.c:579` `view.temperature_raw=temp_raw;` 直接存进视图（`dashboard.h:17` 的 `int16_t accel_raw[3],gyro_raw[3],temperature_raw;`）；
- `dashboard.c:200` `sprintf(text,"TEMP RAW %d",v->temperature_raw);` 在 RAW DATA 页按**计数原值**显示——单位不是摄氏度，文案自己写了 `RAW`。

而屏幕与日志里**所有真正的温度都来自 DHT11**，与这条链无关：

- CLIMATE 页：`const DHT11_Data *d=DHT11_Get();`（`dashboard.c:143`）→ `large_fixed(text,d->temperature_x10);`（`dashboard.c:146`）；
- 日志：`r.temperature_x10=d->temperature_x10;r.humidity_x10=d->humidity_x10;`（`main.c:150`）；
- DHT11 自己的换算：`t = (int16_t)(b[2]*10 + (b[3] & 0x7F));`、负号由 `if (b[3] & 0x80) t = -t;` 给出（`dht11.c:26-27`），送入 `DHT11_Decode` 的两个出参（`dht11.c:96`）。

所以本章列为问题 10 的理由**不是**"现在有 0.53 °C 误差"，而是：工程里躺着一个量纲和注释都明确、名字看起来随时可以调用的公开 API，而它从未被验证过——第一次被接通时，测试者会以为它是对的。修法与"删掉未用 API"或"修正换算"二选一，但无论选哪个，都必须先承认它当前是**被链接器裁掉的死代码**，而不是一条在跑的误差源。

13.10 误差总账与替代方案

13.10.1 五项误差的量级排序

#	误差源	大小	出处
1	常数取整（5730 vs 5729.578）	**+0.007%**	第 7 章 7.8 节（`main.c:586-587`）

#	误差源	大小	出处
2	小角度线性化（`sinθ ≈ θ`）	−0.51% @10°、−2.02% @20°、−4.51% @30°	第 7 章 7.8 节误差表
3	**基准倾斜导致的比例放大 `1/cosθ₀`**	+1.54% @θ₀=10°，**+104.8% @θ₀=60.77°**	本章新增，见 13.10.2
4	Q8 极限环（13.3）	±3.6×10⁻⁶ g ≈ 0.0002°	本章 13.3
5	显示迟滞 + 量化（13.5）	±0.15°（迟滞）+ ±0.05°（量化）	本章 13.5

第 1、2 项已在第 7 章推完并给出误差表，本章只引用。**把它们和 3、4、5 放在一起排，结论会变**：

- 在 **10° 以内**，误差由第 5 项（±0.2°）支配——而这是水平仪最常用的量程；
- 在 **20° 以外**，误差由第 2 项支配（30° 处 −1.35°）；
- **第 3 项一旦被激活，它压倒其它所有项**，而它恰恰是唯一一项**由开机那一秒的姿态决定、之后再也改不掉**的误差。

13.10.2 `MIN_CAL_Z` 允许的开机姿态能把读数放大到 2.05 倍

标定的 Z 均值判据是 `cal_sum[2]/cal_good >= 8000`（`main.c:20`、`main.c:125`）。第 7 章 7.4 节已经算出它等价于"**允许板子在 Z 轴朝上的姿态附近最多倾斜约 60.8°**"。本章要接着算的是**这件事对读数的后果**，这是前文没有覆盖的部分。

角度公式的分母是 `base_az`（`main.c:586-587`）。物理上 `base_az = g·cos θ₀`（θ₀ = 标定时的倾角），所以：

> 读数 ∝ Δa_x / base_az = Δa_x / (g·cos θ₀)

分母比重力模长小 `cos θ₀` 倍，于是整条读数被放大 `1/cos θ₀` 倍。 `MIN_CAL_Z` 允许的最小 `base_az` 是 8000 LSB：

标定时倾角 θ₀	`base_az` / g	放大倍数 `1/cos θ₀`	相对误差
0°（水平放置）	1.0000	1.000	—
10°	0.9848	1.0154	**+1.54%**
30°	0.8660	1.1547	+15.5%
60.77°（`MIN_CAL_Z` 的上限）	0.4883	**2.048**	**+104.8%**

`θ₀ = 60.77°` 是从 8000/16384 = 0.48828 反解出来的（`arccos(0.48828) = 60.77°`），也就是**判据刚好通过的那一点。**

更麻烦的是**放大之外还有不对称。**完整的读数是

> pitch_text ≈ 573 · (sinθ − sinθ₀) / cosθ₀　　（θ = 当前真实倾角）

θ₀ = 0 时它退化成 `573·sinθ`：正负对称，天花板 57.3°（`sinθ ≤ 1`），也就是**在水平标定的常用情形下，屏幕读数存在一个 57.3° 的硬天花板**——板子侧立（90°）时读数显示 57.3°，而 `pose_check` 只在 `raw_z < 0` 时才判 `POSE_FLIPPED`（`main.c:141`），Z 在 (0, base_az/4) 区间判 `POSE_RANGE`（`main.c:142`）。

θ₀ = 60.77° 时它变成：

真实倾角 θ	`pitch_text` 显示值
+90°	**+149**（+14.9°）
+60.77°（标定点）	0（正确）
0°（水平）	−992（−99.2°）
−90°	**−2197**（−219.7°）

同一个物理姿态，标定歪了之后读数能差 20 倍以上，而且上下不对称。 屏幕对此毫无提示：`dashboard.c:117-118` 在 `!v->mpu_valid` 时才会显示 `PLACE FLAT` / `FLIP BOARD` / `TILT RANGE`，而标定**成功**之后 `mpu_valid` 为真，那三行提示就再也不出现了。用户看到的是一个自信的、错误的数字。

13.10.2.1 读数的取值范围上限：±2188

§13.6 与 §13.8 都引用了 `|pitch_x10| ≤ 2188` 这个上界，这里把它推出来。它由三个已经出现过的量共同决定：

约束	来源	数值
重力在 ±2g 量程下的满幅	`ACCEL_CONFIG=0x00`（`mpu6050.c:44`）	16384 LSB
标定的 Z 门限	`MIN_CAL_Z`（`main.c:20`、`main.c:125`）	`base_az ≥ 8000`
姿态有效性门限	`pose_check` 的 `base_az/4`（`main.c:142`）	即时 `z ≥ 2000`

于是（静态、`|a| ≤ 1 g`，同号相减取反向极端）：

```
|base_ax|    ≤ √(16384² − 8000²) = 14294
|lpf[0]/256| ≤ √(16384² − 2000²) = 16261
|Δ|          ≤ 14294 + 16261     = 30555
|px100|      ≤ 5730 × 30555 / 8000 = 21885
|pitch_x10|  ≤ 2188
```

2188 比 3000 低 27%——这是 §13.8 判定 `dashboard.c:77-78` 为死代码的第一条依据，与第二条（缩放不变）相互独立。换用第 4 章实测的克隆件（1 g ≈ 18400 LSB）重算，上界只抬到 `5730 × (16571 + 18291) / 8000 / 10 ≈ 2497`，**仍低于 3000 约 17%**，所以这个结论对替换件同样成立。

顺带回答一个代码阅读中的自然疑问：`dashboard.c:137/139` 的 `pitch_text>999||pitch_text< -999?" OVER"` 分支**不是死代码**——在 θ₀=0 的情形下它不可达（天花板 573），但上表说明 θ₀ 接近 60.77° 时它可以被触发（−2197）。它是一个"只在标定已经歪到极致时才生效"的兜底，而那时它显示的是 `OVER` 而**不是**"标定有问题"。

建议（选一即可，按代价从小到大）：

1. 把 `MIN_CAL_Z` 从 8000 提到 `base_az` 与重力模长成比例的无量纲判据，例如要求 `base_az ≥ 0.9·|a|`（等价于 θ₀ ≤ 25.8°）——只改一个常量，把放大倍数压到 1.11 倍以内；
2. 或者把分母换成即时模长 `sqrt(ax²+ay²+az²)`，从根本上消掉 `1/cos θ₀`（代价：三次乘法、两次加法、一次整数开方——13.6 的 `root()` 现成可用，第 13.6 节的复制品正好可以在这里被复用）；
3. 至少在标定成功时记录 `base_az`，并在 `base_az < 0.9·16384` 时于 LEVEL 页给一行"基准偏斜"提示（`dashboard.c:106-110` 的 `center_small()` 已经具备居中显示一行字的能力）。

13.10.3 若要覆盖大角度：四种替代方案

当前方案的本质是**"用最省的方式只保证小角度"**。如果产品要求覆盖到 60°（比如做成坡度计），可选路径如下：

方案	Flash 成本	运行时间	精度	备注
现状：`5730·Δa/base_az`	0	~3 次乘法、1 次除法	10° 处 −0.51%，30° 处 −4.51%	已在用
`asin` 查表	574 项 × 2 B ≈ **1.15 KB**（0.1° 步进到 57.3°）	O(1)，一次除法求索引	可到 0.1°	最省事的正确解；用 `sin` 索引可省掉反查
CORDIC	~50 行代码	12…16 次迭代 × (1 移位 + 1 加 + 1 比较)	0.1°	无乘法，但迭代次数固定占用时间
定点 `atan2` 多项式	~200 B 表 + 若干乘法	一次 32/32 除法（M3 有硬件除法）+ 3…4 次乘加	0.1°	能同时处理大角度与象限
浮点 `asinf`	会拉进 `__aeabi_f*` / `__aeabi_d*`，**数 KB**	视实现	最高	本工程刻意避开；map 里当前没有任何 `__aeabi_f*` 符号

结论：如果只是想把范围做大，最小改动是**加一张 574 项的 `asin` 表**——它不需要浮点、不需要除法、不影响现有架构，而且能把 13.10.2 的第 2、3 项误差一起解决（因为查表可以同时对 `Δa/base_az` 和模长归一化两种输入建表）。**真正不该做的是"为了准而引入浮点"**：Cortex-M3 没有 FPU，代价是代码尺寸和中断延迟同时上升，而这个工程从头到尾没有一行浮点。

`level\MDK-ARM\List\level.map` 把最后这句话从"grep 无匹配"升级成了链接期确证，而且证据比想象的更彻底——**连浮点库的目标文件都没进映像**：

- 同一组 `printf` 入口有十个变体（`printf0.o` … `printf8.o`、`printfa.o`、`printfb.o`）。工程里 9 处 `sprintf` 全部解析到**纯整数**的 `printf8.o`（`level.map:489`、`:519`、`:528`、`:533`、`:541`、`:560`、`:566`、`:576`、`:585`，即 `__2sprintf → printf8.o(i.__0sprintf$8)`），映像里也只有它的五个段（`level.map:2465-2473`），组件表一行 `1160 50 0 0 0 420`（Code 1160 B、内联数据 50 B、Debug 420 B，`level.map:2673`）。
- 带浮点的那个变体是另一种状态：`level.map:1457` `../clib/microlib/printf/printf.c 0x00000000 Number 0 printfa.o ABSOLUTE` ——**尺寸 0、地址 0**，整份文件没有进入映像。它的 `_fp_digits` 引用的正是那 6 个 double 例程（`level.map:904-908`：`__aeabi_dmul`、`__aeabi_ddiv`、`__aeabi_cdrcmple`、`__aeabi_dadd`、`__aeabi_d2ulz`），而这 6 个例程与它一起被摘掉：`dadd.o(.text) 334`、`dmul.o(.text) 228`、`ddiv.o(.text) 222`、`dfixul.o(.text) 48`、`cdrcmple.o(.text) 48`、`depilogue.o(.text) 186` ——**合计 1066 字节**（`level.map:1411-1416`）。连 `%e` / `%f` / `%g` 的格式入口也只剩 `stubs.o` 里 0 字节的 ABSOLUTE 符号（`_printf_fp_dec`、`_printf_fp_hex`，`level.map:2064-2065`）。
- 于是"这个工程没有浮点"有了三重证据：C 源里没有一行浮点写法、MicroLIB 的浮点 printf 变体尺寸为 0、double 运行库六个目标文件被链接器摘除。反过来说，**一旦有人在 `sprintf` 里写下一个 `%f`，链接器就必须把这 1066 字节拉回映像**——这是可测量的代价，而不是风格问题。

这三重证据里，前一条来自源级 grep（对 `level/USER` 全树搜 `\b(float|double)\b|\d+\.\d+f|\(float\)`，无匹配），后两条来自链接器：grep 只能证明"没有写"，链接器才能证明"即使写了也没有进映像"。

本节引用的 `level.map` 行号取自工作区中 `20261011F` 那一次构建；`20261010E` 构型的同一批符号行号见「摘要」末节「版本边界：本书描述哪一版代码」。

本章发现的问题与建议

问题 1：`bubble_target` 的 ±3000 钳位是死代码；气泡的满量程是 15.0°，超出后只表方向。

- **现象**：`dashboard.c:77-82` 先把 pitch/roll 钳到 ±3000，再用 `max([模长], 150)` 作分母。钳位在**两个独立意义上**都不生效：量程上够不到，且缩放上不改变输出。
- **证据**：`*x=roll*20L*256/(int32_t)d;*y=-pitch*20L*256/(int32_t)d;`（`dashboard.c:82`），`d` 由 `dashboard.c:79-81` 逐步给出（`d=root(n)` → `if(d*d<n)d++` → `if(d<150)d=150`）。闭式 `|(x,y)| = 20·256·√(p²+r²) / max([√(p²+r²)], 150)`。量程依据：`|pitch_x10| ≤ 2188`（13.10.2），比 3000 低 27%。
- **影响**：映射有两段行为、一个精确临界角——径向 <15.0° 时半径从 0 线性涨到 20 px（1.333 px/°），≥15.0° 时锁死 20 px、只表方向；而 15.0°–75.5°（`pose_check` 的清屏门限，`main.c:142`）这 5 倍范围全部压在同一圆周上。另外，钳位会让人以为"输入已经限幅"，而 `root()` 的溢出安全性实际依赖的是那个真实范围、不是钳位（13.6）。
- **建议**：删掉 `dashboard.c:77-78`（`dashboard.c:27-28` 同样不可达，一并删），把 15.0° 抽成与 `controls.c` 共享的宏（见问题 2）；若想要更大的气泡量程，只改这一个宏。

问题 2：15.0° 这个量程常量以字面量 150 写在两个文件的三个地方，没有单一来源。

- **现象**：`controls.c:213-214`（逐轴钳位）、`controls.c:218`（径向封顶）、`dashboard.c:81`（分母下限）三处是同一个物理常量 15.0°，行为彼此一致，却没有任何一处是宏或具名常量；唯一带注释的声明（`controls.h:16` 的 `/* Radial degrees x10, capped at 15deg. */`）挂在 `Controls_LevelDistance()` 这个零调用接口上。
- **证据**：`if(pitch>150)pitch=150;else if(pitch< -150)pitch=-150;`（`controls.c:213-214`）、`level_distance=(uint16_t)(result>150?150:result);`（`controls.c:218`）、`if(d<150)d=150;`（`dashboard.c:81`）。
- **影响**：改量程（例如从 ±15° 放到 ±45°）要同时改三处且分属两个文件；漏掉 `dashboard.c:81` 会让气泡 15° 就饱和而灯到 45° 才满红，漏掉 `controls.c:218` 则相反。这不是 bug，是量程常量缺少单一来源——**而真正被写成显式数字的 ±3000 反倒是死代码（问题 1）**。
- **建议**：`#define TILT_FULL_SCALE_X10 150`（放 `controls.h` 或新建 `tilt.h`），三处引用，并把那句 `capped at 15deg` 的注释从零调用接口挪到宏上。

问题 3：滤波器的 α 按采样次数定义，主循环被拖慢时时间常数被等比放大；工程已经测出这个间隔却没有用它。

- **现象**：`filter_feed` 每次调用固定乘 `2^-shift`（`main.c:71`），而调用节拍由 `if(due(now,sample_next))` 决定（`main.c:570`），`old=now` 与 `now` 的差值可以是 5 ms，也可以在整屏刷新被推迟时变成几十毫秒。
- **证据**：`sample_next=now+SAMPLE_INTERVAL_MS`（`main.c:574`）只是"至少 5 ms"；间隔被实测并存进 `view.max_sample_gap_ms`（`main.c:571-573`）；而第 9 章量过整屏刷新是 37…48 ms 量级。
- **影响**：整屏刷新期间等效 τ 从 80 ms 涨到接近 1 s，倾斜响应出现可感知的滞后；而 `max_sample_gap_ms` 已经把这个事实暴露在屏幕上，代码却没有任何回馈。
- **建议**：把更新式改成时间加权形式 `lpf[k] += (target-lpf[k])*(int32_t)elapsed/(int32_t)(elapsed+tau)`（`tau = 80` 或 `20`，与当前两档的 τ 一致），或在 `elapsed` 明显大于 `SAMPLE_INTERVAL_MS` 时降低这一拍的权重。前者每次一次除法，性能可接受。

问题 4：`calibrated` 置位后从不复位，现场没有重新标定的入口。

- **现象**：全树 `calibrated` 的写点只有 `main.c:130` 一处，其余五处全是读（`main.c:583/604/636/644/647/650`）。主循环的重试条件是 `!calibrated && ...`（`main.c:604`）。
- **证据**：`filter_valid=calibrated=1;`（`main.c:130`）是唯一赋值；`grep calibrated` 无第二处赋值。
- **影响**：板子换台面、或通电时姿态不对（标定把歪姿态记成基准），唯一恢复手段是断电。结合问题 5 的放大效应，一次歪标定会一直生效到下一次上电。整机没有任何"长按重置基准"的入口。
- **建议**：给一个显式的重标定入口（例如同页长按 X 键，复用 `events_pause` / `history_pause` 那套暂停原语），在里面清 `calibrated/filter_valid` 并调 `cal_begin(now)`；或至少把"开机 5 s 内检测到倾角突变则重标定"作为兜底。

问题 5：`MIN_CAL_Z 8000` 允许的标定姿态会把此后所有角度读数放大到 2.05 倍，且上下不对称、屏幕无提示。

- **现象**：判据是 Z 均值 ≥8000 LSB（`main.c:20`、`main.c:125`），而角度公式的分母正是这个 `base_az`（`main.c:586-587`）。
- **证据**：`8000/16384 = 0.48828 → θ₀ = 60.77°`，放大倍数 `1/cos θ₀ = 2.048`；θ₀=60.77° 时 `pitch_text ≈ 573·(sinθ−0.8727)/0.4883`，正方向天花板 +149、负方向到 −2197。屏幕只在 `!v->mpu_valid` 时才给 `PLACE FLAT` / `TILT RANGE` 提示（`dashboard.c:117-118`），标定成功后该分支不再进入。
- **影响**：最坏情况下读数误差 +104.8%（而线性化误差在 30° 处才 −4.51%），且用户看到的数字自信而错误。这是本章量级最大的误差项，也是唯一一项由"开机那一秒"锁定、之后改不掉的误差。
- **建议**：把判据改成无量纲形式（要求 `base_az ≥ 0.9·|a|`，等价 θ₀ ≤ 25.8°，把放大压到 1.11 倍内）；或把分母换成即时模长 `sqrt(ax²+ay²+az²)`，这时 13.6 的 `root()` 可以直接复用；或至少记录 `base_az` 并在偏斜时于 LEVEL 页加一行提示（`center_small()` 现成）。

问题 6：标定没有异常值剔除，一次有效而畸变的采样就会让整轮 1 秒白跑。

- **现象**：运动判据用的是三轴**峰峰**（`cal_hi[k]-cal_lo[k] > CAL_MOTION_LSB`，`main.c:124`），峰峰取极差；`cal_good>=90` 只统计"I2C 读有没有成功"，读到却畸变的值照样进 `cal_lo/cal_hi`（`main.c:119-121`）。
- **证据**：`cal_lo/cal_hi` 的更新在 `if(valid)` 内（`main.c:119`），没有中位数/分位/去极值的处理；失败后 `cal_retry=now+1000`（`main.c:131`）意味着代价是整整 1 s。
- **影响**：标定偶发失败率被单点噪声抬高；在电平噪声较大的克隆件上（工作区记录实测 1 g≈18400 LSB）这一条更容易触发。
- **建议**：用"去掉最大最小各 5 点"或 16 点滑窗中位数代替峰峰；或者更便宜的做法——把峰峰判据改成"超过阈值的样本数 ≤5"，单点畸变就只算一次违规。

问题 7：`cal_lo/cal_hi` 的哨兵初值在"零有效样本"假设下会让静止判据恒定通过。

- **现象**：`cal_lo[k]=32767; cal_hi[k]=-32768;`（`main.c:112`）。若一次有效样本都没有，`(int32_t)cal_hi[k]-cal_lo[k]` 得 **−65535**，而 `-65535 > 1200` 为假——判据"通过"。
- **证据**：`main.c:112` 与 `main.c:124`；同时 `cal_sum[2]/cal_good`（`main.c:125`）会除零。
- **影响**：当前不可达，因为 `ok=cal_good>=CAL_MIN_GOOD` 先判假（`main.c:123`）短路了整段。但如果有人把 `CAL_MIN_GOOD` 改成 0（听起来像"先别卡这么严"的正常调整），会**同时**引入"静止判据恒过"与"除零"两个错误。这是一条只由语句顺序保证的隐式不变量。
- **建议**：把哨兵初始化改成显式判空——`if(cal_good==0){cal_error=1;cal_active=0;return;}` 放在 `cal_attempts<CAL_SAMPLES` 那行之后；或者在 `main.c:124` 前加一句 `if(cal_good<CAL_MIN_GOOD){cal_error=1;cal_active=0;return;}`，让两个判据都不依赖短路。

问题 8：同一个工程里有两份逐字节相同的整数开方，两份的舍入约定还在调用点上分了叉。

- **现象**：`root()` 是 `dashboard.c` 的 `static` 函数（`dashboard.c:53-59`），`controls.c` 在 `Controls_SetLevelOffset` 里就地内联了同一段算法（`controls.c:212`、`controls.c:216-217`）。两份**强等价**（同算法、同类型、同初值，对 `n < 2³²` 全域都返回 `⌊√n⌋`）；差别只在下游：`dashboard.c:80` 加了 `if(d*d<n)d++;` 做**上**取整，`controls.c` 不做（**下**取整）。
- **证据（链接期实测）**：`level.map:1821-1822` `root` Thumb Code **44 字节**；`level.map:2189` `Controls_SetLevelOffset` **104 字节**；`level.map:508` `dashboard.o(i.bubble_target) refers to dashboard.o(i.root) for root`（说明 `root` 是映像里真实存在的函数），而 `level.map:293` 中 `Controls_SetLevelOffset` 的引用列表**只有** `.data for level_distance`、**没有**对 `root` 的引用（`root` 是 `static`，`controls.c` 根本调不到它——这正是复制发生的原因）。
- **影响**：按 13.6 的拆解，共享能省的净字节约 **28–40 B**（64 KB 的 0.05% 以下），**不足以构成重构理由。**真正的代价有两条：内联那份不共享改进（补 Newton 步或加上界保护都只改一边），以及同一个函数在两处被赋予不同的舍入约定、而差异只写在 `dashboard.c:80` 的一句注释里。另外 `root()` 对接近 2³² 的 `n` 不设防（`d++` 后 `d*d` 翻转），当前的安全性来自 `|pitch_x10| ≤ 2188` 这个数值范围本身（13.10.2），**不是** `dashboard.c:77-78` 的钳位（那段是死代码，问题 1）——删钳位不会造成风险，但引入重标定入口（问题 4）会。
- **建议**：把 `root()` 提升为公共函数（`extern` 或移到工具文件），让它只承诺 `⌊√n⌋`，上取整留在气泡调用点；同时在函数入口加 `if(n>0xFFFFFFFEUL)return 65535UL;` 这类显式上界，让安全性不再依赖调用方的取值范围。

问题 9：`Controls_LevelDistance()` 是公开 API，但被链接器丢弃——接口层的死代码。

- **现象**：`controls.h:16` 声明、`controls.c:220` 定义、全树无调用点；模块内部用的是私有变量 `level_distance`（`controls.c:11`，消费点 `controls.c:126-129`）。它与 `static` 函数的关键差别是：**编译器不会因为一个公开 API 没被调用而告警，只有链接器知道。**

- **证据（链接期实测）**：`level.map:958` `Removing controls.o(i.Controls_LevelDistance), (12 bytes).`——它与 `MPU6050_TempToC`（`level.map:957`）出现在同一张 `Unused input sections` 清单里（`level.map:947` 起）。**这不是 grep 推断，而是映像里确实没有这 12 字节。**
- **影响**：与第 15 章登记的 5 个死宏同属"对外承诺了不存在的接口"，但更容易误导——`controls.h:16` 那句 `/* Radial degrees x10, capped at 15deg. */` 是全工程**唯一**写明 15° 量程的注释，读者会以为量程由这个接口对外发布，而它自己已经被删掉了。变量本身在用，所以只是接口层死代码。
- **建议**：删掉声明与定义，并把 `capped at 15deg` 的注释移到问题 2 建议的 `TILT_FULL_SCALE_X10` 宏上；或者反过来让 `controls.c` 内部也走这个 getter——那就从"死接口"变成"有用的封装"，12 字节换一次封装一致性，值得。

问题 10：`MPU6050_TempToC` 被链接器删除——一条从不换算的温度通路，和它身上 0.53 °C 的隐雷。

- **现象**：`mpu6050.c:96-99` 的换算写成 `temp_raw / 340 + 36`，注释引的却是数据手册的 `+ 36.53`。它零调用，且**不是 static**（`mpu6050.h:43` 有声明），所以编译器不会告警——又是一条只有链接器能发现的死代码。
- **证据（链接期实测）**：`level.map:957` `Removing mpu6050.o(i.MPU6050_TempToC), (16 bytes).`、`level.map:956` `Removing mpu6050.o(i.MPU6050_ReadAccel), (78 bytes).`。本工程自有模块被丢弃的还有 `i2c.o` 的 4 个（`SoftI2C_Begin/End/ReadBytes/SendByte`，20+8+108+12 B）、`oled.o` 的 5 个（`OLED_ShowChar/ShowFixed1/ShowNum/ShowSignedNum/ShowString`，144+130+80+66+60 B）、`controls.o` 的 1 个（`Controls_LevelDistance`，12 B，见问题 9）——`level/USER` 自带源被摘的一共 **12 个函数 / 734 字节**（`level.map:947-958`）。若把项目自带的 `system_stm32f10x.o(i.SystemCoreClockUpdate)` 也算进来（**164 字节**，`level.map:1409`，见 13.10.3 与第 14 章 14.4），合计 **13 个 / 898 字节**；`level.map:959` 的 `misc.o` 之后全属 SPL 库条目，不计入项目函数。在跑的路径是原样透传：`main.c:579` `view.temperature_raw=temp_raw;` →（`dashboard.h:17`）→ `dashboard.c:200` `sprintf(text,"TEMP RAW %d",...)`。屏幕与日志里所有真正的温度都来自 DHT11：`dashboard.c:143` `DHT11_Get()` → `dashboard.c:146`、`main.c:150` `r.temperature_x10=d->temperature_x10;`、`dht11.c:26-27` 的 `b[2]*10+(b[3]&0x7F)` 与符号位、`dht11.c:96`。
- **影响**：当前**没有任何偏差到达用户**，因为这段代码不在映像里。风险有两层：一是它带着 −0.53 °C 的单向偏差躺在头文件里（`mpu6050.h:43`），名字正常、量纲明确，未来接线时会先被信任；二是同一文件里 `MPU6050_ReadAll` 传出的 `temp_raw` **没有任何量纲注释**，而全工程唯一给出量纲的地方正好是这个已被删掉的函数——**量纲信息活在死代码里。**
- **建议**：二选一——删掉声明与定义（与第 4 章"驱动层量纲缺位"的建议合并处理），或修正为补偿形式后真正接上一处消费点。修正的代价其实和现状**完全相同**，只需把常数折进分子：`return (int16_t)((temp_raw + 12420) / 340);`（`36.53 × 340 = 12420.2`，取 12420 后误差 0.0006 °C）。仍是纯整数、仍是一次除法，但量纲与注释一致。

问题 11：`large_fixed` 的溢出保护在所有调用点都由上游范围兜住，屏上另有两处重复判断。

- **现象**：`large_fixed` 在 `length>5` 时输出 `" OVER"`（`dashboard.c:70`），而 `level_page` 在调用点又写了一遍同样的条件：`v->pitch_text>999||v->pitch_text<-999?" OVER":text`（`dashboard.c:137`、`dashboard.c:139`）。
- **证据**：`fixed()` 的输出对 `|value| ≥ 10000`（1000.0°）才会超过 5 个字符；而 `pitch_text` 的实际范围由 13.10.2 决定（θ_0=0 时 ≤573，θ_0=60.77° 时最小 −2197），**只有标定歪到极致才够得到 10000。**温度 ≤800、湿度 ≤1000 也够不到。
- **影响**：两条保护**都不是死代码，但都在"标定已经出错"或"上游范围失控"时才生效**，而那时显示的是 `OVER` 而不是"传感器/标定异常"，把问题暴露成了量程问题。三层判断（`large_fixed` 内一层 + 两个调用点各一层）语义重叠。
- **建议**：保留 `large_fixed` 里的一层（它是函数自保所必需的），删掉 `dashboard.c:137/139` 两层重复判断；把"值超量程"与"标定可疑"区分成两种不同文案。

本章与既有章节的分工：`5730 = 180/π×100` 的推导、小角度误差表（5°/10°/15°/20°/30° 两列）、除零审计四行表、`pose_check` 的翻转/超范围判据，均已在第 7 章 7.7-7.9 节完成，本章只引用不重推。第 4 章 4.x 节已覆盖 MPU6050 的寄存器值、量程与 `5730L` 的位置问题，本章在其上补的是**离散化参数（τ/f_c/极限环）、整数统计的取舍、两处重复实现、以及基准倾斜放大 2.05 倍这一项未登记误差。**

第 14 章　专题四：开机自检 POST 与故障处置

本章要点

- POST 是 11 步、3 屏、最多 52 次整屏刷新的一段**同步自旋**代码：`post_startup()`（`main.c:363-486`）自己在原地等，推进时间的唯一函数是 `post_pump()`（`main.c:224-232`）。第 1、2、6 章已写过它的流程与代价，本章只做纵深：每一格显示的到底是什么证据。
- 11 项自检的证据强度并不相同。按"是否在 CPU 之外产生一次往返"分级：**6 项真总线事务**（OLED、MPU6050、DHT11、W25QXX、LOG AREA、EVENT）、**2 项活体时基**（SYSTICK、TIM2）、**3 项纯配置回读**（CLOCK、ENCODER、OUTPUTS）。显示上它们一样是 `DONE`。
- 第 1 项 CLOCK 的第一个合取项是**恒真式**：`SystemCoreClock==72000000UL`（`main.c:238`），而 `SystemCoreClock` 在 `system_stm32f10x.c:130` 被静态初始化成 72000000，唯一的运行期写入者 `SystemCoreClockUpdate()` 已被链接器整个摘除（`level.map:1409`）。这一项无法失败，也无法为下一项提供独立证据。
- 两项时基检查只证明"时间在走"，不证明"走得准"：SYSTICK 只跨一个 tick 并要求 `after-before==1`（`main.c:251`），TIM2 只有 `>=100µs` 的下界（`main.c:267`）。晶振换成 8 MHz、或 2 倍频配错的板子，这两项都照样 `DONE`。
- 第 10 项 OUTPUTS 有一处**与硬件无关的确定性假阴性**：`controls.c:151` 把 `GPIO_Speed_2MHz` 同时写给编码器引脚，`controls.c:161-162` 复用同一个结构体只改了 Pin 与 Mode，于是 RGB 的 `GPIOA->CRH` 实际是 **0x0AAA**，而自检要求 **0x0BBB**（`main.c:278`）。RGB 工作完全正常，屏幕上却永远显示 `ERR`。
- 6.5 秒窗口可以逐项拆开：有界部分是"整屏刷新次数 × 一屏 37 ms"+"MPU6050 的 120 ms 复位"+"DHT11 的 2000 ms 预热"，最坏合计约 **8.5 秒**；而其中 **约 2.9 秒是结构性的硬下界**（DHT 的 2 s 预热占了 2/3）。
- 第 8 步与第 11 步的等待循环用的是**自旋计数而不是时间界**（`main.c:440`、`main.c:468` 的 `guard=120000UL`），与第 6 步的"毫秒 + 微秒双时基"（`main.c:410-411`）形成对照。同一个 `120000` 对应的真实时长可以从 **0.24 秒到 139 秒**——因为它取决于每次迭代里 `OLED_RefreshStep()` 是否有活干。
- 两个进度细节是"标签即公式"：第 7 步显示 `64KB READ`、第 11 步显示 `8KB READ`（`main.c:333`、`main.c:336`），这两个数字是 `LOGGER_SLOTS`×32 与 `EVENT_BYTES` 的乘积（`logger.h:4`、`eventlog.c:8`），但在代码里只是字符串字面量——改宏不会改标签。

14.1 三个执行体与一条时间线

POST 期间系统里同时活着**三个**执行体，理解它们的分工才能读懂后面的时长分析。

主流程：`post_startup()`（`main.c:363-486`）。它是一个直落的函数体，11 步按顺序走完，中间没有任何事件循环——所有等待都写成"在这里原地转"。它返回后，控制权交给 `main()` 的主循环（`main.c:531`）。

推进器：`post_pump()`（`main.c:224-232`）。它是 POST 期间唯一推进其它模块的东西：

```
static void post_pump(void)
{
    uint32_t now=delay_millis();
    if(post_dht_started)DHT11_Task(now);
    if(post_scan_started)Logger_Task(now);
    if(post_event_started)EventLog_Task(now,now/1000UL);
    if(!DHT11_Busy())OLED_Service(now);
    OLED_RefreshStep();
}
```

四个任务都是**可选**的，由三个 `post_*_started` 标志（`main.c:222`）逐步打开：第 3 步 `post_timebase_ok` 成立时 `DHT11_Init()`（`main.c:392`），第 8 步扫描开始时 `post_scan_started=1`（`main.c:439`），第 11 步 `post_event_started=1`（`main.c:467`）。在标志打开之前，`post_pump()` 只剩最后两行——**OLED 的对比度服务与一个刷新步**。

中断体：`SysTick_Handler`（`stm32f10x_it.c:136-140`）不因 POST 而停止：

```
void SysTick_Handler(void)
{
  delay_tick();
  Controls_Tick();
}
```

`Controls_Tick()` 在 1 kHz 上推进编码器灰度解码、按键消抖与 RGB 灯效（`controls.c:168-194`）。这意味着**POST 期间输入与灯效是在动的**，`main.c:484` 的 `(void)Controls_TakeTurn(); (void)Controls_TakeButton();` 正是在 POST 结束时**清空**这段时间里被中断体累积起来的转轮与按键事件。开机的 3 秒里转轮是完全"通电但无人接收"的。

时间线因此可以分成三段：

阶段	位置	内容	备注
外设基础	`main.c:508-509`	`Buzzer_Init/LED_Init/Controls_Init/delay_init`、开机蜂鸣 60 ms、`SoftI2C_Init/BusRecover`、`delay_ms(300)` 等 OLED 上电	蜂鸣与 300 ms 都是阻塞的
OLED 就绪循环	`main.c:510-516`	探测 OLED 与 MPU，失败则闪灯后重试	**没有次数上限**（见 14.12）
POST 本体	`main.c:517`	`post_startup()` 11 步	本章主体

`main.c:517-521` 是 POST 的出口，也是故障处置的分叉：

```
    dht_ready=post_startup(&sensor_ready,(uint8_t)(reset_flags>>24));
    if(!post_timebase_ok){
        volatile uint32_t i;
        while(1){LED_On();for(i=0;i<250000UL;i++);LED_Off();for(i=0;i<1000000UL;i++);}
    }
```

时基三项（CLOCK/SYSTICK/TIM2）里任何一项失败，`post_timebase_ok` 就是 0，于是第 5、6、7、8、11 步全部退化成 `SKIP`，函数返回后程序停在这段**永不返回的闪灯循环**里：屏幕停在最后那一帧（`main.c:303-304` 的 `TIMEBASE FAULT`），日志系统没有被初始化，复位原因也没有被记入事件日志。这是全工程唯一一处"设计上承认无法继续"的路径。

`post_timebase_ok` 的这层含义值得单独说明：它不是一个"时基可不可用"的诊断量，而是**后 5 项自检的开关。**一次 SYSTICK 或 TIM2 失败会让 11 项里的 5 项从"实测"直接降级为 `SKIP`，而这两项本身只验证"计时器在走"（14.5）。换句话说，POST 有近一半的检查项挂在整个系统里最弱的两个断言上。

14.1.1 复位原因的生与死

`main()` 的第一批语句就把复位原因读了出来，但**在得到任何一次机会把它记下来之前就清掉了标志位**：

```
uint32_t reset_flags=RCC->CSR;
memset(&view,0,sizeof(view));Dashboard_Init();
view.reset_reason=reset_reason(reset_flags);RCC_ClearFlag();
```

`RCC->CSR` 的高位是复位标志组（POR/PIN/SFT/IWDG/WWDG/LPWR），`reset_reason()`（`main.c:50`）把它翻译成一个字符串，存进 `view.reset_reason`（`dashboard.h:22`）供仪表盘显示——`dashboard.c:176` 的 `sprintf(text,"SYS %s",v->reset_reason?v->reset_reason:"?");` 就是它的出口之一。同一时刻 `RCC_ClearFlag()` 把标志位全部清零，日志的字节也在一处被留了位：`eventlog.c:79` 把 `reset_reason` 写进记录的第 22 字节（`raw[22]=reset_reason;`），值来自 `main.c:517` 传给 POST 的 `(uint8_t)(reset_flags>>24)`，即 **`RCC->CSR` 的第 31..24 位原样搬移**——正好是那六个标志所在的区间，所以日志里存的是 ST 的位序，读的人需要知道这一点。

问题在于清除的时刻。`RCC_ClearFlag()` 在 `main.c:507`，位于 POST 与整个主循环之前，而工程里两条**可能永久停留**的路径都在它之后：

路径	位置	复位原因的去向
OLED 就绪循环	`main.c:511-515`	**彻底销毁**：标志已清，屏幕尚未初始化，事件日志尚未开始扫描，`view.reset_reason` 只在内存里且永不显示
时基故障闪灯死循环	`main.c:518-521`	屏上只显示 `TIMEBASE FAULT`（`main.c:304`）这一行固定文案，**不含原因**；第 11 项自检是 `SKIP`（`main.c:481`），日志里没有记录

也就是说，恰恰在最需要复位原因的两种故障里，断电即失：一种把原因抹掉且从不显示，另一种只在屏上留一句与原因无关的文案。第 11 项自检把 `reset_flags>>24` 传给了 `EventLog_Init`（`main.c:467`、`main.c:90` 接收），说明作者本来就打算把复位原因落盘——`RCC_ClearFlag()` 的当前位置让这个打算在故障路径上失效了。

建议：把 `RCC_ClearFlag()` 从 `main.c:507` 挪到第 11 项自检成功落盘之后（它除了这三处之外没有别的消费者，移动成本为零）；两条故障路径各自增加一次"原因已存"的机会，至少让 `reset_reason()` 的字符串在第 11 项里也写进日志。

14.1.2 POST 之后：一个返回值、一个补做的初始化

`post_startup` 的返回值是 `post_dht_started`（`main.c:485`），在 `main()` 里叫 `dht_ready`。它的唯一用途是主循环第一行的一道闸：

```
while(1){
    now=delay_millis();if(dht_ready)DHT11_Task(now);
```

也就是说，"DHT11 是否已经初始化"这件事被 POST 的返回值带进了主循环——如果时基故障，DHT11 从头到尾不会被 `DHT11_Task` 推进（虽然那时也不会走到主循环）。

另外两处交接都值得一提：

```
event_init_attempted=post_event_started;
...
if(sensor_ready)cal_begin(now);
```

`event_init_attempted` 取的是 POST 里第 11 项是否真的启动了事件日志；如果没有（例如第 7 项 W25QXX 是 `MISS`、`Logger_State()` 不是 `LOG_READY`），主循环会在 `main.c:612-614` 用同样的一行重试：

```
    if(!event_init_attempted&&Logger_State()==LOG_READY){
        EventLog_Init(now,(uint8_t)(reset_flags>>24));event_init_attempted=1;}
```

这是全书少见的"开机没做成、运行期补做"的写法，而且它复用局部变量 `reset_flags`（`main.c:505`），所以补做时写进日志的复位原因仍然正确——这一点比上面 `RCC_ClearFlag()` 的位置处理得好。`cal_begin(now)` 则把姿态标定直接接到 POST 成功之后（`sensor_ready` 来自第 5 项 MPU6050 自检），于是主循环一开始就处在"正在标定"状态，LEVEL 页会显示 `HOLD STILL`（`dashboard.c:111-118` 的 `mpu_valid` 分支）——这是第 13 章 13.4 节的 100×10 ms 标定窗。

14.2 十一步怎么被推进：三种节拍

按"这一步要花多久、由谁决定"来分，11 步只有三种节拍。

第一种：立即返回（第 1、2、3、9、10 步）。`post_clock_check`（`main.c:234-241`）、`post_systick_check`（`main.c:243-252`）、`post_tim2_check`（`main.c:254-268`）、`post_encoder_config`（`main.c:270-273`）、`post_outputs_config`（`main.c:275-284`）。其中 SYSTICK 与 TIM2 会等待时间流逝（≤1 ms 与 ≥100 µs），其余三项是纯寄存器读取，微秒级。

第二种：真事务（第 4、5、7、11 步）。`SoftI2C_Probe(OLED_ADDR7)`（`main.c:396`）、`MPU6050_Init()`（`main.c:400`，内部含 120 ms 延时），以及三个由 `post_pump` 驱动的状态机（DHT11、Logger、EventLog）。

第三种：等待循环（第 6、8、11 步的收尾）。这三处的写法完全不同，是本章后面两节的主题。

每一步的视觉表现都由 `post_present()`（`main.c:354-361`）决定：

```
static void post_present(uint8_t current,uint8_t completed,char *results[POST_STEP_COUNT],
    const char *dht_detail,uint32_t flash_id)
{
    while(OLED_RefreshBusy())post_pump();
    post_draw(current,completed,results,dht_detail,flash_id);
    OLED_StartRefresh();
    while(OLED_RefreshBusy())post_pump();
}
```

一次 `post_present` 的语义是**一整屏**：`post_draw` 先 `OLED_Clear()` 再重画（`main.c:299`），因此 32 个块几乎全部"变化"，刷新要推满 32 块。按第 9 章 9.9 节的时间预算，一屏在约 250 kHz 的软件 I2C 下是 48.4 ms，而 `i2c.h:33` 记录的实测值是**约 37 ms**（对应 SCL ≈327 kHz）；第 11 章 11.2 节按实测反推得**每块约 1.16 ms**。本章沿用这两个数：**一次 `post_present` ≈ 37 ms**。

于是"每步两次 present（先 `TEST` 再写结果）"就等价于"每步约 74 ms 的显示开销"，与这一步检查本身的耗时无关。

14.2.1 post_draw 的三页布局与 11 条细节字符串

`post_draw`（`main.c:292-352`）是一台不保留状态的打印机：每帧都 `OLED_Clear()` 后从头画。页号由当前步直接除出来：

```
    page=current==0xFFU?2:(uint8_t)(current/POST_DHT_STEP);
```

`POST_DHT_STEP` 是 5（`main.c:219`），所以第 1–5 步在第 1 页、第 6–10 步在第 2 页——**第 11 步独占第 3 页**。每页最多 5 行，行距 8 像素：

```
        uint8_t step=(uint8_t)(i+page*POST_DHT_STEP);
        const char *detail="";
        if(step>=POST_STEP_COUNT)break;
        y=(uint8_t)(21U+i*8U);
        ...
        OLED_ShowSmallString(0,y,post_names[step]);
        OLED_ShowSmallString(48,y,detail);
        OLED_ShowSmallString(99,y,results[step]);
        if(step==current)OLED_DrawRect(0,(uint8_t)(y-1U),127,(uint8_t)(y+7U),1);
```

三列固定为 x=0（名称）、x=48（细节）、x=99（结果），当前步外面套一个 128 像素宽的反框。表头是 `STM32 SYSTEM POST` 加 `1/3`、`2/3`、`3/3`（`main.c:300-302`），下面一行在正常帧显示 `CHECKS nn/11 STEP mm`，在收尾帧（`current==0xFFU`）换成 `CHECKS COMPLETE` 或 `TIMEBASE FAULT`（`main.c:303-312`）。

细节列的 11 条字符串几乎全是硬编码的"常量说明"：

step	细节字符串	含义	依据
0	`72MHZ`	目标主频	`main.c:320`
1	`1MS TICK`	1 kHz 节拍	`main.c:321`
2	`100US`	µs 时基	`main.c:322`
3	`I2C 3C`	OLED 的 7 位地址	`main.c:323`
4	`I2C CFG`	传感器的配置回读	`main.c:324`
5	`dht_detail`	可变：`WAIT` / `CRC OK` / `E<n>` / `NO FRAME` / `NO TIMER`	`main.c:325`、`374`、`420-424`
6	6 位十六进制 JEDEC ID	`W25Q_ID()` 原样打印	`main.c:326-332`
7	`64KB READ`	容量标签（见 14.10）	`main.c:333`
8	`PB0 PB1`	编码器引脚	`main.c:334`
9	`PWM+BUZ`	PWM 三路加蜂鸣器	`main.c:335`
10	`8KB READ`	容量标签（见 14.10）	`main.c:336`

其中第 7 项（W25QXX）的细节是**唯一一处运行期数据**：它把 `flash_id` 的第 20..0 位按大写十六进制打成 6 个字符（`"0123456789ABCDEF"`，`main.c:327-332`）。这正好是 JEDEC ID 的完整宽度（厂商 + 存储类型 + 容量），所以屏上出现一个与预期不同的 ID 就是换片或仿片的直接证据；但它不做任何译码，读屏的人需要自己查表。其余 10 条在编译期就确定了。

进度条走的是另一套算术（`main.c:342-351`）：`progress = completed*100/11`，再对第 8、11 两步做一次"把子进度并进当前步"的修正：

```
    if(current==POST_SCAN_STEP&&Logger_State()==LOG_SCANNING)
        progress=(uint8_t)(((uint32_t)completed*100UL+
            (uint32_t)Logger_ScanProgress())/POST_STEP_COUNT);
```

`Logger_ScanProgress()` 与 `EventLog_ScanProgress()` 都返回 0–100 的**百分比**而不是槽号——`logger.c:72` 是 `{return (uint8_t)((uint32_t)scan_slot*100/LOGGER_SLOTS);}`，`eventlog.c:103` 是 `{return (uint8_t)((uint32_t)scan_slot*100UL/EVENT_SLOTS);}`——除以 11 之后正好填满当前这一步约 9% 的格宽。也就是说这里连着做了两次刻度换算（槽号 → 百分比 → 11 步格宽），最后一步刚好对齐。这是本节里少见的"两套刻度被有意对齐"的地方（可与第 13 章 13.8 节里 15° 与 ±150 的对齐相较）；两处除法的分母（2048 与 256 个槽）相差 8 倍，却都被压进同一格 9% 的宽度里。

14.2.2 结果行在屏上的存活时间

把这台打印机与 `post_startup` 的调用顺序放在一起看，会得到一个不太直观的结论：**每一项自检的结论只在屏上停留一帧。**

原因是相邻两次 `post_present` 之间没有任何延时。以第 1 步为例（`main.c:380-386`）：

```
    step=0;results[step]=(char *)"TEST";post_present(step,completed,results,dht_detail,flash_id);
    clock_ok=post_clock_check();results[step]=(char *)(clock_ok?"DONE":"ERR");completed++;
    post_present(step,completed,results,dht_detail,flash_id);
    step=1;results[step]=(char *)"TEST";post_present(step,completed,results,dht_detail,flash_id);
```

第 1 步的结果帧刚推完（`post_present` 结束时会等刷新结束），第 2 步就把同一行改写成 `TEST`。所以 `DONE` / `ERR` 这一格的可读时间就是一次整屏刷新，37 ms 量级。真正可读的是**当前步的进行中状态**：第 6 步的 `WAIT` 会停约 2 秒（见 14.8），第 8 步的 `SCAN` 会停到扫描结束。

收尾帧更极端。`main.c:483` 画的是第 3 页，而第 3 页只放得下第 11 步一行（见 14.2.1），所以 `CHECKS COMPLETE` 这一帧里**只有第 11 项的结果**，前 10 项的 `DONE` / `ERR` 全部不在这张屏上。而它同样活不过一帧：`main.c:526` 把 `frame_next` 设成 `now`，主循环第一次进入 `main.c:649` 的门就成立，仪表盘立刻开始推送并把它覆盖。

两件事合起来：POST 屏对现场人员是一台**进度指示器**，而不是一份报告——它显示"正在检查什么、进度到哪"，但"哪一项失败了"无法目视获取。11 项结论各显示 37 ms 左右，收尾汇总同样 37 ms 左右；能稳定读到的是第 6、8、11 步的进行中状态，而这三步恰好是耗时最长的三步，于是屏上留下的信息与自检的实际价值正好错位。

建议：在 `main.c:483` 之后加一次有界停留——最省事的是复用一个已有的可读信号，例如 `while(!Controls_ButtonDown())delay_ms(10);`（按键由 1 kHz 的 `Controls_Tick()` 维护，见 14.1），或退一步用 `delay_ms(1500)`；同时把第 3 页改成汇总行（例如把 `ERR` 的项数与第 11 项并排显示），否则即使停下来，屏上也只有 11 项里的 1 项。

14.3 11 项自检的证据强度分级

判断一项自检"证明了什么"，只需要问一句：**它有没有在 CPU 之外产生一次往返？** 按这个判据，11 项分成三类。

#	名称	关键代码	类别	它究竟证明了什么
1	CLOCK	`RCC_GetClocksFreq(&clocks)`（`main.c:237`）	纯配置推导	`RCC->CFGR` 的位域能被重算成 72/72/36/72 MHz（且第一项恒真，见 14.4）
2	SYSTICK	`CTRL&7`、`LOAD+1==72000`，再等 1 个 tick（`main.c:247-251`）	活体时基	计数器在走，且走了一个 tick
3	TIM2	`CR1`、`PSC` 回读，再量 ≥100 µs（`main.c:263-267`）	活体时基	微秒时基在走
4	OLED	`SoftI2C_Probe(OLED_ADDR7)`（`main.c:396`）	**真事务**	地址 0x3C 在总线上应答
5	MPU6050	`MPU6050_Init()`（`main.c:400`）	**真事务**	复位、5 个寄存器写入、WHO_AM_I、5 个回读逐项相符
6	DHT11	`DHT11_Init` + 轮询 `DHT11_Task`（`main.c:392`、`main.c:227`）	**真事务**	单总线上完成一次 40 位帧（含校验和）
7	W25QXX	`Logger_Init()` + `W25Q_ID()`（`main.c:429`）	**真事务**	SPI 上读回 JEDEC ID，容量 ≥64 KB
8	LOG AREA	`Logger_Task` 扫描到底（`main.c:441-445`）	**真事务**	最多 2048 槽 × 32 B = 64 KB 的整片读
9	ENCODER	`GPIOB->CRL&0xFF==0x88`、`ODR&3==3`（`main.c:272`）	纯配置回读	第 150-151 行写进去的上拉配置能读回来
10	OUTPUTS	8 个条件（`main.c:278-283`）	纯配置回读	TIM1 与 GPIOA/B 的配置位；**含一处恒定假阴性**（见 14.7）
11	EVENT	`EventLog_Init/Task` 直到启动记录落盘（`main.c:467-475`）	**真事务**	8 KB 扫描 + 一次 32 字节写入 + 提交字节校验

统计：**6 项真事务、2 项活体时基、3 项纯配置回读。**第 9 项与第 10 项最容易被误读——它们检查的是"配置写没写进去"，不是"器件能不能工作"。

第 11 项反倒是最强的一项：它的循环条件里带着 `EventLog_BootRecorded()`（`main.c:470`），也就是说 POST 只有在**开机记录真的写进 Flash 并被读回确认**之后才算通过。这是全书唯一一处把"写入 → 提交 → 回读"整条链路纳入开机门槛的自检，代价是 POST 的时长被绑在 Flash 的页编程时间上（见 14.8）。

14.4 第 1 项 CLOCK 的第一个合取项是恒真式

`post_clock_check` 的返回表达式有五个合取项（`main.c:238-240`）：

```
    return (uint8_t)(SystemCoreClock==72000000UL&&
        clocks.SYSCLK_Frequency==72000000UL&&clocks.HCLK_Frequency==72000000UL&&
        clocks.PCLK1_Frequency==36000000UL&&clocks.PCLK2_Frequency==72000000UL);
```

后四项来自 `RCC_GetClocksFreq(&clocks)`（`main.c:237`），它按 `RCC->CFGR` 的位域现算频率——这是真正的配置推导。第一项不同：`SystemCoreClock` 是一个**变量**。

它的值来自 `system_stm32f10x.c:74-84` 的编译期开关与 `:119-133` 的静态初始化：

```
#if defined (STM32F10X_LD_VL) || (defined STM32F10X_MD_VL) || (defined STM32F10X_HD_VL)
/* #define SYSCLK_FREQ_HSE    HSE_VALUE */
 #define SYSCLK_FREQ_24MHz  24000000
#else
/* #define SYSCLK_FREQ_HSE    HSE_VALUE */
/* #define SYSCLK_FREQ_24MHz  24000000 */
/* #define SYSCLK_FREQ_36MHz  36000000 */
/* #define SYSCLK_FREQ_48MHz  48000000 */
/* #define SYSCLK_FREQ_56MHz  56000000 */
#define SYSCLK_FREQ_72MHz  72000000
#endif
```

本工程走 `#else` 分支，`SYSCLK_FREQ_72MHz` 是打开的，于是：

```
#elif defined SYSCLK_FREQ_72MHz
  uint32_t SystemCoreClock         = SYSCLK_FREQ_72MHz;        /*!< System Clock Frequency (Core Clock) */
```

`SystemCoreClock` 的另一个写入者是 `SystemCoreClockUpdate()`（`system_stm32f10x.c:245`），它按 `RCC->CFGR` 重算这个变量。全工程搜索该函数名，只命中它自己的定义与注释——**没有任何调用点**；而链接器给出的结论比"没人调用"更硬：`level.map:1409` 一行 `Removing system_stm32f10x.o(i.SystemCoreClockUpdate), (164 bytes).`——**这 164 字节根本没进固件映像**，`main.c:238` 里那个合取项连"调用失败"的可能都不存在，它比较的是两个编译期常量。

同一张 `Unused input sections` 清单里还躺着两条同类旁证：`level.map:1408` 摘除 `core_cm3.o(.emb_text), (32 bytes).`、`level.map:1410` 摘除 `startup_stm32f10x_md.o(HEAP), (512 bytes).`（堆未使用）。也就是说，"配置检查"这一类的退化不只是逻辑上的：凡是只被"看起来该有"的东西引用的代码，链接器都会先一步把它删掉。

结论：`SystemCoreClock` 恒等于 72000000，`SystemCoreClock==72000000UL` **永远是真**。第 1 项 CLOCK 实际只有四个有效合取项，而且它们与真正的时基活性无关——晶振没起振、PLL 配错这类故障，只要 `RCC->CFGR` 的位被写成了预期的样子，判断照样通过（真正会因此失败的是第 1 步之后那些依赖时基的检查和 `main.c:518` 的闪灯死循环，但那时 CLOCK 一格已经显示 `DONE`）。

这一项还牵连下一项：`post_systick_check` 用 `SysTick->LOAD+1UL!=SystemCoreClock/1000UL`（`main.c:247`）作为期望值，即 72000。这个期望值同样建立在一个常量上，而不是建立在实际计数结果上。

建议：把第一项换成对 `RCC->CFGR` 的独立验算，或在 `SystemInit()` 之后调用一次 `SystemCoreClockUpdate()` 再比较——后者只需一行，就能让这一项从"常量比常量"变成"按寄存器重算后与配置目标比对"。

14.5 两项时基检查只证明"在走"，不证明"走得准"

`post_systick_check` 的活性部分是：

```
    before=delay_millis();
    while(delay_millis()==before&&guard)guard--;
    after=delay_millis();
    return (uint8_t)(after-before==1UL);
```

它只等到毫秒计数器**变化一次**。`after-before==1` 是个很弱的断言：计数器停在原值会被 `guard` 兜住（`guard=1000000UL`，`main.c:246`），但计数器**走得快两倍或慢两倍**都满足这个条件。判据里唯一的"速率"信息是 `SysTick->LOAD+1UL==SystemCoreClock/1000UL`（`main.c:247`），也就是"重装载值被写成了 72000"——仍然是配置，不是测量。

`post_tim2_check` 的活性部分是：

```
    start=delay_micros16();
    while((uint16_t)(delay_micros16()-start)<100U&&guard)guard--;
    return (uint8_t)((uint16_t)(delay_micros16()-start)>=100U);
```

它比 SYSTICK 强的地方是用上了微秒分辨率的 `delay_micros16()`，是全书仅有的两处"用 µs 计数验证 µs 计数"的地方之一（另一处在 14.1 的 DHT 等待里）。弱点在于**只有下界没有上界**：`>=100U` 允许 100 µs 到 65535 µs 之间的任何值通过，慢十倍也看不出来。而且 `timer_clock%1000000UL` 这个条件（`main.c:264`）本身就要求分频后是整数 MHz，把 `PSC` 配错的常见形态排除掉了——所以 TIM2 这一项实际验证的是"PSC 写对了 + 计数器在走"。

两项都不到位的原因可以理解：POST 的目标是"能不能继续跑"，不是"校准"。但如果要让这两项具备独立证据价值，代价很小——SYSTICK 可以改成跨 N 个 tick（例如 `N=100`，配合 `delay_ms` 的已知延时），并要求 `after-before==N`；TIM2 可以加一个上界（例如 `>=100U && <=200U`）。**建议**：至少加上界，把"慢"这一侧从"永远通过"变成"可判"。

14.6 第 9 项 ENCODER：被验证的是"配置写进去了"

```
static uint8_t post_encoder_config(void)
{
    return (uint8_t)((GPIOB->CRL&0xFFUL)==0x88UL&&(GPIOB->ODR&3UL)==3UL);
}
```

这一项读的是 `GPIOB->CRL` 低 8 位（PB0、PB1 的配置）与 `ODR` 低 2 位。`0x8` 是"输入 + 上拉/下拉"，而在输入模式下 `ODR` 的那一位**读回来就是上拉/下拉的选择位**，而不是引脚电平。所以 `ODR&3==3` 的含义是"两个引脚选择了上拉"，它验证的正是 `controls.c:150-152` 写下的东西：

```
    gpio.GPIO_Pin = GPIO_Pin_0 | GPIO_Pin_1 | GPIO_Pin_10;
    gpio.GPIO_Mode = GPIO_Mode_IPU; gpio.GPIO_Speed = GPIO_Speed_2MHz;
    GPIO_Init(GPIOB, &gpio);
```

这与"编码器能用"是两件事。机械旋转编码器在旋转时是两个触点对地的断续接通；上拉只是让空闲电平确定，触点的抖动、接触电阻、开路都不是这项检查能看见的——它甚至无法区分"编码器接上了"与"PA0/PB0 悬空但上拉生效"。第 10 章写过编码器解码的灰度表与消抖（`controls.c:168-194`）；从自检的角度，**这两格的证据强度属于最低一档。**

同样的判据也适用于第 3 步 OLED 的"配置"与第 4 步 MPU 的"复位+回读"之间的差别：`MPU6050_Init` 里有回读比对（`mpu6050.c:50-53`），而 OLED 与编码器只有地址应答或配置回读。

14.7 第 10 项 OUTPUTS 的确定性假阴性：0xAAA 与 0xBBB

第 10 项有 8 个条件（`main.c:277-283`），第一个就是把 RGB 三路 PWM 引脚的配置与期望值比对：

```
    uint32_t channels=TIM_CCER_CC1E|TIM_CCER_CC2E|TIM_CCER_CC3E;
    return (uint8_t)((GPIOA->CRH&0x0FFFUL)==0x0BBBUL&&
```

期望值 0xBBB 永远不会成立。 机制分三步。

第一步：`Speed` 与 `Mode` 被写在同一处。

```
    gpio.GPIO_Pin = GPIO_Pin_0 | GPIO_Pin_1 | GPIO_Pin_10;
    gpio.GPIO_Mode = GPIO_Mode_IPU; gpio.GPIO_Speed = GPIO_Speed_2MHz;
    GPIO_Init(GPIOB, &gpio);
    turns=button_event=light_mode=key_ticks=long_sent=0;
```

第二步：RGB 那一路复用了同一个结构体，只改了两个字段。

```
    GPIO_ResetBits(GPIOA, GPIO_Pin_8 | GPIO_Pin_9 | GPIO_Pin_10);
    gpio.GPIO_Pin = GPIO_Pin_8 | GPIO_Pin_9 | GPIO_Pin_10;
    gpio.GPIO_Mode = GPIO_Mode_AF_PP; GPIO_Init(GPIOA, &gpio);
```

`gpio.GPIO_Speed` 仍然是上一次留下的 `GPIO_Speed_2MHz`。

第三步：SPL 只在输出/复用模式下把 `Speed` 并进配置位。

```
  /*---------------------------- GPIO Mode Configuration -----------------------*/
  currentmode = ((uint32_t)GPIO_InitStruct->GPIO_Mode) & ((uint32_t)0x0F);
  if ((((uint32_t)GPIO_InitStruct->GPIO_Mode) & ((uint32_t)0x10)) != 0x00)
  {
    /* Check the parameters */
    assert_param(IS_GPIO_SPEED(GPIO_InitStruct->GPIO_Speed));
    /* Output mode */
    currentmode |= (uint32_t)GPIO_InitStruct->GPIO_Speed;
  }
```

（`stm32f10x_gpio.c:183-188`。）枚举值是 `GPIO_Speed_10MHz=1`、`GPIO_Speed_2MHz=2`、`GPIO_Speed_50MHz=3`（`stm32f10x_gpio.h`），所以 PA8/PA9/PA10 的 CRH 半字节是

```
    (GPIO_Mode_AF_PP & 0x0F) | GPIO_Speed_2MHz = 0x8 | 0x2 = 0xA
```

即 `GPIOA->CRH & 0x0FFF` = **0x0AAA**，而自检要求 **0x0BBB**（0x8|0x3）。每一个时钟周期的显示都是 `ERR`（`main.c:461`），与 RGB 的实际工作状况无关。

为什么 RGB 仍然是好的：`0xA` 只是把引脚的速度档位设成 2 MHz，功能与 50 MHz 档完全相同，只是边沿速率较低。三路 PWM 由 TIM1 的 `CCR1/2/3`（`controls.c:141-143`）驱动，通道使能与周期在同一个函数里逐项检查（`main.c:279-282`），而那些条件都成立。

为什么蜂鸣器那一路能过：`buzzer.c` 显式写了速度。

```
        GPIO_InitStructure.GPIO_Pin   = BUZZER_PIN;
        GPIO_InitStructure.GPIO_Mode  = GPIO_Mode_Out_PP;
        GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
```

于是 PB8 的 CRH 半字节是 `0x0|0x3 = 0x3`，与 `main.c:283` 的 `(GPIOB->CRH&0x0FUL)==0x03UL` 相符。**同一个工程、两个模块、两种写法**：一个显式声明速度并顺利通过自检，一个继承上一次的值并恒定失败。

影响：这是 11 项里唯一一项"与硬件状态无关、恒定误报"的自检。它的危害不在于误报本身（RGB 是好的），而在于它给出的是一格**永久性的**红色，等于把第 10 项的判别力整体降到零——真出现 TIM1 或 GPIO 故障时，这一格不会有任何变化。

两种修法（都很小）：

1. 在 `controls.c:162` 之前补一行 `gpio.GPIO_Speed = GPIO_Speed_50MHz;`，让期望值 0xBBB 成立。这也是与 `buzzer.c:15` 一致的写法。
2. 若确实想用 2 MHz 档，把 `main.c:278` 的期望值改成 `0x0AAAUL`。

第 1 种改法更合本工程的意图（作者把 `0xBBB` 写成期望值，说明本意就是 50 MHz），而且顺带修掉一个真实（虽然无害）的不一致：RGB 与蜂鸣器的速度档位不同。

14.8 6.5 秒窗口：最坏路径逐项累加

POST 的时长由三样东西构成：**整屏刷新的次数、几个固定延时、三个等待循环。**

整屏刷新：`post_present` 的调用点共有 22 个（其中 3 个带前置条件），另外有两处由扫描进度触发。逐点统计如下。

步	调用点	次数
UI 1–6、9、10	`main.c:380/382`、`384/386`、`388/393`、`395/397`、`399/403`、`408/425`、`456/458`、`460/462`	每步 2 次 = 16
UI 7 W25QXX	`main.c:427`、`434`	2
UI 8 LOG AREA	`main.c:438`（仅扫描）、`454`，加 `444`（每 +5% 进度一次，≤20）	2–22
UI 11 EVENT	`main.c:466`（仅就绪）、`483`（收尾），加 `473`（每 +10%，≤10）	2–12

固定 22 次（16+2+2+2），最坏 52 次。按一次 37 ms 计算：**典型约 0.81 秒，最坏约 1.92 秒。**

固定延时：`MPU6050_Init()` 内部是 `delay_ms(100)` + 两次 `delay_ms(10)`（`mpu6050.c:23-28`、`mpu6050.c:46`），合计 **120 ms**，与 `main.c:599` 注释里"120ms MPU 复位"的说法一致。开机蜂鸣 `Buzzer_Beep(BOOT_BEEP_MS)` 是 60 ms 的阻塞（`main.c:508`、`buzzer.c:36-38`），OLED 上电沉降是 `delay_ms(300)`（`main.c:509`）——这两项在 POST 之外。

三个等待循环：

步	位置	判据	时间语义
6 DHT11	`main.c:410-417`	`!dht->attempts||DHT11_Busy()` 且 `<6500 ms` 且 `<6500000 µs` 且 `guard`	**双时基**：毫秒 + 微秒，都有界
8 LOG AREA	`main.c:440-445`	`Logger_State()==LOG_SCANNING` 且 `guard`	**只有自旋计数**
11 EVENT	`main.c:468-475`	`guard` 且（扫描中 ‖ 忙 ‖ 已就绪但启动记录未落盘）	**只有自旋计数**

把有界部分加总（最坏）：

```
整屏刷新（52 次 × 37 ms）        1.92 s
MPU6050 复位                      0.12 s
DHT11 等待（毫秒时基上限）        6.50 s
-----------------------------------------
有界合计                          8.54 s
```

再加上第 8、11 步两段没有时间界（见 14.9）的循环。

但 6.5 秒在实际中够不着。 `DHT11_Init` 在 `dht11.c:50` 把首次读取安排在 2000 ms 之后：

```
    next_read=now+2000; data.error=0;
```

`DHT11_Task` 在此之前不会发起任何一次尝试，所以 `dht->attempts` 一直是 0，第 6 步的循环必然至少转满"到第 2 步 `DHT11_Init` 那一刻起 2000 ms"的剩余部分。第 2 步与第 6 步之间只隔了第 3、4、5 步（两次 present 各约 74 ms，加上一次探测与 120 ms 的 MPU 初始化），所以**这一格的实测等待约 1.7–2.0 秒**，占 POST 总时长的一半以上。6.5 秒只在状态机彻底卡死时才会用满，而第 11 章已经证明 DHT11 的脉冲状态机自身有约 30 ms 的硬上界。

下界：把三项结构性成本相加——DHT 的 2000 ms 预热（`dht11.c:50`）、MPU 的 120 ms 复位、以及 22 次 unavoidable 的整屏刷新（0.81 s）——得到约 **2.9 秒**。这就是"开机大约 3 秒"这个体感的来源。要显著缩短它，只能改变设计：把 DHT11 的预热窗口与更多检查重叠（作者已经把它放在时基刚被证明的第 2 步，是**最早可能的合法位置**，因为 `DHT11_Task` 依赖 `delay_millis()`），或者放弃这一格的实测而只做"存在性"判断。

14.9 两个扫描等待的 guard 是自旋计数，不是时间界

第 8 步与第 11 步的循环只用一个递减变量兜底：

```
        guard=120000UL;
        while(Logger_State()==LOG_SCANNING&&guard){
            guard--;post_pump();scan_progress=Logger_ScanProgress();
            if(scan_progress>=last_scan+5U){last_scan=scan_progress;
                post_present(step,completed,results,dht_detail,flash_id);}
        }
```

```
        guard=120000UL;
        while(guard&&(EventLog_State()==EVENT_SCANNING||EventLog_Busy()||
            (EventLog_State()==EVENT_READY&&!EventLog_BootRecorded()))){
```

对照第 6 步的写法（`main.c:409-411`）：

```
        dht=DHT11_Get();timer_last=delay_micros16();timer_elapsed=0;guard=30000000UL;
        while((!dht->attempts||DHT11_Busy())&&
            (uint32_t)(delay_millis()-now)<6500UL&&timer_elapsed<6500000UL&&guard){
```

第 6 步有**两个时间界**（毫秒与微秒各一，互为独立证据）加一个 3 千万次的自旋兜底；第 8、11 步**只有一个 12 万次的自旋计数。**

同一个自旋计数对应的真实时长取决于每次迭代的成本，而 `post_pump()` 的迭代成本跨度极大：

- **下限**约微秒级。`OLED_RefreshStep()` 的第一行是 `if (!refresh_busy) return;`（`oled.c:184`），所以在没有帧在推的时候，`post_pump()` 只是四个可能为空的模块任务加一层循环判断。
- **上限**是**一个变化块的一次阻塞事务。** `OLED_RefreshStep()` 每轮最多发出一个 32 字节块（`oled.c:196-197` 的两笔 `SoftI2C_WriteBytes`，`oled.c:203` 的 `break` 是硬证据），第 9 章 9.9 节与第 11 章 11.2 节把它定在约 1.16–1.5 ms。

于是 `guard=120000` 的时长范围是：

```
120000 × 2 µs     ≈ 0.24 s
120000 × 1.16 ms ≈ 139 s（约 2 分 19 秒）
```

同一个数字，两个相差近六百倍的答案。 这不是理论上的界限差别——`post_present` 内部会自己把刷新推完（`main.c:357`、`main.c:360` 的两个 `while(OLED_RefreshBusy())post_pump();`），所以在正常路径下循环之外的迭代都很便宜；但这恰恰说明这个 `guard` 的值**不是按时间选的**，读代码的人无法从中读出"这段最多等多久"。

另一个方向看，12 万这个数字对工作量是**极度宽松**的：第 8 步要扫的是最多 `LOGGER_SLOTS` =2048 个槽（`logger.h:4`），第 11 步的 `EventLog` 只有 256 个槽（`eventlog.c:9`），即使状态机每次调用只推进一步，12 万也分别有 58 倍与 469 倍的余量。也就是说 `guard` **永远不会是那个先到达的约束**——除非状态机干脆不前进，而这正是它唯一存在的意义。

建议：把两处 `guard` 换成时间界，与第 6 步保持同一种写法：

```
        uint32_t deadline = delay_millis() + 3000UL;   /* 或者复用一个具名宏 */
        while (Logger_State() == LOG_SCANNING && !due(delay_millis(), deadline)) { ... }
```

这样既保留了兜底语义，又让"最多等多久"可以从代码直接读出；同时与第 6 步的双时基写法形成一致风格。第 6 步的 `guard=30000000UL` 同样可以去掉——它的两个时间界已经覆盖了所有正常与异常路径。

14.10 两处"标签即公式"：64KB READ 与 8KB READ

`post_draw` 为每一点显示一行细节字符串（`main.c:320-336`），其中两条是关于容量与时长的：

```
        }else if(step==7)detail="64KB READ";
```

```
        else detail="8KB READ";
```

这两个数字不是测出来的，也不是算出来的，而是**两个宏的乘积被手工写成了字面量**：

显示	实际含义	依据
`64KB READ`（第 8 步）	`LOGGER_SLOTS` 2048 × 32 B = 65536	`logger.h:4`、`REGION_BYTES 65536UL`（`logger.c:8`）
`8KB READ`（第 11 步）	`EVENT_BYTES` 8192	`eventlog.c:8`，槽数 `EVENT_SLOTS`（`eventlog.c:9`）

字符串与宏之间没有任何编译期联系。改 `LOGGER_SLOTS`、改槽宽、或者把 `REGION_BYTES` 换成按容量动态计算的逻辑，屏幕上的字都不会变——它会继续显示一个已经不对的容量。这与第 12 章 12.x 节里"容量常数分散在多处"的问题是同一类：**常量有多个副本，其中一份是给人看的字符串。**

同一判断也适用于第 6 步的三条可变细节（`main.c:420-424` 的 `CRC OK`、`E<n>`、`NO FRAME`）与第 4 步的 `I2C CFG`：它们描述的是**代码路径**而不是测量值，读屏的人无法区分"读过 64 KB 没发现问题"与"读了 3 个槽就找到了有效魔数"。

建议：把两处字面量改成由宏拼出的字符串（例如把容量写进一个 `const char *`，用 `#define` 的字符串化拼装），或者在显示层用运行期算出的槽数格式化。代价是引入格式化代码，收益是标签不再可能与实现脱钩。

14.11 结果字面量的全集与两处边界

11 格结果列的取值来自 `char *results[POST_STEP_COUNT]`（`main.c:372`），全部是三个大写字母或短串。全集如下。

字面量	出现位置	含义
`--`	`main.c:375-376` 的初值 `pending`	尚未到达
`TEST`	各步第一行 `results[step]=(char *)"TEST"`	正在执行
`WAIT`	`main.c:405`	第 6 步等待中（初值 `dht_detail="WAIT"`，`main.c:374`）
`SCAN`	`main.c:438`、`main.c:466`	第 8、11 步扫描中
`DONE`	`main.c:381`、`385`、`389`、`396`、`401`、`420`、`431`、`446`、`450`、`476`	通过
`ERR`	`main.c:381` 等同一批的三元分支、`main.c:421`、`449`、`453`、`457`、`461`、`478`	失败
`MISS`	`main.c:432`	第 7 步：`Logger_State()!=LOG_OFFLINE`、ID 与容量三项中有一项不满足
`LOCK`	`main.c:447`、`451`、`477`	第 8、11 步：遇到陌生数据，模块永久锁定（见第 12 章与第 8 章）
`TIME`	`main.c:423`、`448`、`480`	超时/未完成
`SKIP`	`main.c:402`、`424`、`433`、`452`、`479`、`481`	前序条件不成立
`CFG`	`main.c:457`、`461`	第 9、10 步：配置回读通过

两点边界值得记录。

第一，`TEST` / `WAIT` / `SCAN` 是"进行中"，其余是终态，而 `--` 表示从未到达。所以第 9、10 步的 `CFG` 与第 8 步的 `DONE` 在语义上并不对等：前者说的是"我读回来的位与写进去的一致"，后者说的是"整片扫描完成"。屏上没有区分。

第二，DHT11 的错误码是取模显示的：

```
            else{results[step]=(char *)"ERR";dht_error_text[0]='E';
                dht_error_text[1]=(char)('0'+(dht->error%10U));dht_error_text[2]=0;dht_detail=dht_error_text;}
```

`dht_error_text` 是 4 字节缓冲（`main.c:373`），只放得下一个 `E` 加**一位**数字，于是 `%10U` 让错误码 0 与 10、1 与 11 在屏上无法区分。当前 `DHT11_Decode` 只会返回 1 和 2（`dht11.c:23-28` 的两处 `return 1/2`），所以这个隐患没有被触发；但错误码空间一旦扩到两位数，屏上就会出现撞码。

建议：`dht_error_text` 扩到 5 字节并去掉取模（`'E'` + 两位数字 + 终止符），或者在 `%10U` 处加注释说明"错误码当前只有 1、2"。

14.12 本章发现的问题与建议

问题 1：第 1 项 CLOCK 的第一个合取项是恒真式，无法失败。

- **现象**：`main.c:238` 的 `SystemCoreClock==72000000UL` 在编译期就被确定。
- **证据**：`SystemCoreClock` 在 `system_stm32f10x.c:130` 由 `SYSCLK_FREQ_72MHz` 静态初始化（该宏在 `system_stm32f10x.c:83` 打开）；唯一的运行期更新函数 `SystemCoreClockUpdate()`（`system_stm32f10x.c:245`）在全工程没有调用点，而且链接器已经把它整个摘除——`level.map:1409` `Removing system_stm32f10x.o(i.SystemCoreClockUpdate), (164 bytes).`，从源到映像这段代码都不存在（见 14.4）。
- **影响**：11 项自检的第一项少了一条独立证据链，且这一格显示的是"配置推导"而非"时基在跑"。`post_systick_check` 的期望值 `SystemCoreClock/1000`（`main.c:247`）同样建立在这个常量上，两项因此共享同一个不可失败的来源。
- **建议**：`SystemInit()` 之后调用一次 `SystemCoreClockUpdate()`，让这一项变成"按 `RCC->CFGR` 重算后与 72 MHz 比对"；或者直接删掉这一项，把 CLOCK 的证据合并到 SYSTICK/TIM2 的活体检查里。

问题 2：两项时基检查都只有下界（SYSTICK 连下界都很松），无法发现"走得不准"。

- **现象**：SYSTICK 只要求跨过一个 tick（`main.c:251` 的 `after-before==1UL`）；TIM2 只要求跨过 100 µs（`main.c:267` 的 `>=100U`）。
- **证据**：两处的 `guard` 分别是 `1000000UL`（`main.c:246`、`main.c:259`），只用于避免死等；没有任何上界断言。
- **影响**：晶振频率错误、PLL 倍频配置错误这类故障（除了完全不起振）不会被这两项发现，而它们直接影响 5 ms 采样节拍、DHT11 的脉宽判据与整机计时。
- **建议**：SYSTICK 改为跨 N 个 tick 并与一个已知延时对照（例如 `delay_ms(100)` 后要求 `after-before==100`，容差 ±1）；TIM2 加下上界（`>=100U && <=200U`）。

问题 3（高）：第 10 项 OUTPUTS 存在与硬件无关的确定性假阴性，RGB 正常也永远显示 `ERR`。

- **现象**：`main.c:278` 要求 `GPIOA->CRH&0x0FFFUL==0x0BBBUL`，实际恒为 `0x0AAA`。
- **证据**：`controls.c:151` 把 `gpio.GPIO_Speed=GPIO_Speed_2MHz` 与 `GPIO_Mode_IPU` 一起写给 GPI0B；`controls.c:161-162` 复用同一结构体改 `GPIO_Pin` / `GPIO_Mode` 后调用 `GPIO_Init(GPIOA,&gpio)`，**没有重设 `GPIO_Speed`**；SPL 在 `stm32f10x_gpio.c:183-188` 只在 `GPIO_Mode & 0x10` 成立时把 `Speed` 并进模式位，故半字节为 `0x8|0x2=0xA`。对照：`buzzer.c:14-15` 显式写了 `Speed_50MHz`，所以 `main.c:283` 的 `(GPIOB->CRH&0x0FUL)==0x03UL` 成立。
- **影响**：这一格是**永久性**的红色，与 RGB 是否工作无关。危害不在误报本身，而在于第 10 项的判别力被整体清零——真出现 TIM1 通道或 GPIO 配置故障时屏上没有任何变化。这是 11 项里唯一"恒定误报"的项。
- **建议**：在 `controls.c:162` 之前补 `gpio.GPIO_Speed = GPIO_Speed_50MHz;`（与 `buzzer.c:15` 一致，也符合作者写出 `0xBBB` 期望值的本意）；若确实要用 2 MHz 档，则把期望值改为 `0x0AAAUL`。

问题 4（中）：第 8、11 步的等待循环用自旋计数而非时间界，真实时长不可读。

- **现象**：`main.c:440` 与 `main.c:468` 都是 `guard=120000UL`，循环条件里没有任何 `delay_millis()`。
- **证据**：对比第 6 步的双时基写法（`main.c:410-411`：`<6500UL` 毫秒与 `<6500000UL` 微秒）。迭代成本取决于 `post_pump()`（`main.c:231` 的 `OLED_RefreshStep()`）：无帧在推时是微秒级，有帧在推时是一个变化块的阻塞事务（`oled.c:184`、`oled.c:196-197`、`oled.c:203`），按第 9 章 9.9 节与第 11 章 11.2 节约 1.16–1.5 ms。折算范围约 0.24 秒到 139 秒。
- **影响**：读代码无法判断"自检最长会停多久"；与第 6 步的严谨写法形成明显不一致。同时 120000 对 2048/256 个槽是 58/469 倍余量，说明它不是按工作量选的。
- **建议**：与第 6 步统一，改成 `delay_millis()` 的 deadline；第 6 步的 `guard=30000000UL` 可以删除，它的两个时间界已覆盖全部路径。

问题 5（低）：两处容量标签是手工写成的字符串，与宏脱钩。

- **现象**：`main.c:333` 的 `"64KB READ"`、`main.c:336` 的 `"8KB READ"`。
- **证据**：前者对应 `LOGGER_SLOTS` 2048（`logger.h:4`）与 `REGION_BYTES 65536UL`（`logger.c:8`），后者对应 `EVENT_BYTES 8192`（`eventlog.c:8`）。
- **影响**：改容量常数不会改标签，屏上会持续显示错误容量；这也是"常量多副本"问题的一种（见第 12 章）。
- **建议**：用字符串化宏或在显示层运行期格式化。

问题 6（低）：DHT11 错误码被取模成一位，错误码 ≥10 时屏上撞码。

- **现象**：`main.c:421-422` 的 `(char)('0'+(dht->error%10U))`，缓冲只有 4 字节（`main.c:373`）。
- **证据**：`DHT11_Decode` 当前只返回 1 与 2（`dht11.c:23-28`），所以未触发。
- **影响**：错误码空间一扩到两位数，`E1` 会同时表示 1 和 11，现场无法归因。
- **建议**：缓冲扩到 5 字节并去掉 `%10U`，或加注释说明当前上限。

问题 7（中）：OLED 就绪循环没有次数上限，屏坏时会永久停在这一段。

- **现象**：`main.c:511-515` 的 `while(!oled_ok){...}` 只在探测成功后退出。
- **证据**：每轮是"3 或 5 次 80+120 ms 闪灯 + `delay_ms(1200)` + `SoftI2C_BusRecover` + 两次 `SoftI2C_Probe`"，即约 1.8–2.2 秒一轮；`mpu_ok` 只用于决定闪 3 次还是 5 次（`main.c:511`），此后不再使用（`main.c:514` 是它最后一次出现）。
- **影响**：OLED 或 I2C 总线故障时，系统会永久停在点亮 LED 的循环里，既不记录复位原因也不进入主循环；而它发生在 `post_startup` **之前**，所以 11 项自检的屏显永远不会出现——现场看到的是"黑屏 + LED 慢闪"，容易被误判为电源问题。
- **建议**：给循环加上限（例如 10 轮后进入一个可识别的故障态并继续跑，或至少把结果记入下一次复位日志）。

问题 8（中）：复位原因在两条故障路径上被销毁或从不显示。

- **现象**：`main.c:507` 在 `main()` 的前 20 行内就调用 `RCC_ClearFlag()`，而两条可能永久停留的路径（`main.c:511-515` 的 OLED 就绪循环、`main.c:518-521` 的时基故障死循环）都在它之后。
- **证据**：`main.c:505` 读 `RCC->CSR`，`main.c:50` 的 `reset_reason()` 把它译成字符串存入 `view.reset_reason`（`dashboard.h:22`，出口之一是 `dashboard.c:176`），紧接着 `RCC_ClearFlag()` 把标志清空。OLED 就绪循环里屏幕尚未初始化、事件日志尚未开始扫描；时基故障路径里第 11 项自检是 `SKIP`（`main.c:481`），屏上只有 `TIMEBASE FAULT` 这句固定文案（`main.c:304`），不含原因。落盘那条路本来是用 `(uint8_t)(reset_flags>>24)` 传给 `EventLog_Init` 的（`main.c:517` → `main.c:467` → `eventlog.c:90`，记录第 22 字节 `eventlog.c:79`），在故障路径上根本不会执行。
- **影响**：最需要复位原因的两种故障恰恰是断电即失的场合——现场只能看到"黑屏 + LED 慢闪"或一行与原因无关的 `TIMEBASE FAULT`，无法归因。这一条与问题 7 是同一段代码的两个侧面。
- **建议**：把 `RCC_ClearFlag()` 移到第 11 项自检成功落盘之后（该标志除这三处外没有别的消费者，移动成本为零）；两条故障路径各补一次"原因已存"的机会。

问题 9（中）：11 项自检的结论在屏上各只停留一帧，收尾汇总屏只列 11 项中的 1 项。

- **现象**：每次 `post_present` 返回后立刻进入下一步并改写同一行（如 `main.c:380-386`），中间没有任何延时；收尾帧 `main.c:483` 画的是第 3 页，而第 3 页只放得下第 11 步一行（`main.c:298` 的页号、`main.c:316-318` 的越界退出）。
- **证据**：一次 `post_present` = 一整屏（37 ms 量级，见 14.2 与第 9 章 9.9 节）；`main.c:526` 把 `frame_next` 设为 `now`，主循环第一次进入 `main.c:649` 的门就成立，仪表盘立即开始推送并覆盖收尾帧。真正可读的只有当前步的进行中状态（第 6 步 `WAIT` 约 2 秒、第 8 步 `SCAN` 到扫描结束）。
- **影响**：POST 屏是进度指示器而不是报告，"哪一项 ERR"无法目视获取；而第 10 项 OUTPUTS 会恒定显示 `ERR`（问题 3），二者组合正是"屏上确实有异常、却看不出异常在哪"的最坏形态。
- **建议**：收尾帧加一次有界停留（复用 `Controls_ButtonDown()` 或直接 `delay_ms(1500)`），并把第 3 页改成汇总行（`ERR` 计数或关键项摘录）。

第 15 章　全局缺陷清单、改进路线与工程结论

本章要点

- 前 14 章按模块拆完了这套固件；本章不再重复推导，只做两件事：把散落在各章的缺陷收成一张可排序的总表（15.3 节，**第一批 24 条 + 15.3.2 节第二批 12 条，共 36 条**），再把整本书的判断收成一个结论（15.7 节）。本章的全部行号在定稿前又经一轮 `read` / `grep` 逐条复核，其中第 10 条**据链接器映像被更正过一次**（见该条）。
- 严重度最高的四条缺陷有一个共同点：它们都让设备**失去自恢复能力**——四个 Cortex 异常处理是 `while(1)` 死循环（`stm32f10x_it.c:57-102`），全工程没有看门狗也没有 `__WFI`，两类 Flash 日志各有不可自愈的锁死态，`bus_error` 只有一个清零点（`w25qxx.c:31`）。
- 这套固件真正的长处集中在**协议与预算**：日志的"写 31 字节 → 回读比对 → 单独写第 32 字节提交标志 → 再回读"三段式（`logger.c:123-138`、`eventlog.c:171-189`），以及把一帧切成 32 个可让位的短事务的差分刷新（`oled.c:179-206`）。15.4 节把这类可复用的工程资产列了 9 项。
- 全工程 42 处 `sprintf`、0 处 `snprintf`（全在 `dashboard.c`），最大缓冲 40 字节（`dashboard.c:154`）；逐条估算当前最长输出约 22 字符，**今天不溢出**，但余量靠人工核算——而溢出的后果正好落在缺陷 1 的死循环上。
- 有一类"缺陷"其实是刻意的取舍，本章一律如实标注：不做 DMA、不做动态内存、不做 RTOS、不做 wear-leveling。判断它们的标准不是"先不先进"，而是"取舍的代价有没有被算清楚"——第 11 章 11.5 节和本章 15.3 节的第 18、23 条就是在算这笔账。
- 本章所有行号都经过 `read` / `grep` 逐条复核。凡在实物上无法只靠静态阅读确认的推断，本章明确标为"待实物确认"，并集中放在本章末尾。

15.1 总评

这份固件的手艺在协议层：持久化全部用「写 31 字节 → 回读比对 → 单独写第 32 字节提交标志 → 再回读」的三段式（`logger.c:123-138`、`eventlog.c:171-189`），撕裂写永远不可见；OLED 把一帧切成 32 个可让位的短事务（`oled.c:179-206`），让共享总线上的长操作有了让位点；姿态用"相对开机基准"绕开克隆 MPU6050 偏高 12.4% 的误差（`main.c:583-587`）。

最该补的是**恢复能力**：四个 Cortex 异常处理全是死循环（`stm32f10x_it.c:57-102`），全工程无看门狗、无 `__WFI`，一次瞬时错误就要人工拔电；两类 Flash 日志各有不可自愈的锁死态（`logger.c:98` + `main.c:633`、`main.c:612-614`）；唯一的在线诊断通道还停在注释里（`stm32f10x_conf.h:15`）。

15.2 按严重度排序的前 10 条

排名	缺陷（一句话）	严重度	证据
1	四个 Cortex 异常处理都是 `while(1)` 死循环，不记录现场也不复位	高	`stm32f10x_it.c:57-102`
2	没有看门狗（无 IWDG/WWDG 调用）、没有 `__WFI`，卡死不可自恢复	高	见 15.3 第 2 条的全项目搜索证据
3	`LOG_FOREIGN` 与 `EVENT_LOCKED` / `EVENT_ERROR` 没有回到就绪态的路径	高	`logger.c:98`、`main.c:633`、`main.c:612-614`
4	Flash `bus_error` 全项目只有一个清零点，且在初始化函数里	高	`w25qxx.c:5,12,14,17,31`
5	`POST` 在主循环之外自建了一套小调度器，最长一次等待窗口 6.5 s	中	`main.c:405-417`、`main.c:224-228`、`main.c:484`
6	`OLED_GRAM` 的并发契约只写在注释里，12 个绘图 API 都不自查	中	`oled.c:132-134`、`oled.c:173-178`
7	共享总线上的"四道互锁"全靠人手写对条件，没有任何所有权断言	中	`main.c:600`、`main.c:649`、`main.c:661`
8	42 处 `sprintf`、0 处 `snprintf`，越界余量靠人工核算	中	`dashboard.c:154,244,289`
9	`main.c:586-587` 的除法没有零检查，安全性依赖 450 行外的隐含不变量	中	`main.c:44,125,128,130`
10	没有串口/SWO，唯一无屏诊断通道是不可数的闪灯循环	中	`stm32f10x_conf.h:15`、`main.c:512,520`

15.3 缺陷总表

严重度判据：**高** = 会导致功能永久失效或需要人工断电，且在正常使用路径上可触发；**中** = 只在异常路径上失效、或需要人工核算才安全、或可观测性缺失导致排障困难；**低** = 可维护性、一致性、文档问题，对当前功能无影响。

#	现象	严重度	证据（文件:行号）	影响
1	`HardFault_Handler`、`MemManage_Handler`、`BusFault_Handler`、`UsageFault_Handler` 的函数体都是 `while (1) { }`	高	`stm32f10x_it.c:57-63`、`:70-76`、`:83-89`、`:96-102`	任何一次空指针写入、未对齐访问或除零都让设备永久静看起来"还在工作"，只能拔电
2	全项目没有任何喂狗代码，主循环也没有睡眠指令	高	在 `level/USER` 的 15 个 .c 与 15 个 .h 上搜索 `malloc|calloc|realloc|WWDG|IWDG|DMA_|osThread|mutex|Semaphore|vTask|FreeRTOS` 只有 2 处命中，且都是复位原因**解码**：`main.c:52` `if(flags&RCC_CSR_IWDGRSTF)return "IWD";`、`main.c:53` `if(flags&RCC_CSR_WWDGRSTF)return "WWD";`；另搜 `__WFI|WFI` 为 0 命中	跑飞或卡死（例如缺陷 1 的死循环）没有任何自动恢复手断电；CPU 永远全速空转，功耗与发热没有优化空间（另

#	现象	严重度	证据（文件:行号）	影响
3	两类日志都能进入没有出口的状态	高	`LOG_FOREIGN`： `logger.c:98` 置位（`if(get32(b)!=MAGIC){state=LOG_FOREIGN;return;}`）； `logger.c:91` 的 `Logger_Task` 闸门只放行 `LOG_SCANNING` / `LOG_WRITING`； `main.c:633` `if(Logger_State()==LOG_READY||Logger_State()==LOG_FOREIGN)flash_retry=0;` 主动关掉唯一的重试通道。 `EVENT_LOCKED` / `EVENT_ERROR`： `eventlog.c:119`、`:131`、`:156` 置 LOCKED， `eventlog.c:108,128,141,144,164,168,171,176,178,182,199` 置 ERROR；而 `EventLog_Init` 每次上电最多调用一次（`main.c:612-614` 的 `!event_init_attempted`）	换板或换固件后 Flash 上半区不是本机格式是**正常场景**，日志；事件区一旦出错，SESSIONS 页与复位溯源一起失信息
4	`bus_error` 是锁存器，唯一清零点在 `W25Q_Init()` 内	高	`w25qxx.c:5` `static uint8_t bus_error;`； `:14` / `:17` 在 1000 µs 超时后置 1； `:12` `if(bus_error) return 0xFF;` 之后所有 `transfer()` 直接短路返回 `0xFF`；清零只在 `:31` `capacity=jedec=0; bus_error=0;`（`W25Q_Init()` 内）；读它的是 `:44`、`:52`、`:53`、`:61`、`:67`、`:76`、`:81`	总线上一次瞬时错误（供电跌落、边沿抖动）就把"总线坏返回 `0xFF`；配合缺陷 3 的终态，本次上电无法恢复
5	`POST` 在主循环之外实现了第二套调度器，最长一次等待 6.5 s	中	等待循环 `main.c:409-417`（`guard=30000000UL` 与 `<6500UL`、`<6500000UL` 三重上界）； `post_pump()` 定义 `main.c:224-230`（其中 `:227` `if(post_dht_started)DHT11_Task(now);`、`:229` `if(post_event_started)EventLog_Task(now,now/1000UL);`）； `post_present()` 在 `main.c:354-360` 里用 `while(OLED_RefreshBusy())post_pump();` 等刷新	这 6.5 s 内主循环的全部服务都不在跑：不做姿态采样（志（`main.c:611`）、不发事件心跳（`main.c:615`）、（`main.c:541`），而且 `main.c:484` 明确丢弃了这段的得"按住按钮上电没反应"
6	`OLED_GRAM` 的并发契约只存在于注释里	中	契约为 `oled.c:132-134` 原文："差分短块刷新：已发送缓存仅用于比较，不是 OLED 的硬件双缓冲。每次发一个 32 字节块，主循环可在块间读取 MPU6050。刷新期间不要改变 OLED_GRAM；完成后才允许绘制下一帧。"而唯一的自查在 `OLED_StartRefresh`（`oled.c:173-178`）， `oled.c:213` 起的全部绘图 API 都不看 `refresh_busy`	当前靠"主循环非抢占 + `main.c:649` 的 `!OLED_Refres变量既没有断言也没有强制手段；一旦绘制被放进中断或裂，且症状难以归因
7	共享总线（PB6/PB7）上没有所有权断言，"互锁"是四处手写条件	中	`main.c:575` 读 MPU6050 与 `main.c:662` 的 `OLED_RefreshStep()` 在同一轮主循环里访问同一条总线； `main.c:599-600` 为 DHT11 的 20 ms 独占期让路； `main.c:649` 用 `!OLED_RefreshBusy()` 门住重绘； `main.c:661` `if(!DHT11_Busy())OLED_Service(now);`	加一个新的总线用户时，没有任何机制能阻止遗漏其中一偶发丢帧，最难复现（第 11 章 11.7 节解释了为什么全工
8	全部格式化输出都用无边界版本	中	全项目 `sprintf` 42 处、 `snprintf` 0 处，42 处全在 `dashboard.c`；缓冲区声明最大 40 字节： `dashboard.c:154` `char text[40],a[16],b[16];`、 `:244`、 `:289`；另有 `number[16]`（`:68` / `:231`）、 `text[16]`（`:113` / `:143`）、 `text[24]`（`:175` / `:190` / `:231`）、 `text[32]`（`:337`）	逐条估算当前最长输出约 22 字符（`dashboard.c:305` `%02lu:%02lu:%02lu` 在 32 位上界是 `RUN 49710D 23:` 安全性完全依赖人工核算，改一次格式串就可能栈上越界的死循环上
9	姿态换算的除法没有零检查，安全性依赖 450 行外的隐含不变量	中	除数为 `base_az` / `base_ay`： `main.c:586-587`；变量声明 `main.c:44` `static int32_t base_ax, base_ay, base_az;`（文件作用域，初值 0）；只在标定成功分支赋值 `main.c:128`，同时 `main.c:130` 才置 `calibrated=1`；而该分支的前置检查是 `main.c:125` `if(cal_sum[2]/cal_good<MIN_CAL_Z)ok=0;`（`MIN_CAL_Z 8000` 见 `main.c:20`），执行这两行前又有 `main.c:583` 的 `if(calibrated)`	**经核对当前不存在除零**： `calibrated` 只在 `main.c:13` `base_az≥8000`。 `main.c:125` / `:128` 自己用到的 `cal`（`cal_good==0` 时 `main.c:123` 的 `ok` 已为假， `:12` 点离不变量太远，改动一处就会崩，而崩的表现是死循环
10	**（已按链接器映像更正）** `MPU6050_TempToC()` 是死代码，而在用的路径根本没有温度换算	中	函数体 `mpu6050.c:96-99` `return (int16_t)(temp_raw / 340 + 36);`，注释 `mpu6050.c:94-95` 写的是 `TEMP = TEMP_OUT/340 + 36.53` ——但它**没有任何调用者**：链接器已把它删除， `level.map:943` `Removing mpu6050.o(i.MPU6050_TempToC), (16 bytes).`。在用的路径是 `main.c:575` `valid=sensor_ready&&MPU6050_ReadAll(acc,gyro,&temp_raw)==0;` → `main.c:579` `view.temperature_raw=temp_raw;` → `dashboard.c:200` `sprintf(text,"TEMP RAW %d",v->temperature_raw);`（DEBUG 页）。CLIMATE 页的温度来自 **DHT11 而不是 MPU**： `dashboard.c:146` `large_fixed(text,d->temperature_x10);` ← `dht11.c:96` ← `DHT11_Decode`（`dht11.c:25-29`， `t=b[2]*10+(b[3]&0x7F)`，0.1 ℃ 分辨率的整数传感器值）	本书初版把这一条写成"CLIMATE 页恒偏低约 0.53 ℃"是个传感器，与 `MPU6050_TempToC` 无关。真实后果有三层换算成摄氏度，DEBUG 页那一行只有原始量（`main.c:` `dashboard.c:200` 的 `TEMP RAW` 字样是这个事实的诚实受影响（只用加速度计），但"MPU 也有温度计"在界面上有人按 `mpu6050.h:43` 的公开声明去调用 `MPU6050_Te` 0.53 ℃、且因整数除法向零截断而在负温时误差更大的结
11	`flags` 低两位都为 0 时整条记录被拒写	中	`logger.c:82` `if(state!=LOG_READY || !(record->flags&3)) return 1;`；位定义 `logger.h:6-7`（`LOG_ENV_VALID 1`、 `LOG_LEVEL_VALID 2`）	一条"运行时长有效、但温湿度与姿态都无效"的记录（例如但设备在正常工作）被完全丢弃，HISTORY 页因此看不到检查点形成的下界"这个设计目标相冲突
12	编码器只能软件解码，快转时的表现是"不翻页"	中	软件解码在 `controls.c:170-179`（16 项格雷码表 + 非法跳变清零，其中 `controls.c:173` `if ((ab ^ old_ab) == 3) quarter_steps = 0;`）；每步 4 个边沿 `controls.h:5` `ENCODER_EDGES_PER_STEP 4`；调用发生在 1 ms 的 SysTick 里（`stm32f10x_it.c:136` `139`， `:139` 为 `Controls_Tick();`）；TIM1 已被 RGB 的 PWM 占用（`controls.c:49-72`）	用不了定时器的硬件编码器模式，1 ms 采样周期限制了可到"页不翻"而不是"翻过头"——方向是安全的，但没有任了
13	没有任何可执行的串口或 SWO 通道	中	在 `level/USER` 全部 30 个源文件上搜索 `USART|UART_|ITM_|SWO|DBGMCU` 只有 2 处命中，都在 `stm32f10x_conf.h`： `:7` 的中文注释"以后要加串口打印，把 usart 那行放开即可"与 `:15` 被注释掉的 `/* #include "stm32f10x_usart.h" */`；对比之下 23 个 SPL 驱动源都被编进工程（`level.uvprojx` 535-645 行）	现场故障只能拿到屏幕和闪灯两条信息。而唯一的无屏通`19`），它的报码由忙等循环实现： `main.c:512` `{LED_On();delay_ms(80);LED_Off();delay_ms(120` `while(1)` `{LED_On();for(i=0;i<250000UL;i++);LED_Off();fo` ——节奏依赖 CPU 速度，无法表达多位错误码
14	DHT11 的 20 ms 独占期只有调用方知道	中	`dht11.c:78-82` 状态 0 把 PA1 拉低 20 ms（`deadline=now+20`）并设 `next_read=now+2500`；数据 6 s 后自动失效（`dht11.c:102` `if (data.valid && (uint32_t)(now-data.last_good_ms)>6000) data.valid=0;`）；让位条件写在调用侧： `main.c:599-600`（注释原文"避免 120ms MPU 复位跨越 DHT 的 20ms 起始脉冲"）与 `main.c:661`	这 20 ms 内不能发起长耗时的共享 I2C 事务，但"必须避 `dht11.h` 的契约，第 4 章 4.9 节称之为"全工程的准互
15	页名表与页宏不一致，7 个页号是裸数字	中	页名唯一权威表 `dashboard.c:321`（10 项 `titles[]`）； `dashboard.h:4` 只给 `DASHBOARD_PAGE_COUNT 10`， `:5-7` 只命名了 `STROBE_PAGE 7` / `RUNTIME_PAGE 8` / `EVENT_PAGE 9`；分派对 0-6 用裸数字（`dashboard.c:328-334`，只有 `:335` / `:336` 两处用了宏）	页顺序号在 `titles[]`、分派链、以及各处 `page==N` 的插一页要同时改多处，编译器查不出漏改

#	现象	严重度	证据（文件:行号）	影响
16	事件日志被传感器日志的就绪状态绑架	中	`eventlog.c:91` `if(Logger_State()!=LOG_READY||W25Q_Capacity()<SENSOR_BYTES+EVENT_BYTES)return;`；读取路径同样检查（`eventlog.c:107` 的 `state!=EVENT_READY||Logger_State()!=LOG_READY`）；启动条件 `main.c:612` 只在 `Logger_State()==LOG_READY` 时成立	两块区域在 Flash 上独立（`SENSOR_BYTES 65536`、`EV` `eventlog.c:7-8`），却共用一个启动条件：传感器区掉电下界
17	所有 SPL 参数断言在发布构建里被编译掉，工程自己的不变量也没有替代品	中	`stm32f10x_conf.h:17-22`：未定义 `USE_FULL_ASSERT` 时 `#define assert_param(expr) ((void)0)`；且该头只打开了 5 个 SPL 模块（`:9-13`：rcc、gpio、spi、exti、misc）	传入非法引脚、非法 SPI 参数这类错误在运行期完全静默 6 的 `OLED_GRAM` 契约、缺陷 7 的总线所有权）也都是注 查是 POST，而它只在启动时跑一次
18	没有 wear-leveling，只有顺序环形覆盖	低	`logger.c:84` `write_slot=count || latest_seq ? (latest_slot+1)%LOGGER_SLOTS : 0;`；擦除粒度 `logger.c:113` `if(!(write_slot%128))` → 每 128 条记录擦一个 4 KB 扇区；`LOGGER_SLOTS 2048`（`logger.h:4`）、`REGION_BYTES 65536`（`logger.c:8`）、`per_sector[16]`（`logger.c:12`）	60 s 一条记录 ⇒ 约 2.1 h 擦一次扇区，16 个扇区轮流被 分散开了（与第 5 章问题 7 的算术结论一致）。真正的景 **同时丢掉该扇区里的 128 条旧记录**，掉电落在 `WRITE_E` （`logger.c:115` `timeout=now+1500`）里会把它们一
19	`i2c_fault` 的语义没有导出，上层分不清"设备没应答"和"总线被拉死"	低	`i2c.c:11` 文件内 static；每个事务在 `i2c_start()` 里清零（`i2c.c:85`）；置位后同一事务内后续字节快速失败（`i2c.c:115` `if(i2c_fault)return 1;`、`i2c.c:146`）；返回值把两种失败混在一起（`i2c.c:182` `return (ack == 0 && !i2c_fault) ? 1 : 0;`）。全项目搜索 `SoftI2C_Fault` **0 命中**	NACK 可以原地重试，总线被拉死必须调用 `SoftI2C_Bu` `81`），两者需要的动作不同；但上层只拿到同一个 `0`， `OLED_Service` 的 500 ms 重扫（`oled.c:143-172`）
20	5 个配置宏是死的，全项目只有定义处	低	在全部源码上搜索 `BUBBLE_X_DIR`、`BUBBLE_Y_DIR`、`HUD_MAX_OFFSET`、`HUD_FS_X10`、`MPU6050_ACCEL_LSB_PER_G`，5 处命中全是定义本身：`main.c:38`、`main.c:39`、`main.c:40`、`main.c:41`、`mpu6050.h:31`	读者会以为气泡方向、HUD 偏移、HUD 字号、加速度计 翻遍代码找不到使用点。`MPU6050_ACCEL_LSB_PER_G` 更 量程换算（真正的换算是 `main.c:586-587` 的 5730 与
21	显示名与状态名不一致，最需要用户介入的那个状态名最不可读	低	`dashboard.c:45-52`：`LOG_FOREIGN` 显示为 `"LOCKED"`，而 `LOG_ERROR` 显示 `"ERROR"`、`LOG_SCANNING` 显示 `"SCAN"`	现场看到 `LOCKED` 时无法对应到状态机里的 `LOG_FOREI` 要用户处置的那个（换板/换固件后的正常场景）。`"LOC`
22	与编译优化等级绑定的常数没有编译期保护	低	`i2c.h:36` `#define I2C_DELAY_LOOPS 8`，注释 `i2c.h:29-35` 明确写了它只在"72MHz、-O0 下"标定过（"每减小 1，SCL 大约快 15kHz"）；Keil 工程的优化等级是 `level.uvprojx:317` 的 `<Optim>1</Optim>`	改一次构建配置（优化等级、时钟）就会让软件 I2C 的实 没有任何地方会发现这件事—— `I2C_DELAY_LOOPS` 是约
23	没有 DMA（刻意取舍），但它把刷屏和采样绑在同一个执行流上	低	`DMA_` 前缀在 `level/USER` 0 命中（与缺陷 2 同一份搜索）；`stm32f10x_conf.h:9-13` 甚至没有打开 `stm32f10x_dma.h`，尽管 `stm32f10x_dma.c` 被编进工程（`level.uvprojx:590`）	这是被算过账的取舍（第 11 章 11.5 节给出 32 字节块的 是：每新增一个总线用户或加长一帧，时间预算都要重新 醒
24	存储路径的所有失败都返回 `1`，调用方无法区分故障种类	低	`w25qxx.c` 的公开 API 统一返回 `bus_error` 或 `1`（`:44`、`:52`、`:53`、`:61`、`:67`、`:76`、`:81`）；`logger.c:82`、`:85` 等也一律返回 `1`	上层的 `toast` 只能显示"SAVE ERROR"（`main.c:622` 域被占"、"总线死了"三种完全不同的处置

15.3.1 两段值得逐字看的代码

第 1 条缺陷的原文（其余三个 handler 与它同构，只有注释里的异常名不同）：

```
void HardFault_Handler(void)
{
  /* Go to infinite loop when Hard Fault exception occurs */
  while (1)
  {
  }
}
```

以上摘自 `stm32f10x_it.c:57-63`。

把它和第 3 条缺陷、以及 15.4 节要讲的提交协议放在一起看，就能看出这套固件的分界线在哪里：同一个工程里，一处把"出错"定义成永久停住，另一处把"出错"定义成"记录自动不可见"。差别只在下面这几行：

```
    } else if(write_phase==WRITE_BODY) {
        if(W25Q_Read(address(write_slot),b,32) || memcmp(b,raw,31)) {state=LOG_ERROR;return;}
        b[0]=0xA5;
        if(W25Q_ProgramStart(address(write_slot)+31,b,1)) {state=LOG_ERROR;return;}
        write_phase=WRITE_COMMIT;timeout=now+50;
    } else {
        if(W25Q_Read(address(write_slot),b,32) || Logger_Decode(b,&record)) {state=LOG_ERROR;return;}
        latest_slot=write_slot;latest_seq=pending.sequence;count++;per_sector[write_slot/128]++;
        add_recent(write_slot,pending.sequence);state=LOG_READY;
    }
```

以上摘自 `logger.c:130-138`。这段代码的顺序是硬的：先写 31 字节数据（普通路径 `logger.c:123`，擦除后的路径 `logger.c:128`）、回读并 `memcmp` 校验、然后才把提交标志 `0xA5` 单独写到第 32 字节（`logger.c:132-133`）、再回读并用 `Logger_Decode()` 复核（`logger.c:136`），全部通过才更新 `latest_slot` / `latest_seq` 与 `recent` 环并回到 `LOG_READY`。任何一步断电，记录的第 32 字节都不是 `0xA5`，扫描阶段就会把它当成空槽——**失败的方向是"看不见"，而不是"看见错的"。**

15.3.2 第二批汇总（编号 25 – 36）

上面 24 条是按各章章末问题汇总的第一批。复核第 1–14 章的章末清单后，另有 12 条此前只散落在各章、没有进入总表。它们全部经本轮逐条 `read` / `grep` 复核过行号，**严重度全部落在"中"与"低"**，因此不改变 15.2 节的排名（前 10 条按现状不动），只在 15.6 节末尾追加对应的落地项。

#	现象	严重度	证据（文件:行号）	影响	建议改法
25	`W25Q_EraseStart` 的 4 KiB 对齐完全靠两个常量巧合	中	驱动的守卫是 `w25qxx.c:80` `if(!capacity || address>=capacity || (address&4095) || write_enable()) return 1;`；调用侧只在"槽号是 128 的倍数"时才擦：`logger.c:113` `if(!(write_slot%128))`、`eventlog.c:198` `if(slot_state==2)`。4 KiB 成立的三个前提全是硬编码的：32 B/槽（`logger.c:27` 的 `address(slot)=region_base+(uint32_t)slot*32`）、`LOGGER_SLOTS 2048` 与 `REGION_BYTES 65536`（`logger.h:4`、`logger.c:8`）满足 `2048*32==65536`、`EVENT_SLOTS 256` 与 `EVENT_BYTES 8192`（`eventlog.c:9,8`）满足 `256*32==8192`；而两个区首也都必须是 4096 的倍数：`logger.c:66` `region_base=W25Q_Capacity()-REGION_BYTES`、`eventlog.c:92` `base=W25Q_Capacity()-SENSOR_BYTES-EVENT_BYTES`（`65536+8192=73728`，`eventlog.c:7-8`）。全工程 `_Static_assert` **0 处**	把 `SENSOR_BYTES` / `EVENT_BYTES` 改成 9000（一个看起来完全无害的调整）会让区首 65536+9000=74536 不再是 4096 的倍数，`(address&4095)` 恒非零 ⇒ **每一次擦除都返回 1** ⇒ 立即 `LOG_ERROR` / `EVENT_ERROR` 并永久停在缺陷 3 描述的终态。症状与"Flash 芯片坏了"无法区分，而根因是一行常量	加三条编译期断言，位置放在各自的 `_Static_assert((SENSOR_BYTES+ base must be 4KiB aligned");`、`_Static_assert(EVENT_BYTES%40 sectors");`、`_Static_assert(LO size must match region size");` 过"
26	DHT11 互锁保护错了对象：门住的是短事务，长事务反而无门控	中	`main.c:661` `if(!DHT11_Busy())OLED_Service(now);`，紧接着 `main.c:662` `OLED_RefreshStep();` **没有任何条件。**被门住的 `OLED_Service`（`oled.c:143-172`）每次只发 3 字节对比度命令（`oled.c:145` `uint8_t cmd[3]={0x00,0x81,0};`、`:168`）或一条恢复序列命令（`oled.c:151`）；而未被门控的 `OLED_RefreshStep`（`oled.c:179-206`）一次发 7 字节窗口命令 + 33 字节像素数据（`oled.c:183` `uint8_t data[33];`、`oled.c:196-197` 两笔 `SoftI2C_WriteBytes`），合计约 40 字节。按时值折算：名义口径 32×9 位×4.00 µs ≈ **1.15 ms**，实测口径 32×28.9 µs ≈ **0.92 ms**——比被门住的事务高一个数量级	本条**不是"忘了写门"，而是门写在了便宜的那一侧**：`OLED_RefreshStep` 的设计本意确实是"短块 + 块间可让位"（契约 `oled.c:132-134`，主循环侧另有 `main.c:649` 的 `!OLED_RefreshBusy()` 门），所以作者显然清楚它的代价；但真正需要与 DHT11 的 20 ms 起始独占窗口互斥的正是它，而它恰好完全敞开。DHT11 的脉冲窗口见 `dht11.c:78-82`，失效后果是"温湿度偶发读到 `--`"，属于最难归因的那一类症状	把条件移到长事务上：`main.c:662` `if(!DHT11_Busy())OLED_Refresh` 检查并直接 `return`），`OLED_Serv` 且不破坏"块间让位"的设计
27	5 个 8×16 文本函数被链接器剔除，但它们的字库仍占 1520 字节 ROM	低	`level.map:937-941` 的删除清单：`Removing oled.o(i.OLED_ShowChar), (144 bytes).`、`OLED_ShowFixed1` (130)、`OLED_ShowNum` (80)、`OLED_ShowSignedNum` (66)、`OLED_ShowString` (60)——共 480 字节代码。但 `level.map:2272` 显示 `OLED_F8x16 0x080071dc Data 1520 oled.o(.constdata)` **仍在映像里**：链接器只能整段保留 const 数组，取字节的代码（`oled.c:334`、`:340` 的 `t = OLED_F8x16[(uint8_t)ch][i];`）被删了，字库本身删不掉。同一份清单里还有整帧流式 I2C API 四件套：`level.map:933-936` `SoftI2C_Begin` (20)、`SoftI2C_End` (8)、`SoftI2C_SendByte` (12)、`SoftI2C_ReadBytes` (108) 共 148 字节，而它们的用法说明仍写在 `i2c.h:47` 的注释里	1520 字节约占映像代码+只读数据（28,912 + 2,524 字节）的 **4.8%**，为一个没有任何调用者的功能付出；对一块总容量 64 KB 的 Flash 而言这是可回收的实质空间。更实际的影响是阅读成本：`oledfont.h:5` 的 `OLED_F8x16[][16]` 是工程里最长的数据块，读者会默认它是显示路径的一部分	删掉 `oledfont.h:5` 起的 `OLED_F8` 可回收 1520 字节；同时清掉 `i2c.h` 用"，就在 `oledfont.h` 顶部加一行'
28	小字体只有 43 个字形，小写字母静默渲染成空白	低	`oled.c:67-76` 的 `small_bits()` 依次处理 `'A'..'Z'` (26 个)、`'0'..'9'` (10 个)、`'-'`、`'+'`、`'.'`、`':'`、`'/'`、`'%'`、`'#'`，合计 **43 个字形**，其余字符一律在 `oled.c:76` `return 0;`。`small_string`（`oled.c:78-…`）不做大小写替换、也不报错，把 0 逐点画出去（`oled.c:82-85` 的 6×7 循环）	任何一处 `OLED_ShowSmallString` 传入小写字母都会得到一段**空白**而不是错字，肉眼看来像"这一行的字没显示出来"。当前所有调用点都恰好只传大写与数字（`dashboard.c:249-250` 的 `"R%4u G%4u B%4u"`、`"AB%u K%u S%u F%u"` 等），所以缺陷是潜伏的——下一次有人写一句小写提示语才会暴露	两种都便宜：在 `oled.c:76` 之前加 `'a'+'A');`（映射到大写，一行）；失可见而不是无声
29	2000 ms 扫描期限只在"总线忙"分支里被检查，触发即丢弃已扫结果	中	`logger.c:66` 在进入扫描态时一次性设定期限：`region_base=W25Q_Capacity()-REGION_BYTES;state=LOG_SCANNING;timeout=now+2000;`。唯一的判据在 `logger.c:95`：`if(state==LOG_SCANNING) { if(busy) {if((int32_t)(now-timeout)>=0)state=LOG_ERROR;return;} …`——期限**只在 `busy` 为真时被查询。**而 `busy==2`（总线错误）已在 `logger.c:93` 提前返回，所以这条期限实际只在芯片持续报告 WIP（`busy==1`）时才有意义；正常的扫描路径（`logger.c:96-104`，每次调用前进一个槽）跑多久都不会触发。真触发时 `state=LOG_ERROR`，重试路径 `main.c:629` `Logger_Init(now)` → `logger.c:63-64` 会把 `count` / `scan_slot` / `latest_slot` / `per_sector` 全部清零重扫	这个 2 s 期限度量的是"等待忙芯片的时长"而不是"扫描耗时"，因此它保护不了它看起来要保护的东西。而一旦触发，已经数出来的 `count`（最多 2047 条）被下一次 `Logger_Init` 的复位丢掉并从头再扫——若触发条件复现，日志**永远到不了** `LOG_READY`，每次重试都在白扫一遍 64 KiB	把期限检查移出 `if(busy)`，让它成否扫完"解耦：超时后**保留已得结果并**条"只读不写的降级模式永远要保留"[
30	BOOT 编号在上电提交成功之前就已自增，失败即永久少记一次上电	低	`eventlog.c:157-159`：扫描结束时 `if(have_latest)boot_count=latest.boot_count;` 然后 **无条件** `boot_count++;pending=EVENT_BOOT;state=EVENT_READY;`；而 `boot_recorded` 只在 BOOT 记录**成功提交之后**才置位（`eventlog.c:185` `if(event.kind==EVENT_BOOT)boot_recorded=1;`，其前置是 `:184` 的复核已通过）。中间任何一步失败（`eventlog.c:171` / `:176` / `:178` / `:183` → `EVENT_ERROR`）都会留下"计数已进位、记录不存在"的状态。该计数被 SESSIONS 页当作会话身份使用：`dashboard.c:302` / `:304` / `:317` 用它区分 `CURRENT SESSION` 与 `SAVED MINIMUM`，`dashboard.c:338` 直接显示 `BOOT #%lu`	下一次上电又从 `latest.boot_count` 重新推算（`eventlog.c:157`）并把编号**重新发放**给这一次上电，于是那个"孤儿编号"被复用而不是跳过。净效果：**可见的 BOOT 编号永远小于真实上电次数，差额等于历次 BOOT 提交失败的次数**，且没有任何地方记录这个差额——一个把"第几次上电"当作会话主键的界面，其主键会静默偏移	把 `boot_count++` 从 `eventlog.c:` `boot_recorded=1` 同一处），或在 数。前者更简单，代价是"扫描期就知

#	现象	严重度	证据（文件:行号）	影响	建议改法
31	注释声明"中断只计时"，实际 SysTick ISR 跑完了整个灯光状态机并写 TIM1 比较寄存器	中	`main.c:16` 原文：`/* 中断只计时；共用 I2C 的采样和短块刷新均在主循环执行。*/`。第二句属实（`main.c:575` 的 MPU 读与 `main.c:662` 的 `OLED_RefreshStep` 确实在主循环），**第一句不成立**：`stm32f10x_it.c:136-140` 的 `SysTick_Handler` 除 `delay_tick()`（`:138`）外还调用 `Controls_Tick()`（`:139`），而 `Controls_Tick` 在 `controls.c:141-143` 直接写 `TIM1->CCR1/2/3 = pwm_duty(light_value[k]);`（另有 `controls.c:118-120` 的频闪分支），并在 `controls.c:111-143` 里跑完 200 Hz 包络分频、淡入淡出插值（`controls.c:132-139`）与 `pwm_duty()` 换算	这条注释是"中断里能做什么"的**唯一成文契约**，它错了会让两类读者同时受害：想加中断处理的人以为中断里只有计时、于是放心在里面做长操作；想理解时序的人以为照明是主循环驱动的、于是在主循环里找它。它同时也让人低估了 ISR 的最坏执行时间（`Controls_Tick` 含整数除法与 3 次循环）	把 `main.c:16` 改成准确的职责描述 TIM1 比较寄存器更新）；共用 I2C 的 照明本不该进 ISR，则这是 15.5 节第 搬出去——但**注释至少要先停止说谎**
32	`due()` 被 `static` 关在 `main.c` 里，同一个判据在另外 6 个文件中被逐字重写 12 次	低	定义：`main.c:49` `static uint8_t due(uint32_t now, uint32_t deadline) { return (int32_t)(now - deadline) >= 0; }`。`due(` 的全部 12 次出现都在 `main.c`（定义 `:49`，调用 `:117` / `:538` / `:539` / `:540` / `:570` / `:600` / `:604` / `:616` / `:628` / `:634` / `:649`）。而同一个判据在其它文件中以 `(int32_t)(now - X) >= 0` 的形式被重写 **12 次**：`dashboard.c:38`、`dht11.c:76` / `:83` / `:90`、`eventlog.c:164` / `:175` / `:182` / `:191`、`logger.c:95` / `:108`、`oled.c:150` / `:163`	12 处重写里有 3 处（`logger.c:95`、`oled.c:150`、`dashboard.c:38`）处在超时判定这种"写错方向就下线"的位置；`int32_t` 环绕语义（`now - deadline` 的无符号相减再转有符号，正是这套代码处理回绕的方式）每抄一遍就有一次抄错的机会，而编译器不会提醒。这也是 15.5 节第 6 条"时基与调度合并"的计量依据	把 `due()` 从 `main.c` 移到 `delay.` 后把 12 处内联判据逐处替换。改动相 定义
33	`delay_ms` 依赖 SysTick 中断，但这个前提没有写进头文件	中	`delay.c:29-33`：`void delay_ms(uint32_t ms) { uint32_t start = delay_millis(); while ((uint32_t)(delay_millis() - start) < ms) { } }`。而 `tick_ms` 只由 `delay_tick()` 递增（`delay.c:17` `void delay_tick(void) { tick_ms++; }`），`delay_tick` 只在 `SysTick_Handler` 里被调用（`stm32f10x_it.c:138`）。`delay.h:6-11` 的六个声明里没有任何前置说明——唯一的暗示是写在**另一个函数**上的注释：`delay.h:11` `void delay_tick(void); /* SysTick_Handler 专用 */`。对照之下 `delay_us` / `delay_micros16` 用的是 TIM2 自由运行计数器（`delay.c:19-27`），完全不依赖中断	**只要 SysTick 中断被关闭（或长时间被高优先级中断屏蔽），`delay_ms` 就是一个死循环**——与缺陷 1、2 同一种"卡住就回不来"的失败模式，却藏在一个看起来无害的延时函数里。它被 `main.c:509`（初始化 300 ms 等待）与 `main.c:512-513`（OLED 未就绪的重试节奏）等路径调用，属于上电必经路径	在 `delay.h` 的 `delay_ms` 声明处写 时间屏蔽"），或把实现改为基于 TIM 65.536 ms 回绕需要按 `delay.c:23` 是一小时的工作
34	`delay_init` 不校验 1 MHz 整除性，这项校验被推迟到 POST 并且失败时进死循环	低	`delay.c:12` `TIM2->CR1 = 0; TIM2->PSC = timer_clock / 1000000UL - 1;`——既不检查 `timer_clock % 1000000UL`，也不检查 `timer_clock >= 1000000UL`（后者会让表达式下溢成 `0xFFFFFFFF`，截断进 16 位 PSC 后得到一个完全错误的时基）。`delay_init` 只做了 `delay.c:9` 的 `if (clocks.PCLK1_Frequency != clocks.HCLK_Frequency) timer_clock *= 2;`。真正检查这件事的是 POST 的时基项：`main.c:263-264` `if(!(TIM2->CR1&TIM_CR1_CEN) || timer_clock<1000000UL || (timer_clock%1000000UL) || TIM2->PSC+1UL!=timer_clock/1000000UL) return 0;`——判据齐全，但失败后果是把设备锁进 `main.c:518-521` 的闪灯死循环	这是全工程把"校验"与"失败处置"分家的典型：校验写得很好（`main.c:263-264` 四个条件一个不少），但放在离原因 250 行之外的 POST 里，且唯一的失败动作是永不停机的闪灯。结果是"改了一次时钟配置"这种正常开发动作会让设备变砖，而不是在 `delay_init()` 处就地报错	把 `main.c:263-264` 的四个判据前移 码，或写进一块 POST 之前就存在的 是**发现**问题。这样失败点紧邻原因，t
35	`main.c` 越过驱动层直读外设寄存器，把驱动内部的期望值抄成字面量	低	POST 的寄存器断言全是**字面量副本**：`main.c:247` 读 `SysTick->CTRL` / `LOAD`；`:263-264` 读 `TIM2->CR1` / `PSC` 并与 `timer_clock` 比对；`:272` `if((GPIOB->CRL&0xFFUL)==0x88UL && (GPIOB->ODR&3UL)==3UL)`（PB0/PB1 编码器两路输入上拉 + 两线此刻都读到高）；`:278` `if((GPIOA->CRH&0x0FFFUL)==0x0BBBUL)`（PA8/PA9/PA10 三路 RGB PWM 的复用推挽配置，见 `controls.c:59` 的注释）；`:279-283` `TIM1->CR1` / `BDTR` / `CCER` / `ARR+1==3600UL` / `CCMR1&0x6868UL` / `CCMR2&0x0068UL`（20 kHz PWM）外加 `(GPIOB->CRH&0x0FUL)==0x03UL`；`:505` 读 `RCC->CSR` 解码复位原因。运行期另有"监视量"性质的直读：`dashboard.c:249` 读 `TIM1->CCR1/2/3`（RGB 占空比）、`dashboard.c:250` 与 `controls.c:164-184` 读 `GPIOB->IDR`（编码器 A/B）、`dht11.c:78` 读 `GPIOA->IDR`	上面那些字面量是 `controls.c:49-72`、`delay.c:12-13`、`oled.c:30-40` 里配置值的**副本**，改驱动参数时没有任何机制提醒同步 `main.c`；而失败动作是把设备锁进 `main.c:518-521` 的闪灯死循环（与缺陷 34 同一失败模式）。具体例子：`main.c:279-283` 断言 `TIM1->ARR+1==3600UL`，而 3600 这个数由 `controls.c` 的 PWM 周期决定——改一次调光频率就让 POST 报 `ERR` 并把设备锁死。两个各自"能跑就对"的部分因此互相绑死	让驱动导出自己的自检结论（例如 `Controls_PostSelfCheck()` / `Del` 各返回一个布尔或错误码），`main.` 第 4 项"驱动自检下沉到驱动层"要解 动：把 `:278` 与 `:279-283` 的魔数 `controls.h` 注明"改 PWM 周期必 **（`controls.c:151` 的 Speed 让 `m`**

#	现象	严重度	证据（文件:行号）	影响	建议改法
36	POST 的 `OUTPUTS` 项今天就在误报：`controls.c:151` 设的 `GPIO_Speed_2MHz` 被 RGB 复用，`GPIOA->CRH` 恒为 `0xAAA` 而断言要求 `0xBBB`	中	`controls.c:148` 声明 `GPIO_InitTypeDef gpio;`，**整个函数只有这一个结构体实例**。第一次用它配 GPIOB 的编码器与按键：`controls.c:150-152` `gpio.GPIO_Pin = GPIO_Pin_0|GPIO_Pin_1|GPIO_Pin_10; gpio.GPIO_Mode = GPIO_Mode_IPU; gpio.GPIO_Speed = GPIO_Speed_2MHz; GPIO_Init(GPIOB,&gpio);`。随后**复用同一个结构体**配 RGB：`controls.c:161-162` `gpio.GPIO_Pin = GPIO_Pin_8|GPIO_Pin_9|GPIO_Pin_10; gpio.GPIO_Mode = GPIO_Mode_AF_PP; GPIO_Init(GPIOA,&gpio);` ——只改了 `Pin` 与 `Mode`，**`GPIO_Speed` 仍是 2 MHz**。而 SPL 在输出模式下确实会把 Speed 写进配置位：`level/Libraries/STM32F10x_StdPeriph_Driver/src/stm32f10x_gpio.c:182-189`（`currentmode = GPIO_Mode & 0x0F;` 之后 `if ((GPIO_Mode & 0x10) != 0) currentmode |= GPIO_Speed;`）。取 `GPIO_Mode_AF_PP = 0x18`（`stm32f10x_gpio.h:78`）与 `GPIO_Speed_2MHz`（`stm32f10x_gpio.h:59-61` 的枚举从 `GPIO_Speed_10MHz = 1` 起算，故其值为 2），得 `currentmode = (0x18&0x0F)|0x02 = 0x0A`，PA8/PA9/PA10 三个 nibble 全为 `0xA` ⇒ `GPIOA->CRH & 0x0FFF == 0xAAA`。POST 第 10 项却要求 `0xBBB`（`main.c:278` `(GPIOA->CRH&0x0FFFUL)==0x0BBBUL`），于是 `main.c:461` `results[step]=(char *)(post_outputs_config()?"CFG":"ERR");` 恒取 `"ERR"` ——它在 `main.c:286-290` 的 `post_names` 里就是第 10 项 **`OUTPUTS`**	这是一条**开箱即有的确定性假阴性**：RGB 三路 PWM 功能完全正常（`controls.c:60-61` 把 TIM1 配成 20 kHz，而 MODE 位只管输出驱动的速度、不影响波形，2 MHz 对 20 kHz 方波绰绰有余），但每次开机都在 POST 里出现一个 `ERR`。真实代价不是功能，而是**把 POST 从"可信的健康检查"降级成"总有一个红叉的装饰品"**——用户会学会忽略它，于是真正该看的 `W25QXX` / `LOG AREA` / `EVENT` 三项失败也一起被忽略。与缺陷 35 完全同类（POST 把实现细节抄成期望值），但比 35 更硬：35 要有人改参数才发作，36 **今天就在误报。**反向证据也说明这是"不一致"而非"设计如此"：`buzzer.c:14-15` 明确写了 `GPIO_Mode_Out_PP` + `GPIO_Speed_50MHz`，所以 `main.c:283` 对 PB8 的 `(GPIOB->CRH&0x0FUL)==0x03UL` 断言**能过**	二选一，与缺陷 10 同一口径：**要么让**一行 `gpio.GPIO_Speed = GPIO_Sp`致、也与 `main.c:283` 对蜂鸣器的 `main.c:278` 的 `0x0BBBUL` 改成 `0x`**迹**：下一个读代码的人会把 `0xAAA` 当

15.4 可复用的工程资产

这是本章最想留下的一段：这套固件里有一部分东西与"十页仪表"这个具体需求无关，换一个项目仍然值得照搬。

资产	位置	为什么值得复用
提交字节 + 回读的三段式日志协议	`logger.c:123-138`、`eventlog.c:171-189`、`eventlog.c:11`（`EVENT_COMMITTED 0xA5U`）、`logger.c:5-6` 的设计注释	断电发生在一行 Flash 写入中间的撕裂写，是嵌入式存储最容易被忽略的失败模式。这里用"数据不完整但提交标志没写 → 记录自动不可见"把问题消掉，而且写入顺序是硬的：先校验数据回读一致，再单独写第 32 字节
32 字节差分块刷新 + 已发送缓存	`oled.c:179-206`、契约注释 `oled.c:132-134`	把一次显示刷新切成 32 个独立小事务，任何两个小事务之间都可以让出 CPU；配合"只发与已发送缓存不同的块"，在静态画面上几乎不产生总线流量。这套结构可以平移到任何一个"慢设备 + 需要实时采样"的场合
软件 I2C + 卡总线恢复	`i2c.c:55-81`、`i2c.c:83-99`、速率标定注释 `i2c.h:29-36`	开漏 + 直接读 IDR 实现双向线，遇到"从机把 SDA 拉死"时用一串时钟脉冲把它顶出来（`SoftI2C_BusRecover()` 让 OLED 掉电重建能自愈）。比"多引一条复位线"更省引脚，也是本工程能在 PB6/PB7 上挂两个从机的前提
相对开机基准的标定	`main.c:108-132`	100 点采样、至少 90 点有效、逐轴峰峰值不超过 `CAL_MOTION_LSB 1200` 拒绝"边动边标"、`MIN_CAL_Z 8000` 拒绝非水平放置。它同时解决了两个问题：克隆件的灵敏度偏差，以及"绝对零点漂移"这个无法在出厂时定标的问题
Q8.8 定点滤波 + 先乘后除的换算	`main.c:129`（`lpf[k]=cal_sum[k]/cal_good*256`）、`main.c:586-587`	全程不用浮点，整数乘除顺序被刻意安排成"先放大再相除"，既省掉 FPU 又避免了小整数相除的精度崩塌。在 M3 这类没有硬件浮点的核上是标准做法，值得写成模板
格雷码查表解码 + 非法跳变清零	`controls.c:170-179`、`controls.h:5`	机械编码器在 1 ms 轮询下必然出现非法状态跳变；这里用 16 项表 + "跳变非法就清零累计"把噪声变成"不响应"而不是"乱翻页"，是安全方向的失败模式
依赖运行时时钟反推的 PWM 初始化	`controls.c:49-72`（`PSC` 由 `RCC_GetClocksFreq()` 反推，不写死 72）	定时器参数不写死时钟频率，改主频或换晶振时不必重新推导分频；这个习惯与缺陷 22 正好相反，值得对照
非阻塞单总线状态机 + EXTI 位解码	`dht11.c:73-103`（三态状态机）、`dht11.c:53-71`（中断里按边沿间隔解码）	单总线协议最容易被写成"拉低 → 忙等若干毫秒 → 读"的阻塞函数；这里把它拆成三个状态并在中断里只做时间戳与位拼接，使 20 ms 的独占期变成可让位的阶段
启动自检 + 屏幕逐项显示	`main.c:218`（`POST_STEP_COUNT 11`）、`main.c:286-290`（名称表）、`main.c:354-360`（画一帧并等刷新）	把"哪一项过了、哪一项没过"直接显示在屏幕上，比"闪一下灯表示失败"可诊断得多。11 项自检的结构（真实通信/回读/扫描/写入校验完成才显示结果）值得照搬

15.5 如果重写：架构层面的六个决定

1. **把总线做成一个有所有权的事务接口，而不是一组约定。** 现在是四处手写让位条件（缺陷 7）；重写时让 `bus_acquire(owner)/bus_release()` 成为唯一入口，断言"没人持有时才能拿"，并把"谁能占多久"写成参数。这一步能同时解掉缺陷 6、7、14。
2. **任何状态机都不允许有终态。** 每个状态机都必须提供 `*_Recover()`，并且由超时驱动进入（缺陷 3）。配套规则是：**"只读不写"的降级模式永远要保留**——即使日志区坏了，HISTORY/SESSIONS 仍应能显示"区域不可用"，而不是让整个页失效。
3. **错误通道分级。** 最低一级是错误码寄存器（一块 RAM，POST 后不清零），第二级是可数的闪灯模式（而不是 `for` 循环计数的忙等），第三级是 UART 或 SWO（缺陷 13、17）。有了前两级，缺陷 1 的现场就有了可读的出口。
4. **页号、页名、页面函数单点定义。** 一张 `{page_id, name, draw_fn}` 表（缺陷 15），页数只出现在表的长度里。这与本工程已有的 `DashboardView` 数据契约（`dashboard.h:12-25`）正好互补：数据已经集中了，控制流还没有。
5. **异常处理统一为"记录现场 + 复位"。** 四个 handler 共用一段汇编/C 混合的现场保存例程（缺陷 1），记录 `CFSR/HFSR/BFAR` 与 `MSP/PSP`，然后 `NVIC_SystemReset()`；复位后由 `reset_reason()` 的既有通道把它显示出来——这条链路本工程已经有一半（`main.c:505-507` 在读 `RCC->CSR`）。

6. **时基与调度合并。** 现在 `delay`（第 2 章）提供三种取时间的方式，各模块再用散落的 `due(now, next)` 自己排程；重写时把它换成"一个软件定时器轮 + 每轮遍历一次到期项"，主循环里那些 `xxx_next` 变量就能收进表里（缺陷 5 的 `post_pump()` 也会自然消失）。

15.6 三阶段改进路线

第一阶段：低风险、可独立验证（建议 1 天）

#	改什么	为什么值得	验收方式
1	四个 Cortex 异常处理改为"记录现场 + `NVIC_SystemReset()`"（`stm32f10x_it.c:57-102`）	缺陷 1 是全表唯一"一次错误就要求人拔电"的项，改动局限在一个文件	用调试器手动改 `PC` 指向非法地址制造 HardFault；板子应自动复位并把 `RESET` 显示为 SOFT 而不是停在黑屏
2	在 `main.c:586-587` 前加显式零守卫（缺陷 9）	把跨越 450 行的隐含不变量变成就地可读的代码	临时把 `MIN_CAL_Z` 改成 0 使标定可通过但 `base_az` 为 0，两行不应被执行；恢复后正常标定仍工作
3	MPU 温度这条路**二选一**：删净（`mpu6050.c:96-99` + `mpu6050.h:43`）或接通（在 `main.c:579` 之后真正调用，并按 `temp_raw*100L/340+3653` 先算百分度）（缺陷 10）	现状既不是"能用"也不是"没有"：`MPU6050_TempToC()` 已被链接器删除（`level.map:943`），DEBUG 页只透传原始量（`dashboard.c:200`），而 CLIMATE 页的温度本来就来自 DHT11（`dashboard.c:146` ← `dht11.c:96`）——两条路径互不相干	若选"接通"：DEBUG 页应显示换算后的摄氏度，与已知温度计对比偏差小于 1 ℃；若选"删净"：全项目搜索 `MPU6050_TempToC` 命中数应为 0，且 DEBUG 页保留 `TEMP RAW` 字样、CLIMATE 页注明温度来源为 DHT11。**两种选法都不应留下"声明存在、实现被链接器删掉"的中间态**
4	`logger.c:82` 允许 `flags` 为 0 的记录写入（缺陷 11）	让"只有运行时长有效"的检查点进入 HISTORY，与设计目的一致	断开 DHT11 后运行 3 分钟，HISTORY 页应出现新记录且温湿度字段显示为无效而不是整条缺失
5	删除 5 个死宏或加注释（缺陷 20）	消除"看起来可调其实无效"的陷阱	全项目搜索这 5 个名字，命中数应从 5 降为 0（或只剩带说明的注释）
6	`dashboard.c:45-52` 把 `"LOCKED"` 改回 `"FOREIGN"`（缺陷 21）	现场一次就能把屏幕文字与状态机状态对应起来	人为把 Flash 上半区写成非本机格式后上电，HISTORY 页显示应为 `FOREIGN`

第二阶段：结构性修补（建议 3–5 天）

#	改什么	为什么值得	验收方式
1	加入 IWDG 喂狗，喂狗条件用模块位掩码（缺陷 2）	把缺陷 1 的"复位"从被动变主动，卡死也能自愈	在主循环里插入一段人为死循环，板子应在约 1～2 s 内自动复位，并在 SYSTEM 页显示复位原因为 IWD
2	给两类日志各加 `*_Recover()` 并允许终态低频重试（缺陷 3）	解掉"换板/换固件后本次上电再也不写日志"这个正常场景下的永久失效	制造 `LOG_FOREIGN`：上电后应显示 FOREIGN，且约 5 分钟后自动重新初始化；制造 `EVENT_ERROR` 后 SESSIONS 页应能恢复
3	42 处 `sprintf` 换 `snprintf`（缺陷 8）	把栈上越界这类"偶发死循环"的根因去掉	全部页面逐页浏览一遍不再出现异常；用 `sizeof` 与最长格式串做一次静态清查，不应有格式化输出超过缓冲声明
4	给 `bus_error` 加重同步清零路径（缺陷 4）	一次瞬时总线错误不再固化到下次上电	在 Flash 读写过程中人为拉低 Flash 供电再恢复，日志应能在若干次重试后继续写入而不是永久 ERROR
5	补齐 7 个页宏并把分派改成表（缺陷 15）	页号只出现在一处，插页不再需要改多处	在表里临时插入一页，编译应通过、翻页顺序正确、无遗漏的裸数字判断
6	事件日志解除对 `Logger_State()` 的启动依赖（缺陷 16）	传感器日志坏掉时，复位溯源仍然可用	制造传感器日志 ERROR，SESSIONS 页与 BOOT 记录应仍能读取与写入

第三阶段：重写级（作为下一版的地基）

#	改什么	为什么值得	验收方式
1	总线所有权接口（缺陷 6、7、14）	让"谁在用总线"从注释约定变成可断言的状态	新增一个总线用户时，编译器/断言应阻止忘记让位的写法；长时间运行不出现显示撕裂
2	分级错误通道：错误码寄存器 + 可数闪灯 + UART/SWO（缺陷 13、17）	把"只能看屏幕"变成"能取证"	人为制造每类故障，能在串口（或错误码寄存器）里读到不同的错误码
3	统一调度：软件定时器轮替代散落的 `due()`（缺陷 5）	`POST` 的第二套调度器会自然消失，主循环结构变简单	主循环里不再出现 `xxx_next` 变量；`POST` 的等待窗口里按键仍被采样
4	驱动自检下沉到驱动层，`main.c` 只做调度（缺陷 35、36，与第 1 章问题 2 同源）	改驱动参数时不会因为忘记同步 `main.c` 的期望值而把设备锁死在闪灯循环里	修改 `controls.c` 的 PWM 周期后，POST 应由驱动自己报告不一致，而不是 `main.c` 里的字面量比较失败。**注意缺陷 36 的那一行修法（`controls.c:162` 前补 `gpio.GPIO_Speed = GPIO_Speed_50MHz;`）不必等这一项落地，应立刻做**——它今天就让人每次开机看到一个假 `ERR`

第二批追加项（对应 15.3.2 的编号 25–34）

15.3.2 节的 10 条没有进入上面的三张表，因为它们全部能塞进既有的档位、且严重度都在中/低。逐条归位如下，不改变前三阶段的定义。

并入第一阶段（低风险、可独立验证，合计约半天）

#	改什么	为什么值得	验收方式
25	在 `SENSOR_BYTES` / `EVENT_BYTES` / `LOGGER_SLOTS` 的 `#define` 之后加三条 `_Static_assert`	把"改常量导致擦除永久失败"从运行期静默故障变成编译期报错，成本是三行	故意把 `EVENT_BYTES` 改成 9000，编译必须失败并给出断言文字；改回后正常通过
32	把 `due()` 移出 `main.c`（放进 `delay.h` 去掉 `static`），替换 12 处内联判据	超时判据只剩一个定义，`int32_t` 环绕语义不会再被抄错	全项目搜索 `(int32_t)(now -` 命中数应从 12 降为 1（只剩 `due()` 自身）；逐页翻动界面，超时行为不变
33	在 `delay.h` 的 `delay_ms` 声明处写明"要求 SysTick 中断已使能"	一行注释，挡掉"在临界区里调用 `delay_ms` "这一类必挂的用法	注释到位即可验收；可选：临时在 `__disable_irq()` 之后调用 `delay_ms`，确认现象与注释一致

#	改什么	为什么值得	验收方式
27	删除 `OLED_F8x16`（1520 字节）与 `i2c.h:47-50` 的流式 API 声明	回收约 5% 的映像空间，并移走工程里最长的死数据块	重新构建后 `Program Size` 的 ROM 应减少约 1520 字节；`level.map` 的 `Removing` 清单里不再出现 `oled.o(.constdata)` 被引用的痕迹
28	`small_bits()` 把 `'a'..'z'` 映射到大写（或返回缺字方块）	一行代码，消除"小写字母渲染成空白"这个潜伏症状	DEBUG 页临时显示一句小写字符串，应显示为大写而不是空白
31	修正 `main.c:16` 的注释，使其与 `SysTick_Handler` 的实际职责一致	注释是"中断里能做什么"的唯一成文契约，必须停止说谎	逐字比对 `main.c:16`、`stm32f10x_it.c:136-140`、`controls.c:141-143`，三者描述一致

并入第二阶段（结构性修补）

#	改什么	为什么值得	验收方式
26	把 `!DHT11_Busy()` 门从 `OLED_Service` 移到 `OLED_RefreshStep`（`main.c:661-662`）	让互锁只挡住真正的长事务（约 1 ms），短命令事务可以放行	连续运行 10 分钟，CLIMATE 页不应再出现偶发 `--`；OLED 对比度渐变仍正常
29	扫描期限移出 `if(busy)`，并让超时保留已扫结果转 `LOG_READY`	期限变成真正的进度看门狗；触发时不再丢弃最多 2047 条已经数出的记录	人为让扫描中途超时：`Logger_Count()` 应保留超时前已统计的记录数，而不是回到 0
30	把 `boot_count++` 移到 `eventlog.c:185` 的成功分支	让 BOOT 编号重新等于真实上电次数，SESSIONS 页的主键不再静默偏移	人为让 BOOT 提交失败一次（写入过程中断电），下一次上电的 `BOOT #n` 应比上一次大一，不跳号也不复用
34	把 `main.c:263-264` 的四个时基判据前移进 `delay_init()`	让"改时钟配置"的失败点紧邻原因，且不再触发闪灯死循环	临时把 `SystemCoreClock` 改成非整兆赫值：`delay_init()` 应就地报错，而不是在 `main.c:520` 的死循环里闪灯

随第三阶段（重写级）一起处理

- 编号 31 的注释只是止血；只要照明状态机还留在 ISR 里，"中断里能做什么"就仍然是一个需要人工遵守的约定。15.5 节第 6 条（时基与调度合并）把它与 `post_pump()` 一起收进统一的软件定时器轮，才是终态。

15.7 工程结论

规模与能力不匹配得恰到好处。 这套固件用 28,912 字节代码 + 2,524 字节只读数据（`level/MDK-ARM/build.log` 的 `Program Size` 行）撑起了十页界面、两块独立的 Flash 日志、一条挂了两个从机的软件 I2C 总线，以及一条共享 SPI 总线上的持久化路径。ROM 侧只有约 31 KB，RAM 侧 420 + 7,628 = 8,048 字节——这里面没有任何 RTOS、任何动态内存、任何 DMA、任何互斥锁（15.3 节第 2 条的搜索证据）与任何第三方库。这个事实本身就是结论：**在一个 64 KB Flash / 20 KB RAM 的 Cortex-M3 上，把"实时显示 + 掉电安全持久化 + 非阻塞传感器"三件事同时做对，并不需要引入上面那些机制。**

代价被诚实地摊在了每一处。 不用 RTOS，就换来"主循环里所有让位条件都要手写"（缺陷 7）和"POST 必须自建第二套调度器"（缺陷 5）；不用互斥锁，就换来"不受保护的显存契约"（缺陷 6）；不做 DMA，就换来"刷屏和采样共享一个执行流"（缺陷 23）。本书逐章标出了这些手写规则的位置——这正是同类项目最容易失传的部分：代码还在，但"为什么这两件事不能同时做"只存在于作者的短期记忆里。第 11 章 11.3 节把四处让位条件写成"三道（实为四道）互锁"，就是在给这类知识留档。

它比多数爱好者固件做得好的地方，是把失败模式当成了设计对象。 日志的三段式提交协议（`logger.c:123-138`、`eventlog.c:171-189`）正视了撕裂写；`SoftI2C_BusRecover()`（`i2c.c:55-81`）正视了从机拉死总线；`bus_error` 超时熔断（`w25qxx.c:9-19`）正视了 SPI 永久等待；`DHT11_Busy()` 的 20 ms 独占期（`dht11.c:78-82`）正视了单总线协议的物理要求；32 字节差分块（`oled.c:179-206`）正视了慢设备与实时采样的冲突。这五个机制合起来，解释了这个项目为什么能在没有操作系统的情况下稳定显示十页内容。

它比多数爱好者固件缺的，是恢复与可观测（recovability & observability）。 三个高严重度缺陷指向同一个空白：设备一旦进入坏状态，既不会自己回到好状态（缺陷 3、4），也不会复位重来（缺陷 1、2），更不会告诉人发生了什么（缺陷 13）。这不是"高级功能"，而是三条最低限度的工程要求：**能复位、能降级、能取证。** 15.6 节的第一、第二阶段基本就是在补这三件事，而它们的实现成本远低于本工程已经投入在协议层的成本——换句话说，这套固件已经付出了 90% 的复杂度去把事情做对，只差最后 10% 去保证"做错时能回来"。

给接手者的最后一句： 阅读顺序仍建议按第 1 章 1.7 节；动手改之前先读 15.6 节的第一阶段并做完——那六项的改动量加起来约一天，但它们决定了后面所有调试工作的效率，因为在那之前，任何一次崩溃都只能靠拔电和猜测。

15.8 最小验收清单

下面是"把设备接到板子上就能做、每条都有可观测判据"的十项。前六项验证本书的结论，后四项验证前三阶段的改动没有引入回归。

#	动作	可观测判据
1	上电后不触碰任何东西，等 10 秒	POST 11 项全部显示结果（真实通信/回读成功才出现 `DONE`），LED 进入 1 s 心跳（`main.c:540`），随后 LEVEL 页稳定显示倾角
2	依次旋转编码器走完 10 页	页名顺序与 `dashboard.c:321` 的 `titles[]` 完全一致；LEVEL 页刷新率高于其它页（`main.c:659` 的 33 ms / 50 ms 两档）
3	在任意页短按按钮一次	屏上出现 `SAVING`，1.5 s 内变为 `SAVED`（`main.c:621`）；HISTORY 页多一条记录
4	拔掉 DHT11 后连续运行 3 分钟，再拔掉 MPU6050 运行 1 分钟	CLIMATE 页显示 `--` 与 `WAIT` / `OLD`；LEVEL 页显示 `--` 而不是残留数值；LED 转入错误指示（`main.c:644-646`）；拔掉 MPU6050 后不应出现界面卡死
5	运行 2 分钟后立刻断电，重新上电	SESSIONS 页显示上一会话的 `PREV >= n MIN`（下界精度约一分钟）；若从未写出检查点则显示 `NO CHECKPOINT`
6	在每次空闲时打开 RUNTIME 页并记下时长，连续观察 5 分钟	时长单调递增、无跳变、无回绕；同时拨打页面不应出现花屏或残留像素（这是缺陷 6 的现场观察点）
7	制造 HardFault（调试器改 PC 指向非法地址）	加入第一阶段的第 1 项之后：设备应自动复位，`RESET` 显示为 SOFT；改动之前：设备停在最后一帧不再响应——两者都要记录，作为改进的对照
8	在主循环插入一段人为死循环	加入看门狗之后：设备应在 1 ~ 2 s 内自动复位并在 SYSTEM 页显示复位原因为 IWD
9	用烧录器把 Flash 上半区（记录区）写成随机数据后上电	传感器日志应显示 `FOREIGN`（改动后）而不是 `LOCKED`；5 分钟内应自动重新初始化；期间 SESSIONS 页仍可读取

#	动作	可观测判据
10	在 Flash 读写过程中人为拉低 Flash 供电再恢复	加入第二阶段第 4 项之后：日志应在若干次退避重试后恢复写入；屏幕在恢复前后都不应卡死

本章发现的问题与建议

本章的结论来自对源码的静态阅读与逐行核对，**没有一条在实物上复现过**。按本书的规矩，这里如实登记本章自身的局限，以及需要在板上才能定论的候选。

局限 1：本章的严重度是"静态可达性"推断，不是实测频率。

- 现象：15.3 节把缺陷 1～4 判为高，依据是"一旦发生就永久失效、且需要人工断电"，而不是"实际发生过多频繁"。例如 `bus_error`（缺陷 4）与 `LOG_FOREIGN`（缺陷 3）都需要外部条件触发（供电跌落、换板），正常使用下可能一次都不出现。
- 影响：读者可能把"高"误读成"经常发生"。这两种排序方法（影响严重度 vs 发生频率）在本章被合并成了一个轴。
- 建议：把本章第 4 项的验证动作（人为拉低 Flash 供电）纳入实机验收，用真实触发难度校正严重度；若长期不触发，可降级为中。

局限 2：42 处 `sprintf` 的"当前不溢出"是估算，不是证明。

- 现象：15.3 节第 8 条的长输出估算基于 `dashboard.c:305` 的 `RUN %luD %02lu:%02lu:%02lu` 在 32 位范围内的上界。估算覆盖了每个格式串的类型宽度，但没有逐条构造极值输入实测。
- 影响：若某处 `%s` 的实际取值为异常长字符串（例如 `v->reset_reason` 来自 `main.c:50-58` 的固定表，风险低；但 `toast` 的来源较分散），估算结论不成立。
- 建议：与其继续论证，不如直接做第二阶段的第 3 项（换 `snprintf`）——成本低于验证成本。

局限 3：缺陷 12（编码器快转丢步）与缺陷 14（20 ms 独占期）都需要手上的转速/时序才能定论。

- 现象：本章只能断定"1 ms 采样周期存在上限"，无法给出"每秒多少格以内可靠"。
- 影响：这条描述对用户不够可操作。
- 建议：在板上从慢到快旋转，记录"开始出现不翻页"的转速与当时 DEBUG 页的非法跳变计数（若按第二阶段建议把该计数暴露出来），把结论补成一个具体数字。

局限 4：缺陷 17 的"断言被编译掉"没有检查 `USE_FULL_ASSERT` 是否在其它地方被定义。

- 现象：`stm32f10x_conf.h:17-22` 显示未定义时 `assert_param` 退化为空；但工程的编译选项（`level.uvprojx` 的 `<Define>` 字段）本章没有逐字读完，因此"发布构建下所有 SPL 断言都被编译掉"这个判断的前提是"`USE_FULL_ASSERT` 未定义"。
- 影响：若该宏实际被定义，本条应从"断言失效"改为"断言生效但 `assert_failed()` 未实现"（后者会更糟：SPL 默认的 `assert_failed` 是死循环）。
- 建议：接手者先 `grep USE_FULL_ASSERT level/MDK-ARM/level.uvprojx` 核实一次再动手；这条不影响第二阶段的其它项。（对照第 1 章对 `level.uvprojx` 的逐字段核对结果，该工程的编译选项以 `-O1`、C99 为特征，本章未在其 Define 列表里看到 `USE_FULL_ASSERT`。）

局限 5：本书的"死代码/映像占用"结论必须以链接器映像为终点判据——静态阅读源码永远看不出哪些函数真的进了二进制。

- 现象：本章有三条结论是靠链接器映像（`level/MDK-ARM/List/level.map`）才定下来的，而不是靠读源码：`MPU6050_TempToC()` 是死代码（`level.map:943 Removing mpu6050.o(i.MPU6050_TempToC), (16 bytes).`——它直接推翻了本书初版把该函数当成在用路径的判断，见缺陷 10）；`OLED_F8x16` 的 1520 字节仍留在映像里（`level.map:2272 OLED_F8x16 0x080071dc Data 1520 oled.o(.constdata)`，见缺陷 27）；`SoftI2C_Begin` / `SendByte` / `End` / `ReadBytes` 共 148 字节被剔除（`level.map:933-936`）。缺陷 27 那一行里引用的映像构成（代码 28,912 字节 + 只读数据 2,524 字节）同样只可能来自 `.map`。
- 影响：任何"这个函数没人调用"的说法，在只读源码时都只是**推断**——源码里可能有函数指针、可能有未编进当前构型的条件编译，也可能像 `MPU6050_TempToC` 这样被链接器的段级垃圾回收顺手删掉。反过来，"这段代码在源码里存在"也不等于"它占用了 Flash"。本书凡是涉及**死代码、映像占用、体积**的结论，判据都在 `.map`；这一点必须在方法论上说清楚，否则读者会以为它们是源码阅读的产物，进而在改动后失去复核手段。
- 建议：读者接手后第一件可自己做的事就是复核这份映像——`level/MDK-ARM/List/level.map` 在每次构建时重新生成，搜 `Removing` 得到全部被剔除的符号，搜 `Image component sizes`（本次在 `level.map:2585`）得到映像构成。**注意工程里还有一份过期副本** `level_dashboard_2026-10-09/MDK-ARM/List/level.map`，用它去查会得到另一套答案（同一符号在同一副本里是 `(18 bytes)` 而非 `(20 bytes)`，`OLED_DrawRect` 也在删除清单里，`OLED_F8x16` 的地址是 `0x08004e78`）——**必须先确认自己在读哪一次构建的映像**。把"哪些代码真的存在"当作一个需要取证的问题，而不是一个可以靠阅读回答的问题：缺陷 10 那次更正之所以必要，错不在推理，而在**没有第一时间去查那个能给出确定答案的文件**。